Codigo Fuente

NomiCore - Sistema de Gestion de Nomina

Proyecto: ProyFinal_LPI_Eq01_NomiCore

Tecnologia: C# 7.3, Windows Forms .NET Framework 4.8 y SQLite

Tema: Manejo de campos en las clases y PrintDocument

Contenido del documento

96 archivos C# incluidos

15,597 lineas de codigo y herramientas

Se excluyen: bin, obj, .git, .vs, packages y salidas de exportacion.

Generado el 15/07/2026 01:55.

Data/AbsenceRequestRepository.cs

```
using System;
using System.Collections.Generic;
using System.Data.SQLite;
using SistemaGestionNomina.Models;

namespace SistemaGestionNomina.Data
{
    public sealed class AbsenceRequestRepository
    {
        private const string SelectSql = @"SELECT s.*, e.Codigo AS CodigoEmpleado,
                e.Nombre || ' ' || e.Apellido AS EmpleadoNombre,
                d.Nombre AS DepartamentoNombre
            FROM SolicitudesAusencia s
            INNER JOIN Empleados e ON e.IdEmpleado = s.IdEmpleado
            INNER JOIN Departamentos d ON d.IdDepartamento = e.IdDepartamento ";

        public List<SolicitudAusencia> Search(AbsenceQuery query, int? forcedEmployeeId,
            int? scopeDepartmentId)
        {
            query = query ?? new AbsenceQuery();
            List<SolicitudAusencia> items = new List<SolicitudAusencia>();
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteCommand command = new SQLiteCommand(SelectSql + @"
                WHERE (@forcedEmployee = 0 OR s.IdEmpleado = @forcedEmployee)
                  AND (@empleado = 0 OR s.IdEmpleado = @empleado)
                  AND (@departamento = 0 OR e.IdDepartamento = @departamento)
                  AND (@scopeDep = 0 OR e.IdDepartamento = @scopeDep)
                  AND (@tipo = '' OR s.Tipo = @tipo)
                  AND (@estado = '' OR s.Estado = @estado)
                  AND (@desde = '' OR s.FechaFin >= @desde)
                  AND (@hasta = '' OR s.FechaInicio <= @hasta)
                  AND (@search = '' OR e.Codigo LIKE @like OR e.Nombre LIKE @like
                       OR e.Apellido LIKE @like OR s.Motivo LIKE @like)
                ORDER BY s.FechaCreacion DESC, s.IdSolicitud DESC;", connection))
            {
                command.Parameters.AddWithValue("@forcedEmployee", forcedEmployeeId ?? 0);
                command.Parameters.AddWithValue("@empleado", query.EmployeeId ?? 0);
                command.Parameters.AddWithValue("@departamento", query.DepartmentId ?? 0);
                command.Parameters.AddWithValue("@scopeDep", scopeDepartmentId ?? 0);
                command.Parameters.AddWithValue("@tipo", query.Type ?? string.Empty);
                command.Parameters.AddWithValue("@estado", query.Status ?? string.Empty);
                command.Parameters.AddWithValue("@desde", query.From.HasValue ? query.From.Value.ToString("yyyy-MM-dd") : string.Empty);
                command.Parameters.AddWithValue("@hasta", query.To.HasValue ? query.To.Value.ToString("yyyy-MM-dd") : string.Empty);
                command.Parameters.AddWithValue("@search", query.Search ?? string.Empty);
                command.Parameters.AddWithValue("@like", "%" + (query.Search ?? string.Empty) + "%");
                using (SQLiteDataReader reader = command.ExecuteReader())
                {
                    while (reader.Read()) items.Add(Map(reader));
                }
            }
            return items;
        }

        public SolicitudAusencia GetById(int requestId)
        {
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            {
                return GetById(connection, null, requestId);
            }
        }
```

Data/AbsenceRequestRepository.cs (continuacion)

```
        public SolicitudAusencia GetById(SQLiteConnection connection, SQLiteTransaction transaction,
            int requestId)
        {
            using (SQLiteCommand command = new SQLiteCommand(SelectSql +
                " WHERE s.IdSolicitud = @id LIMIT 1;", connection, transaction))
            {
                command.Parameters.AddWithValue("@id", requestId);
                using (SQLiteDataReader reader = command.ExecuteReader())
                {
                    return reader.Read() ? Map(reader) : null;
                }
            }
        }

        public int Add(SQLiteConnection connection, SQLiteTransaction transaction, SolicitudAusencia item)
        {
            using (SQLiteCommand command = new SQLiteCommand(@"INSERT INTO SolicitudesAusencia
                (IdEmpleado, Tipo, FechaInicio, FechaFin, Estado, Motivo,
                 UsuarioSolicitante, FechaCreacion)
                VALUES (@empleado, @tipo, @inicio, @fin, @estado, @motivo, @usuario, @creacion);
                SELECT last_insert_rowid();", connection, transaction))
            {
                command.Parameters.AddWithValue("@empleado", item.IdEmpleado);
                command.Parameters.AddWithValue("@tipo", item.Tipo);
                command.Parameters.AddWithValue("@inicio", item.FechaInicio.ToString("yyyy-MM-dd"));
                command.Parameters.AddWithValue("@fin", item.FechaFin.ToString("yyyy-MM-dd"));
                command.Parameters.AddWithValue("@estado", AbsenceStates.Pending);
                command.Parameters.AddWithValue("@motivo", item.Motivo ?? string.Empty);
                command.Parameters.AddWithValue("@usuario", item.UsuarioSolicitante ?? string.Empty);
                command.Parameters.AddWithValue("@creacion", item.FechaCreacion.ToString("yyyy-MM-dd HH:mm:ss"));
                return Convert.ToInt32(command.ExecuteScalar());
            }
        }

        public void UpdateState(SQLiteConnection connection, SQLiteTransaction transaction, int requestId,
            string expectedState, string newState, string resolvedBy, DateTime resolvedAt, string observation)
        {
            using (SQLiteCommand command = new SQLiteCommand(@"UPDATE SolicitudesAusencia
                SET Estado = @nuevo,
                    AprobadoPor = CASE WHEN @resolver = '' THEN AprobadoPor ELSE @resolver END,
                    FechaAprobacion = CASE WHEN @resolver = '' THEN FechaAprobacion ELSE @fecha END,
                    ObservacionResolucion = @observacion
                WHERE IdSolicitud = @id AND Estado = @anterior;", connection, transaction))
            {
                command.Parameters.AddWithValue("@nuevo", newState);
                command.Parameters.AddWithValue("@resolver", resolvedBy ?? string.Empty);
                command.Parameters.AddWithValue("@fecha", resolvedAt.ToString("yyyy-MM-dd HH:mm:ss"));
                command.Parameters.AddWithValue("@observacion", observation ?? string.Empty);
                command.Parameters.AddWithValue("@id", requestId);
                command.Parameters.AddWithValue("@anterior", expectedState);
                if (command.ExecuteNonQuery() != 1)
                    throw new InvalidOperationException("La solicitud cambió de estado antes de completar la operación.");
            }
        }

        public bool HasApprovedOverlap(SQLiteConnection connection, SQLiteTransaction transaction,
            int employeeId, DateTime start, DateTime end, int ignoreRequestId)
        {
            using (SQLiteCommand command = new SQLiteCommand(@"SELECT COUNT(1) FROM SolicitudesAusencia
```

Data/AbsenceRequestRepository.cs (continuacion)

```
                WHERE IdEmpleado = @empleado AND Estado = 'Aprobada' AND IdSolicitud <> @ignorar
                  AND FechaInicio <= @fin AND FechaFin >= @inicio;", connection, transaction))
            {
                command.Parameters.AddWithValue("@empleado", employeeId);
                command.Parameters.AddWithValue("@inicio", start.ToString("yyyy-MM-dd"));
                command.Parameters.AddWithValue("@fin", end.ToString("yyyy-MM-dd"));
                command.Parameters.AddWithValue("@ignorar", ignoreRequestId);
                return Convert.ToInt32(command.ExecuteScalar()) > 0;
            }
        }

        public bool AttendanceExists(SQLiteConnection connection, SQLiteTransaction transaction,
            int employeeId, DateTime date)
        {
            using (SQLiteCommand command = new SQLiteCommand(@"SELECT COUNT(1) FROM Asistencias
                WHERE IdEmpleado = @empleado AND Fecha = @fecha;", connection, transaction))
            {
                command.Parameters.AddWithValue("@empleado", employeeId);
                command.Parameters.AddWithValue("@fecha", date.ToString("yyyy-MM-dd"));
                return Convert.ToInt32(command.ExecuteScalar()) > 0;
            }
        }

        public void AddGeneratedAttendance(SQLiteConnection connection, SQLiteTransaction transaction,
            int requestId, int employeeId, DateTime date, string status)
        {
            using (SQLiteCommand command = new SQLiteCommand(@"INSERT INTO Asistencias
                (IdEmpleado, Fecha, HoraEntrada, HoraSalida, HorasTrabajadas, Estado, IdSolicitudAusencia)
                VALUES (@empleado, @fecha, NULL, NULL, 0, @estado, @solicitud);", connection, transaction))
            {
                command.Parameters.AddWithValue("@empleado", employeeId);
                command.Parameters.AddWithValue("@fecha", date.ToString("yyyy-MM-dd"));
                command.Parameters.AddWithValue("@estado", status);
                command.Parameters.AddWithValue("@solicitud", requestId);
                command.ExecuteNonQuery();
            }
        }

        public int DeleteGeneratedAttendance(SQLiteConnection connection, SQLiteTransaction transaction,
            int requestId)
        {
            using (SQLiteCommand command = new SQLiteCommand(
                "DELETE FROM Asistencias WHERE IdSolicitudAusencia = @solicitud;", connection, transaction))
            {
                command.Parameters.AddWithValue("@solicitud", requestId);
                return command.ExecuteNonQuery();
            }
        }

        private static SolicitudAusencia Map(SQLiteDataReader reader)
        {
            return new SolicitudAusencia
            {
                IdSolicitud = Convert.ToInt32(reader["IdSolicitud"]),
                IdEmpleado = Convert.ToInt32(reader["IdEmpleado"]),
                CodigoEmpleado = Convert.ToString(reader["CodigoEmpleado"]),
                EmpleadoNombre = Convert.ToString(reader["EmpleadoNombre"]),
                DepartamentoNombre = Convert.ToString(reader["DepartamentoNombre"]),
                Tipo = Convert.ToString(reader["Tipo"]),
                FechaInicio = DateTime.Parse(Convert.ToString(reader["FechaInicio"])),
```

Data/AbsenceRequestRepository.cs (continuacion)

```
                FechaFin = DateTime.Parse(Convert.ToString(reader["FechaFin"])),
                Estado = Convert.ToString(reader["Estado"]),
                Motivo = Convert.ToString(reader["Motivo"]),
                AprobadoPor = Convert.ToString(reader["AprobadoPor"]),
                FechaAprobacion = reader["FechaAprobacion"] == DBNull.Value ||
                    string.IsNullOrWhiteSpace(Convert.ToString(reader["FechaAprobacion"]))
                    ? (DateTime?)null : DateTime.Parse(Convert.ToString(reader["FechaAprobacion"])),
                ObservacionResolucion = Convert.ToString(reader["ObservacionResolucion"]),
                UsuarioSolicitante = Convert.ToString(reader["UsuarioSolicitante"]),
                FechaCreacion = DateTime.Parse(Convert.ToString(reader["FechaCreacion"]))
            };
        }
    }
}
```

Data/AuditRepository.cs

```
using System;
using System.Collections.Generic;
using System.Data.SQLite;
using SistemaGestionNomina.Models;

namespace SistemaGestionNomina.Data
{
    public sealed class AuditRepository
    {
        public void Add(AuditRecord record)
        {
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            {
                Add(connection, null, record);
            }
        }

        public void Add(SQLiteConnection connection, SQLiteTransaction transaction, AuditRecord record)
        {
            if (connection == null) throw new ArgumentNullException("connection");
            if (record == null) throw new ArgumentNullException("record");

            using (SQLiteCommand command = new SQLiteCommand(@"INSERT INTO Auditoria
                (Usuario, Modulo, Accion, Detalle, Fecha)
                VALUES (@usuario, @modulo, @accion, @detalle, @fecha);", connection, transaction))
            {
                command.Parameters.AddWithValue("@usuario", record.Usuario);
                command.Parameters.AddWithValue("@modulo", record.Modulo);
                command.Parameters.AddWithValue("@accion", record.Accion);
                command.Parameters.AddWithValue("@detalle", record.Detalle ?? string.Empty);
                command.Parameters.AddWithValue("@fecha", record.Fecha.ToString("yyyy-MM-dd HH:mm:ss"));
                command.ExecuteNonQuery();
            }
        }

        public List<AuditRecord> Search(AuditQuery query)
        {
            List<AuditRecord> records = new List<AuditRecord>();
            int limit = query == null || query.Limit <= 0 ? 100 : Math.Min(query.Limit, 500);
            int offset = query == null ? 0 : Math.Max(0, query.Offset);
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteCommand command = new SQLiteCommand(@"SELECT * FROM Auditoria
                WHERE (@usuario = '' OR Usuario LIKE @usuarioLike)
                  AND (@modulo = '' OR Modulo = @modulo)
                  AND (@accion = '' OR Accion = @accion)
                  AND (@detalle = '' OR Detalle LIKE @detalleLike)
                  AND (@desde = '' OR Fecha >= @desde)
                  AND (@hasta = '' OR Fecha < @hasta)
                ORDER BY Fecha DESC, IdAuditoria DESC LIMIT @limit OFFSET @offset;", connection))
            {
                string user = query == null ? string.Empty : query.Usuario ?? string.Empty;
                string module = query == null ? string.Empty : query.Modulo ?? string.Empty;
                string action = query == null ? string.Empty : query.Accion ?? string.Empty;
                string detail = query == null ? string.Empty : query.Detalle ?? string.Empty;
                command.Parameters.AddWithValue("@usuario", user);
                command.Parameters.AddWithValue("@usuarioLike", "%" + user + "%");
                command.Parameters.AddWithValue("@modulo", module);
                command.Parameters.AddWithValue("@accion", action);
                command.Parameters.AddWithValue("@detalle", detail);
                command.Parameters.AddWithValue("@detalleLike", "%" + detail + "%");
```

Data/AuditRepository.cs (continuacion)

```
                command.Parameters.AddWithValue("@desde", query != null && query.FechaDesde.HasValue
                    ? query.FechaDesde.Value.Date.ToString("yyyy-MM-dd HH:mm:ss") : string.Empty);
                command.Parameters.AddWithValue("@hasta", query != null && query.FechaHasta.HasValue
                    ? query.FechaHasta.Value.Date.AddDays(1).ToString("yyyy-MM-dd HH:mm:ss") : string.Empty);
                command.Parameters.AddWithValue("@limit", limit);
                command.Parameters.AddWithValue("@offset", offset);
                using (SQLiteDataReader reader = command.ExecuteReader())
                {
                    while (reader.Read()) records.Add(Map(reader));
                }
            }

            return records;
        }

        public List<string> GetDistinct(string column)
        {
            if (column != "Modulo" && column != "Accion") throw new ArgumentException("Columna de auditoría no válida.");
            List<string> values = new List<string>();
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteCommand command = new SQLiteCommand(
                "SELECT DISTINCT " + column + " FROM Auditoria ORDER BY " + column + ";", connection))
            using (SQLiteDataReader reader = command.ExecuteReader())
            {
                while (reader.Read()) values.Add(Convert.ToString(reader[0]));
            }
            return values;
        }

        private static AuditRecord Map(SQLiteDataReader reader)
        {
            return new AuditRecord
            {
                IdAuditoria = Convert.ToInt32(reader["IdAuditoria"]),
                Usuario = Convert.ToString(reader["Usuario"]),
                Modulo = Convert.ToString(reader["Modulo"]),
                Accion = Convert.ToString(reader["Accion"]),
                Detalle = Convert.ToString(reader["Detalle"]),
                Fecha = DateTime.Parse(Convert.ToString(reader["Fecha"]))
            };
        }
    }
}
```

Data/AuthenticationRepository.cs

```
using System;
using System.Data.SQLite;
using SistemaGestionNomina.Models;

namespace SistemaGestionNomina.Data
{
    public sealed class AuthenticationRepository
    {
        public Usuario GetByUsername(string username)
        {
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteCommand command = new SQLiteCommand(
                "SELECT * FROM Usuarios WHERE NombreUsuario = @username LIMIT 1;", connection))
            {
                command.Parameters.AddWithValue("@username", username);
                using (SQLiteDataReader reader = command.ExecuteReader())
                {
                    return reader.Read() ? Map(reader) : null;
                }
            }
        }

        public bool UpdatePassword(int userId, string passwordHash)
        {
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteCommand command = new SQLiteCommand(@"UPDATE Usuarios SET PasswordHash = @hash
                WHERE IdUsuario = @id AND Estado = 'Activo';", connection))
            {
                command.Parameters.AddWithValue("@hash", passwordHash);
                command.Parameters.AddWithValue("@id", userId);
                return command.ExecuteNonQuery() == 1;
            }
        }

        public void RegisterFailedAttempt(int userId, int attempts, bool blocked, DateTime? blockedAt)
        {
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteCommand command = new SQLiteCommand(@"UPDATE Usuarios SET
                IntentosFallidos = @attempts, Bloqueado = @blocked, FechaBloqueo = @blockedAt
                WHERE IdUsuario = @id;", connection))
            {
                command.Parameters.AddWithValue("@attempts", attempts);
                command.Parameters.AddWithValue("@blocked", blocked ? 1 : 0);
                command.Parameters.AddWithValue("@blockedAt", blockedAt.HasValue
                    ? (object)blockedAt.Value.ToString("yyyy-MM-dd HH:mm:ss") : DBNull.Value);
                command.Parameters.AddWithValue("@id", userId);
                command.ExecuteNonQuery();
            }
        }

        public void CompleteLogin(int userId, DateTime lastAccess, string upgradedHash)
        {
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteTransaction transaction = connection.BeginTransaction())
            using (SQLiteCommand command = new SQLiteCommand(@"UPDATE Usuarios SET
                UltimoAcceso = @access, IntentosFallidos = 0, Bloqueado = 0, FechaBloqueo = NULL,
                PasswordHash = CASE WHEN @hash = '' THEN PasswordHash ELSE @hash END
                WHERE IdUsuario = @id;", connection, transaction))
            {
                command.Parameters.AddWithValue("@access", lastAccess.ToString("yyyy-MM-dd HH:mm:ss"));
```

Data/AuthenticationRepository.cs (continuacion)

```
                command.Parameters.AddWithValue("@hash", upgradedHash ?? string.Empty);
                command.Parameters.AddWithValue("@id", userId);
                command.ExecuteNonQuery();
                transaction.Commit();
            }
        }

        private static Usuario Map(SQLiteDataReader reader)
        {
            return new Usuario
            {
                IdUsuario = Convert.ToInt32(reader["IdUsuario"]),
                NombreUsuario = Convert.ToString(reader["NombreUsuario"]),
                PasswordHash = Convert.ToString(reader["PasswordHash"]),
                Rol = Convert.ToString(reader["Rol"]),
                Estado = Convert.ToString(reader["Estado"]),
                IdEmpleado = reader["IdEmpleado"] == DBNull.Value ? (int?)null : Convert.ToInt32(reader["IdEmpleado"]),
                UltimoAcceso = ParseNullableDate(reader["UltimoAcceso"]),
                IntentosFallidos = Convert.ToInt32(reader["IntentosFallidos"]),
                Bloqueado = Convert.ToInt32(reader["Bloqueado"]) == 1,
                FechaBloqueo = ParseNullableDate(reader["FechaBloqueo"])
            };
        }

        private static DateTime? ParseNullableDate(object value)
        {
            if (value == null || value == DBNull.Value || string.IsNullOrWhiteSpace(Convert.ToString(value))) return null;
            DateTime parsed;
            return DateTime.TryParse(Convert.ToString(value), out parsed) ? parsed : (DateTime?)null;
        }
    }
}
```

Data/DatabaseInitializer.cs

```
using System;
using System.Data.SQLite;
using System.IO;
using SistemaGestionNomina.Helpers;

namespace SistemaGestionNomina.Data
{
    public static class DatabaseInitializer
    {
        public static void Initialize()
        {
            string folder = Path.GetDirectoryName(SQLiteConnectionFactory.DatabasePath);
            if (!Directory.Exists(folder))
            {
                Directory.CreateDirectory(folder);
            }

            bool existed = File.Exists(SQLiteConnectionFactory.DatabasePath);
            if (!existed)
            {
                SQLiteConnection.CreateFile(SQLiteConnectionFactory.DatabasePath);
            }

            if (existed)
            {
                using (SQLiteConnection versionConnection = SQLiteConnectionFactory.CreateConnection())
                {
                    if (DatabaseMigrationRunner.GetCurrentVersion(versionConnection) < DatabaseMigrationRunner.LatestVersion)
                    {
                        versionConnection.Close();
                        DatabaseMigrationRunner.BackupBeforeMigration(SQLiteConnectionFactory.DatabasePath);
                    }
                }
            }

            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            {
                Execute(connection, "PRAGMA foreign_keys = ON;");
                CreateTables(connection);
                DatabaseMigrationRunner.RunPending(connection);
                SeedData(connection);
            }
        }

        private static void CreateTables(SQLiteConnection connection)
        {
            Execute(connection, @"CREATE TABLE IF NOT EXISTS Usuarios (
                IdUsuario INTEGER PRIMARY KEY AUTOINCREMENT,
                NombreUsuario TEXT NOT NULL UNIQUE,
                PasswordHash TEXT NOT NULL,
                Rol TEXT NOT NULL,
                Estado TEXT NOT NULL
            );");

            Execute(connection, @"CREATE TABLE IF NOT EXISTS Departamentos (
                IdDepartamento INTEGER PRIMARY KEY AUTOINCREMENT,
                Nombre TEXT NOT NULL UNIQUE
            );");

            Execute(connection, @"CREATE TABLE IF NOT EXISTS Empleados (
```

Data/DatabaseInitializer.cs (continuacion)

```
                IdEmpleado INTEGER PRIMARY KEY AUTOINCREMENT,
                Codigo TEXT NOT NULL UNIQUE,
                Nombre TEXT NOT NULL,
                Apellido TEXT NOT NULL,
                Cedula TEXT NOT NULL UNIQUE,
                Cargo TEXT NOT NULL,
                IdDepartamento INTEGER NOT NULL,
                SalarioBase REAL NOT NULL,
                Estado TEXT NOT NULL,
                FechaIngreso TEXT NOT NULL,
                FOREIGN KEY(IdDepartamento) REFERENCES Departamentos(IdDepartamento)
            );");

            Execute(connection, @"CREATE TABLE IF NOT EXISTS Asistencias (
                IdAsistencia INTEGER PRIMARY KEY AUTOINCREMENT,
                IdEmpleado INTEGER NOT NULL,
                Fecha TEXT NOT NULL,
                HoraEntrada TEXT,
                HoraSalida TEXT,
                HorasTrabajadas REAL,
                Estado TEXT NOT NULL,
                FOREIGN KEY(IdEmpleado) REFERENCES Empleados(IdEmpleado)
            );");

            Execute(connection, @"CREATE TABLE IF NOT EXISTS PeriodosNomina (
                IdPeriodo INTEGER PRIMARY KEY AUTOINCREMENT,
                Nombre TEXT NOT NULL,
                FechaInicio TEXT NOT NULL,
                FechaFin TEXT NOT NULL,
                Estado TEXT NOT NULL
            );");

            Execute(connection, @"CREATE TABLE IF NOT EXISTS Nominas (
                IdNomina INTEGER PRIMARY KEY AUTOINCREMENT,
                IdPeriodo INTEGER NOT NULL,
                FechaCalculo TEXT NOT NULL,
                TotalIngresos REAL NOT NULL,
                TotalDeducciones REAL NOT NULL,
                TotalNeto REAL NOT NULL,
                Estado TEXT NOT NULL,
                FOREIGN KEY(IdPeriodo) REFERENCES PeriodosNomina(IdPeriodo)
            );");

            Execute(connection, @"CREATE TABLE IF NOT EXISTS NominaDetalle (
                IdDetalle INTEGER PRIMARY KEY AUTOINCREMENT,
                IdNomina INTEGER NOT NULL,
                IdEmpleado INTEGER NOT NULL,
                SueldoBase REAL NOT NULL,
                Bonos REAL NOT NULL,
                HorasExtra REAL NOT NULL,
                MontoHorasExtra REAL NOT NULL,
                TotalIngresos REAL NOT NULL,
                TotalDeducciones REAL NOT NULL,
                NetoPagar REAL NOT NULL,
                FOREIGN KEY(IdNomina) REFERENCES Nominas(IdNomina),
                FOREIGN KEY(IdEmpleado) REFERENCES Empleados(IdEmpleado)
            );");

            Execute(connection, @"CREATE TABLE IF NOT EXISTS Comprobantes (
                IdComprobante INTEGER PRIMARY KEY AUTOINCREMENT,
```

Data/DatabaseInitializer.cs (continuacion)

```
                IdNomina INTEGER NOT NULL,
                IdEmpleado INTEGER NOT NULL,
                NumeroComprobante TEXT NOT NULL UNIQUE,
                FechaGeneracion TEXT NOT NULL,
                RutaPdf TEXT,
                FOREIGN KEY(IdNomina) REFERENCES Nominas(IdNomina),
                FOREIGN KEY(IdEmpleado) REFERENCES Empleados(IdEmpleado)
            );");

            Execute(connection, @"CREATE TABLE IF NOT EXISTS ConfiguracionNomina (
                IdConfiguracion INTEGER PRIMARY KEY AUTOINCREMENT,
                NombreParametro TEXT NOT NULL UNIQUE,
                Valor REAL NOT NULL,
                Descripcion TEXT
            );");

            Execute(connection, @"CREATE TABLE IF NOT EXISTS ReportesGenerados (
                IdReporte INTEGER PRIMARY KEY AUTOINCREMENT,
                NombreReporte TEXT NOT NULL,
                Tipo TEXT NOT NULL,
                GeneradoPor TEXT NOT NULL,
                FechaGeneracion TEXT NOT NULL,
                RutaArchivo TEXT
            );");

            Execute(connection, @"CREATE TABLE IF NOT EXISTS Auditoria (
                IdAuditoria INTEGER PRIMARY KEY AUTOINCREMENT,
                Usuario TEXT NOT NULL,
                Modulo TEXT NOT NULL,
                Accion TEXT NOT NULL,
                Detalle TEXT,
                Fecha TEXT NOT NULL
            );");

            Execute(connection, @"CREATE TABLE IF NOT EXISTS MigracionesLog (
                IdMigracion INTEGER PRIMARY KEY AUTOINCREMENT,
                Version INTEGER NOT NULL,
                Nombre TEXT NOT NULL,
                FechaAplicacion TEXT NOT NULL,
                Detalle TEXT
            );");
        }

        private static void SeedData(SQLiteConnection connection)
        {
            object adminExists = ExecuteScalar(connection,
                "SELECT COUNT(1) FROM Usuarios WHERE NombreUsuario = @usuario;",
                new SQLiteParameter("@usuario", "admin"));
            if (Convert.ToInt32(adminExists) == 0)
            {
                ExecuteWithParameters(connection,
                    "INSERT INTO Usuarios (NombreUsuario, PasswordHash, Rol, Estado) VALUES (@u, @p, @r, @e);",
                    new SQLiteParameter("@u", "admin"),
                    new SQLiteParameter("@p", PasswordHelper.HashPassword("admin123")),
                    new SQLiteParameter("@r", "Admin"),
                    new SQLiteParameter("@e", "Activo"));
            }

            string[] departments = { "Tecnología", "Ventas", "Recursos Humanos", "Finanzas", "Operaciones" };
            for (int i = 0; i < departments.Length; i++)
```

Data/DatabaseInitializer.cs (continuacion)

```
            {
                ExecuteWithParameters(connection,
                    "INSERT OR IGNORE INTO Departamentos (Nombre) VALUES (@nombre);",
                    new SQLiteParameter("@nombre", departments[i]));
            }

            SeedEmployee(connection, "EMP-1001", "Carlos", "Mendoza", "8-765-123", "Desarrollador Senior", "Tecnología", 2450);
            SeedEmployee(connection, "EMP-1002", "Ana", "López", "8-432-987", "Ejecutiva de Ventas", "Ventas", 1850);
            SeedEmployee(connection, "EMP-1003", "Sofía", "Gómez", "9-111-222", "Analista de RRHH", "Recursos Humanos", 1700);
            SeedEmployee(connection, "EMP-1004", "Javier", "Ramírez", "4-555-321", "Supervisor de Operaciones", "Operaciones", 2100);

            SeedDemoUser(connection, "rrhh", "Rrhh2026*", "RRHH", null);
            SeedDemoUser(connection, "contador", "Conta2026*", "Contabilidad", null);
            SeedDemoUser(connection, "supervisor", "Super2026*", "Supervisor", "EMP-1004");
            SeedDemoUser(connection, "trabajador", "Trabaja2026*", "Trabajador", "EMP-1001");

            SeedConfig(connection, "SeguroSocial", 9.75m, "Porcentaje académico de seguro social.");
            SeedConfig(connection, "ISR", 10, "Porcentaje académico de impuesto sobre la renta.");
            SeedConfig(connection, "SeguroEducativo", 1.25m, "Porcentaje académico de seguro educativo.");
            SeedConfig(connection, "RecargoHoraExtra", 1.25m, "Multiplicador académico de horas extra.");
            SeedConfig(connection, "HorasMensualesBase", 160, "Horas mensuales base para cálculo por hora.");
        }

        private static void SeedEmployee(SQLiteConnection connection, string code, string name, string lastName, string cedula, string role, string  \
    department, decimal salary)
        {
            object departmentId = ExecuteScalar(connection, "SELECT IdDepartamento FROM Departamentos WHERE Nombre = @nombre;",
                new SQLiteParameter("@nombre", department));

            ExecuteWithParameters(connection, @"INSERT OR IGNORE INTO Empleados
                (Codigo, Nombre, Apellido, Cedula, Cargo, IdDepartamento, SalarioBase, Estado, FechaIngreso)
                VALUES (@codigo, @nombre, @apellido, @cedula, @cargo, @departamento, @salario, 'Activo', @fecha);",
                new SQLiteParameter("@codigo", code),
                new SQLiteParameter("@nombre", name),
                new SQLiteParameter("@apellido", lastName),
                new SQLiteParameter("@cedula", cedula),
                new SQLiteParameter("@cargo", role),
                new SQLiteParameter("@departamento", Convert.ToInt32(departmentId)),
                new SQLiteParameter("@salario", salary),
                new SQLiteParameter("@fecha", DateTime.Today.AddMonths(-8).ToString("yyyy-MM-dd")));
        }

        private static void SeedConfig(SQLiteConnection connection, string name, decimal value, string description)
        {
            ExecuteWithParameters(connection,
                "INSERT OR IGNORE INTO ConfiguracionNomina (NombreParametro, Valor, Descripcion) VALUES (@n, @v, @d);",
                new SQLiteParameter("@n", name),
                new SQLiteParameter("@v", value),
                new SQLiteParameter("@d", description));
        }

        private static void SeedDemoUser(SQLiteConnection connection, string username, string password,
            string role, string employeeCode)
        {
            object existing = ExecuteScalar(connection,
                "SELECT COUNT(1) FROM Usuarios WHERE NombreUsuario = @usuario;",
                new SQLiteParameter("@usuario", username));
            if (Convert.ToInt32(existing) > 0) return;

            object employeeId = DBNull.Value;
```

Data/DatabaseInitializer.cs (continuacion)

```
            if (!string.IsNullOrWhiteSpace(employeeCode))
            {
                employeeId = ExecuteScalar(connection,
                    "SELECT IdEmpleado FROM Empleados WHERE Codigo = @codigo AND Estado = 'Activo' LIMIT 1;",
                    new SQLiteParameter("@codigo", employeeCode));
                if (employeeId == null) employeeId = DBNull.Value;
                if (employeeId != DBNull.Value)
                {
                    object associated = ExecuteScalar(connection,
                        "SELECT COUNT(1) FROM Usuarios WHERE IdEmpleado = @empleado;",
                        new SQLiteParameter("@empleado", employeeId));
                    if (Convert.ToInt32(associated) > 0) return;
                }
            }

            ExecuteWithParameters(connection, @"INSERT OR IGNORE INTO Usuarios
                (NombreUsuario, PasswordHash, Rol, Estado, IdEmpleado, IntentosFallidos, Bloqueado)
                VALUES (@usuario, @password, @rol, 'Activo', @empleado, 0, 0);",
                new SQLiteParameter("@usuario", username),
                new SQLiteParameter("@password", PasswordHelper.HashPassword(password)),
                new SQLiteParameter("@rol", role),
                new SQLiteParameter("@empleado", employeeId));
        }

        private static void Execute(SQLiteConnection connection, string sql)
        {
            using (SQLiteCommand command = new SQLiteCommand(sql, connection))
            {
                command.ExecuteNonQuery();
            }
        }

        private static void ExecuteWithParameters(SQLiteConnection connection, string sql, params SQLiteParameter[] parameters)
        {
            using (SQLiteCommand command = new SQLiteCommand(sql, connection))
            {
                command.Parameters.AddRange(parameters);
                command.ExecuteNonQuery();
            }
        }

        private static object ExecuteScalar(SQLiteConnection connection, string sql, params SQLiteParameter[] parameters)
        {
            using (SQLiteCommand command = new SQLiteCommand(sql, connection))
            {
                command.Parameters.AddRange(parameters);
                return command.ExecuteScalar();
            }
        }
    }
}
```

Data/DatabaseMigrationRunner.cs

```
using System;
using System.Data.SQLite;
using System.Globalization;
using System.IO;
using System.Security.Cryptography;
using System.Text;

namespace SistemaGestionNomina.Data
{
    public static class DatabaseMigrationRunner
    {
        public const int LatestVersion = 6;

        public static int GetCurrentVersion(SQLiteConnection connection)
        {
            using (SQLiteCommand command = new SQLiteCommand("PRAGMA user_version;", connection))
            {
                return Convert.ToInt32(command.ExecuteScalar());
            }
        }

        public static string BackupBeforeMigration(string databasePath)
        {
            // Se conserva una copia verificable antes de modificar una base existente.
            string folder = Path.Combine(Path.GetDirectoryName(databasePath), "Backups", "Migrations");
            Directory.CreateDirectory(folder);
            string destination = Path.Combine(folder,
                "nomina_pre_migracion_" + DateTime.Now.ToString("yyyyMMdd_HHmmss_fff") + ".db");
            File.Copy(databasePath, destination, false);

            string hash = ComputeSha256(destination);
            File.WriteAllText(destination + ".sha256", hash, Encoding.ASCII);
            return destination;
        }

        public static void RunPending(SQLiteConnection connection)
        {
            int version = GetCurrentVersion(connection);
            if (version < 1) ApplyMigration(connection, 1, "Seguridad de usuarios", MigrateUsers);
            if (version < 2) ApplyMigration(connection, 2, "Índices e integridad de asistencia", MigrateIndexesAndAttendance);
            if (version < 3) ApplyMigration(connection, 3, "Validaciones críticas", MigrateIntegrityRules);
            if (version < 4) ApplyMigration(connection, 4, "Cierre, pago y anulación de nómina", MigrateNominaRobusta);
            if (version < 5) ApplyMigration(connection, 5, "Historial laboral y protección histórica", MigrateEmployeeHistoryAndPayrollProtection);
            if (version < 6) ApplyMigration(connection, 6, "Solicitudes de ausencia", MigrateAbsenceRequests);
        }

        private static void ApplyMigration(SQLiteConnection connection, int version, string name,
            Action<SQLiteConnection, SQLiteTransaction, StringBuilder> migration)
        {
            // El numero de version y el registro de auditoria avanzan solo si toda la migracion termina bien.
            using (SQLiteTransaction transaction = connection.BeginTransaction())
            {
                StringBuilder detail = new StringBuilder();
                migration(connection, transaction, detail);
                Execute(connection, transaction, "PRAGMA user_version = " + version + ";");
                Execute(connection, transaction, @"INSERT INTO MigracionesLog
                    (Version, Nombre, FechaAplicacion, Detalle)
                    VALUES (@version, @nombre, @fecha, @detalle);",
                    new SQLiteParameter("@version", version),
                    new SQLiteParameter("@nombre", name),
```

Data/DatabaseMigrationRunner.cs (continuacion)

```
                    new SQLiteParameter("@fecha", DateTime.Now.ToString("yyyy-MM-dd HH:mm:ss")),
                    new SQLiteParameter("@detalle", detail.ToString()));
                transaction.Commit();
            }
        }

        private static void MigrateUsers(SQLiteConnection connection, SQLiteTransaction transaction, StringBuilder detail)
        {
            AddColumnIfMissing(connection, transaction, "Usuarios", "IdEmpleado",
                "INTEGER NULL REFERENCES Empleados(IdEmpleado)");
            AddColumnIfMissing(connection, transaction, "Usuarios", "UltimoAcceso", "TEXT NULL");
            AddColumnIfMissing(connection, transaction, "Usuarios", "IntentosFallidos", "INTEGER NOT NULL DEFAULT 0");
            AddColumnIfMissing(connection, transaction, "Usuarios", "Bloqueado", "INTEGER NOT NULL DEFAULT 0");
            AddColumnIfMissing(connection, transaction, "Usuarios", "FechaBloqueo", "TEXT NULL");
            Execute(connection, transaction,
                "CREATE INDEX IF NOT EXISTS IX_Usuarios_IdEmpleado ON Usuarios(IdEmpleado);");
            detail.Append("Columnas de autenticación e índice de empleado agregados.");
        }

        private static void MigrateIndexesAndAttendance(SQLiteConnection connection, SQLiteTransaction transaction, StringBuilder detail)
        {
            Execute(connection, transaction, "CREATE INDEX IF NOT EXISTS IX_Empleados_Codigo ON Empleados(Codigo);");
            Execute(connection, transaction, "CREATE INDEX IF NOT EXISTS IX_Empleados_Cedula ON Empleados(Cedula);");
            Execute(connection, transaction, "CREATE INDEX IF NOT EXISTS IX_Asistencias_EmpleadoFecha ON Asistencias(IdEmpleado, Fecha);");
            Execute(connection, transaction, "CREATE INDEX IF NOT EXISTS IX_Nominas_IdPeriodo ON Nominas(IdPeriodo);");
            Execute(connection, transaction, "CREATE INDEX IF NOT EXISTS IX_Comprobantes_IdEmpleado ON Comprobantes(IdEmpleado);");
            Execute(connection, transaction, "CREATE INDEX IF NOT EXISTS IX_Auditoria_Fecha ON Auditoria(Fecha DESC);");

            int duplicateGroups = Convert.ToInt32(Scalar(connection, transaction, @"SELECT COUNT(1) FROM (
                SELECT IdEmpleado, Fecha FROM Asistencias
                GROUP BY IdEmpleado, Fecha HAVING COUNT(1) > 1);"));

            if (duplicateGroups == 0)
            {
                Execute(connection, transaction,
                    "CREATE UNIQUE INDEX IF NOT EXISTS UX_Asistencias_EmpleadoFecha ON Asistencias(IdEmpleado, Fecha);");
                detail.Append("No se detectaron asistencias duplicadas; se creó el índice único. ");
            }
            else
            {
                detail.Append("Se detectaron " + duplicateGroups +
                    " grupos históricos duplicados; se conservaron sin cambios. ");
            }

            Execute(connection, transaction, @"CREATE TRIGGER IF NOT EXISTS TR_Asistencias_NoDuplicar_Insert
                BEFORE INSERT ON Asistencias
                WHEN EXISTS (SELECT 1 FROM Asistencias
                    WHERE IdEmpleado = NEW.IdEmpleado AND Fecha = NEW.Fecha)
                BEGIN
                    SELECT RAISE(ABORT, 'Ya existe una asistencia para el empleado en esa fecha.');
                END;");
            Execute(connection, transaction, @"CREATE TRIGGER IF NOT EXISTS TR_Asistencias_NoDuplicar_Update
                BEFORE UPDATE OF IdEmpleado, Fecha ON Asistencias
                WHEN EXISTS (SELECT 1 FROM Asistencias
                    WHERE IdEmpleado = NEW.IdEmpleado AND Fecha = NEW.Fecha
                      AND IdAsistencia <> OLD.IdAsistencia)
                BEGIN
                    SELECT RAISE(ABORT, 'Ya existe una asistencia para el empleado en esa fecha.');
                END;");
            detail.Append("Se instalaron protecciones para impedir nuevas duplicaciones.");
```

Data/DatabaseMigrationRunner.cs (continuacion)

```
        }

        private static void MigrateIntegrityRules(SQLiteConnection connection, SQLiteTransaction transaction, StringBuilder detail)
        {
            Execute(connection, transaction, @"CREATE TRIGGER IF NOT EXISTS TR_Usuarios_Validar_Insert
                BEFORE INSERT ON Usuarios
                WHEN NEW.Rol NOT IN ('Admin','RRHH','Contabilidad','Supervisor','Trabajador')
                  OR (NEW.Rol IN ('Supervisor','Trabajador') AND NEW.IdEmpleado IS NULL)
                BEGIN
                    SELECT RAISE(ABORT, 'El rol o la asociación del usuario no es válida.');
                END;");
            Execute(connection, transaction, @"CREATE TRIGGER IF NOT EXISTS TR_Usuarios_Validar_Update
                BEFORE UPDATE OF Rol, IdEmpleado ON Usuarios
                WHEN NEW.Rol NOT IN ('Admin','RRHH','Contabilidad','Supervisor','Trabajador')
                  OR (NEW.Rol IN ('Supervisor','Trabajador') AND NEW.IdEmpleado IS NULL)
                BEGIN
                    SELECT RAISE(ABORT, 'El rol o la asociación del usuario no es válida.');
                END;");
            int duplicateUserLinks = Convert.ToInt32(Scalar(connection, transaction, @"SELECT COUNT(1) FROM (
                SELECT IdEmpleado FROM Usuarios WHERE IdEmpleado IS NOT NULL
                GROUP BY IdEmpleado HAVING COUNT(1) > 1);"));
            if (duplicateUserLinks == 0)
            {
                Execute(connection, transaction, @"CREATE UNIQUE INDEX IF NOT EXISTS UX_Usuarios_IdEmpleado
                    ON Usuarios(IdEmpleado) WHERE IdEmpleado IS NOT NULL;");
            }
            else
            {
                detail.Append("Se conservaron " + duplicateUserLinks + " asociaciones históricas duplicadas. ");
            }
            Execute(connection, transaction, @"CREATE TRIGGER IF NOT EXISTS TR_Usuarios_NoDuplicarEmpleado_Insert
                BEFORE INSERT ON Usuarios
                WHEN NEW.IdEmpleado IS NOT NULL AND EXISTS
                    (SELECT 1 FROM Usuarios WHERE IdEmpleado = NEW.IdEmpleado)
                BEGIN
                    SELECT RAISE(ABORT, 'El empleado ya tiene un usuario asociado.');
                END;");
            Execute(connection, transaction, @"CREATE TRIGGER IF NOT EXISTS TR_Usuarios_NoDuplicarEmpleado_Update
                BEFORE UPDATE OF IdEmpleado ON Usuarios
                WHEN NEW.IdEmpleado IS NOT NULL AND EXISTS
                    (SELECT 1 FROM Usuarios WHERE IdEmpleado = NEW.IdEmpleado AND IdUsuario <> OLD.IdUsuario)
                BEGIN
                    SELECT RAISE(ABORT, 'El empleado ya tiene un usuario asociado.');
                END;");

            Execute(connection, transaction, @"CREATE TRIGGER IF NOT EXISTS TR_Empleados_Validar_Insert
                BEFORE INSERT ON Empleados
                WHEN NEW.SalarioBase <= 0 OR NEW.IdDepartamento <= 0
                BEGIN
                    SELECT RAISE(ABORT, 'El salario y el departamento del empleado deben ser válidos.');
                END;");
            Execute(connection, transaction, @"CREATE TRIGGER IF NOT EXISTS TR_Empleados_Validar_Update
                BEFORE UPDATE OF SalarioBase, IdDepartamento ON Empleados
                WHEN NEW.SalarioBase <= 0 OR NEW.IdDepartamento <= 0
                BEGIN
                    SELECT RAISE(ABORT, 'El salario y el departamento del empleado deben ser válidos.');
                END;");

            Execute(connection, transaction, @"CREATE TRIGGER IF NOT EXISTS TR_Asistencias_Validar_Insert
                BEFORE INSERT ON Asistencias
```

Data/DatabaseMigrationRunner.cs (continuacion)

```
                WHEN NEW.HorasTrabajadas < 0
                  OR (NEW.HoraEntrada IS NOT NULL AND NEW.HoraSalida IS NOT NULL AND NEW.HoraSalida <= NEW.HoraEntrada)
                BEGIN
                    SELECT RAISE(ABORT, 'Las horas de asistencia no son válidas.');
                END;");
            Execute(connection, transaction, @"CREATE TRIGGER IF NOT EXISTS TR_Asistencias_Validar_Update
                BEFORE UPDATE OF HoraEntrada, HoraSalida, HorasTrabajadas ON Asistencias
                WHEN NEW.HorasTrabajadas < 0
                  OR (NEW.HoraEntrada IS NOT NULL AND NEW.HoraSalida IS NOT NULL AND NEW.HoraSalida <= NEW.HoraEntrada)
                BEGIN
                    SELECT RAISE(ABORT, 'Las horas de asistencia no son válidas.');
                END;");
            detail.Append("Roles, asociaciones, salarios, departamentos y horas protegidos mediante índices y triggers.");
        }

        private static void MigrateNominaRobusta(SQLiteConnection connection, SQLiteTransaction transaction, StringBuilder detail)
        {
            AddColumnIfMissing(connection, transaction, "PeriodosNomina", "Cerrado",
                "INTEGER NOT NULL DEFAULT 0");
            AddColumnIfMissing(connection, transaction, "PeriodosNomina", "FechaCierre", "TEXT NULL");
            AddColumnIfMissing(connection, transaction, "PeriodosNomina", "CerradoPor", "TEXT NULL");

            AddColumnIfMissing(connection, transaction, "Nominas", "FechaConfirmacion", "TEXT NULL");
            AddColumnIfMissing(connection, transaction, "Nominas", "ConfirmadaPor", "TEXT NULL");
            AddColumnIfMissing(connection, transaction, "Nominas", "FechaPago", "TEXT NULL");
            AddColumnIfMissing(connection, transaction, "Nominas", "PagadaPor", "TEXT NULL");
            AddColumnIfMissing(connection, transaction, "Nominas", "FechaAnulacion", "TEXT NULL");
            AddColumnIfMissing(connection, transaction, "Nominas", "AnuladaPor", "TEXT NULL");
            AddColumnIfMissing(connection, transaction, "Nominas", "MotivoAnulacion", "TEXT NULL");

            Execute(connection, transaction, @"CREATE TABLE IF NOT EXISTS NominaVersiones (
                IdVersion INTEGER PRIMARY KEY AUTOINCREMENT,
                IdNominaOriginal INTEGER NOT NULL,
                IdNominaNueva INTEGER NULL,
                MotivoCambio TEXT NOT NULL,
                UsuarioResponsable TEXT NOT NULL,
                FechaCambio TEXT NOT NULL,
                FOREIGN KEY(IdNominaOriginal) REFERENCES Nominas(IdNomina),
                FOREIGN KEY(IdNominaNueva) REFERENCES Nominas(IdNomina)
            );");

            Execute(connection, transaction,
                "CREATE INDEX IF NOT EXISTS IX_Nominas_IdPeriodo_Estado ON Nominas(IdPeriodo, Estado);");
            Execute(connection, transaction,
                "CREATE INDEX IF NOT EXISTS IX_NominaVersiones_IdNominaOriginal ON NominaVersiones(IdNominaOriginal);");

            Execute(connection, transaction, @"CREATE TRIGGER IF NOT EXISTS TR_Asistencias_PeriodoCerrado_Insert
                BEFORE INSERT ON Asistencias
                WHEN EXISTS (SELECT 1 FROM PeriodosNomina
                    WHERE Cerrado = 1
                      AND FechaInicio <= NEW.Fecha
                      AND FechaFin >= NEW.Fecha)
                BEGIN
                    SELECT RAISE(ABORT, 'No se puede registrar asistencia en un período cerrado.');
                END;");

            detail.Append("PeriodosNomina (Cerrado,FechaCierre,CerradoPor), Nominas (fechas/usuarios de cada estado), ");
            detail.Append("NominaVersiones, índices y trigger de bloqueo de asistencia creados.");
        }

```

Data/DatabaseMigrationRunner.cs (continuacion)

```
        private static void MigrateEmployeeHistoryAndPayrollProtection(SQLiteConnection connection,
            SQLiteTransaction transaction, StringBuilder detail)
        {
            AddColumnIfMissing(connection, transaction, "Empleados", "FechaEfectivaLaboral", "TEXT NULL");
            Execute(connection, transaction, @"UPDATE Empleados
                SET FechaEfectivaLaboral = FechaIngreso
                WHERE FechaEfectivaLaboral IS NULL OR FechaEfectivaLaboral = '';" );

            AddColumnIfMissing(connection, transaction, "NominaDetalle", "CodigoEmpleadoSnapshot", "TEXT NULL");
            AddColumnIfMissing(connection, transaction, "NominaDetalle", "NombreEmpleadoSnapshot", "TEXT NULL");
            AddColumnIfMissing(connection, transaction, "NominaDetalle", "CargoEmpleadoSnapshot", "TEXT NULL");
            AddColumnIfMissing(connection, transaction, "NominaDetalle", "DepartamentoSnapshot", "TEXT NULL");
            Execute(connection, transaction, @"UPDATE NominaDetalle
                SET CodigoEmpleadoSnapshot = COALESCE(CodigoEmpleadoSnapshot,
                        (SELECT Codigo FROM Empleados WHERE IdEmpleado = NominaDetalle.IdEmpleado)),
                    NombreEmpleadoSnapshot = COALESCE(NombreEmpleadoSnapshot,
                        (SELECT Nombre || ' ' || Apellido FROM Empleados WHERE IdEmpleado = NominaDetalle.IdEmpleado)),
                    CargoEmpleadoSnapshot = COALESCE(CargoEmpleadoSnapshot,
                        (SELECT Cargo FROM Empleados WHERE IdEmpleado = NominaDetalle.IdEmpleado)),
                    DepartamentoSnapshot = COALESCE(DepartamentoSnapshot,
                        (SELECT d.Nombre FROM Empleados e INNER JOIN Departamentos d
                         ON d.IdDepartamento = e.IdDepartamento WHERE e.IdEmpleado = NominaDetalle.IdEmpleado));");

            Execute(connection, transaction, @"CREATE TABLE IF NOT EXISTS HistorialEmpleado (
                IdHistorial INTEGER PRIMARY KEY AUTOINCREMENT,
                IdEmpleado INTEGER NOT NULL,
                CampoModificado TEXT NOT NULL,
                ValorAnterior TEXT NULL,
                ValorNuevo TEXT NULL,
                ValorAnteriorTecnico TEXT NULL,
                ValorNuevoTecnico TEXT NULL,
                FechaCambio TEXT NOT NULL,
                FechaEfectiva TEXT NOT NULL,
                UsuarioResponsable TEXT NOT NULL,
                Motivo TEXT NULL,
                Aplicado INTEGER NOT NULL DEFAULT 1,
                FechaAplicacion TEXT NULL,
                FOREIGN KEY(IdEmpleado) REFERENCES Empleados(IdEmpleado)
            );");
            Execute(connection, transaction,
                "CREATE INDEX IF NOT EXISTS IX_HistorialEmpleado_EmpleadoFecha ON HistorialEmpleado(IdEmpleado, FechaEfectiva DESC);");
            Execute(connection, transaction,
                "CREATE INDEX IF NOT EXISTS IX_HistorialEmpleado_Pendientes ON HistorialEmpleado(Aplicado, FechaEfectiva);");

            // Los tres triggers cubren insercion, actualizacion y eliminacion de asistencia en periodos cerrados.
            Execute(connection, transaction, "DROP TRIGGER IF EXISTS TR_Asistencias_PeriodoCerrado_Insert;");
            Execute(connection, transaction, @"CREATE TRIGGER TR_Asistencias_PeriodoCerrado_Insert
                BEFORE INSERT ON Asistencias
                WHEN EXISTS (SELECT 1 FROM PeriodosNomina
                    WHERE Cerrado = 1 AND FechaInicio <= NEW.Fecha AND FechaFin >= NEW.Fecha)
                BEGIN
                    SELECT RAISE(ABORT, 'No se puede registrar asistencia en un periodo cerrado.');
                END;");
            Execute(connection, transaction, "DROP TRIGGER IF EXISTS TR_Asistencias_PeriodoCerrado_Update;");
            Execute(connection, transaction, @"CREATE TRIGGER TR_Asistencias_PeriodoCerrado_Update
                BEFORE UPDATE ON Asistencias
                WHEN EXISTS (SELECT 1 FROM PeriodosNomina
                    WHERE Cerrado = 1
                      AND ((FechaInicio <= OLD.Fecha AND FechaFin >= OLD.Fecha)
                        OR (FechaInicio <= NEW.Fecha AND FechaFin >= NEW.Fecha)))
```

Data/DatabaseMigrationRunner.cs (continuacion)

```
                BEGIN
                    SELECT RAISE(ABORT, 'No se puede modificar asistencia relacionada con un periodo cerrado.');
                END;");
            Execute(connection, transaction, "DROP TRIGGER IF EXISTS TR_Asistencias_PeriodoCerrado_Delete;");
            Execute(connection, transaction, @"CREATE TRIGGER TR_Asistencias_PeriodoCerrado_Delete
                BEFORE DELETE ON Asistencias
                WHEN EXISTS (SELECT 1 FROM PeriodosNomina
                    WHERE Cerrado = 1 AND FechaInicio <= OLD.Fecha AND FechaFin >= OLD.Fecha)
                BEGIN
                    SELECT RAISE(ABORT, 'No se puede eliminar asistencia relacionada con un periodo cerrado.');
                END;");

            Execute(connection, transaction, @"CREATE TRIGGER IF NOT EXISTS TR_PeriodosNomina_Rango_Insert
                BEFORE INSERT ON PeriodosNomina WHEN NEW.FechaInicio > NEW.FechaFin
                BEGIN SELECT RAISE(ABORT, 'La fecha inicial del periodo no puede superar la fecha final.'); END;");
            Execute(connection, transaction, @"CREATE TRIGGER IF NOT EXISTS TR_PeriodosNomina_Rango_Update
                BEFORE UPDATE OF FechaInicio, FechaFin ON PeriodosNomina WHEN NEW.FechaInicio > NEW.FechaFin
                BEGIN SELECT RAISE(ABORT, 'La fecha inicial del periodo no puede superar la fecha final.'); END;");
            // Dos rangos se solapan cuando el inicio de uno es anterior al fin del otro y viceversa.
            Execute(connection, transaction, @"CREATE TRIGGER IF NOT EXISTS TR_PeriodosNomina_Solapado_Insert
                BEFORE INSERT ON PeriodosNomina
                WHEN (NEW.Cerrado = 1 OR NEW.Estado IN ('Confirmado','Pagado'))
                  AND EXISTS (SELECT 1 FROM PeriodosNomina
                    WHERE (Cerrado = 1 OR Estado IN ('Confirmado','Pagado'))
                      AND FechaInicio <= NEW.FechaFin AND FechaFin >= NEW.FechaInicio)
                BEGIN SELECT RAISE(ABORT, 'El periodo se solapa con otro periodo cerrado o confirmado.'); END;");
            Execute(connection, transaction, @"CREATE TRIGGER IF NOT EXISTS TR_PeriodosNomina_Solapado_Update
                BEFORE UPDATE OF FechaInicio, FechaFin, Estado, Cerrado ON PeriodosNomina
                WHEN (NEW.Cerrado = 1 OR NEW.Estado IN ('Confirmado','Pagado'))
                  AND EXISTS (SELECT 1 FROM PeriodosNomina
                    WHERE IdPeriodo <> OLD.IdPeriodo
                      AND (Cerrado = 1 OR Estado IN ('Confirmado','Pagado'))
                      AND FechaInicio <= NEW.FechaFin AND FechaFin >= NEW.FechaInicio)
                BEGIN SELECT RAISE(ABORT, 'El periodo se solapa con otro periodo cerrado o confirmado.'); END;");

            Execute(connection, transaction, @"CREATE TRIGGER IF NOT EXISTS TR_HistorialEmpleado_PeriodoCerrado_Insert
                BEFORE INSERT ON HistorialEmpleado
                WHEN EXISTS (SELECT 1 FROM PeriodosNomina
                    WHERE Cerrado = 1 AND FechaFin >= NEW.FechaEfectiva)
                BEGIN SELECT RAISE(ABORT, 'El cambio laboral afectaria un periodo de nomina cerrado.'); END;");
            Execute(connection, transaction, @"CREATE TRIGGER IF NOT EXISTS TR_Empleados_CambioLaboral_PeriodoCerrado
                BEFORE UPDATE OF SalarioBase, Cargo, IdDepartamento, Estado ON Empleados
                WHEN (OLD.SalarioBase <> NEW.SalarioBase OR OLD.Cargo <> NEW.Cargo
                      OR OLD.IdDepartamento <> NEW.IdDepartamento OR OLD.Estado <> NEW.Estado)
                  AND (NEW.FechaEfectivaLaboral IS NULL OR EXISTS (SELECT 1 FROM PeriodosNomina
                    WHERE Cerrado = 1 AND FechaFin >= NEW.FechaEfectivaLaboral))
                BEGIN SELECT RAISE(ABORT, 'El cambio laboral afectaria un periodo de nomina cerrado.'); END;");

            Execute(connection, transaction, @"CREATE TRIGGER IF NOT EXISTS TR_Nominas_Estado_Insert
                BEFORE INSERT ON Nominas
                WHEN NEW.Estado NOT IN ('Borrador','Calculada','Confirmada','Pagada','Anulada')
                BEGIN SELECT RAISE(ABORT, 'El estado de nomina no es valido.'); END;");
            Execute(connection, transaction, @"CREATE TRIGGER IF NOT EXISTS TR_Nominas_Estado_Update
                BEFORE UPDATE OF Estado ON Nominas
                WHEN OLD.Estado <> NEW.Estado AND NOT (
                    (OLD.Estado = 'Borrador' AND NEW.Estado = 'Calculada') OR
                    (OLD.Estado = 'Calculada' AND NEW.Estado = 'Confirmada') OR
                    (OLD.Estado = 'Confirmada' AND NEW.Estado IN ('Pagada','Anulada')) OR
                    (OLD.Estado = 'Pagada' AND NEW.Estado = 'Anulada'))
                BEGIN SELECT RAISE(ABORT, 'La transicion de estado de nomina no es valida.'); END;");
```

Data/DatabaseMigrationRunner.cs (continuacion)

```
            Execute(connection, transaction, @"CREATE TRIGGER IF NOT EXISTS TR_Nominas_ProtegerDatos_Update
                BEFORE UPDATE OF IdPeriodo, TotalIngresos, TotalDeducciones, TotalNeto ON Nominas
                WHEN OLD.Estado IN ('Confirmada','Pagada','Anulada') AND
                    (OLD.IdPeriodo <> NEW.IdPeriodo OR OLD.TotalIngresos <> NEW.TotalIngresos
                     OR OLD.TotalDeducciones <> NEW.TotalDeducciones OR OLD.TotalNeto <> NEW.TotalNeto)
                BEGIN SELECT RAISE(ABORT, 'Una nomina confirmada, pagada o anulada no puede editarse.'); END;");
            Execute(connection, transaction, @"CREATE TRIGGER IF NOT EXISTS TR_NominaDetalle_Proteger_Update
                BEFORE UPDATE ON NominaDetalle
                WHEN EXISTS (SELECT 1 FROM Nominas WHERE IdNomina = OLD.IdNomina
                    AND Estado IN ('Confirmada','Pagada','Anulada'))
                BEGIN SELECT RAISE(ABORT, 'El detalle de una nomina cerrada no puede editarse.'); END;");
            Execute(connection, transaction, @"CREATE TRIGGER IF NOT EXISTS TR_NominaDetalle_Proteger_Delete
                BEFORE DELETE ON NominaDetalle
                WHEN EXISTS (SELECT 1 FROM Nominas WHERE IdNomina = OLD.IdNomina
                    AND Estado IN ('Confirmada','Pagada','Anulada'))
                BEGIN SELECT RAISE(ABORT, 'El detalle de una nomina cerrada no puede eliminarse.'); END;");

            detail.Append("Historial laboral, cambios efectivos, snapshots, bloqueos por rango y estados de nomina instalados.");
        }

        private static void MigrateAbsenceRequests(SQLiteConnection connection,
            SQLiteTransaction transaction, StringBuilder detail)
        {
            Execute(connection, transaction, @"CREATE TABLE IF NOT EXISTS SolicitudesAusencia (
                IdSolicitud INTEGER PRIMARY KEY AUTOINCREMENT,
                IdEmpleado INTEGER NOT NULL,
                Tipo TEXT NOT NULL CHECK (Tipo IN ('Vacaciones','Permiso remunerado','Permiso no remunerado',
                    'Incapacidad médica','Licencia','Ausencia justificada','Ausencia injustificada','Suspensión')),
                FechaInicio TEXT NOT NULL,
                FechaFin TEXT NOT NULL,
                Estado TEXT NOT NULL CHECK (Estado IN ('Pendiente','Aprobada','Rechazada','Cancelada')),
                Motivo TEXT NULL,
                AprobadoPor TEXT NULL,
                FechaAprobacion TEXT NULL,
                ObservacionResolucion TEXT NULL,
                UsuarioSolicitante TEXT NULL,
                FechaCreacion TEXT NOT NULL,
                FOREIGN KEY(IdEmpleado) REFERENCES Empleados(IdEmpleado)
            );");
            AddColumnIfMissing(connection, transaction, "Asistencias", "IdSolicitudAusencia",
                "INTEGER NULL REFERENCES SolicitudesAusencia(IdSolicitud)");
            Execute(connection, transaction,
                "CREATE INDEX IF NOT EXISTS IX_SolicitudesAusencia_EmpleadoFechas ON SolicitudesAusencia(IdEmpleado, FechaInicio, FechaFin);");
            Execute(connection, transaction,
                "CREATE INDEX IF NOT EXISTS IX_SolicitudesAusencia_Estado ON SolicitudesAusencia(Estado, FechaInicio);");
            Execute(connection, transaction,
                "CREATE INDEX IF NOT EXISTS IX_Asistencias_IdSolicitudAusencia ON Asistencias(IdSolicitudAusencia);");

            Execute(connection, transaction, @"CREATE TRIGGER IF NOT EXISTS TR_SolicitudesAusencia_Validar_Insert
                BEFORE INSERT ON SolicitudesAusencia
                WHEN NEW.FechaInicio > NEW.FechaFin OR NEW.Estado <> 'Pendiente'
                BEGIN SELECT RAISE(ABORT, 'La solicitud debe tener fechas validas e iniciar pendiente.'); END;");
            Execute(connection, transaction, @"CREATE TRIGGER IF NOT EXISTS TR_SolicitudesAusencia_Transicion_Update
                BEFORE UPDATE OF Estado ON SolicitudesAusencia
                WHEN OLD.Estado <> NEW.Estado AND NOT (
                    (OLD.Estado = 'Pendiente' AND NEW.Estado IN ('Aprobada','Rechazada','Cancelada')) OR
                    (OLD.Estado = 'Aprobada' AND NEW.Estado = 'Cancelada'))
                BEGIN SELECT RAISE(ABORT, 'La transicion de la solicitud de ausencia no es valida.'); END;");
            Execute(connection, transaction, @"CREATE TRIGGER IF NOT EXISTS TR_SolicitudesAusencia_PeriodoCerrado_Insert
                BEFORE INSERT ON SolicitudesAusencia
```

Data/DatabaseMigrationRunner.cs (continuacion)

```
                WHEN EXISTS (SELECT 1 FROM PeriodosNomina WHERE Cerrado = 1
                    AND FechaInicio <= NEW.FechaFin AND FechaFin >= NEW.FechaInicio)
                BEGIN SELECT RAISE(ABORT, 'La ausencia afecta un periodo de nomina cerrado.'); END;");
            Execute(connection, transaction, @"CREATE TRIGGER IF NOT EXISTS TR_SolicitudesAusencia_PeriodoCerrado_Update
                BEFORE UPDATE ON SolicitudesAusencia
                WHEN EXISTS (SELECT 1 FROM PeriodosNomina WHERE Cerrado = 1
                    AND ((FechaInicio <= OLD.FechaFin AND FechaFin >= OLD.FechaInicio)
                      OR (FechaInicio <= NEW.FechaFin AND FechaFin >= NEW.FechaInicio)))
                BEGIN SELECT RAISE(ABORT, 'La ausencia afecta un periodo de nomina cerrado.'); END;");
            Execute(connection, transaction, @"CREATE TRIGGER IF NOT EXISTS TR_SolicitudesAusencia_AprobadaSolapada_Insert
                BEFORE INSERT ON SolicitudesAusencia
                WHEN NEW.Estado = 'Aprobada' AND EXISTS (SELECT 1 FROM SolicitudesAusencia
                    WHERE IdEmpleado = NEW.IdEmpleado AND Estado = 'Aprobada'
                      AND FechaInicio <= NEW.FechaFin AND FechaFin >= NEW.FechaInicio)
                BEGIN SELECT RAISE(ABORT, 'Ya existe una ausencia aprobada que se solapa con este rango.'); END;");
            Execute(connection, transaction, @"CREATE TRIGGER IF NOT EXISTS TR_SolicitudesAusencia_AprobadaSolapada_Update
                BEFORE UPDATE OF Estado, FechaInicio, FechaFin, IdEmpleado ON SolicitudesAusencia
                WHEN NEW.Estado = 'Aprobada' AND EXISTS (SELECT 1 FROM SolicitudesAusencia
                    WHERE IdSolicitud <> OLD.IdSolicitud AND IdEmpleado = NEW.IdEmpleado AND Estado = 'Aprobada'
                      AND FechaInicio <= NEW.FechaFin AND FechaFin >= NEW.FechaInicio)
                BEGIN SELECT RAISE(ABORT, 'Ya existe una ausencia aprobada que se solapa con este rango.'); END;");

            detail.Append("Solicitudes de ausencia, alcance de asistencia, indices y reglas de integridad instalados.");
        }

        private static void AddColumnIfMissing(SQLiteConnection connection, SQLiteTransaction transaction,
            string table, string column, string definition)
        {
            if (ColumnExists(connection, transaction, table, column)) return;
            Execute(connection, transaction, "ALTER TABLE " + table + " ADD COLUMN " + column + " " + definition + ";");
        }

        private static bool ColumnExists(SQLiteConnection connection, SQLiteTransaction transaction, string table, string column)
        {
            using (SQLiteCommand command = new SQLiteCommand("PRAGMA table_info(" + table + ");", connection, transaction))
            using (SQLiteDataReader reader = command.ExecuteReader())
            {
                while (reader.Read())
                {
                    if (string.Equals(Convert.ToString(reader["name"]), column, StringComparison.OrdinalIgnoreCase))
                    {
                        return true;
                    }
                }
            }

            return false;
        }

        private static object Scalar(SQLiteConnection connection, SQLiteTransaction transaction, string sql)
        {
            using (SQLiteCommand command = new SQLiteCommand(sql, connection, transaction))
            {
                return command.ExecuteScalar();
            }
        }

        private static void Execute(SQLiteConnection connection, SQLiteTransaction transaction, string sql,
            params SQLiteParameter[] parameters)
        {
```

Data/DatabaseMigrationRunner.cs (continuacion)

```
            using (SQLiteCommand command = new SQLiteCommand(sql, connection, transaction))
            {
                if (parameters != null && parameters.Length > 0) command.Parameters.AddRange(parameters);
                command.ExecuteNonQuery();
            }
        }

        private static string ComputeSha256(string path)
        {
            using (SHA256 sha = SHA256.Create())
            using (FileStream stream = File.OpenRead(path))
            {
                byte[] hash = sha.ComputeHash(stream);
                StringBuilder builder = new StringBuilder(hash.Length * 2);
                for (int i = 0; i < hash.Length; i++) builder.Append(hash[i].ToString("x2", CultureInfo.InvariantCulture));
                return builder.ToString();
            }
        }
    }
}
```

Data/EmployeeHistoryRepository.cs

```
using System;
using System.Collections.Generic;
using System.Data.SQLite;
using SistemaGestionNomina.Models;

namespace SistemaGestionNomina.Data
{
    public sealed class EmployeeHistoryRepository
    {
        private const string SelectSql = @"SELECT h.*, e.Codigo AS CodigoEmpleado,
                e.Nombre || ' ' || e.Apellido AS EmpleadoNombre
            FROM HistorialEmpleado h
            INNER JOIN Empleados e ON e.IdEmpleado = h.IdEmpleado ";

        public int Add(SQLiteConnection connection, SQLiteTransaction transaction, HistorialEmpleado item)
        {
            using (SQLiteCommand command = new SQLiteCommand(@"INSERT INTO HistorialEmpleado
                (IdEmpleado, CampoModificado, ValorAnterior, ValorNuevo,
                 ValorAnteriorTecnico, ValorNuevoTecnico, FechaCambio, FechaEfectiva,
                 UsuarioResponsable, Motivo, Aplicado, FechaAplicacion)
                VALUES (@empleado, @campo, @anterior, @nuevo, @anteriorTecnico, @nuevoTecnico,
                 @cambio, @efectiva, @usuario, @motivo, @aplicado, @aplicacion);
                SELECT last_insert_rowid();", connection, transaction))
            {
                command.Parameters.AddWithValue("@empleado", item.IdEmpleado);
                command.Parameters.AddWithValue("@campo", item.CampoModificado);
                command.Parameters.AddWithValue("@anterior", item.ValorAnterior ?? string.Empty);
                command.Parameters.AddWithValue("@nuevo", item.ValorNuevo ?? string.Empty);
                command.Parameters.AddWithValue("@anteriorTecnico", item.ValorAnteriorTecnico ?? item.ValorAnterior ?? string.Empty);
                command.Parameters.AddWithValue("@nuevoTecnico", item.ValorNuevoTecnico ?? item.ValorNuevo ?? string.Empty);
                command.Parameters.AddWithValue("@cambio", item.FechaCambio.ToString("yyyy-MM-dd HH:mm:ss"));
                command.Parameters.AddWithValue("@efectiva", item.FechaEfectiva.ToString("yyyy-MM-dd"));
                command.Parameters.AddWithValue("@usuario", item.UsuarioResponsable);
                command.Parameters.AddWithValue("@motivo", item.Motivo ?? string.Empty);
                command.Parameters.AddWithValue("@aplicado", item.Aplicado ? 1 : 0);
                command.Parameters.AddWithValue("@aplicacion", item.FechaAplicacion.HasValue
                    ? (object)item.FechaAplicacion.Value.ToString("yyyy-MM-dd HH:mm:ss") : DBNull.Value);
                return Convert.ToInt32(command.ExecuteScalar());
            }
        }

        public List<HistorialEmpleado> Search(EmployeeHistoryQuery query, int? scopeDepartmentId)
        {
            query = query ?? new EmployeeHistoryQuery();
            List<HistorialEmpleado> items = new List<HistorialEmpleado>();
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteCommand command = new SQLiteCommand(SelectSql + @"
                WHERE (@empleado = 0 OR h.IdEmpleado = @empleado)
                  AND (@scopeDep = 0 OR e.IdDepartamento = @scopeDep)
                  AND (@campo = '' OR h.CampoModificado = @campo)
                  AND (@usuario = '' OR h.UsuarioResponsable LIKE @usuarioLike)
                  AND (@desde = '' OR h.FechaEfectiva >= @desde)
                  AND (@hasta = '' OR h.FechaEfectiva <= @hasta)
                ORDER BY h.FechaEfectiva DESC, h.FechaCambio DESC, h.IdHistorial DESC;", connection))
            {
                command.Parameters.AddWithValue("@empleado", query.EmployeeId ?? 0);
                command.Parameters.AddWithValue("@scopeDep", scopeDepartmentId ?? 0);
                command.Parameters.AddWithValue("@campo", query.Field ?? string.Empty);
                command.Parameters.AddWithValue("@usuario", query.User ?? string.Empty);
                command.Parameters.AddWithValue("@usuarioLike", "%" + (query.User ?? string.Empty) + "%");
```

Data/EmployeeHistoryRepository.cs (continuacion)

```
                command.Parameters.AddWithValue("@desde", query.From.HasValue ? query.From.Value.ToString("yyyy-MM-dd") : string.Empty);
                command.Parameters.AddWithValue("@hasta", query.To.HasValue ? query.To.Value.ToString("yyyy-MM-dd") : string.Empty);
                using (SQLiteDataReader reader = command.ExecuteReader())
                {
                    while (reader.Read()) items.Add(Map(reader));
                }
            }
            return items;
        }

        public List<HistorialEmpleado> GetByEmployee(int employeeId)
        {
            List<HistorialEmpleado> items = new List<HistorialEmpleado>();
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteCommand command = new SQLiteCommand(SelectSql + @"
                WHERE h.IdEmpleado = @empleado
                ORDER BY h.FechaEfectiva ASC, h.FechaCambio ASC, h.IdHistorial ASC;", connection))
            {
                command.Parameters.AddWithValue("@empleado", employeeId);
                using (SQLiteDataReader reader = command.ExecuteReader())
                {
                    while (reader.Read()) items.Add(Map(reader));
                }
            }
            return items;
        }

        public List<HistorialEmpleado> GetPendingDue(SQLiteConnection connection,
            SQLiteTransaction transaction, DateTime effectiveThrough)
        {
            List<HistorialEmpleado> items = new List<HistorialEmpleado>();
            using (SQLiteCommand command = new SQLiteCommand(SelectSql + @"
                WHERE h.Aplicado = 0 AND h.FechaEfectiva <= @fecha
                ORDER BY h.FechaEfectiva, h.FechaCambio, h.IdHistorial;", connection, transaction))
            {
                command.Parameters.AddWithValue("@fecha", effectiveThrough.ToString("yyyy-MM-dd"));
                using (SQLiteDataReader reader = command.ExecuteReader())
                {
                    while (reader.Read()) items.Add(Map(reader));
                }
            }
            return items;
        }

        public void ApplyChange(SQLiteConnection connection, SQLiteTransaction transaction,
            HistorialEmpleado item, DateTime appliedAt)
        {
            string column;
            if (item.CampoModificado == EmployeeHistoryFields.BaseSalary) column = "SalarioBase";
            else if (item.CampoModificado == EmployeeHistoryFields.Position) column = "Cargo";
            else if (item.CampoModificado == EmployeeHistoryFields.Department) column = "IdDepartamento";
            else if (item.CampoModificado == EmployeeHistoryFields.Status) column = "Estado";
            else throw new InvalidOperationException("El campo de historial laboral no es compatible.");

            using (SQLiteCommand command = new SQLiteCommand("UPDATE Empleados SET " + column +
                " = @valor, FechaEfectivaLaboral = @efectiva WHERE IdEmpleado = @empleado;", connection, transaction))
            {
                object value = item.ValorNuevoTecnico ?? item.ValorNuevo;
                if (item.CampoModificado == EmployeeHistoryFields.BaseSalary)
                    value = decimal.Parse(Convert.ToString(value), System.Globalization.CultureInfo.InvariantCulture);
```

Data/EmployeeHistoryRepository.cs (continuacion)

```
                else if (item.CampoModificado == EmployeeHistoryFields.Department)
                    value = int.Parse(Convert.ToString(value), System.Globalization.CultureInfo.InvariantCulture);
                command.Parameters.AddWithValue("@valor", value);
                command.Parameters.AddWithValue("@efectiva", item.FechaEfectiva.ToString("yyyy-MM-dd"));
                command.Parameters.AddWithValue("@empleado", item.IdEmpleado);
                if (command.ExecuteNonQuery() != 1) throw new InvalidOperationException("No se pudo aplicar el cambio laboral programado.");
            }

            using (SQLiteCommand command = new SQLiteCommand(@"UPDATE HistorialEmpleado
                SET Aplicado = 1, FechaAplicacion = @fecha WHERE IdHistorial = @id AND Aplicado = 0;",
                connection, transaction))
            {
                command.Parameters.AddWithValue("@fecha", appliedAt.ToString("yyyy-MM-dd HH:mm:ss"));
                command.Parameters.AddWithValue("@id", item.IdHistorial);
                if (command.ExecuteNonQuery() != 1) throw new InvalidOperationException("El cambio laboral ya fue procesado.");
            }
        }

        private static HistorialEmpleado Map(SQLiteDataReader reader)
        {
            return new HistorialEmpleado
            {
                IdHistorial = Convert.ToInt32(reader["IdHistorial"]),
                IdEmpleado = Convert.ToInt32(reader["IdEmpleado"]),
                CodigoEmpleado = Convert.ToString(reader["CodigoEmpleado"]),
                EmpleadoNombre = Convert.ToString(reader["EmpleadoNombre"]),
                CampoModificado = Convert.ToString(reader["CampoModificado"]),
                ValorAnterior = Convert.ToString(reader["ValorAnterior"]),
                ValorNuevo = Convert.ToString(reader["ValorNuevo"]),
                ValorAnteriorTecnico = Convert.ToString(reader["ValorAnteriorTecnico"]),
                ValorNuevoTecnico = Convert.ToString(reader["ValorNuevoTecnico"]),
                FechaCambio = DateTime.Parse(Convert.ToString(reader["FechaCambio"])),
                FechaEfectiva = DateTime.Parse(Convert.ToString(reader["FechaEfectiva"])),
                UsuarioResponsable = Convert.ToString(reader["UsuarioResponsable"]),
                Motivo = Convert.ToString(reader["Motivo"]),
                Aplicado = Convert.ToInt32(reader["Aplicado"]) == 1,
                FechaAplicacion = reader["FechaAplicacion"] == DBNull.Value ||
                    string.IsNullOrWhiteSpace(Convert.ToString(reader["FechaAplicacion"]))
                    ? (DateTime?)null : DateTime.Parse(Convert.ToString(reader["FechaAplicacion"]))
            };
        }
    }
}
```

Data/Repositories.cs

```
using System;
using System.Collections.Generic;
using System.Data.SQLite;
using SistemaGestionNomina.Models;

namespace SistemaGestionNomina.Data
{
    public class UsuarioRepository
    {
        public Usuario GetByUsername(string username)
        {
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteCommand command = new SQLiteCommand("SELECT * FROM Usuarios WHERE NombreUsuario = @u AND Estado = 'Activo';", connection))
            {
                command.Parameters.AddWithValue("@u", username);
                using (SQLiteDataReader reader = command.ExecuteReader())
                {
                    if (reader.Read())
                    {
                        return new Usuario
                        {
                            IdUsuario = Convert.ToInt32(reader["IdUsuario"]),
                            NombreUsuario = Convert.ToString(reader["NombreUsuario"]),
                            PasswordHash = Convert.ToString(reader["PasswordHash"]),
                            Rol = Convert.ToString(reader["Rol"]),
                            Estado = Convert.ToString(reader["Estado"]),
                            IdEmpleado = reader["IdEmpleado"] == DBNull.Value ? (int?)null : Convert.ToInt32(reader["IdEmpleado"]),
                            UltimoAcceso = ParseNullableDate(reader["UltimoAcceso"]),
                            IntentosFallidos = Convert.ToInt32(reader["IntentosFallidos"]),
                            Bloqueado = Convert.ToInt32(reader["Bloqueado"]) == 1,
                            FechaBloqueo = ParseNullableDate(reader["FechaBloqueo"])
                        };
                    }
                }
            }

            return null;
        }

        private static DateTime? ParseNullableDate(object value)
        {
            if (value == null || value == DBNull.Value || string.IsNullOrWhiteSpace(Convert.ToString(value))) return null;
            DateTime parsed;
            return DateTime.TryParse(Convert.ToString(value), out parsed) ? parsed : (DateTime?)null;
        }
    }

    public class DepartamentoRepository
    {
        public List<Departamento> GetAll()
        {
            List<Departamento> items = new List<Departamento>();
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteCommand command = new SQLiteCommand("SELECT IdDepartamento, Nombre FROM Departamentos ORDER BY Nombre;", connection))
            using (SQLiteDataReader reader = command.ExecuteReader())
            {
                while (reader.Read())
                {
                    items.Add(new Departamento
                    {
```

Data/Repositories.cs (continuacion)

```
                        IdDepartamento = Convert.ToInt32(reader["IdDepartamento"]),
                        Nombre = Convert.ToString(reader["Nombre"])
                    });
                }
            }

            return items;
        }
    }

    public class EmpleadoRepository
    {
        public List<Empleado> GetAll(string search, int? departmentId, string status)
        {
            return GetAll(search, departmentId, status, null);
        }

        public List<Empleado> GetAll(string search, int? departmentId, string status, int? scopeDepartmentId)
        {
            List<Empleado> items = new List<Empleado>();
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteCommand command = new SQLiteCommand(@"SELECT e.*, d.Nombre AS DepartamentoNombre
                FROM Empleados e
                INNER JOIN Departamentos d ON d.IdDepartamento = e.IdDepartamento
                WHERE (@search = '' OR e.Codigo LIKE @like OR e.Nombre LIKE @like OR e.Apellido LIKE @like OR e.Cedula LIKE @like)
                  AND (@dep = 0 OR e.IdDepartamento = @dep)
                  AND (@scopeDep = 0 OR e.IdDepartamento = @scopeDep)
                  AND (@estado = '' OR e.Estado = @estado)
                ORDER BY e.Nombre, e.Apellido;", connection))
            {
                command.Parameters.AddWithValue("@search", search ?? string.Empty);
                command.Parameters.AddWithValue("@like", "%" + (search ?? string.Empty) + "%");
                command.Parameters.AddWithValue("@dep", departmentId.HasValue ? departmentId.Value : 0);
                command.Parameters.AddWithValue("@scopeDep", scopeDepartmentId.HasValue ? scopeDepartmentId.Value : 0);
                command.Parameters.AddWithValue("@estado", status ?? string.Empty);
                using (SQLiteDataReader reader = command.ExecuteReader())
                {
                    while (reader.Read())
                    {
                        items.Add(MapEmpleado(reader));
                    }
                }
            }

            return items;
        }

        public List<Empleado> GetActiveByDepartment(int? departmentId)
        {
            return GetAll(string.Empty, departmentId, "Activo");
        }

        public List<Empleado> GetActiveByDepartment(int? departmentId, int? scopeDepartmentId)
        {
            return GetAll(string.Empty, departmentId, "Activo", scopeDepartmentId);
        }

        public List<Empleado> GetAllForEffectiveDate(int? departmentId)
        {
            return GetAll(string.Empty, departmentId, string.Empty, null);
```

Data/Repositories.cs (continuacion)

```
        }

        public Empleado GetById(int id)
        {
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            {
                return GetById(connection, null, id);
            }
        }

        public Empleado GetById(SQLiteConnection connection, SQLiteTransaction transaction, int id)
        {
            using (SQLiteCommand command = new SQLiteCommand(@"SELECT e.*, d.Nombre AS DepartamentoNombre
                FROM Empleados e INNER JOIN Departamentos d ON d.IdDepartamento = e.IdDepartamento
                WHERE e.IdEmpleado = @id;", connection, transaction))
            {
                command.Parameters.AddWithValue("@id", id);
                using (SQLiteDataReader reader = command.ExecuteReader())
                {
                    return reader.Read() ? MapEmpleado(reader) : null;
                }
            }
        }

        public int? GetDepartmentId(int employeeId)
        {
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteCommand command = new SQLiteCommand(
                "SELECT IdDepartamento FROM Empleados WHERE IdEmpleado = @id LIMIT 1;", connection))
            {
                command.Parameters.AddWithValue("@id", employeeId);
                object value = command.ExecuteScalar();
                return value == null || value == DBNull.Value ? (int?)null : Convert.ToInt32(value);
            }
        }

        public bool ExistsCode(string code, int ignoreId)
        {
            return Exists("Codigo", code, ignoreId);
        }

        public bool ExistsCedula(string cedula, int ignoreId)
        {
            return Exists("Cedula", cedula, ignoreId);
        }

        public int Add(Empleado empleado)
        {
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteCommand command = new SQLiteCommand(@"INSERT INTO Empleados
                (Codigo, Nombre, Apellido, Cedula, Cargo, IdDepartamento, SalarioBase, Estado, FechaIngreso, FechaEfectivaLaboral)
                VALUES (@codigo, @nombre, @apellido, @cedula, @cargo, @dep, @salario, @estado, @fecha, @efectiva);
                SELECT last_insert_rowid();", connection))
            {
                FillEmpleadoParameters(command, empleado);
                return Convert.ToInt32(command.ExecuteScalar());
            }
        }

        public void Update(Empleado empleado)
```

Data/Repositories.cs (continuacion)

```
        {
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteTransaction transaction = connection.BeginTransaction())
            {
                Update(connection, transaction, empleado);
                transaction.Commit();
            }
        }

        public void Update(SQLiteConnection connection, SQLiteTransaction transaction, Empleado empleado)
        {
            using (SQLiteCommand command = new SQLiteCommand(@"UPDATE Empleados SET
                Codigo = @codigo, Nombre = @nombre, Apellido = @apellido, Cedula = @cedula,
                Cargo = @cargo, IdDepartamento = @dep, SalarioBase = @salario,
                Estado = @estado, FechaIngreso = @fecha, FechaEfectivaLaboral = @efectiva
                WHERE IdEmpleado = @id;", connection, transaction))
            {
                FillEmpleadoParameters(command, empleado);
                command.Parameters.AddWithValue("@id", empleado.IdEmpleado);
                if (command.ExecuteNonQuery() != 1)
                    throw new InvalidOperationException("No se pudo actualizar el empleado.");
            }
        }

        public string GetDepartmentName(int departmentId)
        {
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteCommand command = new SQLiteCommand(
                "SELECT Nombre FROM Departamentos WHERE IdDepartamento = @id LIMIT 1;", connection))
            {
                command.Parameters.AddWithValue("@id", departmentId);
                object value = command.ExecuteScalar();
                return value == null || value == DBNull.Value ? string.Empty : Convert.ToString(value);
            }
        }

        public void Deactivate(int id)
        {
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteCommand command = new SQLiteCommand("UPDATE Empleados SET Estado = 'Inactivo' WHERE IdEmpleado = @id;", connection))
            {
                command.Parameters.AddWithValue("@id", id);
                command.ExecuteNonQuery();
            }
        }

        private bool Exists(string field, string value, int ignoreId)
        {
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteCommand command = new SQLiteCommand("SELECT COUNT(1) FROM Empleados WHERE " + field + " = @value AND IdEmpleado <> @id;",  \
    connection))
            {
                command.Parameters.AddWithValue("@value", value);
                command.Parameters.AddWithValue("@id", ignoreId);
                return Convert.ToInt32(command.ExecuteScalar()) > 0;
            }
        }

        private static Empleado MapEmpleado(SQLiteDataReader reader)
        {
```

Data/Repositories.cs (continuacion)

```
            return new Empleado
            {
                IdEmpleado = Convert.ToInt32(reader["IdEmpleado"]),
                Codigo = Convert.ToString(reader["Codigo"]),
                Nombre = Convert.ToString(reader["Nombre"]),
                Apellido = Convert.ToString(reader["Apellido"]),
                Cedula = Convert.ToString(reader["Cedula"]),
                Cargo = Convert.ToString(reader["Cargo"]),
                IdDepartamento = Convert.ToInt32(reader["IdDepartamento"]),
                DepartamentoNombre = Convert.ToString(reader["DepartamentoNombre"]),
                SalarioBase = Convert.ToDecimal(reader["SalarioBase"]),
                Estado = Convert.ToString(reader["Estado"]),
                FechaIngreso = DateTime.Parse(Convert.ToString(reader["FechaIngreso"])),
                FechaEfectivaLaboral = reader["FechaEfectivaLaboral"] == DBNull.Value ||
                    string.IsNullOrWhiteSpace(Convert.ToString(reader["FechaEfectivaLaboral"]))
                    ? (DateTime?)null : DateTime.Parse(Convert.ToString(reader["FechaEfectivaLaboral"]))
            };
        }

        private static void FillEmpleadoParameters(SQLiteCommand command, Empleado empleado)
        {
            command.Parameters.AddWithValue("@codigo", empleado.Codigo);
            command.Parameters.AddWithValue("@nombre", empleado.Nombre);
            command.Parameters.AddWithValue("@apellido", empleado.Apellido);
            command.Parameters.AddWithValue("@cedula", empleado.Cedula);
            command.Parameters.AddWithValue("@cargo", empleado.Cargo);
            command.Parameters.AddWithValue("@dep", empleado.IdDepartamento);
            command.Parameters.AddWithValue("@salario", empleado.SalarioBase);
            command.Parameters.AddWithValue("@estado", empleado.Estado);
            command.Parameters.AddWithValue("@fecha", empleado.FechaIngreso.ToString("yyyy-MM-dd"));
            command.Parameters.AddWithValue("@efectiva", (empleado.FechaEfectivaLaboral ?? empleado.FechaIngreso).ToString("yyyy-MM-dd"));
        }
    }

    public class AsistenciaRepository
    {
        public bool Exists(int employeeId, DateTime date)
        {
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteCommand command = new SQLiteCommand(@"SELECT COUNT(1) FROM Asistencias
                WHERE IdEmpleado = @empleado AND Fecha = @fecha;", connection))
            {
                command.Parameters.AddWithValue("@empleado", employeeId);
                command.Parameters.AddWithValue("@fecha", date.ToString("yyyy-MM-dd"));
                return Convert.ToInt32(command.ExecuteScalar()) > 0;
            }
        }

        public int Add(Asistencia asistencia)
        {
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteCommand command = new SQLiteCommand(@"INSERT INTO Asistencias
                (IdEmpleado, Fecha, HoraEntrada, HoraSalida, HorasTrabajadas, Estado)
                VALUES (@empleado, @fecha, @entrada, @salida, @horas, @estado);
                SELECT last_insert_rowid();", connection))
            {
                command.Parameters.AddWithValue("@empleado", asistencia.IdEmpleado);
                command.Parameters.AddWithValue("@fecha", asistencia.Fecha.ToString("yyyy-MM-dd"));
                command.Parameters.AddWithValue("@entrada", asistencia.HoraEntrada.HasValue ? asistencia.HoraEntrada.Value.ToString(@"hh\:mm") : null);
                command.Parameters.AddWithValue("@salida", asistencia.HoraSalida.HasValue ? asistencia.HoraSalida.Value.ToString(@"hh\:mm") : null);
```

Data/Repositories.cs (continuacion)

```
                command.Parameters.AddWithValue("@horas", asistencia.HorasTrabajadas);
                command.Parameters.AddWithValue("@estado", asistencia.Estado);
                return Convert.ToInt32(command.ExecuteScalar());
            }
        }

        public List<Asistencia> GetAll(DateTime? start, DateTime? end, int? employeeId, string status)
        {
            return GetAll(start, end, employeeId, status, null);
        }

        public List<Asistencia> GetAll(DateTime? start, DateTime? end, int? employeeId, string status,
            int? scopeDepartmentId)
        {
            List<Asistencia> items = new List<Asistencia>();
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteCommand command = new SQLiteCommand(@"SELECT a.*, e.Nombre || ' ' || e.Apellido AS EmpleadoNombre
                FROM Asistencias a
                INNER JOIN Empleados e ON e.IdEmpleado = a.IdEmpleado
                WHERE (@inicio = '' OR a.Fecha >= @inicio)
                  AND (@fin = '' OR a.Fecha <= @fin)
                  AND (@empleado = 0 OR a.IdEmpleado = @empleado)
                  AND (@scopeDep = 0 OR e.IdDepartamento = @scopeDep)
                  AND (@estado = '' OR a.Estado = @estado)
                ORDER BY a.Fecha DESC, EmpleadoNombre;", connection))
            {
                command.Parameters.AddWithValue("@inicio", start.HasValue ? start.Value.ToString("yyyy-MM-dd") : string.Empty);
                command.Parameters.AddWithValue("@fin", end.HasValue ? end.Value.ToString("yyyy-MM-dd") : string.Empty);
                command.Parameters.AddWithValue("@empleado", employeeId.HasValue ? employeeId.Value : 0);
                command.Parameters.AddWithValue("@scopeDep", scopeDepartmentId.HasValue ? scopeDepartmentId.Value : 0);
                command.Parameters.AddWithValue("@estado", status ?? string.Empty);
                using (SQLiteDataReader reader = command.ExecuteReader())
                {
                    while (reader.Read())
                    {
                        items.Add(MapAsistencia(reader));
                    }
                }
            }

            return items;
        }

        public List<Asistencia> GetByEmployee(int employeeId, DateTime? start, DateTime? end, string status)
        {
            List<Asistencia> items = new List<Asistencia>();
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteCommand command = new SQLiteCommand(@"SELECT a.*, e.Nombre || ' ' || e.Apellido AS EmpleadoNombre
                FROM Asistencias a
                INNER JOIN Empleados e ON e.IdEmpleado = a.IdEmpleado
                WHERE a.IdEmpleado = @empleado
                  AND (@inicio = '' OR a.Fecha >= @inicio)
                  AND (@fin = '' OR a.Fecha <= @fin)
                  AND (@estado = '' OR a.Estado = @estado)
                ORDER BY a.Fecha DESC;", connection))
            {
                command.Parameters.AddWithValue("@empleado", employeeId);
                command.Parameters.AddWithValue("@inicio", start.HasValue ? start.Value.ToString("yyyy-MM-dd") : string.Empty);
                command.Parameters.AddWithValue("@fin", end.HasValue ? end.Value.ToString("yyyy-MM-dd") : string.Empty);
                command.Parameters.AddWithValue("@estado", status ?? string.Empty);
```

Data/Repositories.cs (continuacion)

```
                using (SQLiteDataReader reader = command.ExecuteReader())
                {
                    while (reader.Read())
                    {
                        items.Add(MapAsistencia(reader));
                    }
                }
            }

            return items;
        }

        public decimal GetExtraHours(int employeeId, DateTime start, DateTime end)
        {
            decimal total = 0;
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteCommand command = new SQLiteCommand(@"SELECT HorasTrabajadas FROM Asistencias
                WHERE IdEmpleado = @id AND Fecha BETWEEN @inicio AND @fin AND Estado IN ('Puntual','Tardanza');", connection))
            {
                command.Parameters.AddWithValue("@id", employeeId);
                command.Parameters.AddWithValue("@inicio", start.ToString("yyyy-MM-dd"));
                command.Parameters.AddWithValue("@fin", end.ToString("yyyy-MM-dd"));
                using (SQLiteDataReader reader = command.ExecuteReader())
                {
                    while (reader.Read())
                    {
                        decimal hours = Convert.ToDecimal(reader["HorasTrabajadas"]);
                        if (hours > 8)
                        {
                            total += hours - 8;
                        }
                    }
                }
            }

            return total;
        }

        private static Asistencia MapAsistencia(SQLiteDataReader reader)
        {
            string entrada = Convert.ToString(reader["HoraEntrada"]);
            string salida = Convert.ToString(reader["HoraSalida"]);
            return new Asistencia
            {
                IdAsistencia = Convert.ToInt32(reader["IdAsistencia"]),
                IdEmpleado = Convert.ToInt32(reader["IdEmpleado"]),
                EmpleadoNombre = Convert.ToString(reader["EmpleadoNombre"]),
                Fecha = DateTime.Parse(Convert.ToString(reader["Fecha"])),
                HoraEntrada = string.IsNullOrWhiteSpace(entrada) ? (TimeSpan?)null : TimeSpan.Parse(entrada),
                HoraSalida = string.IsNullOrWhiteSpace(salida) ? (TimeSpan?)null : TimeSpan.Parse(salida),
                HorasTrabajadas = Convert.ToDecimal(reader["HorasTrabajadas"]),
                Estado = Convert.ToString(reader["Estado"]),
                IdSolicitudAusencia = reader["IdSolicitudAusencia"] == DBNull.Value
                    ? (int?)null : Convert.ToInt32(reader["IdSolicitudAusencia"])
            };
        }
    }

    public class NominaRepository
    {
```

Data/Repositories.cs (continuacion)

```
        public int CreatePeriodo(PeriodoNomina periodo)
        {
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteCommand command = new SQLiteCommand(@"INSERT INTO PeriodosNomina (Nombre, FechaInicio, FechaFin, Estado)
                VALUES (@nombre, @inicio, @fin, @estado); SELECT last_insert_rowid();", connection))
            {
                command.Parameters.AddWithValue("@nombre", periodo.Nombre);
                command.Parameters.AddWithValue("@inicio", periodo.FechaInicio.ToString("yyyy-MM-dd"));
                command.Parameters.AddWithValue("@fin", periodo.FechaFin.ToString("yyyy-MM-dd"));
                command.Parameters.AddWithValue("@estado", periodo.Estado);
                return Convert.ToInt32(command.ExecuteScalar());
            }
        }

        public int CreateNomina(Nomina nomina)
        {
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteTransaction transaction = connection.BeginTransaction())
            {
                using (SQLiteCommand command = new SQLiteCommand(@"INSERT INTO Nominas
                    (IdPeriodo, FechaCalculo, TotalIngresos, TotalDeducciones, TotalNeto, Estado)
                    VALUES (@periodo, @fecha, @ingresos, @deducciones, @neto, @estado);
                    SELECT last_insert_rowid();", connection, transaction))
                {
                    command.Parameters.AddWithValue("@periodo", nomina.IdPeriodo);
                    command.Parameters.AddWithValue("@fecha", nomina.FechaCalculo.ToString("yyyy-MM-dd HH:mm:ss"));
                    command.Parameters.AddWithValue("@ingresos", nomina.TotalIngresos);
                    command.Parameters.AddWithValue("@deducciones", nomina.TotalDeducciones);
                    command.Parameters.AddWithValue("@neto", nomina.TotalNeto);
                    command.Parameters.AddWithValue("@estado", nomina.Estado);
                    int idNomina = Convert.ToInt32(command.ExecuteScalar());

                    foreach (NominaDetalle detalle in nomina.Detalles)
                    {
                        using (SQLiteCommand detailCommand = new SQLiteCommand(@"INSERT INTO NominaDetalle
                            (IdNomina, IdEmpleado, SueldoBase, Bonos, HorasExtra, MontoHorasExtra, TotalIngresos, TotalDeducciones, NetoPagar)
                            VALUES (@nomina, @empleado, @sueldo, @bonos, @horas, @montoHoras, @ingresos, @deducciones, @neto);", connection, transaction))
                        {
                            detailCommand.Parameters.AddWithValue("@nomina", idNomina);
                            detailCommand.Parameters.AddWithValue("@empleado", detalle.IdEmpleado);
                            detailCommand.Parameters.AddWithValue("@sueldo", detalle.SueldoBase);
                            detailCommand.Parameters.AddWithValue("@bonos", detalle.Bonos);
                            detailCommand.Parameters.AddWithValue("@horas", detalle.HorasExtra);
                            detailCommand.Parameters.AddWithValue("@montoHoras", detalle.MontoHorasExtra);
                            detailCommand.Parameters.AddWithValue("@ingresos", detalle.TotalIngresos);
                            detailCommand.Parameters.AddWithValue("@deducciones", detalle.TotalDeducciones);
                            detailCommand.Parameters.AddWithValue("@neto", detalle.NetoPagar);
                            detailCommand.ExecuteNonQuery();
                        }
                    }

                    transaction.Commit();
                    return idNomina;
                }
            }
        }

        public List<Nomina> GetAll()
        {
            List<Nomina> items = new List<Nomina>();
```

Data/Repositories.cs (continuacion)

```
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteCommand command = new SQLiteCommand(@"SELECT n.*, p.Nombre AS PeriodoNombre
                FROM Nominas n INNER JOIN PeriodosNomina p ON p.IdPeriodo = n.IdPeriodo
                WHERE n.Estado <> 'Anulada'
                ORDER BY n.FechaCalculo DESC;", connection))
            using (SQLiteDataReader reader = command.ExecuteReader())
            {
                while (reader.Read())
                {
                    items.Add(MapNomina(reader));
                }
            }

            return items;
        }

        public List<NominaDetalle> GetDetalles(int idNomina)
        {
            List<NominaDetalle> items = new List<NominaDetalle>();
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteCommand command = new SQLiteCommand(@"SELECT nd.*,
                    COALESCE(NULLIF(nd.CodigoEmpleadoSnapshot, ''), e.Codigo) AS Codigo,
                    COALESCE(NULLIF(nd.NombreEmpleadoSnapshot, ''), e.Nombre || ' ' || e.Apellido) AS EmpleadoNombre,
                    COALESCE(NULLIF(nd.CargoEmpleadoSnapshot, ''), e.Cargo) AS CargoEmpleado,
                    COALESCE(NULLIF(nd.DepartamentoSnapshot, ''), d.Nombre) AS Departamento
                FROM NominaDetalle nd
                INNER JOIN Empleados e ON e.IdEmpleado = nd.IdEmpleado
                INNER JOIN Departamentos d ON d.IdDepartamento = e.IdDepartamento
                WHERE nd.IdNomina = @id
                ORDER BY EmpleadoNombre;", connection))
            {
                command.Parameters.AddWithValue("@id", idNomina);
                using (SQLiteDataReader reader = command.ExecuteReader())
                {
                    while (reader.Read())
                    {
                        items.Add(MapDetalle(reader));
                    }
                }
            }

            return items;
        }

        public Nomina GetById(int idNomina)
        {
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteCommand command = new SQLiteCommand(@"SELECT n.*, p.Nombre AS PeriodoNombre
                FROM Nominas n INNER JOIN PeriodosNomina p ON p.IdPeriodo = n.IdPeriodo
                WHERE n.IdNomina = @id;", connection))
            {
                command.Parameters.AddWithValue("@id", idNomina);
                using (SQLiteDataReader reader = command.ExecuteReader())
                {
                    if (reader.Read()) return MapNomina(reader);
                }
            }
            return null;
        }

```

Data/Repositories.cs (continuacion)

```
        public void UpdateEstadoNomina(int idNomina, string estado,
            string campoFecha, DateTime? fecha,
            string campoUsuario, string usuario,
            string motivo = null)
        {
            string sql = "UPDATE Nominas SET Estado = @estado";
            if (!string.IsNullOrWhiteSpace(campoFecha) && fecha.HasValue)
                sql += ", " + campoFecha + " = @fecha";
            if (!string.IsNullOrWhiteSpace(campoUsuario) && !string.IsNullOrWhiteSpace(usuario))
                sql += ", " + campoUsuario + " = @usuario";
            if (motivo != null)
                sql += ", MotivoAnulacion = @motivo";
            sql += " WHERE IdNomina = @id;";

            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteCommand command = new SQLiteCommand(sql, connection))
            {
                command.Parameters.AddWithValue("@id", idNomina);
                command.Parameters.AddWithValue("@estado", estado);
                if (fecha.HasValue) command.Parameters.AddWithValue("@fecha", fecha.Value.ToString("yyyy-MM-dd HH:mm:ss"));
                if (!string.IsNullOrWhiteSpace(usuario)) command.Parameters.AddWithValue("@usuario", usuario);
                if (motivo != null) command.Parameters.AddWithValue("@motivo", motivo);
                command.ExecuteNonQuery();
            }
        }

        public void CerrarPeriodo(int idPeriodo, string usuario)
        {
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteCommand command = new SQLiteCommand(@"UPDATE PeriodosNomina
                SET Cerrado = 1, FechaCierre = @fecha, CerradoPor = @usuario
                WHERE IdPeriodo = @id;", connection))
            {
                command.Parameters.AddWithValue("@id", idPeriodo);
                command.Parameters.AddWithValue("@fecha", DateTime.Now.ToString("yyyy-MM-dd HH:mm:ss"));
                command.Parameters.AddWithValue("@usuario", usuario);
                command.ExecuteNonQuery();
            }
        }

        public void ReabrirPeriodo(int idPeriodo)
        {
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteCommand command = new SQLiteCommand(@"UPDATE PeriodosNomina
                SET Cerrado = 0, FechaCierre = NULL, CerradoPor = NULL
                WHERE IdPeriodo = @id;", connection))
            {
                command.Parameters.AddWithValue("@id", idPeriodo);
                command.ExecuteNonQuery();
            }
        }

        public PeriodoNomina GetPeriodoById(int idPeriodo)
        {
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteCommand command = new SQLiteCommand("SELECT * FROM PeriodosNomina WHERE IdPeriodo = @id;", connection))
            {
                command.Parameters.AddWithValue("@id", idPeriodo);
                using (SQLiteDataReader reader = command.ExecuteReader())
                {
```

Data/Repositories.cs (continuacion)

```
                    if (reader.Read())
                    {
                        return new PeriodoNomina
                        {
                            IdPeriodo = Convert.ToInt32(reader["IdPeriodo"]),
                            Nombre = Convert.ToString(reader["Nombre"]),
                            FechaInicio = DateTime.Parse(Convert.ToString(reader["FechaInicio"])),
                            FechaFin = DateTime.Parse(Convert.ToString(reader["FechaFin"])),
                            Estado = Convert.ToString(reader["Estado"]),
                            Cerrado = Convert.ToInt32(reader["Cerrado"]) == 1,
                            FechaCierre = reader["FechaCierre"] == DBNull.Value ? (DateTime?)null : DateTime.Parse(Convert.ToString(reader["FechaCierre"])),
                            CerradoPor = reader["CerradoPor"] == DBNull.Value ? null : Convert.ToString(reader["CerradoPor"])
                        };
                    }
                }
            }
            return null;
        }

        public PeriodoNomina GetPeriodoByFechas(DateTime fechaInicio, DateTime fechaFin)
        {
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteCommand command = new SQLiteCommand(@"SELECT * FROM PeriodosNomina
                WHERE FechaInicio <= @fin AND FechaFin >= @inicio
                ORDER BY Cerrado DESC, FechaInicio LIMIT 1;", connection))
            {
                command.Parameters.AddWithValue("@inicio", fechaInicio.ToString("yyyy-MM-dd"));
                command.Parameters.AddWithValue("@fin", fechaFin.ToString("yyyy-MM-dd"));
                using (SQLiteDataReader reader = command.ExecuteReader())
                {
                    if (reader.Read())
                    {
                        return new PeriodoNomina
                        {
                            IdPeriodo = Convert.ToInt32(reader["IdPeriodo"]),
                            Nombre = Convert.ToString(reader["Nombre"]),
                            FechaInicio = DateTime.Parse(Convert.ToString(reader["FechaInicio"])),
                            FechaFin = DateTime.Parse(Convert.ToString(reader["FechaFin"])),
                            Estado = Convert.ToString(reader["Estado"]),
                            Cerrado = Convert.ToInt32(reader["Cerrado"]) == 1
                        };
                    }
                }
            }
            return null;
        }

        public PeriodoNomina GetOverlappingProtectedPeriod(DateTime fechaInicio, DateTime fechaFin)
        {
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteCommand command = new SQLiteCommand(@"SELECT * FROM PeriodosNomina
                WHERE FechaInicio <= @fin AND FechaFin >= @inicio
                  AND (Cerrado = 1 OR Estado IN ('Confirmado','Pagado'))
                ORDER BY FechaInicio LIMIT 1;", connection))
            {
                command.Parameters.AddWithValue("@inicio", fechaInicio.ToString("yyyy-MM-dd"));
                command.Parameters.AddWithValue("@fin", fechaFin.ToString("yyyy-MM-dd"));
                using (SQLiteDataReader reader = command.ExecuteReader())
                {
                    if (reader.Read()) return MapPeriodo(reader);
```

Data/Repositories.cs (continuacion)

```
                }
            }
            return null;
        }

        public bool HasClosedPeriodAffectedByEmployeeChange(DateTime effectiveDate)
        {
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            {
                return HasClosedPeriodAffectedByEmployeeChange(connection, null, effectiveDate);
            }
        }

        public bool HasClosedPeriodAffectedByEmployeeChange(SQLiteConnection connection,
            SQLiteTransaction transaction, DateTime effectiveDate)
        {
            using (SQLiteCommand command = new SQLiteCommand(@"SELECT COUNT(1) FROM PeriodosNomina
                WHERE Cerrado = 1 AND FechaFin >= @efectiva;", connection, transaction))
            {
                command.Parameters.AddWithValue("@efectiva", effectiveDate.ToString("yyyy-MM-dd"));
                return Convert.ToInt32(command.ExecuteScalar()) > 0;
            }
        }

        private static PeriodoNomina MapPeriodo(SQLiteDataReader reader)
        {
            return new PeriodoNomina
            {
                IdPeriodo = Convert.ToInt32(reader["IdPeriodo"]),
                Nombre = Convert.ToString(reader["Nombre"]),
                FechaInicio = DateTime.Parse(Convert.ToString(reader["FechaInicio"])),
                FechaFin = DateTime.Parse(Convert.ToString(reader["FechaFin"])),
                Estado = Convert.ToString(reader["Estado"]),
                Cerrado = Convert.ToInt32(reader["Cerrado"]) == 1,
                FechaCierre = reader["FechaCierre"] == DBNull.Value ? (DateTime?)null :
                    DateTime.Parse(Convert.ToString(reader["FechaCierre"])),
                CerradoPor = reader["CerradoPor"] == DBNull.Value ? null : Convert.ToString(reader["CerradoPor"])
            };
        }

        public int CreateNominaEnTransaccion(SQLiteConnection connection, SQLiteTransaction transaction, Nomina nomina)
        {
            using (SQLiteCommand command = new SQLiteCommand(@"INSERT INTO Nominas
                (IdPeriodo, FechaCalculo, TotalIngresos, TotalDeducciones, TotalNeto, Estado,
                 FechaConfirmacion, ConfirmadaPor)
                VALUES (@periodo, @fecha, @ingresos, @deducciones, @neto, @estado, @fecConf, @usrConf);
                SELECT last_insert_rowid();", connection, transaction))
            {
                command.Parameters.AddWithValue("@periodo", nomina.IdPeriodo);
                command.Parameters.AddWithValue("@fecha", nomina.FechaCalculo.ToString("yyyy-MM-dd HH:mm:ss"));
                command.Parameters.AddWithValue("@ingresos", nomina.TotalIngresos);
                command.Parameters.AddWithValue("@deducciones", nomina.TotalDeducciones);
                command.Parameters.AddWithValue("@neto", nomina.TotalNeto);
                command.Parameters.AddWithValue("@estado", nomina.Estado);
                command.Parameters.AddWithValue("@fecConf", nomina.FechaConfirmacion?.ToString("yyyy-MM-dd HH:mm:ss") ??  \
    DateTime.Now.ToString("yyyy-MM-dd HH:mm:ss"));
                command.Parameters.AddWithValue("@usrConf", nomina.ConfirmadaPor ?? string.Empty);
                return Convert.ToInt32(command.ExecuteScalar());
            }
        }
```

Data/Repositories.cs (continuacion)

```

        private static Nomina MapNomina(SQLiteDataReader reader)
        {
            return new Nomina
            {
                IdNomina = Convert.ToInt32(reader["IdNomina"]),
                IdPeriodo = Convert.ToInt32(reader["IdPeriodo"]),
                PeriodoNombre = Convert.ToString(reader["PeriodoNombre"]),
                FechaCalculo = DateTime.Parse(Convert.ToString(reader["FechaCalculo"])),
                TotalIngresos = Convert.ToDecimal(reader["TotalIngresos"]),
                TotalDeducciones = Convert.ToDecimal(reader["TotalDeducciones"]),
                TotalNeto = Convert.ToDecimal(reader["TotalNeto"]),
                Estado = Convert.ToString(reader["Estado"]),
                FechaConfirmacion = reader["FechaConfirmacion"] == DBNull.Value ? (DateTime?)null :  \
    DateTime.Parse(Convert.ToString(reader["FechaConfirmacion"])),
                ConfirmadaPor = reader["ConfirmadaPor"] == DBNull.Value ? null : Convert.ToString(reader["ConfirmadaPor"]),
                FechaPago = reader["FechaPago"] == DBNull.Value ? (DateTime?)null : DateTime.Parse(Convert.ToString(reader["FechaPago"])),
                PagadaPor = reader["PagadaPor"] == DBNull.Value ? null : Convert.ToString(reader["PagadaPor"]),
                FechaAnulacion = reader["FechaAnulacion"] == DBNull.Value ? (DateTime?)null : DateTime.Parse(Convert.ToString(reader["FechaAnulacion"])),
                AnuladaPor = reader["AnuladaPor"] == DBNull.Value ? null : Convert.ToString(reader["AnuladaPor"]),
                MotivoAnulacion = reader["MotivoAnulacion"] == DBNull.Value ? null : Convert.ToString(reader["MotivoAnulacion"])
            };
        }

        private static NominaDetalle MapDetalle(SQLiteDataReader reader)
        {
            return new NominaDetalle
            {
                IdDetalle = Convert.ToInt32(reader["IdDetalle"]),
                IdNomina = Convert.ToInt32(reader["IdNomina"]),
                IdEmpleado = Convert.ToInt32(reader["IdEmpleado"]),
                CodigoEmpleado = Convert.ToString(reader["Codigo"]),
                EmpleadoNombre = Convert.ToString(reader["EmpleadoNombre"]),
                CargoEmpleado = Convert.ToString(reader["CargoEmpleado"]),
                Departamento = Convert.ToString(reader["Departamento"]),
                SueldoBase = Convert.ToDecimal(reader["SueldoBase"]),
                Bonos = Convert.ToDecimal(reader["Bonos"]),
                HorasExtra = Convert.ToDecimal(reader["HorasExtra"]),
                MontoHorasExtra = Convert.ToDecimal(reader["MontoHorasExtra"]),
                TotalIngresos = Convert.ToDecimal(reader["TotalIngresos"]),
                TotalDeducciones = Convert.ToDecimal(reader["TotalDeducciones"]),
                NetoPagar = Convert.ToDecimal(reader["NetoPagar"])
            };
        }
    }

    public class NominaVersionRepository
    {
        public void Add(NominaVersion version)
        {
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteCommand command = new SQLiteCommand(@"INSERT INTO NominaVersiones
                (IdNominaOriginal, IdNominaNueva, MotivoCambio, UsuarioResponsable, FechaCambio)
                VALUES (@original, @nueva, @motivo, @usuario, @fecha);", connection))
            {
                command.Parameters.AddWithValue("@original", version.IdNominaOriginal);
                command.Parameters.AddWithValue("@nueva", version.IdNominaNueva.HasValue ? (object)version.IdNominaNueva.Value : DBNull.Value);
                command.Parameters.AddWithValue("@motivo", version.MotivoCambio);
                command.Parameters.AddWithValue("@usuario", version.UsuarioResponsable);
                command.Parameters.AddWithValue("@fecha", version.FechaCambio.ToString("yyyy-MM-dd HH:mm:ss"));
```

Data/Repositories.cs (continuacion)

```
                command.ExecuteNonQuery();
            }
        }

        public List<NominaVersion> GetByIdNominaOriginal(int idNominaOriginal)
        {
            List<NominaVersion> items = new List<NominaVersion>();
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteCommand command = new SQLiteCommand(@"SELECT * FROM NominaVersiones
                WHERE IdNominaOriginal = @id ORDER BY FechaCambio DESC;", connection))
            {
                command.Parameters.AddWithValue("@id", idNominaOriginal);
                using (SQLiteDataReader reader = command.ExecuteReader())
                {
                    while (reader.Read())
                    {
                        items.Add(new NominaVersion
                        {
                            IdVersion = Convert.ToInt32(reader["IdVersion"]),
                            IdNominaOriginal = Convert.ToInt32(reader["IdNominaOriginal"]),
                            IdNominaNueva = reader["IdNominaNueva"] == DBNull.Value ? (int?)null : Convert.ToInt32(reader["IdNominaNueva"]),
                            MotivoCambio = Convert.ToString(reader["MotivoCambio"]),
                            UsuarioResponsable = Convert.ToString(reader["UsuarioResponsable"]),
                            FechaCambio = DateTime.Parse(Convert.ToString(reader["FechaCambio"]))
                        });
                    }
                }
            }
            return items;
        }

        public void UpdateIdNominaNueva(int idVersion, int idNominaNueva)
        {
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteCommand command = new SQLiteCommand(@"UPDATE NominaVersiones
                SET IdNominaNueva = @nueva WHERE IdVersion = @id;", connection))
            {
                command.Parameters.AddWithValue("@id", idVersion);
                command.Parameters.AddWithValue("@nueva", idNominaNueva);
                command.ExecuteNonQuery();
            }
        }

        public List<NominaVersion> GetHistorialChain(int idNomina)
        {
            List<NominaVersion> result = new List<NominaVersion>();
            int? currentId = idNomina;
            while (currentId.HasValue)
            {
                NominaVersion version = GetByNominaNueva(currentId.Value);
                if (version == null)
                    break;
                result.Add(version);
                currentId = version.IdNominaOriginal;
            }
            return result;
        }

        private NominaVersion GetByNominaNueva(int idNominaNueva)
        {
```

Data/Repositories.cs (continuacion)

```
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteCommand command = new SQLiteCommand(@"SELECT * FROM NominaVersiones
                WHERE IdNominaNueva = @id LIMIT 1;", connection))
            {
                command.Parameters.AddWithValue("@id", idNominaNueva);
                using (SQLiteDataReader reader = command.ExecuteReader())
                {
                    if (reader.Read())
                    {
                        return new NominaVersion
                        {
                            IdVersion = Convert.ToInt32(reader["IdVersion"]),
                            IdNominaOriginal = Convert.ToInt32(reader["IdNominaOriginal"]),
                            IdNominaNueva = reader["IdNominaNueva"] == DBNull.Value ? (int?)null : Convert.ToInt32(reader["IdNominaNueva"]),
                            MotivoCambio = Convert.ToString(reader["MotivoCambio"]),
                            UsuarioResponsable = Convert.ToString(reader["UsuarioResponsable"]),
                            FechaCambio = DateTime.Parse(Convert.ToString(reader["FechaCambio"]))
                        };
                    }
                }
            }
            return null;
        }
    }

    public class ComprobanteRepository
    {
        private const string ComprobanteSelect = @"SELECT c.*,
                   COALESCE(NULLIF(nd.CodigoEmpleadoSnapshot, ''), e.Codigo) AS Codigo,
                   COALESCE(NULLIF(nd.NombreEmpleadoSnapshot, ''), e.Nombre || ' ' || e.Apellido) AS EmpleadoNombre,
                   p.Nombre AS PeriodoNombre, nd.TotalIngresos, nd.TotalDeducciones, nd.NetoPagar
            FROM Comprobantes c
            INNER JOIN Empleados e ON e.IdEmpleado = c.IdEmpleado
            INNER JOIN Nominas n ON n.IdNomina = c.IdNomina
            INNER JOIN PeriodosNomina p ON p.IdPeriodo = n.IdPeriodo
            INNER JOIN NominaDetalle nd ON nd.IdNomina = c.IdNomina AND nd.IdEmpleado = c.IdEmpleado ";

        public void Add(Comprobante comprobante)
        {
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteCommand command = new SQLiteCommand(@"INSERT OR IGNORE INTO Comprobantes
                (IdNomina, IdEmpleado, NumeroComprobante, FechaGeneracion, RutaPdf)
                VALUES (@nomina, @empleado, @numero, @fecha, @ruta);", connection))
            {
                command.Parameters.AddWithValue("@nomina", comprobante.IdNomina);
                command.Parameters.AddWithValue("@empleado", comprobante.IdEmpleado);
                command.Parameters.AddWithValue("@numero", comprobante.NumeroComprobante);
                command.Parameters.AddWithValue("@fecha", comprobante.FechaGeneracion.ToString("yyyy-MM-dd HH:mm:ss"));
                command.Parameters.AddWithValue("@ruta", comprobante.RutaPdf ?? string.Empty);
                command.ExecuteNonQuery();
            }
        }

        public void UpdateRutaPdf(int idComprobante, string ruta)
        {
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteCommand command = new SQLiteCommand("UPDATE Comprobantes SET RutaPdf = @ruta WHERE IdComprobante = @id;", connection))
            {
                command.Parameters.AddWithValue("@ruta", ruta ?? string.Empty);
                command.Parameters.AddWithValue("@id", idComprobante);
```

Data/Repositories.cs (continuacion)

```
                command.ExecuteNonQuery();
            }
        }

        public bool UpdateRutaPdfForEmployee(int idComprobante, int employeeId, string ruta)
        {
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteCommand command = new SQLiteCommand(@"UPDATE Comprobantes SET RutaPdf = @ruta
                WHERE IdComprobante = @id AND IdEmpleado = @empleado;", connection))
            {
                command.Parameters.AddWithValue("@ruta", ruta ?? string.Empty);
                command.Parameters.AddWithValue("@id", idComprobante);
                command.Parameters.AddWithValue("@empleado", employeeId);
                return command.ExecuteNonQuery() == 1;
            }
        }

        public List<Comprobante> GetAll(string search)
        {
            List<Comprobante> items = new List<Comprobante>();
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteCommand command = new SQLiteCommand(ComprobanteSelect + @"
                WHERE (@search = '' OR c.NumeroComprobante LIKE @like OR e.Nombre LIKE @like OR e.Apellido LIKE @like OR p.Nombre LIKE @like)
                ORDER BY c.FechaGeneracion DESC;", connection))
            {
                command.Parameters.AddWithValue("@search", search ?? string.Empty);
                command.Parameters.AddWithValue("@like", "%" + (search ?? string.Empty) + "%");
                using (SQLiteDataReader reader = command.ExecuteReader())
                {
                    while (reader.Read())
                    {
                        items.Add(MapComprobante(reader));
                    }
                }
            }

            return items;
        }

        public Comprobante GetById(int id)
        {
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteCommand command = new SQLiteCommand(ComprobanteSelect +
                " WHERE c.IdComprobante = @id LIMIT 1;", connection))
            {
                command.Parameters.AddWithValue("@id", id);
                using (SQLiteDataReader reader = command.ExecuteReader())
                {
                    return reader.Read() ? MapComprobante(reader) : null;
                }
            }
        }

        public List<Comprobante> GetByEmployee(int employeeId, string search)
        {
            List<Comprobante> items = new List<Comprobante>();
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteCommand command = new SQLiteCommand(ComprobanteSelect + @"
                WHERE c.IdEmpleado = @empleado
                  AND (@search = '' OR c.NumeroComprobante LIKE @like OR p.Nombre LIKE @like)
```

Data/Repositories.cs (continuacion)

```
                ORDER BY c.FechaGeneracion DESC;", connection))
            {
                command.Parameters.AddWithValue("@empleado", employeeId);
                command.Parameters.AddWithValue("@search", search ?? string.Empty);
                command.Parameters.AddWithValue("@like", "%" + (search ?? string.Empty) + "%");
                using (SQLiteDataReader reader = command.ExecuteReader())
                {
                    while (reader.Read())
                    {
                        items.Add(MapComprobante(reader));
                    }
                }
            }

            return items;
        }

        public Comprobante GetByIdAndEmployee(int payslipId, int employeeId)
        {
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteCommand command = new SQLiteCommand(ComprobanteSelect + @"
                WHERE c.IdComprobante = @id AND c.IdEmpleado = @empleado LIMIT 1;", connection))
            {
                command.Parameters.AddWithValue("@id", payslipId);
                command.Parameters.AddWithValue("@empleado", employeeId);
                using (SQLiteDataReader reader = command.ExecuteReader())
                {
                    return reader.Read() ? MapComprobante(reader) : null;
                }
            }
        }

        private static Comprobante MapComprobante(SQLiteDataReader reader)
        {
            return new Comprobante
            {
                IdComprobante = Convert.ToInt32(reader["IdComprobante"]),
                IdNomina = Convert.ToInt32(reader["IdNomina"]),
                IdEmpleado = Convert.ToInt32(reader["IdEmpleado"]),
                NumeroComprobante = Convert.ToString(reader["NumeroComprobante"]),
                FechaGeneracion = DateTime.Parse(Convert.ToString(reader["FechaGeneracion"])),
                RutaPdf = Convert.ToString(reader["RutaPdf"]),
                CodigoEmpleado = Convert.ToString(reader["Codigo"]),
                EmpleadoNombre = Convert.ToString(reader["EmpleadoNombre"]),
                PeriodoNombre = Convert.ToString(reader["PeriodoNombre"]),
                TotalIngresos = Convert.ToDecimal(reader["TotalIngresos"]),
                TotalDeducciones = Convert.ToDecimal(reader["TotalDeducciones"]),
                NetoPagar = Convert.ToDecimal(reader["NetoPagar"])
            };
        }
    }

    public class ConfiguracionRepository
    {
        public List<ConfiguracionNomina> GetAll()
        {
            List<ConfiguracionNomina> items = new List<ConfiguracionNomina>();
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteCommand command = new SQLiteCommand("SELECT * FROM ConfiguracionNomina ORDER BY NombreParametro;", connection))
            using (SQLiteDataReader reader = command.ExecuteReader())
```

Data/Repositories.cs (continuacion)

```
            {
                while (reader.Read())
                {
                    items.Add(new ConfiguracionNomina
                    {
                        IdConfiguracion = Convert.ToInt32(reader["IdConfiguracion"]),
                        NombreParametro = Convert.ToString(reader["NombreParametro"]),
                        Valor = Convert.ToDecimal(reader["Valor"]),
                        Descripcion = Convert.ToString(reader["Descripcion"])
                    });
                }
            }

            return items;
        }

        public decimal GetValue(string name, decimal fallback)
        {
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteCommand command = new SQLiteCommand("SELECT Valor FROM ConfiguracionNomina WHERE NombreParametro = @n;", connection))
            {
                command.Parameters.AddWithValue("@n", name);
                object result = command.ExecuteScalar();
                return result == null || result == DBNull.Value ? fallback : Convert.ToDecimal(result);
            }
        }

        public void Save(string name, decimal value, string description)
        {
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteCommand command = new SQLiteCommand(@"INSERT INTO ConfiguracionNomina (NombreParametro, Valor, Descripcion)
                VALUES (@n, @v, @d)
                ON CONFLICT(NombreParametro) DO UPDATE SET Valor = excluded.Valor, Descripcion = excluded.Descripcion;", connection))
            {
                command.Parameters.AddWithValue("@n", name);
                command.Parameters.AddWithValue("@v", value);
                command.Parameters.AddWithValue("@d", description);
                command.ExecuteNonQuery();
            }
        }
    }

    public class ReporteRepository
    {
        public int Add(ReporteGenerado reporte)
        {
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteCommand command = new SQLiteCommand(@"INSERT INTO ReportesGenerados
                (NombreReporte, Tipo, GeneradoPor, FechaGeneracion, RutaArchivo)
                VALUES (@nombre, @tipo, @por, @fecha, @ruta); SELECT last_insert_rowid();", connection))
            {
                command.Parameters.AddWithValue("@nombre", reporte.NombreReporte);
                command.Parameters.AddWithValue("@tipo", reporte.Tipo);
                command.Parameters.AddWithValue("@por", reporte.GeneradoPor);
                command.Parameters.AddWithValue("@fecha", reporte.FechaGeneracion.ToString("yyyy-MM-dd HH:mm:ss"));
                command.Parameters.AddWithValue("@ruta", reporte.RutaArchivo ?? string.Empty);
                return Convert.ToInt32(command.ExecuteScalar());
            }
        }
```

Data/Repositories.cs (continuacion)

```
        public List<ReporteGenerado> GetAll()
        {
            List<ReporteGenerado> items = new List<ReporteGenerado>();
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteCommand command = new SQLiteCommand("SELECT * FROM ReportesGenerados ORDER BY FechaGeneracion DESC;", connection))
            using (SQLiteDataReader reader = command.ExecuteReader())
            {
                while (reader.Read())
                {
                    items.Add(new ReporteGenerado
                    {
                        IdReporte = Convert.ToInt32(reader["IdReporte"]),
                        NombreReporte = Convert.ToString(reader["NombreReporte"]),
                        Tipo = Convert.ToString(reader["Tipo"]),
                        GeneradoPor = Convert.ToString(reader["GeneradoPor"]),
                        FechaGeneracion = DateTime.Parse(Convert.ToString(reader["FechaGeneracion"])),
                        RutaArchivo = Convert.ToString(reader["RutaArchivo"])
                    });
                }
            }

            return items;
        }
    }
}
```

Data/SQLiteConnectionFactory.cs

```
using System;
using System.Data.SQLite;
using System.IO;

namespace SistemaGestionNomina.Data
{
    public static class SQLiteConnectionFactory
    {
        public static string DatabasePath
        {
            get
            {
                string overridePath = Environment.GetEnvironmentVariable("NOMINA_DB_PATH");
                return string.IsNullOrWhiteSpace(overridePath)
                    ? Path.Combine(GetApplicationFolder(), "nomina.db")
                    : Path.GetFullPath(overridePath);
            }
        }

        public static SQLiteConnection CreateConnection()
        {
            SQLiteConnection connection = new SQLiteConnection("Data Source=" + DatabasePath + ";Version=3;Foreign Keys=True;");
            connection.Open();
            return connection;
        }

        private static string GetApplicationFolder()
        {
            string location = typeof(SQLiteConnectionFactory).Assembly.Location;
            if (!string.IsNullOrWhiteSpace(location))
            {
                return Path.GetDirectoryName(location);
            }

            return AppDomain.CurrentDomain.BaseDirectory;
        }
    }
}
```

Helpers/ComprobantePrintRenderer.cs

```
using System.Drawing;
using SistemaGestionNomina.Models;

namespace SistemaGestionNomina.Helpers
{
    public static class ComprobantePrintRenderer
    {
        public static void Draw(Graphics graphics, Rectangle bounds, Comprobante comprobante)
        {
            // El mismo diseno se reutiliza en vista previa, impresion fisica y comprobante PDF.
            int y = bounds.Top;
            using (Font titleFont = new Font("Segoe UI", 18F, FontStyle.Bold))
            using (Font headerFont = new Font("Segoe UI", 11F, FontStyle.Bold))
            using (Font bodyFont = new Font("Segoe UI", 10F))
            using (Font totalFont = new Font("Segoe UI", 13F, FontStyle.Bold))
            using (Pen linePen = new Pen(Color.Black, 1F))
            {
                graphics.DrawString("ProyFinal_LPI_Eq01_NomiCore", titleFont, Brushes.Black, bounds.Left, y);
                y += 42;
                graphics.DrawString("Comprobante de Pago", headerFont, Brushes.Black, bounds.Left, y);
                y += 34;
                graphics.DrawLine(linePen, bounds.Left, y, bounds.Right, y);
                y += 22;
                DrawRow(graphics, bodyFont, "Documento:", comprobante.NumeroComprobante, bounds.Left, ref y);
                DrawRow(graphics, bodyFont, "Fecha emision:", comprobante.FechaGeneracion.ToString("dd/MM/yyyy HH:mm"), bounds.Left, ref y);
                DrawRow(graphics, bodyFont, "Periodo:", comprobante.PeriodoNombre, bounds.Left, ref y);
                y += 8;
                graphics.DrawLine(linePen, bounds.Left, y, bounds.Right, y);
                y += 22;
                DrawRow(graphics, bodyFont, "Empleado:", comprobante.EmpleadoNombre, bounds.Left, ref y);
                DrawRow(graphics, bodyFont, "Codigo:", comprobante.CodigoEmpleado, bounds.Left, ref y);
                y += 8;
                graphics.DrawLine(linePen, bounds.Left, y, bounds.Right, y);
                y += 22;
                DrawRow(graphics, bodyFont, "Ingresos:", "B/. " + comprobante.TotalIngresos.ToString("0.00"), bounds.Left, ref y);
                DrawRow(graphics, bodyFont, "Deducciones:", "B/. " + comprobante.TotalDeducciones.ToString("0.00"), bounds.Left, ref y);
                y += 12;
                graphics.DrawString("NETO A PAGAR: B/. " + comprobante.NetoPagar.ToString("0.00"), totalFont, Brushes.Black, bounds.Left, y);
                y += 54;
                graphics.DrawLine(linePen, bounds.Left, y, bounds.Left + 300, y);
                graphics.DrawString("Firma autorizada", bodyFont, Brushes.Black, bounds.Left, y + 8);
            }
        }

        private static void DrawRow(Graphics graphics, Font font, string label, string value, int x, ref int y)
        {
            graphics.DrawString(label, font, Brushes.Black, x, y);
            graphics.DrawString(value ?? string.Empty, font, Brushes.Black, x + 140, y);
            y += 26;
        }
    }
}
```

Helpers/ControlStyleHelper.cs

```
using System;
using System.Drawing;
using System.Drawing.Drawing2D;
using System.Windows.Forms;
using FontAwesome.Sharp;
using MaterialSkin;

namespace SistemaGestionNomina.Helpers
{
    public static class ControlStyleHelper
    {
        private const int PanelRadius = 12;
        private const int ButtonRadius = 8;
        private static bool materialThemeApplied;

        public static void ApplyModernForm(Form form)
        {
            TryApplyMaterialTheme();
            form.BackColor = ThemeHelper.Background;
            form.ForeColor = ThemeHelper.Text;
            form.Font = ThemeHelper.BodyFont(9F, FontStyle.Regular);
            StyleContainer(form);
        }

        public static void StyleContainer(Control parent)
        {
            foreach (Control control in parent.Controls)
            {
                if (control is DataGridView)
                {
                    StyleGrid((DataGridView)control);
                }
                else if (control is IconButton)
                {
                    StyleIconButton((IconButton)control, false);
                }
                else if (control is Button)
                {
                    StyleButton((Button)control, IsPrimaryButton((Button)control));
                }
                else if (control is TextBox)
                {
                    StyleTextBox((TextBox)control);
                }
                else if (control is ComboBox)
                {
                    StyleComboBox((ComboBox)control);
                }
                else if (control is NumericUpDown)
                {
                    StyleNumeric((NumericUpDown)control);
                }
                else if (control is DateTimePicker)
                {
                    StyleDatePicker((DateTimePicker)control);
                }
                else if (control is CheckBox)
                {
                    StyleCheckBox((CheckBox)control);
                }
```

Helpers/ControlStyleHelper.cs (continuacion)

```
                else if (control is Panel)
                {
                    StylePanel((Panel)control);
                }
                else if (control is Label)
                {
                    StyleLabel((Label)control);
                }

                if (control.HasChildren)
                {
                    StyleContainer(control);
                }
            }
        }

        public static void StyleGrid(DataGridView grid)
        {
            grid.BackgroundColor = ThemeHelper.CardDeep;
            grid.BorderStyle = BorderStyle.None;
            grid.EnableHeadersVisualStyles = false;
            grid.RowHeadersVisible = false;
            grid.AllowUserToAddRows = false;
            grid.AllowUserToDeleteRows = false;
            grid.ReadOnly = true;
            grid.SelectionMode = DataGridViewSelectionMode.FullRowSelect;
            grid.MultiSelect = false;
            grid.AutoSizeColumnsMode = DataGridViewAutoSizeColumnsMode.Fill;
            grid.GridColor = ThemeHelper.BorderSoft;
            grid.CellBorderStyle = DataGridViewCellBorderStyle.SingleHorizontal;
            grid.ColumnHeadersBorderStyle = DataGridViewHeaderBorderStyle.None;
            grid.ColumnHeadersDefaultCellStyle.BackColor = ThemeHelper.CardAlt;
            grid.ColumnHeadersDefaultCellStyle.ForeColor = ThemeHelper.Text;
            grid.ColumnHeadersDefaultCellStyle.SelectionBackColor = ThemeHelper.Hover;
            grid.ColumnHeadersDefaultCellStyle.Font = ThemeHelper.BodyFont(9F, FontStyle.Bold);
            grid.ColumnHeadersHeight = 40;
            grid.DefaultCellStyle.BackColor = ThemeHelper.CardDeep;
            grid.DefaultCellStyle.ForeColor = ThemeHelper.Text;
            grid.DefaultCellStyle.SelectionBackColor = ThemeHelper.Hover;
            grid.DefaultCellStyle.SelectionForeColor = Color.White;
            grid.DefaultCellStyle.Font = ThemeHelper.BodyFont(9.25F, FontStyle.Regular);
            grid.AlternatingRowsDefaultCellStyle.BackColor = Color.FromArgb(20, 23, 34);
            grid.AlternatingRowsDefaultCellStyle.ForeColor = ThemeHelper.Text;
            grid.RowTemplate.Height = 36;
        }

        public static void StyleSidebarButton(IconButton button, bool active)
        {
            StyleIconButton(button, active);
            button.TextAlign = ContentAlignment.MiddleLeft;
            button.ImageAlign = ContentAlignment.MiddleLeft;
            button.TextImageRelation = TextImageRelation.ImageBeforeText;
            button.Padding = new Padding(16, 0, 0, 0);
            button.IconSize = 20;
        }

        public static void SetActiveSidebarButton(Control sidebar, IconButton activeButton)
        {
            StyleSidebarButtons(sidebar, activeButton);
        }
```

Helpers/ControlStyleHelper.cs (continuacion)

```

        private static void StyleSidebarButtons(Control parent, IconButton activeButton)
        {
            foreach (Control control in parent.Controls)
            {
                IconButton button = control as IconButton;
                if (button != null)
                {
                    StyleSidebarButton(button, button == activeButton);
                }

                if (control.HasChildren)
                {
                    StyleSidebarButtons(control, activeButton);
                }
            }
        }

        private static void StyleButton(Button button, bool primary)
        {
            button.Cursor = Cursors.Hand;
            button.FlatStyle = FlatStyle.Flat;
            button.FlatAppearance.BorderSize = 1;
            button.FlatAppearance.BorderColor = primary ? ThemeHelper.Accent : ThemeHelper.Border;
            button.BackColor = primary ? ThemeHelper.AccentAlt : ThemeHelper.CardDeep;
            button.ForeColor = ThemeHelper.Text;
            button.Font = ThemeHelper.BodyFont(9F, FontStyle.Bold);
            button.UseVisualStyleBackColor = false;
            button.MouseEnter -= Button_MouseEnter;
            button.MouseLeave -= Button_MouseLeave;
            button.MouseEnter += Button_MouseEnter;
            button.MouseLeave += Button_MouseLeave;
            button.Resize -= Button_Resize;
            button.Resize += Button_Resize;
            button.Tag = primary ? "primary" : "secondary";
            ApplyRoundedRegion(button, ButtonRadius);
        }

        private static void StyleIconButton(IconButton button, bool active)
        {
            button.Cursor = Cursors.Hand;
            button.FlatStyle = FlatStyle.Flat;
            button.FlatAppearance.BorderSize = 0;
            button.BackColor = active ? ThemeHelper.AccentSoft : ThemeHelper.Sidebar;
            button.ForeColor = active ? ThemeHelper.Text : ThemeHelper.TextMuted;
            button.IconColor = active ? ThemeHelper.Accent : ThemeHelper.TextMuted;
            button.Font = ThemeHelper.BodyFont(10F, FontStyle.Bold);
            button.UseVisualStyleBackColor = false;
            button.MouseEnter -= IconButton_MouseEnter;
            button.MouseLeave -= IconButton_MouseLeave;
            button.MouseEnter += IconButton_MouseEnter;
            button.MouseLeave += IconButton_MouseLeave;
            button.Tag = active ? "active" : "inactive";
            ApplyRoundedRegion(button, ButtonRadius);
        }

        private static void StyleTextBox(TextBox textBox)
        {
            textBox.BackColor = ThemeHelper.Input;
            textBox.ForeColor = ThemeHelper.Text;
```

Helpers/ControlStyleHelper.cs (continuacion)

```
            textBox.BorderStyle = BorderStyle.FixedSingle;
            textBox.Font = ThemeHelper.BodyFont(9.5F, FontStyle.Regular);
        }

        private static void StyleComboBox(ComboBox comboBox)
        {
            comboBox.BackColor = ThemeHelper.Input;
            comboBox.ForeColor = ThemeHelper.Text;
            comboBox.FlatStyle = FlatStyle.Flat;
            comboBox.Font = ThemeHelper.BodyFont(9.5F, FontStyle.Regular);
        }

        private static void StyleNumeric(NumericUpDown numeric)
        {
            numeric.BackColor = ThemeHelper.Input;
            numeric.ForeColor = ThemeHelper.Text;
            numeric.BorderStyle = BorderStyle.FixedSingle;
            numeric.Font = ThemeHelper.BodyFont(9.5F, FontStyle.Regular);
        }

        private static void StyleDatePicker(DateTimePicker picker)
        {
            picker.CalendarForeColor = ThemeHelper.Text;
            picker.CalendarMonthBackground = ThemeHelper.Input;
            picker.CalendarTitleBackColor = ThemeHelper.CardAlt;
            picker.CalendarTitleForeColor = ThemeHelper.Text;
            picker.CalendarTrailingForeColor = ThemeHelper.TextMuted;
            picker.Font = ThemeHelper.BodyFont(9.5F, FontStyle.Regular);
        }

        private static void StyleCheckBox(CheckBox checkBox)
        {
            checkBox.BackColor = Color.Transparent;
            checkBox.ForeColor = ThemeHelper.TextMuted;
            checkBox.Font = ThemeHelper.BodyFont(9F, FontStyle.Regular);
            checkBox.Cursor = Cursors.Hand;
            checkBox.UseVisualStyleBackColor = false;
        }

        private static void StylePanel(Panel panel)
        {
            if (IsLayoutPanel(panel))
            {
                panel.ForeColor = ThemeHelper.Text;
                return;
            }

            if (panel.BackColor == Color.Black || panel.BackColor == ThemeHelper.Card || panel.BackColor == ThemeHelper.CardDeep)
            {
                panel.BackColor = ThemeHelper.Card;
            }
            else if (panel.BackColor == ThemeHelper.CardAlt)
            {
                panel.BackColor = ThemeHelper.CardAlt;
            }

            panel.ForeColor = ThemeHelper.Text;
            panel.Paint -= Panel_Paint;
            panel.Resize -= Panel_Resize;
            panel.Paint += Panel_Paint;
```

Helpers/ControlStyleHelper.cs (continuacion)

```
            panel.Resize += Panel_Resize;
            ApplyRoundedRegion(panel, PanelRadius);
        }

        private static void StyleLabel(Label label)
        {
            string name = label.Name.ToLowerInvariant();
            if (label.ForeColor == SystemColors.ControlText)
            {
                label.ForeColor = ThemeHelper.Text;
            }

            if (name.Contains("ingreso"))
            {
                label.ForeColor = ThemeHelper.Success;
            }
            else if (name.Contains("deduccion") || name.Contains("deducciones") || name.Contains("alerta"))
            {
                label.ForeColor = ThemeHelper.Error;
            }
            else if (name.Contains("totalneto") || name.Contains("neto"))
            {
                label.ForeColor = ThemeHelper.Accent;
            }

            label.BackColor = Color.Transparent;
        }

        private static bool IsPrimaryButton(Button button)
        {
            string name = button.Name.ToLowerInvariant();
            return name.Contains("guardar") ||
                   name.Contains("nuevo") ||
                   name.Contains("editar") ||
                   name.Contains("calcular") ||
                   name.Contains("confirmar") ||
                   name.Contains("ingresar") ||
                   name.Contains("registrar") ||
                   name.Contains("procesar") ||
                   name.Contains("imprimir") ||
                   name.Contains("email");
        }

        private static void Button_MouseEnter(object sender, EventArgs e)
        {
            Button button = (Button)sender;
            button.BackColor = ThemeHelper.Hover;
            button.ForeColor = Color.White;
        }

        private static void Button_MouseLeave(object sender, EventArgs e)
        {
            Button button = (Button)sender;
            bool primary = Convert.ToString(button.Tag) == "primary";
            button.BackColor = primary ? ThemeHelper.AccentAlt : ThemeHelper.CardDeep;
            button.ForeColor = ThemeHelper.Text;
        }

        private static void IconButton_MouseEnter(object sender, EventArgs e)
        {
```

Helpers/ControlStyleHelper.cs (continuacion)

```
            IconButton button = (IconButton)sender;
            button.BackColor = ThemeHelper.AccentSoft;
            button.ForeColor = ThemeHelper.Text;
            button.IconColor = ThemeHelper.Accent;
        }

        private static void IconButton_MouseLeave(object sender, EventArgs e)
        {
            IconButton button = (IconButton)sender;
            bool active = Convert.ToString(button.Tag) == "active";
            button.BackColor = active ? ThemeHelper.AccentSoft : ThemeHelper.Sidebar;
            button.ForeColor = active ? ThemeHelper.Text : ThemeHelper.TextMuted;
            button.IconColor = active ? ThemeHelper.Accent : ThemeHelper.TextMuted;
        }

        private static void Panel_Paint(object sender, PaintEventArgs e)
        {
            Panel panel = (Panel)sender;
            if (panel.Width < 2 || panel.Height < 2)
            {
                return;
            }

            e.Graphics.SmoothingMode = SmoothingMode.AntiAlias;
            using (GraphicsPath path = CreateRoundPath(new Rectangle(0, 0, panel.Width - 1, panel.Height - 1), PanelRadius))
            using (Pen pen = new Pen(ThemeHelper.Border, 1))
            {
                e.Graphics.DrawPath(pen, path);
            }
        }

        private static void Panel_Resize(object sender, EventArgs e)
        {
            ApplyRoundedRegion((Control)sender, PanelRadius);
        }

        private static void Button_Resize(object sender, EventArgs e)
        {
            ApplyRoundedRegion((Control)sender, ButtonRadius);
        }

        private static void ApplyRoundedRegion(Control control, int radius)
        {
            if (control.Width <= 0 || control.Height <= 0)
            {
                return;
            }

            using (GraphicsPath path = CreateRoundPath(new Rectangle(0, 0, control.Width, control.Height), radius))
            {
                control.Region = new Region(path);
            }
        }

        private static GraphicsPath CreateRoundPath(Rectangle bounds, int radius)
        {
            int diameter = radius * 2;
            Rectangle rect = new Rectangle(bounds.X, bounds.Y, bounds.Width - 1, bounds.Height - 1);
            GraphicsPath path = new GraphicsPath();
            path.AddArc(rect.X, rect.Y, diameter, diameter, 180, 90);
```

Helpers/ControlStyleHelper.cs (continuacion)

```
            path.AddArc(rect.Right - diameter, rect.Y, diameter, diameter, 270, 90);
            path.AddArc(rect.Right - diameter, rect.Bottom - diameter, diameter, diameter, 0, 90);
            path.AddArc(rect.X, rect.Bottom - diameter, diameter, diameter, 90, 90);
            path.CloseFigure();
            return path;
        }

        private static bool IsLayoutPanel(Panel panel)
        {
            string name = panel.Name.ToLowerInvariant();
            return name == "panelsidebar" ||
                   name == "panelcontent" ||
                   name == "paneltopbar" ||
                   name == "panelbrand" ||
                   name == "flowmenu" ||
                   name == "flowaccount";
        }

        private static void TryApplyMaterialTheme()
        {
            if (materialThemeApplied)
            {
                return;
            }

            try
            {
                MaterialSkinManager manager = MaterialSkinManager.Instance;
                manager.Theme = MaterialSkinManager.Themes.DARK;
                manager.ColorScheme = new ColorScheme(
                    Primary.DeepPurple400,
                    Primary.DeepPurple700,
                    Primary.DeepPurple200,
                    Accent.DeepPurple200,
                    TextShade.WHITE);
                materialThemeApplied = true;
            }
            catch
            {
                // MaterialSkin is optional here; standard WinForms styling keeps the designer stable.
            }
        }
    }
}
```

Helpers/PasswordHelper.cs

```
using System;
using System.Security.Cryptography;
using System.Text;

namespace SistemaGestionNomina.Helpers
{
    public static class PasswordHelper
    {
        private const string Prefix = "pbkdf2-sha256$v1";
        private const int Iterations = 120000;
        private const int SaltSize = 16;
        private const int HashSize = 32;

        public static string HashPassword(string password)
        {
            if (string.IsNullOrEmpty(password))
            {
                throw new ArgumentException("La contraseña es obligatoria.", "password");
            }

            byte[] salt = new byte[SaltSize];
            using (RandomNumberGenerator random = RandomNumberGenerator.Create())
            {
                random.GetBytes(salt);
            }

            byte[] hash = Derive(password, salt, Iterations, HashSize);
            return Prefix + "$" + Iterations + "$" + Convert.ToBase64String(salt) + "$" + Convert.ToBase64String(hash);
        }

        public static bool IsLegacyHash(string storedHash)
        {
            if (string.IsNullOrWhiteSpace(storedHash) || storedHash.Length != 64)
            {
                return false;
            }

            for (int i = 0; i < storedHash.Length; i++)
            {
                if (!Uri.IsHexDigit(storedHash[i]))
                {
                    return false;
                }
            }

            return true;
        }

        public static bool Verify(string password, string hash)
        {
            if (string.IsNullOrEmpty(password) || string.IsNullOrWhiteSpace(hash))
            {
                return false;
            }

            if (IsLegacyHash(hash))
            {
                return FixedTimeEquals(ComputeLegacyHash(password), hash.ToLowerInvariant());
            }

```

Helpers/PasswordHelper.cs (continuacion)

```
            string[] parts = hash.Split('$');
            if (parts.Length != 5 || !string.Equals(parts[0] + "$" + parts[1], Prefix, StringComparison.Ordinal))
            {
                return false;
            }

            int iterations;
            byte[] salt;
            byte[] expected;
            if (!int.TryParse(parts[2], out iterations) || iterations < 10000 || iterations > 1000000)
            {
                return false;
            }

            try
            {
                salt = Convert.FromBase64String(parts[3]);
                expected = Convert.FromBase64String(parts[4]);
            }
            catch (FormatException)
            {
                return false;
            }

            if (salt.Length < SaltSize || expected.Length != HashSize)
            {
                return false;
            }

            byte[] actual = Derive(password, salt, iterations, expected.Length);
            return FixedTimeEquals(actual, expected);
        }

        private static byte[] Derive(string password, byte[] salt, int iterations, int length)
        {
            using (Rfc2898DeriveBytes derive = new Rfc2898DeriveBytes(
                password, salt, iterations, HashAlgorithmName.SHA256))
            {
                return derive.GetBytes(length);
            }
        }

        private static string ComputeLegacyHash(string password)
        {
            using (SHA256 sha = SHA256.Create())
            {
                byte[] bytes = sha.ComputeHash(Encoding.UTF8.GetBytes(password));
                StringBuilder builder = new StringBuilder(bytes.Length * 2);
                for (int i = 0; i < bytes.Length; i++)
                {
                    builder.Append(bytes[i].ToString("x2"));
                }

                return builder.ToString();
            }
        }

        private static bool FixedTimeEquals(string left, string right)
        {
            return FixedTimeEquals(Encoding.UTF8.GetBytes(left ?? string.Empty), Encoding.UTF8.GetBytes(right ?? string.Empty));
```

Helpers/PasswordHelper.cs (continuacion)

```
        }

        private static bool FixedTimeEquals(byte[] left, byte[] right)
        {
            if (left == null || right == null || left.Length != right.Length)
            {
                return false;
            }

            int difference = 0;
            for (int i = 0; i < left.Length; i++)
            {
                difference |= left[i] ^ right[i];
            }

            return difference == 0;
        }
    }
}
```

Helpers/PathHelper.cs

```
using System;
using System.IO;
using System.Windows.Forms;

namespace SistemaGestionNomina.Helpers
{
    public static class PathHelper
    {
        public static string Timestamp()
        {
            return DateTime.Now.ToString("yyyy_MM_dd_HHmm");
        }

        public static string RequestExportPath(string prefix, string extension, string filter)
        {
            string cleanExtension = NormalizeExtension(extension);
            using (SaveFileDialog dialog = new SaveFileDialog())
            {
                dialog.Title = "Guardar exportación";
                dialog.FileName = BuildExportFileName(prefix, cleanExtension);
                dialog.Filter = filter + "|Todos los archivos (*.*)|*.*";
                dialog.DefaultExt = cleanExtension.TrimStart('.');
                dialog.AddExtension = true;
                dialog.OverwritePrompt = true;
                dialog.InitialDirectory = GetSafeInitialDirectory();

                return dialog.ShowDialog() == DialogResult.OK ? dialog.FileName : string.Empty;
            }
        }

        public static string BuildUserDocumentPath(string prefix, string extension)
        {
            string folder = Environment.GetFolderPath(Environment.SpecialFolder.MyDocuments);
            if (string.IsNullOrWhiteSpace(folder))
            {
                folder = Path.GetTempPath();
            }

            Directory.CreateDirectory(folder);
            return Path.Combine(folder, BuildExportFileName(prefix, NormalizeExtension(extension)));
        }

        private static string BuildExportFileName(string prefix, string extension)
        {
            return SanitizeFileName(prefix) + "_" + Timestamp() + extension;
        }

        private static string NormalizeExtension(string extension)
        {
            if (string.IsNullOrWhiteSpace(extension))
            {
                return ".dat";
            }

            return extension.StartsWith(".") ? extension : "." + extension;
        }

        private static string GetSafeInitialDirectory()
        {
            string documents = Environment.GetFolderPath(Environment.SpecialFolder.MyDocuments);
```

Helpers/PathHelper.cs (continuacion)

```
            return Directory.Exists(documents) ? documents : Environment.GetFolderPath(Environment.SpecialFolder.DesktopDirectory);
        }

        private static string SanitizeFileName(string value)
        {
            string safe = string.IsNullOrWhiteSpace(value) ? "Exportacion" : value.Trim();
            foreach (char c in Path.GetInvalidFileNameChars())
            {
                safe = safe.Replace(c, '_');
            }

            return safe.Replace(' ', '_');
        }
    }
}
```

Helpers/RememberedUsernameHelper.cs

```
using System;
using System.IO;

namespace SistemaGestionNomina.Helpers
{
    public static class RememberedUsernameHelper
    {
        private static string FilePath
        {
            get
            {
                return Path.Combine(Environment.GetFolderPath(Environment.SpecialFolder.LocalApplicationData),
                    "ProyFinal_LPI_Eq01_NomiCore", "usuario.recordado");
            }
        }

        public static string Load()
        {
            try
            {
                return File.Exists(FilePath) ? File.ReadAllText(FilePath).Trim() : string.Empty;
            }
            catch
            {
                return string.Empty;
            }
        }

        public static void Save(string username)
        {
            string folder = Path.GetDirectoryName(FilePath);
            Directory.CreateDirectory(folder);
            File.WriteAllText(FilePath, (username ?? string.Empty).Trim());
        }

        public static void Clear()
        {
            if (File.Exists(FilePath)) File.Delete(FilePath);
        }
    }
}
```

Helpers/ThemeHelper.cs

```
using System.Drawing;
using System.Windows.Forms;

namespace SistemaGestionNomina.Helpers
{
    public static class ThemeHelper
    {
        public static readonly Color Background = Color.FromArgb(13, 14, 20);
        public static readonly Color BackgroundAlt = Color.FromArgb(17, 24, 39);
        public static readonly Color Sidebar = Color.FromArgb(11, 15, 25);
        public static readonly Color SidebarAlt = Color.FromArgb(8, 10, 17);
        public static readonly Color Card = Color.FromArgb(24, 25, 34);
        public static readonly Color CardAlt = Color.FromArgb(31, 41, 55);
        public static readonly Color CardDeep = Color.FromArgb(17, 18, 26);
        public static readonly Color Input = Color.FromArgb(20, 23, 35);
        public static readonly Color Border = Color.FromArgb(45, 51, 64);
        public static readonly Color BorderSoft = Color.FromArgb(38, 43, 55);
        public static readonly Color Text = Color.FromArgb(243, 244, 246);
        public static readonly Color TextMuted = Color.FromArgb(161, 161, 170);
        public static readonly Color Accent = Color.FromArgb(139, 92, 246);
        public static readonly Color AccentAlt = Color.FromArgb(124, 58, 237);
        public static readonly Color AccentSoft = Color.FromArgb(46, 32, 84);
        public static readonly Color Hover = Color.FromArgb(91, 62, 168);
        public static readonly Color Success = Color.FromArgb(34, 197, 94);
        public static readonly Color Warning = Color.FromArgb(251, 191, 36);
        public static readonly Color Error = Color.FromArgb(239, 68, 68);
        public static readonly Color Info = Color.FromArgb(59, 130, 246);

        public static Font TitleFont(float size)
        {
            return new Font("Segoe UI", size, FontStyle.Bold, GraphicsUnit.Point);
        }

        public static Font BodyFont(float size, FontStyle style)
        {
            return new Font("Segoe UI", size, style, GraphicsUnit.Point);
        }

        public static void ApplyForm(Form form)
        {
            form.BackColor = Background;
            form.ForeColor = Text;
            form.Font = BodyFont(10F, FontStyle.Regular);
        }

        public static Label Label(string text, int x, int y, int width, int height, float size, FontStyle style, Color color)
        {
            return new Label
            {
                Text = text,
                Location = new Point(x, y),
                Size = new Size(width, height),
                Font = BodyFont(size, style),
                ForeColor = color,
                BackColor = Color.Transparent
            };
        }

        public static TextBox TextBox(string name, int x, int y, int width)
        {
```

Helpers/ThemeHelper.cs (continuacion)

```
            TextBox textBox = new TextBox();
            textBox.Name = name;
            textBox.Location = new Point(x, y);
            textBox.Size = new Size(width, 30);
            textBox.BorderStyle = BorderStyle.FixedSingle;
            textBox.BackColor = CardAlt;
            textBox.ForeColor = Text;
            textBox.Font = BodyFont(10F, FontStyle.Regular);
            return textBox;
        }

        public static ComboBox ComboBox(string name, int x, int y, int width)
        {
            ComboBox combo = new ComboBox();
            combo.Name = name;
            combo.Location = new Point(x, y);
            combo.Size = new Size(width, 30);
            combo.DropDownStyle = ComboBoxStyle.DropDownList;
            combo.BackColor = CardAlt;
            combo.ForeColor = Text;
            combo.FlatStyle = FlatStyle.Flat;
            combo.Font = BodyFont(10F, FontStyle.Regular);
            return combo;
        }

        public static void ApplyGrid(DataGridView grid)
        {
            grid.BackgroundColor = Color.Black;
            grid.BorderStyle = BorderStyle.None;
            grid.EnableHeadersVisualStyles = false;
            grid.AllowUserToAddRows = false;
            grid.AllowUserToDeleteRows = false;
            grid.ReadOnly = true;
            grid.SelectionMode = DataGridViewSelectionMode.FullRowSelect;
            grid.MultiSelect = false;
            grid.RowHeadersVisible = false;
            grid.AutoSizeColumnsMode = DataGridViewAutoSizeColumnsMode.Fill;
            grid.GridColor = Border;
            grid.ColumnHeadersDefaultCellStyle.BackColor = CardAlt;
            grid.ColumnHeadersDefaultCellStyle.ForeColor = TextMuted;
            grid.ColumnHeadersDefaultCellStyle.Font = BodyFont(9F, FontStyle.Bold);
            grid.ColumnHeadersHeight = 38;
            grid.DefaultCellStyle.BackColor = Color.Black;
            grid.DefaultCellStyle.ForeColor = Text;
            grid.DefaultCellStyle.SelectionBackColor = Color.FromArgb(36, 31, 52);
            grid.DefaultCellStyle.SelectionForeColor = Text;
            grid.DefaultCellStyle.Font = BodyFont(9.5F, FontStyle.Regular);
            grid.RowTemplate.Height = 36;
        }
    }
}
```

Helpers/ValidationHelper.cs

```
using System;
using System.Windows.Forms;

namespace SistemaGestionNomina.Helpers
{
    public static class ValidationHelper
    {
        public static bool RequireText(TextBox textBox, string fieldName)
        {
            if (textBox == null || !string.IsNullOrWhiteSpace(textBox.Text))
            {
                return true;
            }

            MessageBox.Show(fieldName + " es obligatorio.", "Validación", MessageBoxButtons.OK, MessageBoxIcon.Warning);
            if (textBox != null)
            {
                textBox.Focus();
            }

            return false;
        }

        public static bool TryPositiveDecimal(TextBox textBox, string fieldName, out decimal value)
        {
            value = 0;
            if (textBox != null && decimal.TryParse(textBox.Text, out value) && value > 0)
            {
                return true;
            }

            MessageBox.Show(fieldName + " debe ser mayor que cero.", "Validación", MessageBoxButtons.OK, MessageBoxIcon.Warning);
            if (textBox != null)
            {
                textBox.Focus();
            }

            return false;
        }

        public static bool ValidateDateRange(DateTime start, DateTime end)
        {
            if (start.Date <= end.Date)
            {
                return true;
            }

            MessageBox.Show("La fecha de inicio debe ser menor o igual que la fecha fin.",
                "Validación", MessageBoxButtons.OK, MessageBoxIcon.Warning);
            return false;
        }

        public static bool ValidateHourRange(TimeSpan entry, TimeSpan exit)
        {
            if (exit > entry)
            {
                return true;
            }

            MessageBox.Show("La hora de salida debe ser mayor que la hora de entrada.",
```

Helpers/ValidationHelper.cs (continuacion)

```
                "Validación", MessageBoxButtons.OK, MessageBoxIcon.Warning);
            return false;
        }
    }
}
```

Models/AbsenceModels.cs

```
using System;
using System.Collections.Generic;

namespace SistemaGestionNomina.Models
{
    public static class AbsenceTypes
    {
        public const string Vacation = "Vacaciones";
        public const string PaidLeave = "Permiso remunerado";
        public const string UnpaidLeave = "Permiso no remunerado";
        public const string MedicalLeave = "Incapacidad médica";
        public const string License = "Licencia";
        public const string JustifiedAbsence = "Ausencia justificada";
        public const string UnjustifiedAbsence = "Ausencia injustificada";
        public const string Suspension = "Suspensión";

        public static readonly string[] All =
        {
            Vacation, PaidLeave, UnpaidLeave, MedicalLeave, License,
            JustifiedAbsence, UnjustifiedAbsence, Suspension
        };

        public static bool IsValid(string value)
        {
            for (int i = 0; i < All.Length; i++)
            {
                if (string.Equals(All[i], value, StringComparison.OrdinalIgnoreCase)) return true;
            }
            return false;
        }

        public static string ToAttendanceStatus(string value)
        {
            if (string.Equals(value, UnpaidLeave, StringComparison.OrdinalIgnoreCase) ||
                string.Equals(value, UnjustifiedAbsence, StringComparison.OrdinalIgnoreCase) ||
                string.Equals(value, Suspension, StringComparison.OrdinalIgnoreCase))
                return "Falta";
            return "Permiso";
        }
    }

    public static class AbsenceStates
    {
        public const string Pending = "Pendiente";
        public const string Approved = "Aprobada";
        public const string Rejected = "Rechazada";
        public const string Cancelled = "Cancelada";

        public static readonly string[] All = { Pending, Approved, Rejected, Cancelled };

        public static bool IsValid(string value)
        {
            for (int i = 0; i < All.Length; i++)
            {
                if (string.Equals(All[i], value, StringComparison.OrdinalIgnoreCase)) return true;
            }
            return false;
        }
    }
```

Models/AbsenceModels.cs (continuacion)

```
    public static class AbsenceStateMachine
    {
        public static bool CanTransition(string currentState, string targetState)
        {
            if (string.Equals(currentState, AbsenceStates.Pending, StringComparison.OrdinalIgnoreCase))
            {
                return string.Equals(targetState, AbsenceStates.Approved, StringComparison.OrdinalIgnoreCase) ||
                    string.Equals(targetState, AbsenceStates.Rejected, StringComparison.OrdinalIgnoreCase) ||
                    string.Equals(targetState, AbsenceStates.Cancelled, StringComparison.OrdinalIgnoreCase);
            }
            return string.Equals(currentState, AbsenceStates.Approved, StringComparison.OrdinalIgnoreCase) &&
                string.Equals(targetState, AbsenceStates.Cancelled, StringComparison.OrdinalIgnoreCase);
        }

        public static void DemandTransition(string currentState, string targetState)
        {
            if (!CanTransition(currentState, targetState))
                throw new InvalidOperationException("La solicitud no puede cambiar de " + currentState + " a " + targetState + ".");
        }
    }

    public sealed class SolicitudAusencia
    {
        public int IdSolicitud { get; set; }
        public int IdEmpleado { get; set; }
        public string CodigoEmpleado { get; set; }
        public string EmpleadoNombre { get; set; }
        public string DepartamentoNombre { get; set; }
        public string Tipo { get; set; }
        public DateTime FechaInicio { get; set; }
        public DateTime FechaFin { get; set; }
        public string Estado { get; set; }
        public string Motivo { get; set; }
        public string AprobadoPor { get; set; }
        public DateTime? FechaAprobacion { get; set; }
        public string ObservacionResolucion { get; set; }
        public string UsuarioSolicitante { get; set; }
        public DateTime FechaCreacion { get; set; }
    }

    public sealed class AbsenceQuery
    {
        public int? EmployeeId { get; set; }
        public int? DepartmentId { get; set; }
        public string Type { get; set; }
        public string Status { get; set; }
        public DateTime? From { get; set; }
        public DateTime? To { get; set; }
        public string Search { get; set; }
    }

    public sealed class OwnAbsenceRequest
    {
        public string Type { get; set; }
        public DateTime StartDate { get; set; }
        public DateTime EndDate { get; set; }
        public string Reason { get; set; }
    }
}
```

Models/AuditModels.cs

```
using System;

namespace SistemaGestionNomina.Models
{
    public sealed class AuditRecord
    {
        public int IdAuditoria { get; set; }
        public string Usuario { get; set; }
        public string Modulo { get; set; }
        public string Accion { get; set; }
        public string Detalle { get; set; }
        public DateTime Fecha { get; set; }
    }

    public sealed class AuditQuery
    {
        public string Usuario { get; set; }
        public string Modulo { get; set; }
        public string Accion { get; set; }
        public string Detalle { get; set; }
        public DateTime? FechaDesde { get; set; }
        public DateTime? FechaHasta { get; set; }
        public int Offset { get; set; }
        public int Limit { get; set; }

        public AuditQuery()
        {
            Limit = 100;
        }
    }
}
```

Models/AuthenticationModels.cs

```
using System;

namespace SistemaGestionNomina.Models
{
    public enum AuthenticationStatus
    {
        Success,
        InvalidCredentials,
        TemporarilyBlocked
    }

    public sealed class AuthenticationResult
    {
        public AuthenticationStatus Status { get; private set; }
        public Usuario User { get; private set; }
        public string Message { get; private set; }
        public DateTime? BlockedUntil { get; private set; }

        public bool IsSuccess
        {
            get { return Status == AuthenticationStatus.Success && User != null; }
        }

        public static AuthenticationResult Successful(Usuario user)
        {
            return new AuthenticationResult
            {
                Status = AuthenticationStatus.Success,
                User = user,
                Message = string.Empty
            };
        }

        public static AuthenticationResult Invalid()
        {
            return new AuthenticationResult
            {
                Status = AuthenticationStatus.InvalidCredentials,
                Message = "Usuario o contraseña incorrectos."
            };
        }

        public static AuthenticationResult Blocked(DateTime blockedUntil)
        {
            return new AuthenticationResult
            {
                Status = AuthenticationStatus.TemporarilyBlocked,
                BlockedUntil = blockedUntil,
                Message = "El acceso está bloqueado temporalmente. Intente nuevamente después de " +
                    blockedUntil.ToString("HH:mm") + "."
            };
        }
    }
}
```

Models/EmployeeHistoryModels.cs

```
using System;

namespace SistemaGestionNomina.Models
{
    public static class EmployeeHistoryFields
    {
        public const string BaseSalary = "SalarioBase";
        public const string Position = "Cargo";
        public const string Department = "Departamento";
        public const string Status = "Estado";
    }

    public sealed class HistorialEmpleado
    {
        public int IdHistorial { get; set; }
        public int IdEmpleado { get; set; }
        public string CodigoEmpleado { get; set; }
        public string EmpleadoNombre { get; set; }
        public string CampoModificado { get; set; }
        public string ValorAnterior { get; set; }
        public string ValorNuevo { get; set; }
        public string ValorAnteriorTecnico { get; set; }
        public string ValorNuevoTecnico { get; set; }
        public DateTime FechaCambio { get; set; }
        public DateTime FechaEfectiva { get; set; }
        public string UsuarioResponsable { get; set; }
        public string Motivo { get; set; }
        public bool Aplicado { get; set; }
        public DateTime? FechaAplicacion { get; set; }
    }

    public sealed class EmployeeChangeContext
    {
        public DateTime EffectiveDate { get; set; }
        public string Reason { get; set; }
    }

    public sealed class EmployeeSaveResult
    {
        public int EmployeeId { get; set; }
        public int HistoryRecords { get; set; }
        public bool AppliedImmediately { get; set; }
        public bool HasScheduledChanges { get; set; }
    }

    public sealed class EmployeeHistoryQuery
    {
        public int? EmployeeId { get; set; }
        public string Field { get; set; }
        public string User { get; set; }
        public DateTime? From { get; set; }
        public DateTime? To { get; set; }
    }
}
```

Models/EmployeePortalModels.cs

```
using System;
using System.Collections.Generic;

namespace SistemaGestionNomina.Models
{
    public sealed class EmployeeProfileViewModel
    {
        public int IdEmpleado { get; set; }
        public string Codigo { get; set; }
        public string NombreCompleto { get; set; }
        public string Cedula { get; set; }
        public string Cargo { get; set; }
        public string Departamento { get; set; }
        public decimal SalarioBase { get; set; }
        public string Estado { get; set; }
        public DateTime FechaIngreso { get; set; }
    }

    public sealed class EmployeeAttendanceSummaryViewModel
    {
        public int Puntuales { get; set; }
        public int Tardanzas { get; set; }
        public int Faltas { get; set; }
        public int Permisos { get; set; }
        public decimal HorasTrabajadas { get; set; }
        public decimal HorasExtra { get; set; }
    }

    public sealed class EmployeePortalDashboardViewModel
    {
        public EmployeeProfileViewModel Profile { get; set; }
        public Comprobante LatestPayslip { get; set; }
        public List<Asistencia> RecentAttendance { get; set; }
        public EmployeeAttendanceSummaryViewModel AttendanceSummary { get; set; }
    }
}
```

Models/Entities.cs

```
using System;
using System.Collections.Generic;

namespace SistemaGestionNomina.Models
{
    public class Usuario
    {
        public int IdUsuario { get; set; }
        public string NombreUsuario { get; set; }
        public string PasswordHash { get; set; }
        public string Rol { get; set; }
        public string Estado { get; set; }
        public int? IdEmpleado { get; set; }
        public DateTime? UltimoAcceso { get; set; }
        public int IntentosFallidos { get; set; }
        public bool Bloqueado { get; set; }
        public DateTime? FechaBloqueo { get; set; }
    }

    public class Departamento
    {
        public int IdDepartamento { get; set; }
        public string Nombre { get; set; }

        public override string ToString()
        {
            return Nombre;
        }
    }

    public class Empleado
    {
        private int idEmpleado;
        private string codigo;
        private string nombre;
        private string apellido;
        private string cedula;
        private string cargo;
        private int idDepartamento;
        private string departamentoNombre;
        private decimal salarioBase;
        private string estado;
        private DateTime fechaIngreso;
        private DateTime? fechaEfectivaLaboral;

        public int IdEmpleado
        {
            get { return idEmpleado; }
            set { idEmpleado = value; }
        }

        public string Codigo
        {
            get { return codigo; }
            set { codigo = EntityValidation.NormalizeRequired(value, "El codigo del empleado"); }
        }

        public string Nombre
        {
            get { return nombre; }
```

Models/Entities.cs (continuacion)

```
            set { nombre = EntityValidation.NormalizeRequired(value, "El nombre del empleado"); }
        }

        public string Apellido
        {
            get { return apellido; }
            set { apellido = EntityValidation.NormalizeOptional(value); }
        }

        public string Cedula
        {
            get { return cedula; }
            set { cedula = EntityValidation.NormalizeRequired(value, "La cedula del empleado"); }
        }

        public string Cargo
        {
            get { return cargo; }
            set { cargo = EntityValidation.NormalizeRequired(value, "El cargo del empleado"); }
        }

        public int IdDepartamento
        {
            get { return idDepartamento; }
            set { idDepartamento = value; }
        }

        public string DepartamentoNombre
        {
            get { return departamentoNombre; }
            set { departamentoNombre = EntityValidation.NormalizeOptional(value); }
        }

        public decimal SalarioBase
        {
            get { return salarioBase; }
            set
            {
                EntityValidation.RequireNonNegative(value, "El salario base");
                salarioBase = value;
            }
        }

        public string Estado
        {
            get { return estado; }
            set { estado = EntityValidation.NormalizeAllowedRequired(value, "El estado del empleado", "Activo", "Inactivo"); }
        }

        public DateTime FechaIngreso
        {
            get { return fechaIngreso; }
            set
            {
                EntityValidation.RequireValidDate(value, "La fecha de ingreso");
                fechaIngreso = value;
            }
        }

        public DateTime? FechaEfectivaLaboral
```

Models/Entities.cs (continuacion)

```
        {
            get { return fechaEfectivaLaboral; }
            set { fechaEfectivaLaboral = value; }
        }

        public string NombreCompleto
        {
            get { return (Nombre + " " + Apellido).Trim(); }
        }

        public override string ToString()
        {
            return Codigo + " - " + NombreCompleto;
        }
    }

    public class Asistencia
    {
        private int idAsistencia;
        private int idEmpleado;
        private string empleadoNombre;
        private DateTime fecha;
        private TimeSpan? horaEntrada;
        private TimeSpan? horaSalida;
        private decimal horasTrabajadas;
        private string estado;
        private int? idSolicitudAusencia;

        public int IdAsistencia
        {
            get { return idAsistencia; }
            set { idAsistencia = value; }
        }

        public int IdEmpleado
        {
            get { return idEmpleado; }
            set
            {
                EntityValidation.RequirePositiveId(value, "El empleado");
                idEmpleado = value;
            }
        }

        public string EmpleadoNombre
        {
            get { return empleadoNombre; }
            set { empleadoNombre = EntityValidation.NormalizeOptional(value); }
        }

        public DateTime Fecha
        {
            get { return fecha; }
            set
            {
                EntityValidation.RequireValidDate(value, "La fecha de asistencia");
                fecha = value;
            }
        }

```

Models/Entities.cs (continuacion)

```
        public TimeSpan? HoraEntrada
        {
            get { return horaEntrada; }
            set { horaEntrada = value; }
        }

        public TimeSpan? HoraSalida
        {
            get { return horaSalida; }
            set { horaSalida = value; }
        }

        public decimal HorasTrabajadas
        {
            get { return horasTrabajadas; }
            set
            {
                EntityValidation.RequireNonNegative(value, "Las horas trabajadas");
                horasTrabajadas = value;
            }
        }

        public string Estado
        {
            get { return estado; }
            set { estado = EntityValidation.NormalizeAllowedRequired(value, "El estado de asistencia", "Puntual", "Tardanza", "Falta", "Permiso"); }
        }

        public int? IdSolicitudAusencia
        {
            get { return idSolicitudAusencia; }
            set { idSolicitudAusencia = value; }
        }
    }

    public class PeriodoNomina
    {
        public int IdPeriodo { get; set; }
        public string Nombre { get; set; }
        public DateTime FechaInicio { get; set; }
        public DateTime FechaFin { get; set; }
        public string Estado { get; set; }
        public bool Cerrado { get; set; }
        public DateTime? FechaCierre { get; set; }
        public string CerradoPor { get; set; }
    }

    public class Nomina
    {
        private int idNomina;
        private int idPeriodo;
        private string periodoNombre;
        private DateTime fechaCalculo;
        private decimal totalIngresos;
        private decimal totalDeducciones;
        private decimal totalNeto;
        private string estado;
        private DateTime? fechaConfirmacion;
        private string confirmadaPor;
        private DateTime? fechaPago;
```

Models/Entities.cs (continuacion)

```
        private string pagadaPor;
        private DateTime? fechaAnulacion;
        private string anuladaPor;
        private string motivoAnulacion;
        private List<NominaDetalle> detalles;

        public Nomina()
        {
            Detalles = new List<NominaDetalle>();
        }

        public int IdNomina
        {
            get { return idNomina; }
            set { idNomina = value; }
        }

        public int IdPeriodo
        {
            get { return idPeriodo; }
            set { idPeriodo = value; }
        }

        public string PeriodoNombre
        {
            get { return periodoNombre; }
            set { periodoNombre = EntityValidation.NormalizeOptional(value); }
        }

        public DateTime FechaCalculo
        {
            get { return fechaCalculo; }
            set
            {
                EntityValidation.RequireValidDate(value, "La fecha de calculo");
                fechaCalculo = value;
            }
        }

        public decimal TotalIngresos
        {
            get { return totalIngresos; }
            set
            {
                EntityValidation.RequireNonNegative(value, "El total de ingresos");
                totalIngresos = value;
            }
        }

        public decimal TotalDeducciones
        {
            get { return totalDeducciones; }
            set
            {
                EntityValidation.RequireNonNegative(value, "El total de deducciones");
                totalDeducciones = value;
            }
        }

        public decimal TotalNeto
```

Models/Entities.cs (continuacion)

```
        {
            get { return totalNeto; }
            set
            {
                EntityValidation.RequireNonNegative(value, "El total neto");
                totalNeto = value;
            }
        }

        public string Estado
        {
            get { return estado; }
            set { estado = EntityValidation.NormalizeOptional(value); }
        }

        public DateTime? FechaConfirmacion
        {
            get { return fechaConfirmacion; }
            set { fechaConfirmacion = value; }
        }

        public string ConfirmadaPor
        {
            get { return confirmadaPor; }
            set { confirmadaPor = EntityValidation.NormalizeOptional(value); }
        }

        public DateTime? FechaPago
        {
            get { return fechaPago; }
            set { fechaPago = value; }
        }

        public string PagadaPor
        {
            get { return pagadaPor; }
            set { pagadaPor = EntityValidation.NormalizeOptional(value); }
        }

        public DateTime? FechaAnulacion
        {
            get { return fechaAnulacion; }
            set { fechaAnulacion = value; }
        }

        public string AnuladaPor
        {
            get { return anuladaPor; }
            set { anuladaPor = EntityValidation.NormalizeOptional(value); }
        }

        public string MotivoAnulacion
        {
            get { return motivoAnulacion; }
            set { motivoAnulacion = EntityValidation.NormalizeOptional(value); }
        }

        public List<NominaDetalle> Detalles
        {
            get { return detalles; }
```

Models/Entities.cs (continuacion)

```
            set { detalles = value ?? new List<NominaDetalle>(); }
        }
    }

    public class NominaDetalle
    {
        private int idDetalle;
        private int idNomina;
        private int idEmpleado;
        private string codigoEmpleado;
        private string empleadoNombre;
        private string departamento;
        private decimal sueldoBase;
        private decimal bonos;
        private decimal horasExtra;
        private decimal montoHorasExtra;
        private decimal totalIngresos;
        private decimal totalDeducciones;
        private decimal netoPagar;
        private string cargoEmpleado;

        public int IdDetalle
        {
            get { return idDetalle; }
            set { idDetalle = value; }
        }

        public int IdNomina
        {
            get { return idNomina; }
            set { idNomina = value; }
        }

        public int IdEmpleado
        {
            get { return idEmpleado; }
            set { idEmpleado = value; }
        }

        public string CodigoEmpleado
        {
            get { return codigoEmpleado; }
            set { codigoEmpleado = EntityValidation.NormalizeOptional(value); }
        }

        public string EmpleadoNombre
        {
            get { return empleadoNombre; }
            set { empleadoNombre = EntityValidation.NormalizeOptional(value); }
        }

        public string Departamento
        {
            get { return departamento; }
            set { departamento = EntityValidation.NormalizeOptional(value); }
        }

        public string CargoEmpleado
        {
            get { return cargoEmpleado; }
```

Models/Entities.cs (continuacion)

```
            set { cargoEmpleado = EntityValidation.NormalizeOptional(value); }
        }

        public decimal SueldoBase
        {
            get { return sueldoBase; }
            set
            {
                EntityValidation.RequireNonNegative(value, "El sueldo base");
                sueldoBase = value;
            }
        }

        public decimal Bonos
        {
            get { return bonos; }
            set
            {
                EntityValidation.RequireNonNegative(value, "Los bonos");
                bonos = value;
            }
        }

        public decimal HorasExtra
        {
            get { return horasExtra; }
            set
            {
                EntityValidation.RequireNonNegative(value, "Las horas extra");
                horasExtra = value;
            }
        }

        public decimal MontoHorasExtra
        {
            get { return montoHorasExtra; }
            set
            {
                EntityValidation.RequireNonNegative(value, "El monto de horas extra");
                montoHorasExtra = value;
            }
        }

        public decimal TotalIngresos
        {
            get { return totalIngresos; }
            set
            {
                EntityValidation.RequireNonNegative(value, "El total de ingresos");
                totalIngresos = value;
            }
        }

        public decimal TotalDeducciones
        {
            get { return totalDeducciones; }
            set
            {
                EntityValidation.RequireNonNegative(value, "El total de deducciones");
                totalDeducciones = value;
```

Models/Entities.cs (continuacion)

```
            }
        }

        public decimal NetoPagar
        {
            get { return netoPagar; }
            set
            {
                EntityValidation.RequireNonNegative(value, "El neto a pagar");
                netoPagar = value;
            }
        }
    }

    public class Comprobante
    {
        private int idComprobante;
        private int idNomina;
        private int idEmpleado;
        private string numeroComprobante;
        private DateTime fechaGeneracion;
        private string rutaPdf;
        private string empleadoNombre;
        private string codigoEmpleado;
        private string periodoNombre;
        private decimal totalIngresos;
        private decimal totalDeducciones;
        private decimal netoPagar;

        public int IdComprobante
        {
            get { return idComprobante; }
            set { idComprobante = value; }
        }

        public int IdNomina
        {
            get { return idNomina; }
            set { idNomina = value; }
        }

        public int IdEmpleado
        {
            get { return idEmpleado; }
            set { idEmpleado = value; }
        }

        public string NumeroComprobante
        {
            get { return numeroComprobante; }
            set { numeroComprobante = EntityValidation.NormalizeRequired(value, "El numero de comprobante"); }
        }

        public DateTime FechaGeneracion
        {
            get { return fechaGeneracion; }
            set
            {
                EntityValidation.RequireValidDate(value, "La fecha de generacion");
                fechaGeneracion = value;
```

Models/Entities.cs (continuacion)

```
            }
        }

        public string RutaPdf
        {
            get { return rutaPdf; }
            set { rutaPdf = EntityValidation.NormalizeOptional(value); }
        }

        public string EmpleadoNombre
        {
            get { return empleadoNombre; }
            set { empleadoNombre = EntityValidation.NormalizeOptional(value); }
        }

        public string CodigoEmpleado
        {
            get { return codigoEmpleado; }
            set { codigoEmpleado = EntityValidation.NormalizeOptional(value); }
        }

        public string PeriodoNombre
        {
            get { return periodoNombre; }
            set { periodoNombre = EntityValidation.NormalizeOptional(value); }
        }

        public decimal TotalIngresos
        {
            get { return totalIngresos; }
            set
            {
                EntityValidation.RequireNonNegative(value, "El total de ingresos");
                totalIngresos = value;
            }
        }

        public decimal TotalDeducciones
        {
            get { return totalDeducciones; }
            set
            {
                EntityValidation.RequireNonNegative(value, "El total de deducciones");
                totalDeducciones = value;
            }
        }

        public decimal NetoPagar
        {
            get { return netoPagar; }
            set
            {
                EntityValidation.RequireNonNegative(value, "El neto a pagar");
                netoPagar = value;
            }
        }
    }

    public class ConfiguracionNomina
    {
```

Models/Entities.cs (continuacion)

```
        public int IdConfiguracion { get; set; }
        public string NombreParametro { get; set; }
        public decimal Valor { get; set; }
        public string Descripcion { get; set; }
    }

    public class ReporteGenerado
    {
        public int IdReporte { get; set; }
        public string NombreReporte { get; set; }
        public string Tipo { get; set; }
        public string GeneradoPor { get; set; }
        public DateTime FechaGeneracion { get; set; }
        public string RutaArchivo { get; set; }
    }

    public class NominaVersion
    {
        public int IdVersion { get; set; }
        public int IdNominaOriginal { get; set; }
        public int? IdNominaNueva { get; set; }
        public string MotivoCambio { get; set; }
        public string UsuarioResponsable { get; set; }
        public DateTime FechaCambio { get; set; }
    }

    internal static class EntityValidation
    {
        public static string NormalizeRequired(string value, string fieldName)
        {
            if (string.IsNullOrWhiteSpace(value))
            {
                throw new ArgumentException(fieldName + " es obligatorio.");
            }

            return value.Trim();
        }

        public static string NormalizeOptional(string value)
        {
            return value == null ? null : value.Trim();
        }

        public static string NormalizeAllowedRequired(string value, string fieldName, params string[] allowedValues)
        {
            string normalized = NormalizeRequired(value, fieldName);
            for (int i = 0; i < allowedValues.Length; i++)
            {
                if (string.Equals(normalized, allowedValues[i], StringComparison.OrdinalIgnoreCase))
                {
                    return allowedValues[i];
                }
            }

            throw new ArgumentException(fieldName + " debe ser: " + string.Join(", ", allowedValues) + ".");
        }

        public static void RequireNonNegative(decimal value, string fieldName)
        {
            if (value < 0)
```

Models/Entities.cs (continuacion)

```
            {
                throw new ArgumentException(fieldName + " no puede ser negativo.");
            }
        }

        public static void RequirePositiveId(int value, string fieldName)
        {
            if (value <= 0)
            {
                throw new ArgumentException(fieldName + " debe ser mayor que cero.");
            }
        }

        public static void RequireValidDate(DateTime value, string fieldName)
        {
            if (value == DateTime.MinValue)
            {
                throw new ArgumentException(fieldName + " debe ser una fecha valida.");
            }
        }
    }
}
```

Models/PayrollDomain.cs

```
using System;
using System.Collections.Generic;

namespace SistemaGestionNomina.Models
{
    public static class PayrollStates
    {
        public const string Draft = "Borrador";
        public const string Calculated = "Calculada";
        public const string Confirmed = "Confirmada";
        public const string Paid = "Pagada";
        public const string Annulled = "Anulada";

        public static bool IsValid(string state)
        {
            return string.Equals(state, Draft, StringComparison.OrdinalIgnoreCase) ||
                string.Equals(state, Calculated, StringComparison.OrdinalIgnoreCase) ||
                string.Equals(state, Confirmed, StringComparison.OrdinalIgnoreCase) ||
                string.Equals(state, Paid, StringComparison.OrdinalIgnoreCase) ||
                string.Equals(state, Annulled, StringComparison.OrdinalIgnoreCase);
        }
    }

    public static class PayrollPeriodStates
    {
        public const string Open = "Abierto";
        public const string Confirmed = "Confirmado";
        public const string Paid = "Pagado";
        public const string Reopened = "Reabierto";
    }

    public static class PayrollStateMachine
    {
        private static readonly IDictionary<string, string[]> AllowedTransitions =
            new Dictionary<string, string[]>(StringComparer.OrdinalIgnoreCase)
            {
                { PayrollStates.Draft, new[] { PayrollStates.Calculated } },
                { PayrollStates.Calculated, new[] { PayrollStates.Confirmed } },
                { PayrollStates.Confirmed, new[] { PayrollStates.Paid, PayrollStates.Annulled } },
                { PayrollStates.Paid, new[] { PayrollStates.Annulled } },
                { PayrollStates.Annulled, new string[0] }
            };

        public static bool CanTransition(string currentState, string targetState)
        {
            if (string.IsNullOrWhiteSpace(currentState) || string.IsNullOrWhiteSpace(targetState))
                return false;

            string[] targets;
            if (!AllowedTransitions.TryGetValue(currentState.Trim(), out targets)) return false;
            for (int i = 0; i < targets.Length; i++)
            {
                if (string.Equals(targets[i], targetState.Trim(), StringComparison.OrdinalIgnoreCase)) return true;
            }
            return false;
        }

        public static void DemandTransition(string currentState, string targetState)
        {
            if (!CanTransition(currentState, targetState))
```

Models/PayrollDomain.cs (continuacion)

```
            {
                throw new InvalidOperationException("No se permite cambiar la nómina de " +
                    (currentState ?? "sin estado") + " a " + (targetState ?? "sin estado") + ".");
            }
        }
    }
}
```

Program.cs

```
using System;
using System.Windows.Forms;
using SistemaGestionNomina.Data;
using SistemaGestionNomina.UI;
using SistemaGestionNomina.Security;
using SistemaGestionNomina.Services;

namespace SistemaGestionNomina
{
    internal static class Program
    {
        [STAThread]
        private static void Main()
        {
            Application.EnableVisualStyles();
            Application.SetCompatibleTextRenderingDefault(false);

            try
            {
                DatabaseInitializer.Initialize();
                new EmployeeHistoryService().ApplyDueChanges(DateTime.Today);
            }
            catch (Exception ex)
            {
                MessageBox.Show("No se pudo inicializar la base de datos.\n\n" + ex.Message,
                    "Error de inicio", MessageBoxButtons.OK, MessageBoxIcon.Error);
                return;
            }

            bool continueLogin = true;
            while (continueLogin)
            {
                using (FrmLogin login = new FrmLogin())
                {
                    if (login.ShowDialog() != DialogResult.OK)
                    {
                        break;
                    }

                    using (FrmMain main = new FrmMain(login.AuthenticatedUser))
                    {
                        Application.Run(main);
                        continueLogin = main.LogoutRequested;
                        if (SessionContext.IsAuthenticated)
                        {
                            new AuditTrailService().RegistrarAccion("Seguridad", "Cerrar aplicación", string.Empty);
                            SessionContext.Clear();
                        }
                    }
                }
            }
        }
    }
}
```

Properties/Resources.Designer.cs

```
//------------------------------------------------------------------------------
// <auto-generated>
//     This code was generated by a tool.
//     Runtime Version:4.0.30319.42000
//
//     Changes to this file may cause incorrect behavior and will be lost if
//     the code is regenerated.
// </auto-generated>
//------------------------------------------------------------------------------

namespace SistemaGestionNomina.Properties {
    using System;


    /// <summary>
    ///   A strongly-typed resource class, for looking up localized strings, etc.
    /// </summary>
    // This class was auto-generated by the StronglyTypedResourceBuilder
    // class via a tool like ResGen or Visual Studio.
    // To add or remove a member, edit your .ResX file then rerun ResGen
    // with the /str option, or rebuild your VS project.
    [global::System.CodeDom.Compiler.GeneratedCodeAttribute("System.Resources.Tools.StronglyTypedResourceBuilder", "18.0.0.0")]
    [global::System.Diagnostics.DebuggerNonUserCodeAttribute()]
    [global::System.Runtime.CompilerServices.CompilerGeneratedAttribute()]
    internal class Resources {

        private static global::System.Resources.ResourceManager resourceMan;

        private static global::System.Globalization.CultureInfo resourceCulture;

        [global::System.Diagnostics.CodeAnalysis.SuppressMessageAttribute("Microsoft.Performance", "CA1811:AvoidUncalledPrivateCode")]
        internal Resources() {
        }

        /// <summary>
        ///   Returns the cached ResourceManager instance used by this class.
        /// </summary>
        [global::System.ComponentModel.EditorBrowsableAttribute(global::System.ComponentModel.EditorBrowsableState.Advanced)]
        internal static global::System.Resources.ResourceManager ResourceManager {
            get {
                if (object.ReferenceEquals(resourceMan, null)) {
                    global::System.Resources.ResourceManager temp = new global::System.Resources.ResourceManager("SistemaGestionNomina.Properties.Resource \
    s", typeof(Resources).Assembly);
                    resourceMan = temp;
                }
                return resourceMan;
            }
        }

        /// <summary>
        ///   Overrides the current thread's CurrentUICulture property for all
        ///   resource lookups using this strongly typed resource class.
        /// </summary>
        [global::System.ComponentModel.EditorBrowsableAttribute(global::System.ComponentModel.EditorBrowsableState.Advanced)]
        internal static global::System.Globalization.CultureInfo Culture {
            get {
                return resourceCulture;
            }
            set {
                resourceCulture = value;
```

Properties/Resources.Designer.cs (continuacion)

```
            }
        }

        /// <summary>
        ///   Looks up a localized resource of type System.Drawing.Bitmap.
        /// </summary>
        internal static System.Drawing.Bitmap ChatGPT_Image_28_jun_2026__16_41_58 {
            get {
                object obj = ResourceManager.GetObject("ChatGPT Image 28 jun 2026, 16_41_58", resourceCulture);
                return ((System.Drawing.Bitmap)(obj));
            }
        }

        /// <summary>
        ///   Looks up a localized resource of type System.Drawing.Bitmap.
        /// </summary>
        internal static System.Drawing.Bitmap logo_aplicacion {
            get {
                object obj = ResourceManager.GetObject("logo aplicacion", resourceCulture);
                return ((System.Drawing.Bitmap)(obj));
            }
        }

        /// <summary>
        ///   Looks up a localized resource of type System.Drawing.Bitmap.
        /// </summary>
        internal static System.Drawing.Bitmap logo_transparent {
            get {
                object obj = ResourceManager.GetObject("logo transparent", resourceCulture);
                return ((System.Drawing.Bitmap)(obj));
            }
        }
    }
}
```

Security/AuthorizationService.cs

```
using System;
using SistemaGestionNomina.Services;

namespace SistemaGestionNomina.Security
{
    public sealed class AuthorizationService
    {
        private readonly RolePermissionService rolePermissionService = new RolePermissionService();
        private readonly AuditTrailService auditTrailService = new AuditTrailService();

        public bool HasPermission(string permission)
        {
            return SessionContext.IsAuthenticated &&
                rolePermissionService.TienePermiso(SessionContext.Role, permission);
        }

        public void DemandPermission(string permission)
        {
            if (HasPermission(permission)) return;
            auditTrailService.RegistrarAccion("Seguridad", "Acceso rechazado", permission ?? string.Empty);
            throw new UnauthorizedAccessException("No tiene permisos para realizar esta operación.");
        }

        public void DemandAny(params string[] permissions)
        {
            if (permissions != null)
            {
                for (int i = 0; i < permissions.Length; i++)
                {
                    if (HasPermission(permissions[i])) return;
                }
            }

            auditTrailService.RegistrarAccion("Seguridad", "Acceso rechazado",
                permissions == null ? string.Empty : string.Join(", ", permissions));
            throw new UnauthorizedAccessException("No tiene permisos para realizar esta operación.");
        }
    }
}
```

Security/EmployeeScopeService.cs

```
using System;
using SistemaGestionNomina.Data;
using SistemaGestionNomina.Models;

namespace SistemaGestionNomina.Security
{
    public sealed class EmployeeScopeService
    {
        private readonly EmpleadoRepository employeeRepository = new EmpleadoRepository();
        private readonly AuditRepository auditRepository = new AuditRepository();

        public int RequireCurrentEmployeeId()
        {
            if (!SessionContext.IsAuthenticated ||
                !string.Equals(SessionContext.Role, Roles.Trabajador, StringComparison.OrdinalIgnoreCase))
            {
                throw Reject("Esta operacion requiere una sesion de trabajador.", "Rol o sesion no validos");
            }

            if (!SessionContext.EmployeeId.HasValue || SessionContext.EmployeeId.Value <= 0)
            {
                throw Reject("El usuario no tiene un empleado asociado.", "Cuenta sin empleado asociado");
            }

            Empleado employee = employeeRepository.GetById(SessionContext.EmployeeId.Value);
            if (employee == null)
            {
                throw Reject("No se encontro el empleado asociado a la cuenta.", "Empleado asociado inexistente");
            }

            if (!string.Equals(employee.Estado, "Activo", StringComparison.OrdinalIgnoreCase))
            {
                throw Reject("El empleado asociado no se encuentra activo.", "Empleado asociado inactivo");
            }

            return employee.IdEmpleado;
        }

        public void DemandCurrentEmployee(int employeeId)
        {
            int currentEmployeeId = RequireCurrentEmployeeId();
            if (employeeId <= 0 || employeeId != currentEmployeeId)
            {
                throw Reject("Solo puede consultar su propia informacion laboral.", "Intento de acceso a otro empleado");
            }
        }

        private UnauthorizedAccessException Reject(string message, string detail)
        {
            auditRepository.Add(new AuditRecord
            {
                Usuario = SessionContext.IsAuthenticated ? SessionContext.Username : "anonimo",
                Modulo = "Portal trabajador",
                Accion = "Acceso rechazado",
                Detalle = detail,
                Fecha = DateTime.Now
            });
            return new UnauthorizedAccessException(message);
        }

```

Security/EmployeeScopeService.cs (continuacion)

```
        public int? GetDepartmentScope()
        {
            if (!string.Equals(SessionContext.Role, Roles.Supervisor, StringComparison.OrdinalIgnoreCase))
            {
                return null;
            }

            if (!SessionContext.EmployeeId.HasValue)
            {
                throw new UnauthorizedAccessException("El supervisor no tiene un empleado asociado.");
            }

            int? departmentId = employeeRepository.GetDepartmentId(SessionContext.EmployeeId.Value);
            if (!departmentId.HasValue)
            {
                throw new UnauthorizedAccessException("No se encontró el departamento asociado al supervisor.");
            }

            return departmentId;
        }

        public void DemandEmployeeInScope(int employeeId)
        {
            int? scopeDepartmentId = GetDepartmentScope();
            if (!scopeDepartmentId.HasValue) return;

            int? employeeDepartmentId = employeeRepository.GetDepartmentId(employeeId);
            if (!employeeDepartmentId.HasValue || employeeDepartmentId.Value != scopeDepartmentId.Value)
            {
                throw new UnauthorizedAccessException("No puede consultar o modificar empleados de otro departamento.");
            }
        }
    }
}
```

Security/Permissions.cs

```
namespace SistemaGestionNomina.Security
{
    public static class Permissions
    {
        public const string DashboardView = "dashboard.ver";
        public const string EmployeesView = "empleados.ver";
        public const string EmployeesCreate = "empleados.crear";
        public const string EmployeesEdit = "empleados.editar";
        public const string EmployeesDeactivate = "empleados.desactivar";
        public const string EmployeesExport = "empleados.exportar";
        public const string EmployeeHistoryView = "empleados.historial.ver";
        public const string EmployeeHistoryExport = "empleados.historial.exportar";
        public const string EmployeeChangesSchedule = "empleados.cambios.programar";
        public const string AttendanceView = "asistencia.ver";
        public const string AttendanceRegister = "asistencia.registrar";
        public const string AttendanceImport = "asistencia.importar";
        public const string AttendanceExport = "asistencia.exportar";
        public const string PayrollView = "nomina.ver";
        public const string PayrollCalculate = "nomina.calcular";
        public const string PayrollConfirm = "nomina.confirmar";
        public const string PayrollExport = "nomina.exportar";
        public const string PayslipsView = "comprobantes.ver";
        public const string PayslipsExport = "comprobantes.exportar";
        public const string PayslipsPrint = "comprobantes.imprimir";
        public const string PayslipsEmail = "comprobantes.email";
        public const string ReportsView = "reportes.ver";
        public const string ReportsPersonal = "reportes.personal";
        public const string ReportsFinancial = "reportes.financiero";
        public const string ReportsExport = "reportes.exportar";
        public const string ConfigurationView = "configuracion.ver";
        public const string ConfigurationEdit = "configuracion.editar";
        public const string BackupsManage = "backups.gestionar";
        public const string AuditView = "auditoria.ver";
        public const string PortalView = "portal.ver";
        public const string OwnProfileView = "portal.perfil.ver";
        public const string OwnAttendanceView = "portal.asistencia.ver";
        public const string OwnPayslipsView = "portal.comprobantes.ver";
        public const string OwnPayslipsDownload = "portal.comprobantes.descargar";
        public const string OwnPasswordChange = "cuenta.password.cambiar";

        public const string PayrollPay = "nomina.pagar";
        public const string PayrollAnnul = "nomina.anular";
        public const string PayrollRecalculate = "nomina.recalcular";
        public const string PayrollHistoryView = "nomina.historial.ver";

        public const string AbsencesView = "ausencias.ver";
        public const string AbsencesCreate = "ausencias.crear";
        public const string AbsencesApprove = "ausencias.aprobar";
        public const string AbsencesReject = "ausencias.rechazar";
        public const string AbsencesCancel = "ausencias.cancelar";
        public const string AbsencesExport = "ausencias.exportar";
        public const string OwnAbsencesView = "portal.ausencias.ver";
        public const string OwnAbsencesCreate = "portal.ausencias.crear";
        public const string OwnAbsencesCancel = "portal.ausencias.cancelar";
    }
}
```

Security/Roles.cs

```
namespace SistemaGestionNomina.Security
{
    public static class Roles
    {
        public const string Admin = "Admin";
        public const string RRHH = "RRHH";
        public const string Contabilidad = "Contabilidad";
        public const string Supervisor = "Supervisor";
        public const string Trabajador = "Trabajador";

        public static bool IsValid(string role)
        {
            return role == Admin || role == RRHH || role == Contabilidad ||
                role == Supervisor || role == Trabajador;
        }
    }
}
```

Security/SessionContext.cs

```
using System;
using SistemaGestionNomina.Models;

namespace SistemaGestionNomina.Security
{
    public static class SessionContext
    {
        private static readonly object Sync = new object();
        private static Usuario currentUser;

        public static Usuario CurrentUser
        {
            get { lock (Sync) { return currentUser; } }
        }

        public static bool IsAuthenticated
        {
            get { return CurrentUser != null; }
        }

        public static string Username
        {
            get { return CurrentUser == null ? string.Empty : CurrentUser.NombreUsuario; }
        }

        public static string Role
        {
            get { return CurrentUser == null ? string.Empty : CurrentUser.Rol; }
        }

        public static int? EmployeeId
        {
            get { return CurrentUser == null ? null : CurrentUser.IdEmpleado; }
        }

        public static void Begin(Usuario user)
        {
            if (user == null || user.IdUsuario <= 0 || string.IsNullOrWhiteSpace(user.NombreUsuario))
            {
                throw new ArgumentException("El usuario autenticado no es válido.", "user");
            }

            lock (Sync)
            {
                currentUser = user;
            }
        }

        public static void Clear()
        {
            lock (Sync)
            {
                currentUser = null;
            }
        }
    }
}
```

Services/AbsenceRequestService.cs

```
using System;
using System.Collections.Generic;
using System.Data.SQLite;
using SistemaGestionNomina.Data;
using SistemaGestionNomina.Models;
using SistemaGestionNomina.Security;

namespace SistemaGestionNomina.Services
{
    public sealed class AbsenceRequestService
    {
        private readonly AbsenceRequestRepository repository = new AbsenceRequestRepository();
        private readonly EmpleadoRepository employeeRepository = new EmpleadoRepository();
        private readonly EmployeeEffectiveDataService effectiveEmployeeService = new EmployeeEffectiveDataService();
        private readonly EmployeeScopeService employeeScopeService = new EmployeeScopeService();
        private readonly AuthorizationService authorizationService = new AuthorizationService();
        private readonly PayrollPeriodPolicyService periodPolicyService = new PayrollPeriodPolicyService();
        private readonly AuditRepository auditRepository = new AuditRepository();
        private readonly AuditTrailService auditTrailService = new AuditTrailService();

        public List<SolicitudAusencia> Search(AbsenceQuery query)
        {
            query = query ?? new AbsenceQuery();
            ValidateQuery(query);
            int? forcedEmployeeId = null;
            int? scopeDepartmentId = null;
            if (IsWorker())
            {
                authorizationService.DemandPermission(Permissions.OwnAbsencesView);
                forcedEmployeeId = employeeScopeService.RequireCurrentEmployeeId();
                if (query.EmployeeId.HasValue) employeeScopeService.DemandCurrentEmployee(query.EmployeeId.Value);
            }
            else
            {
                authorizationService.DemandPermission(Permissions.AbsencesView);
                if (query.EmployeeId.HasValue) employeeScopeService.DemandEmployeeInScope(query.EmployeeId.Value);
                scopeDepartmentId = employeeScopeService.GetDepartmentScope();
            }

            List<SolicitudAusencia> items = repository.Search(query, forcedEmployeeId, scopeDepartmentId);
            auditTrailService.RegistrarAccion("Ausencias", "Consultar", "Registros=" + items.Count);
            return items;
        }

        public SolicitudAusencia GetById(int requestId)
        {
            if (requestId <= 0) throw new ArgumentException("Seleccione una solicitud válida.");
            SolicitudAusencia item = repository.GetById(requestId);
            if (item == null) throw new InvalidOperationException("La solicitud no existe.");
            DemandRequestScope(item, Permissions.AbsencesView, Permissions.OwnAbsencesView);
            return item;
        }

        public int CreateForEmployee(SolicitudAusencia item)
        {
            authorizationService.DemandPermission(Permissions.AbsencesCreate);
            if (item == null) throw new ArgumentNullException("item");
            employeeScopeService.DemandEmployeeInScope(item.IdEmpleado);
            return Create(item.IdEmpleado, item.Tipo, item.FechaInicio, item.FechaFin, item.Motivo);
        }
```

Services/AbsenceRequestService.cs (continuacion)

```

        public int CreateOwn(OwnAbsenceRequest request)
        {
            authorizationService.DemandPermission(Permissions.OwnAbsencesCreate);
            if (request == null) throw new ArgumentNullException("request");
            int employeeId = employeeScopeService.RequireCurrentEmployeeId();
            return Create(employeeId, request.Type, request.StartDate, request.EndDate, request.Reason);
        }

        public void Approve(int requestId, string observation)
        {
            authorizationService.DemandPermission(Permissions.AbsencesApprove);
            SolicitudAusencia current = repository.GetById(requestId);
            if (current == null) throw new InvalidOperationException("La solicitud no existe.");
            employeeScopeService.DemandEmployeeInScope(current.IdEmpleado);
            AbsenceStateMachine.DemandTransition(current.Estado, AbsenceStates.Approved);
            periodPolicyService.VerificarFechasAbiertas(current.FechaInicio, current.FechaFin);

            List<DateTime> workdays = GetWorkdays(current.FechaInicio, current.FechaFin);
            if (workdays.Count == 0) throw new InvalidOperationException("La solicitud no contiene días laborables de lunes a viernes.");
            string user = CurrentUsername();
            DateTime now = DateTime.Now;

            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteTransaction transaction = connection.BeginTransaction())
            {
                try
                {
                    current = repository.GetById(connection, transaction, requestId);
                    if (current == null) throw new InvalidOperationException("La solicitud no existe.");
                    AbsenceStateMachine.DemandTransition(current.Estado, AbsenceStates.Approved);
                    if (repository.HasApprovedOverlap(connection, transaction, current.IdEmpleado,
                        current.FechaInicio, current.FechaFin, current.IdSolicitud))
                        throw new InvalidOperationException("Ya existe una ausencia aprobada que se solapa con el rango.");

                    // Se valida todo el rango antes de escribir para no reemplazar asistencias existentes.
                    for (int i = 0; i < workdays.Count; i++)
                    {
                        if (repository.AttendanceExists(connection, transaction, current.IdEmpleado, workdays[i]))
                            throw new InvalidOperationException("Ya existe asistencia para " +
                                workdays[i].ToString("dd/MM/yyyy") + ". No se reemplazó ningún registro.");
                    }

                    repository.UpdateState(connection, transaction, requestId, AbsenceStates.Pending,
                        AbsenceStates.Approved, user, now, observation);
                    string attendanceStatus = AbsenceTypes.ToAttendanceStatus(current.Tipo);
                    // La aprobacion refleja la ausencia como asistencia y mantiene el vinculo para poder revertirla.
                    for (int i = 0; i < workdays.Count; i++)
                        repository.AddGeneratedAttendance(connection, transaction, requestId,
                            current.IdEmpleado, workdays[i], attendanceStatus);
                    AddAudit(connection, transaction, user, "Aprobar",
                        "IdSolicitud=" + requestId + ", Días=" + workdays.Count, now);
                    transaction.Commit();
                }
                catch
                {
                    transaction.Rollback();
                    throw;
                }
            }
```

```
        }

        public void Reject(int requestId, string observation)
        {
            authorizationService.DemandPermission(Permissions.AbsencesReject);
            if (string.IsNullOrWhiteSpace(observation))
                throw new ArgumentException("Debe indicar la razón del rechazo.");
            SolicitudAusencia current = repository.GetById(requestId);
            if (current == null) throw new InvalidOperationException("La solicitud no existe.");
            employeeScopeService.DemandEmployeeInScope(current.IdEmpleado);
            AbsenceStateMachine.DemandTransition(current.Estado, AbsenceStates.Rejected);
            periodPolicyService.VerificarFechasAbiertas(current.FechaInicio, current.FechaFin);
            ChangeState(current, AbsenceStates.Rejected, observation.Trim(), "Rechazar");
        }

        public void Cancel(int requestId, string reason)
        {
            if (string.IsNullOrWhiteSpace(reason))
                throw new ArgumentException("Debe indicar el motivo de cancelación.");
            SolicitudAusencia current = repository.GetById(requestId);
            if (current == null) throw new InvalidOperationException("La solicitud no existe.");

            if (IsWorker())
            {
                authorizationService.DemandPermission(Permissions.OwnAbsencesCancel);
                employeeScopeService.DemandCurrentEmployee(current.IdEmpleado);
                if (!string.Equals(current.Estado, AbsenceStates.Pending, StringComparison.OrdinalIgnoreCase))
                    throw new InvalidOperationException("El trabajador solo puede cancelar solicitudes propias pendientes.");
            }
            else
            {
                authorizationService.DemandPermission(Permissions.AbsencesCancel);
                employeeScopeService.DemandEmployeeInScope(current.IdEmpleado);
            }

            AbsenceStateMachine.DemandTransition(current.Estado, AbsenceStates.Cancelled);
            periodPolicyService.VerificarFechasAbiertas(current.FechaInicio, current.FechaFin);
            string user = CurrentUsername();
            DateTime now = DateTime.Now;
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteTransaction transaction = connection.BeginTransaction())
            {
                try
                {
                    if (string.Equals(current.Estado, AbsenceStates.Approved, StringComparison.OrdinalIgnoreCase))
                        // Solo se eliminan las asistencias creadas automaticamente por esta solicitud.
                        repository.DeleteGeneratedAttendance(connection, transaction, current.IdSolicitud);
                    repository.UpdateState(connection, transaction, current.IdSolicitud, current.Estado,
                        AbsenceStates.Cancelled, user, now, reason.Trim());
                    AddAudit(connection, transaction, user, "Cancelar",
                        "IdSolicitud=" + current.IdSolicitud + ", EstadoAnterior=" + current.Estado, now);
                    transaction.Commit();
                }
                catch
                {
                    transaction.Rollback();
                    throw;
                }
            }
        }
```

Services/AbsenceRequestService.cs (continuacion)

```

        private int Create(int employeeId, string type, DateTime start, DateTime end, string reason)
        {
            ValidateRequest(employeeId, type, start, end, reason);
            Empleado employee = employeeRepository.GetById(employeeId);
            Empleado effectiveEmployee = effectiveEmployeeService.GetEmployeeAsOf(employeeId, start.Date);
            if (employee == null || !string.Equals(employee.Estado, "Activo", StringComparison.OrdinalIgnoreCase) ||
                effectiveEmployee == null || !string.Equals(effectiveEmployee.Estado, "Activo", StringComparison.OrdinalIgnoreCase))
                throw new InvalidOperationException("El empleado debe estar activo al crear la solicitud y en su fecha inicial.");

            periodPolicyService.VerificarFechasAbiertas(start.Date, end.Date);
            string user = CurrentUsername();
            DateTime now = DateTime.Now;
            SolicitudAusencia item = new SolicitudAusencia
            {
                IdEmpleado = employeeId,
                Tipo = NormalizeType(type),
                FechaInicio = start.Date,
                FechaFin = end.Date,
                Estado = AbsenceStates.Pending,
                Motivo = reason.Trim(),
                UsuarioSolicitante = user,
                FechaCreacion = now
            };

            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteTransaction transaction = connection.BeginTransaction())
            {
                try
                {
                    int id = repository.Add(connection, transaction, item);
                    AddAudit(connection, transaction, user, "Crear",
                        "IdSolicitud=" + id + ", IdEmpleado=" + employeeId + ", Tipo=" + item.Tipo, now);
                    transaction.Commit();
                    return id;
                }
                catch
                {
                    transaction.Rollback();
                    throw;
                }
            }
        }

        private void ChangeState(SolicitudAusencia current, string newState, string observation, string action)
        {
            string user = CurrentUsername();
            DateTime now = DateTime.Now;
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteTransaction transaction = connection.BeginTransaction())
            {
                try
                {
                    repository.UpdateState(connection, transaction, current.IdSolicitud, current.Estado,
                        newState, user, now, observation);
                    AddAudit(connection, transaction, user, action, "IdSolicitud=" + current.IdSolicitud, now);
                    transaction.Commit();
                }
                catch
                {
```

Services/AbsenceRequestService.cs (continuacion)

```
                    transaction.Rollback();
                    throw;
                }
            }
        }

        private void DemandRequestScope(SolicitudAusencia item, string generalPermission, string ownPermission)
        {
            if (IsWorker())
            {
                authorizationService.DemandPermission(ownPermission);
                employeeScopeService.DemandCurrentEmployee(item.IdEmpleado);
            }
            else
            {
                authorizationService.DemandPermission(generalPermission);
                employeeScopeService.DemandEmployeeInScope(item.IdEmpleado);
            }
        }

        private static void ValidateQuery(AbsenceQuery query)
        {
            if (query.From.HasValue && query.To.HasValue && query.From.Value.Date > query.To.Value.Date)
                throw new ArgumentException("La fecha inicial no puede superar la fecha final.");
            if (!string.IsNullOrWhiteSpace(query.Type) && !AbsenceTypes.IsValid(query.Type))
                throw new ArgumentException("El tipo de ausencia no es válido.");
            if (!string.IsNullOrWhiteSpace(query.Status) && !AbsenceStates.IsValid(query.Status))
                throw new ArgumentException("El estado de ausencia no es válido.");
        }

        private static void ValidateRequest(int employeeId, string type, DateTime start, DateTime end, string reason)
        {
            if (employeeId <= 0) throw new ArgumentException("Debe seleccionar un empleado.");
            if (!AbsenceTypes.IsValid(type)) throw new ArgumentException("Seleccione un tipo de ausencia válido.");
            if (start.Date > end.Date) throw new ArgumentException("La fecha inicial no puede superar la fecha final.");
            if (GetWorkdays(start, end).Count == 0)
                throw new ArgumentException("La solicitud debe incluir al menos un día de lunes a viernes.");
            if (string.IsNullOrWhiteSpace(reason)) throw new ArgumentException("Debe indicar el motivo de la solicitud.");
        }

        private static string NormalizeType(string type)
        {
            for (int i = 0; i < AbsenceTypes.All.Length; i++)
            {
                if (string.Equals(AbsenceTypes.All[i], type, StringComparison.OrdinalIgnoreCase))
                    return AbsenceTypes.All[i];
            }
            return type;
        }

        private static List<DateTime> GetWorkdays(DateTime start, DateTime end)
        {
            List<DateTime> dates = new List<DateTime>();
            for (DateTime date = start.Date; date <= end.Date; date = date.AddDays(1))
            {
                if (date.DayOfWeek != DayOfWeek.Saturday && date.DayOfWeek != DayOfWeek.Sunday)
                    dates.Add(date);
            }
            return dates;
        }
```

Services/AbsenceRequestService.cs (continuacion)

```

        private void AddAudit(SQLiteConnection connection, SQLiteTransaction transaction, string user,
            string action, string detail, DateTime date)
        {
            auditRepository.Add(connection, transaction, new AuditRecord
            {
                Usuario = user,
                Modulo = "Ausencias",
                Accion = action,
                Detalle = detail,
                Fecha = date
            });
        }

        private static bool IsWorker()
        {
            return string.Equals(SessionContext.Role, Roles.Trabajador, StringComparison.OrdinalIgnoreCase);
        }

        private static string CurrentUsername()
        {
            if (!SessionContext.IsAuthenticated || string.IsNullOrWhiteSpace(SessionContext.Username))
                throw new UnauthorizedAccessException("Debe iniciar sesión para gestionar solicitudes.");
            return SessionContext.Username;
        }
    }
}
```

Services/AccountService.cs

```
using System;
using SistemaGestionNomina.Data;
using SistemaGestionNomina.Helpers;
using SistemaGestionNomina.Models;
using SistemaGestionNomina.Security;

namespace SistemaGestionNomina.Services
{
    public sealed class AccountService
    {
        private readonly AuthenticationRepository authenticationRepository = new AuthenticationRepository();
        private readonly AuthorizationService authorizationService = new AuthorizationService();
        private readonly EmployeeScopeService employeeScopeService = new EmployeeScopeService();
        private readonly AuditTrailService auditTrailService = new AuditTrailService();

        public void ChangeOwnPassword(string currentPassword, string newPassword, string confirmation)
        {
            authorizationService.DemandPermission(Permissions.OwnPasswordChange);
            employeeScopeService.RequireCurrentEmployeeId();

            Usuario sessionUser = SessionContext.CurrentUser;
            if (sessionUser == null)
            {
                throw new UnauthorizedAccessException("No existe una sesion autenticada.");
            }

            Usuario storedUser = authenticationRepository.GetByUsername(SessionContext.Username);
            if (storedUser == null || storedUser.IdUsuario != sessionUser.IdUsuario)
            {
                throw new UnauthorizedAccessException("No se pudo validar la cuenta de la sesion.");
            }

            if (!PasswordHelper.Verify(currentPassword, storedUser.PasswordHash))
            {
                AuditRejected("Contrasena actual invalida");
                throw new ArgumentException("La contrasena actual no es correcta.");
            }

            ValidateNewPassword(newPassword, confirmation, storedUser.PasswordHash);
            string newHash = PasswordHelper.HashPassword(newPassword);
            if (!authenticationRepository.UpdatePassword(storedUser.IdUsuario, newHash))
            {
                throw new InvalidOperationException("No se pudo actualizar la contrasena.");
            }

            sessionUser.PasswordHash = newHash;
            auditTrailService.RegistrarAccion("Cuenta", "Cambiar contrasena", "Actualizacion completada");
        }

        private void ValidateNewPassword(string newPassword, string confirmation, string currentHash)
        {
            if (string.IsNullOrEmpty(newPassword))
            {
                AuditRejected("Nueva contrasena vacia");
                throw new ArgumentException("La nueva contrasena es obligatoria.");
            }

            if (!string.Equals(newPassword, confirmation, StringComparison.Ordinal))
            {
                AuditRejected("Confirmacion diferente");
```

Services/AccountService.cs (continuacion)

```
                throw new ArgumentException("La confirmacion no coincide con la nueva contrasena.");
            }

            if (PasswordHelper.Verify(newPassword, currentHash))
            {
                AuditRejected("Contrasena reutilizada");
                throw new ArgumentException("La nueva contrasena debe ser diferente de la actual.");
            }

            bool hasUpper = false;
            bool hasLower = false;
            bool hasDigit = false;
            for (int i = 0; i < newPassword.Length; i++)
            {
                hasUpper |= char.IsUpper(newPassword[i]);
                hasLower |= char.IsLower(newPassword[i]);
                hasDigit |= char.IsDigit(newPassword[i]);
            }

            if (newPassword.Length < 8 || !hasUpper || !hasLower || !hasDigit)
            {
                AuditRejected("Politica de contrasena incumplida");
                throw new ArgumentException("Use al menos 8 caracteres, con mayuscula, minuscula y numero.");
            }
        }

        private void AuditRejected(string reason)
        {
            auditTrailService.RegistrarAccion("Cuenta", "Cambio de contrasena rechazado", reason);
        }
    }
}
```

Services/AdvancedNominaRules.cs

```
using System;
using System.Data.SQLite;
using SistemaGestionNomina.Data;

namespace SistemaGestionNomina.Services
{
    /// <summary>
    /// Contiene reglas avanzadas opcionales de nómina.
    /// </summary>
    public class AdvancedNominaRules
    {
        /// <summary>
        /// Calcula ajustes especiales no incluidos en la nómina académica básica.
        /// </summary>
        public decimal CalcularAjusteEspecial(int idEmpleado, decimal ingresoBase)
        {
            DateTime today = DateTime.Today;
            DateTime start = new DateTime(today.Year, today.Month, 1);
            DateTime end = start.AddMonths(1).AddDays(-1);
            return CalcularAjusteEspecial(idEmpleado, ingresoBase, start, end);
        }

        /// <summary>
        /// Calcula ajuste por antiguedad, asistencia perfecta y descuentos disciplinarios del periodo.
        /// </summary>
        public decimal CalcularAjusteEspecial(int idEmpleado, decimal ingresoBase, DateTime fechaInicio, DateTime fechaFin)
        {
            if (idEmpleado <= 0)
            {
                throw new ArgumentException("Debe indicar un empleado válido.");
            }

            if (ingresoBase < 0)
            {
                throw new ArgumentException("El ingreso base no puede ser negativo.");
            }

            if (fechaInicio.Date > fechaFin.Date)
            {
                throw new ArgumentException("La fecha de inicio no puede ser mayor que la fecha fin.");
            }

            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            {
                DateTime fechaIngreso = ObtenerFechaIngreso(connection, idEmpleado);
                int years = Math.Max(0, (DateTime.Today - fechaIngreso.Date).Days / 365);
                decimal porcentajeAntiguedad = Math.Min(years * 0.005m, 0.03m);

                int faltas;
                int tardanzas;
                ObtenerIndicadoresAsistencia(connection, idEmpleado, fechaInicio, fechaFin, out faltas, out tardanzas);

                decimal bonoAntiguedad = ingresoBase * porcentajeAntiguedad;
                decimal bonoAsistencia = faltas == 0 && tardanzas == 0 ? ingresoBase * 0.01m : 0m;
                decimal descuentoFaltas = ingresoBase * Math.Min(faltas * 0.02m, 0.10m);
                decimal descuentoTardanzas = ingresoBase * Math.Min(tardanzas * 0.005m, 0.03m);

                return Math.Round(bonoAntiguedad + bonoAsistencia - descuentoFaltas - descuentoTardanzas, 2);
            }
        }
```

Services/AdvancedNominaRules.cs (continuacion)

```

        private static DateTime ObtenerFechaIngreso(SQLiteConnection connection, int idEmpleado)
        {
            using (SQLiteCommand command = new SQLiteCommand(
                "SELECT FechaIngreso FROM Empleados WHERE IdEmpleado = @id AND Estado = 'Activo';", connection))
            {
                command.Parameters.AddWithValue("@id", idEmpleado);
                object result = command.ExecuteScalar();
                if (result == null || result == DBNull.Value)
                {
                    throw new ArgumentException("No se encontró un empleado activo para calcular el ajuste.");
                }

                return DateTime.Parse(Convert.ToString(result));
            }
        }

        private static void ObtenerIndicadoresAsistencia(SQLiteConnection connection, int idEmpleado, DateTime fechaInicio, DateTime fechaFin, out int faltas, out int tardanzas)
        {
            faltas = 0;
            tardanzas = 0;

            using (SQLiteCommand command = new SQLiteCommand(@"SELECT Estado, COUNT(1) AS Total
                FROM Asistencias
                WHERE IdEmpleado = @id AND Fecha BETWEEN @inicio AND @fin
                GROUP BY Estado;", connection))
            {
                command.Parameters.AddWithValue("@id", idEmpleado);
                command.Parameters.AddWithValue("@inicio", fechaInicio.ToString("yyyy-MM-dd"));
                command.Parameters.AddWithValue("@fin", fechaFin.ToString("yyyy-MM-dd"));

                using (SQLiteDataReader reader = command.ExecuteReader())
                {
                    while (reader.Read())
                    {
                        string estado = Convert.ToString(reader["Estado"]);
                        int total = Convert.ToInt32(reader["Total"]);
                        if (estado == "Falta")
                        {
                            faltas = total;
                        }
                        else if (estado == "Tardanza")
                        {
                            tardanzas = total;
                        }
                    }
                }
            }
        }
    }
}
```

Services/AdvancedReportBuilder.cs

```
using System;
using System.Data.SQLite;
using System.IO;
using System.Text;
using SistemaGestionNomina.Data;
using SistemaGestionNomina.Helpers;
using SistemaGestionNomina.Security;

namespace SistemaGestionNomina.Services
{
    /// <summary>
    /// Constructor de reportes avanzados personalizados.
    /// </summary>
    public class AdvancedReportBuilder
    {
        private readonly AuthorizationService authorizationService = new AuthorizationService();
        private readonly AuditTrailService auditTrailService = new AuditTrailService();

        /// <summary>
        /// Construye un reporte personalizado según filtros dinámicos.
        /// </summary>
        public string ConstruirReportePersonalizado(string nombreReporte)
        {
            authorizationService.DemandPermission(Permissions.ReportsView);
            if (string.IsNullOrWhiteSpace(nombreReporte))
            {
                throw new ArgumentException("Debe indicar el nombre del reporte.");
            }

            StringBuilder builder = new StringBuilder();
            builder.AppendLine("# " + nombreReporte.Trim());
            builder.AppendLine();
            builder.AppendLine("Generado: " + DateTime.Now.ToString("dd/MM/yyyy HH:mm"));
            builder.AppendLine();

            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            {
                AppendMetric(builder, connection, "Empleados activos", "SELECT COUNT(1) FROM Empleados WHERE Estado = 'Activo';");
                AppendMetric(builder, connection, "Departamentos", "SELECT COUNT(1) FROM Departamentos;");
                AppendMetric(builder, connection, "Nominas vigentes", "SELECT COUNT(1) FROM Nominas WHERE Estado <> 'Anulada';");
                AppendMetric(builder, connection, "Comprobantes generados", "SELECT COUNT(1) FROM Comprobantes;");
                AppendMetric(builder, connection, "Total neto pagado", "SELECT COALESCE(SUM(TotalNeto), 0) FROM Nominas WHERE Estado = 'Pagada';", true);

                builder.AppendLine();
                builder.AppendLine("## Nómina por departamento");
                builder.AppendLine();
                using (SQLiteCommand command = new SQLiteCommand(@"SELECT d.Nombre AS Departamento,
                    COUNT(DISTINCT e.IdEmpleado) AS Empleados,
                    COALESCE(SUM(nd.NetoPagar), 0) AS Neto
                    FROM Departamentos d
                    LEFT JOIN Empleados e ON e.IdDepartamento = d.IdDepartamento AND e.Estado = 'Activo'
                    LEFT JOIN NominaDetalle nd ON nd.IdEmpleado = e.IdEmpleado
                        AND EXISTS (SELECT 1 FROM Nominas n
                            WHERE n.IdNomina = nd.IdNomina AND n.Estado <> 'Anulada')
                    GROUP BY d.Nombre
                    ORDER BY d.Nombre;", connection))
                using (SQLiteDataReader reader = command.ExecuteReader())
                {
                    while (reader.Read())
                    {
```

Services/AdvancedReportBuilder.cs (continuacion)

```
                        builder.AppendLine("- " + Convert.ToString(reader["Departamento"]) +
                            ": " + Convert.ToInt32(reader["Empleados"]) + " empleados, B/. " +
                            Convert.ToDecimal(reader["Neto"]).ToString("0.00"));
                    }
                }

                builder.AppendLine();
                builder.AppendLine("## Asistencia reciente");
                builder.AppendLine();
                using (SQLiteCommand command = new SQLiteCommand(@"SELECT Estado, COUNT(1) AS Total
                    FROM Asistencias
                    WHERE Fecha >= date('now', '-30 day')
                    GROUP BY Estado
                    ORDER BY Estado;", connection))
                using (SQLiteDataReader reader = command.ExecuteReader())
                {
                    bool hasRows = false;
                    while (reader.Read())
                    {
                        hasRows = true;
                        builder.AppendLine("- " + Convert.ToString(reader["Estado"]) + ": " + Convert.ToInt32(reader["Total"]));
                    }

                    if (!hasRows)
                    {
                        builder.AppendLine("- Sin registros de asistencia en los últimos 30 días.");
                    }
                }
            }

            return builder.ToString();
        }

        /// <summary>
        /// Genera un reporte ejecutivo en archivo Markdown para revisión o conversión a PDF.
        /// </summary>
        public string ExportarReportePersonalizado(string nombreReporte)
        {
            authorizationService.DemandPermission(Permissions.ReportsExport);
            string content = ConstruirReportePersonalizado(nombreReporte);
            string path = PathHelper.RequestExportPath(SanitizeFileName(nombreReporte), ".md", "Archivo Markdown (*.md)|*.md");
            if (string.IsNullOrWhiteSpace(path)) return string.Empty;

            File.WriteAllText(path, content, Encoding.UTF8);
            auditTrailService.RegistrarAccion("Reportes", "Exportar personalizado", Path.GetFileName(path));
            return path;
        }

        private static void AppendMetric(StringBuilder builder, SQLiteConnection connection, string label, string sql)
        {
            AppendMetric(builder, connection, label, sql, false);
        }

        private static void AppendMetric(StringBuilder builder, SQLiteConnection connection, string label, string sql, bool currency)
        {
            using (SQLiteCommand command = new SQLiteCommand(sql, connection))
            {
                object value = command.ExecuteScalar();
                if (currency)
                {
```

Services/AdvancedReportBuilder.cs (continuacion)

```
                    builder.AppendLine("- " + label + ": B/. " + Convert.ToDecimal(value).ToString("0.00"));
                }
                else
                {
                    builder.AppendLine("- " + label + ": " + Convert.ToInt32(value));
                }
            }
        }

        private static string SanitizeFileName(string value)
        {
            string safe = string.IsNullOrWhiteSpace(value) ? "ReporteAvanzado" : value.Trim();
            foreach (char c in Path.GetInvalidFileNameChars())
            {
                safe = safe.Replace(c, '_');
            }

            return safe.Replace(' ', '_');
        }
    }
}
```

Services/AttendanceDeviceImportService.cs

```
using System;
using System.Collections.Generic;
using System.Data.SQLite;
using System.Globalization;
using System.IO;
using SistemaGestionNomina.Data;
using SistemaGestionNomina.Security;

namespace SistemaGestionNomina.Services
{
    /// <summary>
    /// Servicio destinado a importar asistencia desde reloj marcador o archivos externos.
    /// </summary>
    public class AttendanceDeviceImportService
    {
        private readonly AuthorizationService authorizationService = new AuthorizationService();
        private readonly AuditTrailService auditTrailService = new AuditTrailService();

        /// <summary>
        /// Importa registros externos de asistencia desde CSV: Codigo,Fecha,HoraEntrada,HoraSalida,Estado.
        /// </summary>
        public int ImportarDesdeArchivo(string rutaArchivo)
        {
            authorizationService.DemandPermission(Permissions.AttendanceImport);
            if (string.IsNullOrWhiteSpace(rutaArchivo))
            {
                throw new ArgumentException("Debe indicar el archivo de asistencia.");
            }

            if (!File.Exists(rutaArchivo))
            {
                throw new FileNotFoundException("No se encontró el archivo de asistencia.", rutaArchivo);
            }

            string[] lines = File.ReadAllLines(rutaArchivo);
            if (lines.Length == 0)
            {
                return 0;
            }

            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteTransaction transaction = connection.BeginTransaction())
            {
                Dictionary<string, int> empleados = CargarEmpleados(connection, transaction);
                int imported = 0;

                for (int i = 0; i < lines.Length; i++)
                {
                    if (string.IsNullOrWhiteSpace(lines[i]))
                    {
                        continue;
                    }

                    string[] values = SplitCsvLine(lines[i]);
                    if (i == 0 && values.Length > 0 && values[0].Trim().Equals("Codigo", StringComparison.OrdinalIgnoreCase))
                    {
                        continue;
                    }

                    if (values.Length < 5)
```

Services/AttendanceDeviceImportService.cs (continuacion)

```
                    {
                        throw new InvalidDataException("Formato inválido en línea " + (i + 1) + ". Use Codigo,Fecha,HoraEntrada,HoraSalida,Estado.");
                    }

                    string codigo = values[0].Trim();
                    if (!empleados.ContainsKey(codigo))
                    {
                        throw new InvalidDataException("Empleado no encontrado en línea " + (i + 1) + ": " + codigo);
                    }

                    DateTime fecha = ParseDate(values[1], i + 1);
                    TimeSpan? entrada = ParseTime(values[2], i + 1, false);
                    TimeSpan? salida = ParseTime(values[3], i + 1, false);
                    string estado = NormalizeStatus(values[4], i + 1);
                    decimal horas = CalculateHours(estado, entrada, salida, i + 1);

                    UpsertAsistencia(connection, transaction, empleados[codigo], fecha, entrada, salida, horas, estado);
                    imported++;
                }

                transaction.Commit();
                auditTrailService.RegistrarAccion("Asistencia", "Importar",
                    Path.GetFileName(rutaArchivo) + ", Registros=" + imported);
                return imported;
            }
        }

        private static Dictionary<string, int> CargarEmpleados(SQLiteConnection connection, SQLiteTransaction transaction)
        {
            Dictionary<string, int> empleados = new Dictionary<string, int>(StringComparer.OrdinalIgnoreCase);
            using (SQLiteCommand command = new SQLiteCommand("SELECT IdEmpleado, Codigo FROM Empleados WHERE Estado = 'Activo';", connection, transaction))
            using (SQLiteDataReader reader = command.ExecuteReader())
            {
                while (reader.Read())
                {
                    empleados[Convert.ToString(reader["Codigo"])] = Convert.ToInt32(reader["IdEmpleado"]);
                }
            }

            return empleados;
        }

        private static string[] SplitCsvLine(string line)
        {
            List<string> values = new List<string>();
            bool insideQuotes = false;
            string current = string.Empty;

            for (int i = 0; i < line.Length; i++)
            {
                char c = line[i];
                if (c == '"')
                {
                    insideQuotes = !insideQuotes;
                }
                else if (c == ',' && !insideQuotes)
                {
                    values.Add(current);
                    current = string.Empty;
                }
```

Services/AttendanceDeviceImportService.cs (continuacion)

```
                else
                {
                    current += c;
                }
            }

            values.Add(current);
            return values.ToArray();
        }

        private static DateTime ParseDate(string value, int lineNumber)
        {
            DateTime date;
            string text = (value ?? string.Empty).Trim();
            string[] formats = { "yyyy-MM-dd", "dd/MM/yyyy", "d/M/yyyy", "MM/dd/yyyy" };
            if (DateTime.TryParseExact(text, formats, CultureInfo.InvariantCulture, DateTimeStyles.None, out date) ||
                DateTime.TryParse(text, out date))
            {
                return date.Date;
            }

            throw new InvalidDataException("Fecha inválida en línea " + lineNumber + ".");
        }

        private static TimeSpan? ParseTime(string value, int lineNumber, bool required)
        {
            string text = (value ?? string.Empty).Trim();
            if (text.Length == 0)
            {
                if (required)
                {
                    throw new InvalidDataException("Hora obligatoria en línea " + lineNumber + ".");
                }

                return null;
            }

            TimeSpan time;
            if (TimeSpan.TryParse(text, out time))
            {
                return time;
            }

            DateTime dateTime;
            if (DateTime.TryParse(text, out dateTime))
            {
                return dateTime.TimeOfDay;
            }

            throw new InvalidDataException("Hora inválida en línea " + lineNumber + ".");
        }

        private static string NormalizeStatus(string value, int lineNumber)
        {
            string status = (value ?? string.Empty).Trim();
            string[] allowed = { "Puntual", "Tardanza", "Falta", "Permiso" };
            for (int i = 0; i < allowed.Length; i++)
            {
                if (string.Equals(status, allowed[i], StringComparison.OrdinalIgnoreCase))
                {
```

Services/AttendanceDeviceImportService.cs (continuacion)

```
                    return allowed[i];
                }
            }

            throw new InvalidDataException("Estado inválido en línea " + lineNumber + ". Use Puntual, Tardanza, Falta o Permiso.");
        }

        private static decimal CalculateHours(string estado, TimeSpan? entrada, TimeSpan? salida, int lineNumber)
        {
            if (estado == "Falta" || estado == "Permiso")
            {
                return 0m;
            }

            if (!entrada.HasValue || !salida.HasValue)
            {
                throw new InvalidDataException("Debe indicar entrada y salida en línea " + lineNumber + ".");
            }

            if (salida.Value <= entrada.Value)
            {
                throw new InvalidDataException("La salida debe ser mayor que la entrada en línea " + lineNumber + ".");
            }

            return Math.Round(Convert.ToDecimal((salida.Value - entrada.Value).TotalHours), 2);
        }

        private static void UpsertAsistencia(SQLiteConnection connection, SQLiteTransaction transaction, int idEmpleado, DateTime fecha, TimeSpan?  \
    entrada, TimeSpan? salida, decimal horas, string estado)
        {
            object existingId;
            using (SQLiteCommand find = new SQLiteCommand(
                "SELECT IdAsistencia FROM Asistencias WHERE IdEmpleado = @empleado AND Fecha = @fecha LIMIT 1;", connection, transaction))
            {
                find.Parameters.AddWithValue("@empleado", idEmpleado);
                find.Parameters.AddWithValue("@fecha", fecha.ToString("yyyy-MM-dd"));
                existingId = find.ExecuteScalar();
            }

            string entradaText = entrada.HasValue ? entrada.Value.ToString(@"hh\:mm") : null;
            string salidaText = salida.HasValue ? salida.Value.ToString(@"hh\:mm") : null;

            if (existingId == null || existingId == DBNull.Value)
            {
                using (SQLiteCommand insert = new SQLiteCommand(@"INSERT INTO Asistencias
                    (IdEmpleado, Fecha, HoraEntrada, HoraSalida, HorasTrabajadas, Estado)
                    VALUES (@empleado, @fecha, @entrada, @salida, @horas, @estado);", connection, transaction))
                {
                    FillParameters(insert, idEmpleado, fecha, entradaText, salidaText, horas, estado);
                    insert.ExecuteNonQuery();
                }
            }
            else
            {
                using (SQLiteCommand update = new SQLiteCommand(@"UPDATE Asistencias SET
                    HoraEntrada = @entrada, HoraSalida = @salida, HorasTrabajadas = @horas, Estado = @estado
                    WHERE IdAsistencia = @id;", connection, transaction))
                {
                    FillParameters(update, idEmpleado, fecha, entradaText, salidaText, horas, estado);
                    update.Parameters.AddWithValue("@id", Convert.ToInt32(existingId));
```

Services/AttendanceDeviceImportService.cs (continuacion)

```
                    update.ExecuteNonQuery();
                }
            }
        }

        private static void FillParameters(SQLiteCommand command, int idEmpleado, DateTime fecha, string entrada, string salida, decimal horas, string estado)
        {
            command.Parameters.AddWithValue("@empleado", idEmpleado);
            command.Parameters.AddWithValue("@fecha", fecha.ToString("yyyy-MM-dd"));
            command.Parameters.AddWithValue("@entrada", (object)entrada ?? DBNull.Value);
            command.Parameters.AddWithValue("@salida", (object)salida ?? DBNull.Value);
            command.Parameters.AddWithValue("@horas", horas);
            command.Parameters.AddWithValue("@estado", estado);
        }
    }
}
```

Services/AuditTrailService.cs

```
using System;
using System.Collections.Generic;
using SistemaGestionNomina.Data;
using SistemaGestionNomina.Models;
using SistemaGestionNomina.Security;

namespace SistemaGestionNomina.Services
{
    public class AuditTrailService
    {
        private readonly AuditRepository repository = new AuditRepository();

        public void RegistrarAccion(string modulo, string accion, string detalle)
        {
            RegistrarCambio(SessionContext.IsAuthenticated ? SessionContext.Username : "sistema",
                modulo, accion, detalle);
        }

        public void RegistrarCambio(string usuario, string modulo, string accion)
        {
            RegistrarCambio(usuario, modulo, accion, string.Empty);
        }

        public void RegistrarCambio(string usuario, string modulo, string accion, string detalle)
        {
            repository.Add(new AuditRecord
            {
                Usuario = Normalize(usuario, "sistema"),
                Modulo = Normalize(modulo, "General"),
                Accion = Normalize(accion, "Acción"),
                Detalle = Normalize(detalle, string.Empty),
                Fecha = DateTime.Now
            });
        }

        public List<AuditRecord> Buscar(AuditQuery query)
        {
            return repository.Search(query ?? new AuditQuery());
        }

        public List<string> ObtenerModulos()
        {
            return repository.GetDistinct("Modulo");
        }

        public List<string> ObtenerAcciones()
        {
            return repository.GetDistinct("Accion");
        }

        public List<string> ObtenerUltimos(int cantidad)
        {
            List<string> items = new List<string>();
            List<AuditRecord> records = repository.Search(new AuditQuery
            {
                Limit = cantidad <= 0 ? 20 : Math.Min(cantidad, 200)
            });
            for (int i = 0; i < records.Count; i++)
            {
                AuditRecord record = records[i];
```

Services/AuditTrailService.cs (continuacion)

```
                items.Add(record.Fecha.ToString("yyyy-MM-dd HH:mm:ss") + " | " + record.Usuario + " | " +
                    record.Modulo + " | " + record.Accion + " | " + record.Detalle);
            }
            return items;
        }

        private static string Normalize(string value, string fallback)
        {
            return string.IsNullOrWhiteSpace(value) ? fallback : value.Trim();
        }
    }
}
```

Services/BackupService.cs

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Security.Cryptography;
using System.Text;
using SistemaGestionNomina.Data;
using SistemaGestionNomina.Security;

namespace SistemaGestionNomina.Services
{
    /// <summary>
    /// Servicio reservado para crear copias de seguridad de la base de datos.
    /// </summary>
    public class BackupService
    {
        private readonly AuditTrailService auditTrailService = new AuditTrailService();
        private readonly AuthorizationService authorizationService = new AuthorizationService();

        /// <summary>
        /// Crea una copia de seguridad de nomina.db en la ruta indicada.
        /// </summary>
        public string CrearBackup(string carpetaDestino)
        {
            authorizationService.DemandPermission(Permissions.BackupsManage);
            string source = SQLiteConnectionFactory.DatabasePath;
            if (!File.Exists(source))
            {
                throw new FileNotFoundException("No se encontró la base de datos para respaldar.", source);
            }

            string destinationFolder = string.IsNullOrWhiteSpace(carpetaDestino)
                ? Path.Combine(AppDomain.CurrentDomain.BaseDirectory, "Backups")
                : carpetaDestino;

            Directory.CreateDirectory(destinationFolder);
            string destination = Path.Combine(destinationFolder, "nomina_" + DateTime.Now.ToString("yyyyMMdd_HHmmss") + ".db");
            File.Copy(source, destination, false);

            string hash = ComputeSha256(destination);
            File.WriteAllText(destination + ".sha256", hash, Encoding.ASCII);
            auditTrailService.RegistrarAccion("Backup", "Crear", Path.GetFileName(destination));
            return destination;
        }

        /// <summary>
        /// Lista respaldos disponibles ordenados del más reciente al más antiguo.
        /// </summary>
        public List<string> ListarBackups(string carpeta)
        {
            authorizationService.DemandPermission(Permissions.BackupsManage);
            string folder = string.IsNullOrWhiteSpace(carpeta)
                ? Path.Combine(AppDomain.CurrentDomain.BaseDirectory, "Backups")
                : carpeta;

            List<string> backups = new List<string>();
            if (!Directory.Exists(folder))
            {
                return backups;
            }

```

Services/BackupService.cs (continuacion)

```
            string[] files = Directory.GetFiles(folder, "nomina_*.db");
            Array.Sort(files);
            Array.Reverse(files);
            backups.AddRange(files);
            return backups;
        }

        /// <summary>
        /// Verifica que el hash del backup coincida con su archivo .sha256.
        /// </summary>
        public bool VerificarBackup(string rutaBackup)
        {
            authorizationService.DemandPermission(Permissions.BackupsManage);
            if (string.IsNullOrWhiteSpace(rutaBackup) || !File.Exists(rutaBackup))
            {
                return false;
            }

            string hashFile = rutaBackup + ".sha256";
            if (!File.Exists(hashFile))
            {
                return false;
            }

            string expected = File.ReadAllText(hashFile).Trim();
            return string.Equals(expected, ComputeSha256(rutaBackup), StringComparison.OrdinalIgnoreCase);
        }

        private static string ComputeSha256(string path)
        {
            using (SHA256 sha = SHA256.Create())
            using (FileStream stream = File.OpenRead(path))
            {
                byte[] hash = sha.ComputeHash(stream);
                StringBuilder builder = new StringBuilder();
                for (int i = 0; i < hash.Length; i++)
                {
                    builder.Append(hash[i].ToString("x2"));
                }

                return builder.ToString();
            }
        }
    }
}
```

Services/BusinessServices.cs

```
using System;
using System.Collections.Generic;
using SistemaGestionNomina.Data;
using SistemaGestionNomina.Helpers;
using SistemaGestionNomina.Models;
using SistemaGestionNomina.Security;

namespace SistemaGestionNomina.Services
{
    public class AuthService
    {
        private const int MaximumAttempts = 5;
        private static readonly TimeSpan LockDuration = TimeSpan.FromMinutes(15);
        private readonly AuthenticationRepository authenticationRepository = new AuthenticationRepository();
        private readonly AuditTrailService auditTrailService = new AuditTrailService();

        public AuthenticationResult Authenticate(string username, string password)
        {
            string normalizedUsername = (username ?? string.Empty).Trim();
            if (normalizedUsername.Length == 0 || string.IsNullOrEmpty(password))
            {
                return AuthenticationResult.Invalid();
            }

            Usuario usuario = authenticationRepository.GetByUsername(normalizedUsername);
            if (usuario == null || !string.Equals(usuario.Estado, "Activo", StringComparison.OrdinalIgnoreCase) ||
                !Roles.IsValid(usuario.Rol))
            {
                auditTrailService.RegistrarCambio(normalizedUsername, "Seguridad", "Inicio de sesión fallido",
                    "Credenciales no válidas.");
                return AuthenticationResult.Invalid();
            }

            DateTime now = DateTime.Now;
            if (usuario.Bloqueado && usuario.FechaBloqueo.HasValue)
            {
                DateTime blockedUntil = usuario.FechaBloqueo.Value.Add(LockDuration);
                if (now < blockedUntil)
                {
                    auditTrailService.RegistrarCambio(normalizedUsername, "Seguridad", "Acceso bloqueado",
                        "Intento durante bloqueo temporal.");
                    return AuthenticationResult.Blocked(blockedUntil);
                }

                authenticationRepository.RegisterFailedAttempt(usuario.IdUsuario, 0, false, null);
                usuario.IntentosFallidos = 0;
                usuario.Bloqueado = false;
                usuario.FechaBloqueo = null;
            }

            if (!PasswordHelper.Verify(password, usuario.PasswordHash))
            {
                int attempts = usuario.IntentosFallidos + 1;
                bool blocked = attempts >= MaximumAttempts;
                DateTime? blockedAt = blocked ? now : (DateTime?)null;
                authenticationRepository.RegisterFailedAttempt(usuario.IdUsuario, attempts, blocked, blockedAt);
                auditTrailService.RegistrarCambio(normalizedUsername, "Seguridad",
                    blocked ? "Usuario bloqueado" : "Inicio de sesión fallido",
                    blocked ? "Se alcanzó el máximo de intentos." : "Intento " + attempts + " de " + MaximumAttempts + ".");
                return blocked ? AuthenticationResult.Blocked(now.Add(LockDuration)) : AuthenticationResult.Invalid();
```

Services/BusinessServices.cs (continuacion)

```
            }

            bool upgradeLegacyHash = PasswordHelper.IsLegacyHash(usuario.PasswordHash);
            string upgradedHash = upgradeLegacyHash ? PasswordHelper.HashPassword(password) : string.Empty;
            authenticationRepository.CompleteLogin(usuario.IdUsuario, now, upgradedHash);
            if (upgradeLegacyHash) usuario.PasswordHash = upgradedHash;
            usuario.UltimoAcceso = now;
            usuario.IntentosFallidos = 0;
            usuario.Bloqueado = false;
            usuario.FechaBloqueo = null;
            SessionContext.Begin(usuario);
            auditTrailService.RegistrarCambio(usuario.NombreUsuario, "Seguridad", "Inicio de sesión correcto",
                upgradeLegacyHash ? "Contraseña actualizada a PBKDF2." : string.Empty);
            return AuthenticationResult.Successful(usuario);
        }
    }

    public class EmpleadoService
    {
        private readonly EmpleadoRepository empleadoRepository = new EmpleadoRepository();
        private readonly DepartamentoRepository departamentoRepository = new DepartamentoRepository();
        private readonly AuditTrailService auditTrailService = new AuditTrailService();
        private readonly AuthorizationService authorizationService = new AuthorizationService();
        private readonly EmployeeScopeService employeeScopeService = new EmployeeScopeService();
        private readonly EmployeeHistoryService employeeHistoryService = new EmployeeHistoryService();

        public List<Empleado> GetAll(string search, int? departmentId, string status)
        {
            authorizationService.DemandPermission(Permissions.EmployeesView);
            employeeHistoryService.ApplyDueChanges(DateTime.Today);
            return empleadoRepository.GetAll(search, departmentId, status, employeeScopeService.GetDepartmentScope());
        }

        public List<Empleado> GetActive(int? departmentId)
        {
            authorizationService.DemandAny(Permissions.EmployeesView, Permissions.AttendanceView,
                Permissions.PayrollView, Permissions.ReportsPersonal);
            employeeHistoryService.ApplyDueChanges(DateTime.Today);
            int? scope = employeeScopeService.GetDepartmentScope();
            return empleadoRepository.GetActiveByDepartment(scope ?? departmentId, scope);
        }

        public Empleado GetById(int id)
        {
            authorizationService.DemandPermission(Permissions.EmployeesView);
            employeeScopeService.DemandEmployeeInScope(id);
            employeeHistoryService.ApplyDueChanges(DateTime.Today);
            return empleadoRepository.GetById(id);
        }

        public List<Departamento> GetDepartments()
        {
            authorizationService.DemandAny(Permissions.EmployeesView, Permissions.AttendanceView,
                Permissions.PayrollView);
            return departamentoRepository.GetAll();
        }

        public int Save(Empleado empleado)
        {
            return Save(empleado, null).EmployeeId;
```

Services/BusinessServices.cs (continuacion)

```
        }

        public EmployeeSaveResult Save(Empleado empleado, EmployeeChangeContext changeContext)
        {
            if (empleado == null)
            {
                throw new ArgumentException("Debe indicar los datos del empleado.");
            }

            authorizationService.DemandPermission(empleado.IdEmpleado == 0
                ? Permissions.EmployeesCreate : Permissions.EmployeesEdit);

            if (empleado.IdDepartamento <= 0)
            {
                throw new ArgumentException("Debe seleccionar un departamento válido.");
            }

            if (empleado.SalarioBase <= 0)
            {
                throw new ArgumentException("El salario base debe ser mayor que cero.");
            }

            if (empleadoRepository.ExistsCode(empleado.Codigo, empleado.IdEmpleado))
            {
                throw new InvalidOperationException("Ya existe un empleado con ese código.");
            }

            if (empleadoRepository.ExistsCedula(empleado.Cedula, empleado.IdEmpleado))
            {
                throw new InvalidOperationException("Ya existe un empleado con esa cédula.");
            }

            if (empleado.IdEmpleado == 0)
            {
                empleado.FechaEfectivaLaboral = empleado.FechaIngreso;
                int id = empleadoRepository.Add(empleado);
                auditTrailService.RegistrarAccion("Empleados", "Crear", empleado.Codigo);
                return new EmployeeSaveResult
                {
                    EmployeeId = id,
                    AppliedImmediately = true
                };
            }

            EmployeeSaveResult result = employeeHistoryService.SaveEmployee(empleado, changeContext);
            if (result.HistoryRecords == 0)
                auditTrailService.RegistrarAccion("Empleados", "Actualizar", empleado.Codigo);
            return result;
        }

        public void Deactivate(int id)
        {
            throw new ArgumentException("Debe indicar la fecha efectiva y el motivo de desactivación.");
        }

        public EmployeeSaveResult Deactivate(int id, EmployeeChangeContext changeContext)
        {
            authorizationService.DemandPermission(Permissions.EmployeesDeactivate);
            Empleado employee = empleadoRepository.GetById(id);
            if (employee == null) throw new InvalidOperationException("El empleado no existe.");
```

Services/BusinessServices.cs (continuacion)

```
            employee.Estado = "Inactivo";
            return employeeHistoryService.SaveEmployee(employee, changeContext);
        }
    }

    public class AsistenciaService
    {
        private readonly AsistenciaRepository asistenciaRepository = new AsistenciaRepository();
        private readonly AuditTrailService auditTrailService = new AuditTrailService();
        private readonly AuthorizationService authorizationService = new AuthorizationService();
        private readonly PayrollPeriodPolicyService periodPolicyService = new PayrollPeriodPolicyService();
        private readonly EmployeeScopeService employeeScopeService = new EmployeeScopeService();

        public int Register(Asistencia asistencia)
        {
            if (asistencia == null)
            {
                throw new ArgumentException("Debe indicar los datos de asistencia.");
            }

            authorizationService.DemandPermission(Permissions.AttendanceRegister);
            employeeScopeService.DemandEmployeeInScope(asistencia.IdEmpleado);
            periodPolicyService.VerificarFechasAbiertas(asistencia.Fecha.Date, asistencia.Fecha.Date);

            if (asistenciaRepository.Exists(asistencia.IdEmpleado, asistencia.Fecha.Date))
            {
                throw new InvalidOperationException("Ya existe una asistencia para el empleado en esa fecha.");
            }

            if (asistencia.Estado == "Falta" || asistencia.Estado == "Permiso")
            {
                asistencia.HoraEntrada = null;
                asistencia.HoraSalida = null;
                asistencia.HorasTrabajadas = 0;
            }
            else if (asistencia.HoraEntrada.HasValue && asistencia.HoraSalida.HasValue)
            {
                // Para asistencia valida, la salida debe cerrar una jornada real.
                if (asistencia.HoraSalida.Value <= asistencia.HoraEntrada.Value)
                {
                    throw new InvalidOperationException("La hora de salida debe ser mayor que la hora de entrada.");
                }

                asistencia.HorasTrabajadas = Convert.ToDecimal((asistencia.HoraSalida.Value - asistencia.HoraEntrada.Value).TotalHours);
            }
            else
            {
                throw new InvalidOperationException("Debe registrar hora de entrada y salida.");
            }

            int id = asistenciaRepository.Add(asistencia);
            auditTrailService.RegistrarAccion("Asistencia", "Registrar",
                "IdEmpleado=" + asistencia.IdEmpleado + ", Fecha=" + asistencia.Fecha.ToString("yyyy-MM-dd"));
            return id;
        }

        public List<Asistencia> GetAll(DateTime? start, DateTime? end, int? employeeId, string status)
        {
            authorizationService.DemandPermission(Permissions.AttendanceView);
            if (start.HasValue && end.HasValue && start.Value.Date > end.Value.Date)
```

Services/BusinessServices.cs (continuacion)

```
            {
                throw new ArgumentException("La fecha inicial no puede ser mayor que la fecha final.");
            }

            if (employeeId.HasValue) employeeScopeService.DemandEmployeeInScope(employeeId.Value);
            return asistenciaRepository.GetAll(start, end, employeeId, status,
                employeeScopeService.GetDepartmentScope());
        }
    }

    public class ConfiguracionService
    {
        private readonly ConfiguracionRepository configuracionRepository = new ConfiguracionRepository();
        private readonly AuthorizationService authorizationService = new AuthorizationService();
        private readonly AuditTrailService auditTrailService = new AuditTrailService();

        public List<ConfiguracionNomina> GetAll()
        {
            authorizationService.DemandPermission(Permissions.ConfigurationView);
            return configuracionRepository.GetAll();
        }

        public decimal GetValue(string name, decimal fallback)
        {
            authorizationService.DemandPermission(Permissions.ConfigurationView);
            return configuracionRepository.GetValue(name, fallback);
        }

        public void SaveDefaults(Dictionary<string, decimal> values)
        {
            authorizationService.DemandPermission(Permissions.ConfigurationEdit);
            if (values == null || !values.ContainsKey("SeguroSocial") || !values.ContainsKey("ISR") ||
                !values.ContainsKey("SeguroEducativo") || !values.ContainsKey("RecargoHoraExtra") ||
                !values.ContainsKey("HorasMensualesBase"))
            {
                throw new ArgumentException("La configuración de nómina está incompleta.");
            }
            configuracionRepository.Save("SeguroSocial", values["SeguroSocial"], "Porcentaje académico de seguro social.");
            configuracionRepository.Save("ISR", values["ISR"], "Porcentaje académico de impuesto sobre la renta.");
            configuracionRepository.Save("SeguroEducativo", values["SeguroEducativo"], "Porcentaje académico de seguro educativo.");
            configuracionRepository.Save("RecargoHoraExtra", values["RecargoHoraExtra"], "Multiplicador académico de horas extra.");
            configuracionRepository.Save("HorasMensualesBase", values["HorasMensualesBase"], "Horas mensuales base para cálculo por hora.");
            auditTrailService.RegistrarAccion("Configuración", "Actualizar", "Parámetros de nómina.");
        }
    }

    public class NominaService
    {
        private readonly EmpleadoRepository empleadoRepository = new EmpleadoRepository();
        private readonly AsistenciaRepository asistenciaRepository = new AsistenciaRepository();
        private readonly ConfiguracionRepository configuracionRepository = new ConfiguracionRepository();
        private readonly NominaRepository nominaRepository = new NominaRepository();
        private readonly ComprobanteRepository comprobanteRepository = new ComprobanteRepository();
        private readonly AuditTrailService auditTrailService = new AuditTrailService();
        private readonly AuthorizationService authorizationService = new AuthorizationService();
        private readonly EmployeeEffectiveDataService effectiveEmployeeService = new EmployeeEffectiveDataService();
        private readonly PayrollPeriodPolicyService periodPolicyService = new PayrollPeriodPolicyService();

        public Nomina CalcularNomina(DateTime fechaInicio, DateTime fechaFin, int? departamentoId)
        {
```

Services/BusinessServices.cs (continuacion)

```
            authorizationService.DemandPermission(Permissions.PayrollCalculate);
            if (fechaInicio.Date > fechaFin.Date)
            {
                throw new InvalidOperationException("La fecha de inicio debe ser menor o igual que la fecha fin.");
            }

            periodPolicyService.VerificarFechasAbiertas(fechaInicio.Date, fechaFin.Date);

            List<Empleado> empleados = effectiveEmployeeService.GetActiveEmployeesAsOf(fechaFin.Date, departamentoId);
            if (empleados.Count == 0)
            {
                throw new InvalidOperationException("No hay empleados activos para calcular la nómina.");
            }

            decimal seguroSocial = configuracionRepository.GetValue("SeguroSocial", 9.75m);
            decimal isr = configuracionRepository.GetValue("ISR", 10m);
            decimal seguroEducativo = configuracionRepository.GetValue("SeguroEducativo", 1.25m);
            decimal recargoHoraExtra = configuracionRepository.GetValue("RecargoHoraExtra", 1.25m);
            decimal horasMensualesBase = configuracionRepository.GetValue("HorasMensualesBase", 160m);

            Nomina nomina = new Nomina();
            nomina.FechaCalculo = DateTime.Now;
            nomina.PeriodoNombre = "Nómina " + fechaInicio.ToString("dd/MM/yyyy") + " - " + fechaFin.ToString("dd/MM/yyyy");
            nomina.Estado = PayrollStates.Calculated;

            foreach (Empleado empleado in empleados)
            {
                // Formula academica: sueldo base + bono + horas extra - deducciones configurables.
                decimal horasExtra = asistenciaRepository.GetExtraHours(empleado.IdEmpleado, fechaInicio, fechaFin);
                decimal pagoHora = horasMensualesBase <= 0 ? 0 : empleado.SalarioBase / horasMensualesBase;
                decimal montoHorasExtra = Math.Round(horasExtra * pagoHora * recargoHoraExtra, 2);
                decimal bonos = Math.Round(empleado.SalarioBase * 0.03m, 2);
                decimal totalIngresos = empleado.SalarioBase + bonos + montoHorasExtra;
                decimal totalDeducciones = Math.Round(
                    totalIngresos * (seguroSocial / 100m) +
                    totalIngresos * (isr / 100m) +
                    totalIngresos * (seguroEducativo / 100m), 2);
                decimal neto = totalIngresos - totalDeducciones;

                NominaDetalle detalle = new NominaDetalle();
                detalle.IdEmpleado = empleado.IdEmpleado;
                detalle.CodigoEmpleado = empleado.Codigo;
                detalle.EmpleadoNombre = empleado.NombreCompleto;
                detalle.CargoEmpleado = empleado.Cargo;
                detalle.Departamento = empleado.DepartamentoNombre;
                detalle.SueldoBase = empleado.SalarioBase;
                detalle.Bonos = bonos;
                detalle.HorasExtra = horasExtra;
                detalle.MontoHorasExtra = montoHorasExtra;
                detalle.TotalIngresos = totalIngresos;
                detalle.TotalDeducciones = totalDeducciones;
                detalle.NetoPagar = neto;
                nomina.Detalles.Add(detalle);

                nomina.TotalIngresos += totalIngresos;
                nomina.TotalDeducciones += totalDeducciones;
                nomina.TotalNeto += neto;
            }

            auditTrailService.RegistrarAccion("Nómina", "Calcular",
```

Services/BusinessServices.cs (continuacion)

```
                fechaInicio.ToString("yyyy-MM-dd") + " a " + fechaFin.ToString("yyyy-MM-dd") +
                ", Empleados=" + nomina.Detalles.Count);

            return nomina;
        }

        private readonly PayrollLifecycleService payrollLifecycleService = new PayrollLifecycleService();

        public int ConfirmarPago(Nomina nomina, DateTime fechaInicio, DateTime fechaFin)
        {
            return payrollLifecycleService.Confirmar(nomina, fechaInicio, fechaFin);
        }

        public void PagarNomina(int idNomina)
        {
            payrollLifecycleService.Pagar(idNomina);
        }

        public void AnularNomina(int idNomina, string motivo)
        {
            payrollLifecycleService.Anular(idNomina, motivo);
        }

        public int RecalcularNomina(int idNominaAnulada, DateTime fechaInicio, DateTime fechaFin, Nomina nominaRecalculada)
        {
            return payrollLifecycleService.Recalcular(idNominaAnulada, fechaInicio, fechaFin, nominaRecalculada);
        }

        public List<NominaVersion> ObtenerHistorialNomina(int idNomina)
        {
            return payrollLifecycleService.ObtenerHistorial(idNomina);
        }

        public List<Nomina> GetNominas()
        {
            authorizationService.DemandPermission(Permissions.PayrollView);
            return nominaRepository.GetAll();
        }

        public List<NominaDetalle> GetDetalles(int idNomina)
        {
            authorizationService.DemandPermission(Permissions.PayrollView);
            return nominaRepository.GetDetalles(idNomina);
        }
    }

    public class ComprobanteService
    {
        private readonly ComprobanteRepository comprobanteRepository = new ComprobanteRepository();
        private readonly AuditTrailService auditTrailService = new AuditTrailService();
        private readonly AuthorizationService authorizationService = new AuthorizationService();

        public List<Comprobante> GetAll(string search)
        {
            authorizationService.DemandPermission(Permissions.PayslipsView);
            return comprobanteRepository.GetAll(search);
        }

        public Comprobante GetById(int id)
        {
```

Services/BusinessServices.cs (continuacion)

```
            authorizationService.DemandPermission(Permissions.PayslipsView);
            return comprobanteRepository.GetById(id);
        }

        public void SaveRutaPdf(int idComprobante, string ruta)
        {
            authorizationService.DemandPermission(Permissions.PayslipsExport);
            comprobanteRepository.UpdateRutaPdf(idComprobante, ruta);
            auditTrailService.RegistrarAccion("Comprobantes", "Guardar PDF",
                "IdComprobante=" + idComprobante);
        }
    }

    public class ReporteService
    {
        private readonly ReporteRepository reporteRepository = new ReporteRepository();
        private readonly AuditTrailService auditTrailService = new AuditTrailService();
        private readonly AuthorizationService authorizationService = new AuthorizationService();

        public List<ReporteGenerado> GetAll()
        {
            authorizationService.DemandPermission(Permissions.ReportsView);
            List<ReporteGenerado> reports = reporteRepository.GetAll();
            if (string.Equals(SessionContext.Role, Roles.Admin, StringComparison.OrdinalIgnoreCase)) return reports;

            List<ReporteGenerado> filtered = new List<ReporteGenerado>();
            bool personalOnly = authorizationService.HasPermission(Permissions.ReportsPersonal) &&
                !authorizationService.HasPermission(Permissions.ReportsFinancial);
            for (int i = 0; i < reports.Count; i++)
            {
                bool isPersonal = (reports[i].Tipo ?? string.Empty).IndexOf("Personal",
                    StringComparison.OrdinalIgnoreCase) >= 0;
                if ((personalOnly && isPersonal) || (!personalOnly && !isPersonal)) filtered.Add(reports[i]);
            }
            return filtered;
        }

        public ReporteGenerado Register(string name, string type, string path)
        {
            authorizationService.DemandPermission(Permissions.ReportsExport);
            ReporteGenerado reporte = new ReporteGenerado();
            reporte.NombreReporte = name;
            reporte.Tipo = type;
            reporte.GeneradoPor = SessionContext.Username;
            reporte.FechaGeneracion = DateTime.Now;
            reporte.RutaArchivo = path;
            reporte.IdReporte = reporteRepository.Add(reporte);
            auditTrailService.RegistrarAccion("Reportes", "Generar", name + " - " + type);
            return reporte;
        }
    }
}
```

Services/EmailService.cs

```
using System;
using System.IO;
using System.Text;
using SistemaGestionNomina.Helpers;
using SistemaGestionNomina.Security;

namespace SistemaGestionNomina.Services
{
    /// <summary>
    /// Servicio responsable de enviar comprobantes por correo electrónico.
    /// </summary>
    public class EmailService
    {
        private readonly AuthorizationService authorizationService = new AuthorizationService();
        private readonly AuditTrailService auditTrailService = new AuditTrailService();

        /// <summary>
        /// Envía un comprobante generado a un destinatario.
        /// </summary>
        public void EnviarComprobante(string destinatario, string rutaPdf)
        {
            CrearBorradorComprobante(destinatario, rutaPdf, "Comprobante de pago");
        }

        /// <summary>
        /// Crea un borrador .eml con el comprobante adjunto para abrirlo con el cliente de correo.
        /// </summary>
        public string CrearBorradorComprobante(string destinatario, string rutaPdf, string asunto)
        {
            authorizationService.DemandPermission(Permissions.PayslipsEmail);
            if (string.IsNullOrWhiteSpace(destinatario) || string.IsNullOrWhiteSpace(rutaPdf))
            {
                throw new ArgumentException("Debe indicar destinatario y ruta del comprobante.");
            }

            if (!IsValidEmail(destinatario))
            {
                throw new ArgumentException("El correo del destinatario no tiene un formato válido.");
            }

            if (!File.Exists(rutaPdf))
            {
                throw new FileNotFoundException("No se encontró el PDF del comprobante.", rutaPdf);
            }

            string fileName = "EmailComprobante_" + DateTime.Now.ToString("yyyyMMdd_HHmmss") + ".eml";
            string path = PathHelper.RequestExportPath(Path.GetFileNameWithoutExtension(fileName), ".eml", "Mensaje de correo (*.eml)|*.eml");
            if (string.IsNullOrWhiteSpace(path)) return string.Empty;

            string boundary = "----=_Nomina_" + Guid.NewGuid().ToString("N");
            string subject = string.IsNullOrWhiteSpace(asunto) ? "Comprobante de pago" : asunto.Trim();

            StringBuilder builder = new StringBuilder();
            builder.AppendLine("To: " + destinatario.Trim());
            builder.AppendLine("Subject: " + subject);
            builder.AppendLine("X-Unsent: 1");
            builder.AppendLine("MIME-Version: 1.0");
            builder.AppendLine("Content-Type: multipart/mixed; boundary=\"" + boundary + "\"");
            builder.AppendLine();
            builder.AppendLine("--" + boundary);
```

Services/EmailService.cs (continuacion)

```
            builder.AppendLine("Content-Type: text/plain; charset=utf-8");
            builder.AppendLine("Content-Transfer-Encoding: 8bit");
            builder.AppendLine();
            builder.AppendLine("Adjunto encontrará su comprobante de pago generado por el Sistema de Gestión de Nómina.");
            builder.AppendLine();
            builder.AppendLine("--" + boundary);
            builder.AppendLine("Content-Type: application/pdf; name=\"" + Path.GetFileName(rutaPdf) + "\"");
            builder.AppendLine("Content-Transfer-Encoding: base64");
            builder.AppendLine("Content-Disposition: attachment; filename=\"" + Path.GetFileName(rutaPdf) + "\"");
            builder.AppendLine();
            builder.AppendLine(ToBase64Lines(File.ReadAllBytes(rutaPdf)));
            builder.AppendLine("--" + boundary + "--");

            File.WriteAllText(path, builder.ToString(), Encoding.UTF8);
            auditTrailService.RegistrarAccion("Comprobantes", "Crear borrador de correo", destinatario.Trim());
            return path;
        }

        private static bool IsValidEmail(string email)
        {
            string value = email.Trim();
            int at = value.IndexOf('@');
            return at > 0 && at < value.Length - 3 && value.LastIndexOf('.') > at + 1;
        }

        private static string ToBase64Lines(byte[] bytes)
        {
            string base64 = Convert.ToBase64String(bytes);
            StringBuilder builder = new StringBuilder();
            for (int i = 0; i < base64.Length; i += 76)
            {
                int length = Math.Min(76, base64.Length - i);
                builder.AppendLine(base64.Substring(i, length));
            }

            return builder.ToString();
        }
    }
}
```

Services/EmployeeHistoryService.cs

```
using System;
using System.Collections.Generic;
using System.Data.SQLite;
using System.Globalization;
using SistemaGestionNomina.Data;
using SistemaGestionNomina.Models;
using SistemaGestionNomina.Security;

namespace SistemaGestionNomina.Services
{
    public sealed class EmployeeHistoryService
    {
        private readonly EmployeeHistoryRepository historyRepository = new EmployeeHistoryRepository();
        private readonly EmpleadoRepository employeeRepository = new EmpleadoRepository();
        private readonly NominaRepository payrollRepository = new NominaRepository();
        private readonly AuditRepository auditRepository = new AuditRepository();
        private readonly AuthorizationService authorizationService = new AuthorizationService();
        private readonly EmployeeScopeService employeeScopeService = new EmployeeScopeService();

        public List<HistorialEmpleado> Search(EmployeeHistoryQuery query)
        {
            authorizationService.DemandPermission(Permissions.EmployeeHistoryView);
            if (query != null && query.EmployeeId.HasValue)
                employeeScopeService.DemandEmployeeInScope(query.EmployeeId.Value);
            if (query != null && query.From.HasValue && query.To.HasValue && query.From.Value.Date > query.To.Value.Date)
                throw new ArgumentException("La fecha inicial no puede superar la fecha final.");
            return historyRepository.Search(query, employeeScopeService.GetDepartmentScope());
        }

        internal EmployeeSaveResult SaveEmployee(Empleado candidate, EmployeeChangeContext context)
        {
            Empleado original = employeeRepository.GetById(candidate.IdEmpleado);
            if (original == null) throw new InvalidOperationException("El empleado no existe.");
            employeeScopeService.DemandEmployeeInScope(candidate.IdEmpleado);

            List<HistorialEmpleado> changes = BuildChanges(original, candidate);
            if (changes.Count == 0)
            {
                candidate.FechaEfectivaLaboral = original.FechaEfectivaLaboral ?? original.FechaIngreso;
                employeeRepository.Update(candidate);
                return new EmployeeSaveResult
                {
                    EmployeeId = candidate.IdEmpleado,
                    AppliedImmediately = true
                };
            }

            if (context == null || string.IsNullOrWhiteSpace(context.Reason))
                throw new ArgumentException("Debe indicar la fecha efectiva y el motivo del cambio laboral.");

            DateTime effectiveDate = context.EffectiveDate.Date;
            if (effectiveDate == DateTime.MinValue.Date)
                throw new ArgumentException("La fecha efectiva del cambio no es válida.");
            if (effectiveDate < original.FechaIngreso.Date)
                throw new ArgumentException("La fecha efectiva no puede ser anterior al ingreso del empleado.");
            if (effectiveDate > DateTime.Today)
                authorizationService.DemandPermission(Permissions.EmployeeChangesSchedule);
            if (payrollRepository.HasClosedPeriodAffectedByEmployeeChange(effectiveDate))
                throw new InvalidOperationException("El cambio laboral afectaría un período de nómina cerrado.");

```

Services/EmployeeHistoryService.cs (continuacion)

```
            bool applyNow = effectiveDate <= DateTime.Today;
            string user = SessionContext.IsAuthenticated ? SessionContext.Username : "sistema";
            DateTime now = DateTime.Now;
            for (int i = 0; i < changes.Count; i++)
            {
                changes[i].FechaCambio = now;
                changes[i].FechaEfectiva = effectiveDate;
                changes[i].UsuarioResponsable = user;
                changes[i].Motivo = context.Reason.Trim();
                changes[i].Aplicado = applyNow;
                changes[i].FechaAplicacion = applyNow ? (DateTime?)now : null;
            }

            Empleado valueToPersist = applyNow ? candidate : PreserveTrackedValues(candidate, original);
            valueToPersist.FechaEfectivaLaboral = applyNow
                ? effectiveDate : original.FechaEfectivaLaboral ?? original.FechaIngreso;

            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteTransaction transaction = connection.BeginTransaction())
            {
                try
                {
                    // Historial, dato vigente y auditoria se guardan juntos para no dejar cambios parciales.
                    for (int i = 0; i < changes.Count; i++) historyRepository.Add(connection, transaction, changes[i]);
                    employeeRepository.Update(connection, transaction, valueToPersist);
                    auditRepository.Add(connection, transaction, new AuditRecord
                    {
                        Usuario = user,
                        Modulo = "Empleados",
                        Accion = applyNow ? "Aplicar cambio laboral" : "Programar cambio laboral",
                        Detalle = "IdEmpleado=" + candidate.IdEmpleado + ", Campos=" + changes.Count +
                            ", FechaEfectiva=" + effectiveDate.ToString("yyyy-MM-dd"),
                        Fecha = now
                    });
                    transaction.Commit();
                }
                catch
                {
                    transaction.Rollback();
                    throw;
                }
            }

            return new EmployeeSaveResult
            {
                EmployeeId = candidate.IdEmpleado,
                HistoryRecords = changes.Count,
                AppliedImmediately = applyNow,
                HasScheduledChanges = !applyNow
            };
        }

        public int ApplyDueChanges(DateTime effectiveThrough)
        {
            int applied = 0;
            DateTime now = DateTime.Now;
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteTransaction transaction = connection.BeginTransaction())
            {
                try
```

Services/EmployeeHistoryService.cs (continuacion)

```
                {
                    List<HistorialEmpleado> pending = historyRepository.GetPendingDue(
                        connection, transaction, effectiveThrough.Date);
                    // Los cambios futuros se aplican por fecha efectiva, no por la fecha en que fueron creados.
                    for (int i = 0; i < pending.Count; i++)
                    {
                        HistorialEmpleado item = pending[i];
                        if (payrollRepository.HasClosedPeriodAffectedByEmployeeChange(
                            connection, transaction, item.FechaEfectiva))
                        {
                            auditRepository.Add(connection, transaction, new AuditRecord
                            {
                                Usuario = "sistema",
                                Modulo = "Empleados",
                                Accion = "Cambio programado bloqueado",
                                Detalle = "IdHistorial=" + item.IdHistorial + ", período cerrado relacionado",
                                Fecha = now
                            });
                            continue;
                        }

                        historyRepository.ApplyChange(connection, transaction, item, now);
                        auditRepository.Add(connection, transaction, new AuditRecord
                        {
                            Usuario = "sistema",
                            Modulo = "Empleados",
                            Accion = "Aplicar cambio programado",
                            Detalle = "IdHistorial=" + item.IdHistorial + ", IdEmpleado=" + item.IdEmpleado,
                            Fecha = now
                        });
                        applied++;
                    }
                    transaction.Commit();
                }
                catch
                {
                    transaction.Rollback();
                    throw;
                }
            }
            return applied;
        }

        private List<HistorialEmpleado> BuildChanges(Empleado original, Empleado candidate)
        {
            List<HistorialEmpleado> changes = new List<HistorialEmpleado>();
            if (original.SalarioBase != candidate.SalarioBase)
            {
                changes.Add(NewChange(candidate.IdEmpleado, EmployeeHistoryFields.BaseSalary,
                    "B/. " + original.SalarioBase.ToString("0.00"),
                    "B/. " + candidate.SalarioBase.ToString("0.00"),
                    original.SalarioBase.ToString(CultureInfo.InvariantCulture),
                    candidate.SalarioBase.ToString(CultureInfo.InvariantCulture)));
            }
            if (!string.Equals(original.Cargo, candidate.Cargo, StringComparison.Ordinal))
            {
                changes.Add(NewChange(candidate.IdEmpleado, EmployeeHistoryFields.Position,
                    original.Cargo, candidate.Cargo, original.Cargo, candidate.Cargo));
            }
            if (original.IdDepartamento != candidate.IdDepartamento)
```

Services/EmployeeHistoryService.cs (continuacion)

```
            {
                changes.Add(NewChange(candidate.IdEmpleado, EmployeeHistoryFields.Department,
                    original.DepartamentoNombre, employeeRepository.GetDepartmentName(candidate.IdDepartamento),
                    original.IdDepartamento.ToString(CultureInfo.InvariantCulture),
                    candidate.IdDepartamento.ToString(CultureInfo.InvariantCulture)));
            }
            if (!string.Equals(original.Estado, candidate.Estado, StringComparison.OrdinalIgnoreCase))
            {
                changes.Add(NewChange(candidate.IdEmpleado, EmployeeHistoryFields.Status,
                    original.Estado, candidate.Estado, original.Estado, candidate.Estado));
            }
            return changes;
        }

        private static HistorialEmpleado NewChange(int employeeId, string field,
            string oldDisplay, string newDisplay, string oldTechnical, string newTechnical)
        {
            return new HistorialEmpleado
            {
                IdEmpleado = employeeId,
                CampoModificado = field,
                ValorAnterior = oldDisplay,
                ValorNuevo = newDisplay,
                ValorAnteriorTecnico = oldTechnical,
                ValorNuevoTecnico = newTechnical
            };
        }

        private static Empleado PreserveTrackedValues(Empleado candidate, Empleado original)
        {
            return new Empleado
            {
                IdEmpleado = candidate.IdEmpleado,
                Codigo = candidate.Codigo,
                Nombre = candidate.Nombre,
                Apellido = candidate.Apellido,
                Cedula = candidate.Cedula,
                Cargo = original.Cargo,
                IdDepartamento = original.IdDepartamento,
                DepartamentoNombre = original.DepartamentoNombre,
                SalarioBase = original.SalarioBase,
                Estado = original.Estado,
                FechaIngreso = candidate.FechaIngreso,
                FechaEfectivaLaboral = original.FechaEfectivaLaboral
            };
        }
    }

    public sealed class EmployeeEffectiveDataService
    {
        private readonly EmployeeHistoryRepository historyRepository = new EmployeeHistoryRepository();
        private readonly EmpleadoRepository employeeRepository = new EmpleadoRepository();
        private readonly EmployeeHistoryService historyService = new EmployeeHistoryService();

        public List<Empleado> GetActiveEmployeesAsOf(DateTime effectiveDate, int? departmentId)
        {
            historyService.ApplyDueChanges(DateTime.Today);
            List<Empleado> source = employeeRepository.GetAllForEffectiveDate(null);
            List<Empleado> result = new List<Empleado>();
            for (int i = 0; i < source.Count; i++)
```

Services/EmployeeHistoryService.cs (continuacion)

```
            {
                Empleado resolved = ResolveAsOf(source[i], effectiveDate.Date);
                if (!string.Equals(resolved.Estado, "Activo", StringComparison.OrdinalIgnoreCase)) continue;
                if (departmentId.HasValue && resolved.IdDepartamento != departmentId.Value) continue;
                result.Add(resolved);
            }
            return result;
        }

        public Empleado GetEmployeeAsOf(int employeeId, DateTime effectiveDate)
        {
            historyService.ApplyDueChanges(DateTime.Today);
            Empleado employee = employeeRepository.GetById(employeeId);
            return employee == null ? null : ResolveAsOf(employee, effectiveDate.Date);
        }

        private Empleado ResolveAsOf(Empleado current, DateTime date)
        {
            Empleado result = Clone(current);
            List<HistorialEmpleado> history = historyRepository.GetByEmployee(current.IdEmpleado);

            for (int i = history.Count - 1; i >= 0; i--)
            {
                HistorialEmpleado item = history[i];
                if (item.Aplicado && item.FechaEfectiva.Date > date)
                    ApplyValue(result, item.CampoModificado, item.ValorAnteriorTecnico);
            }
            for (int i = 0; i < history.Count; i++)
            {
                HistorialEmpleado item = history[i];
                if (!item.Aplicado && item.FechaEfectiva.Date <= date)
                    ApplyValue(result, item.CampoModificado, item.ValorNuevoTecnico);
            }
            result.DepartamentoNombre = employeeRepository.GetDepartmentName(result.IdDepartamento);
            return result;
        }

        private static void ApplyValue(Empleado employee, string field, string value)
        {
            if (field == EmployeeHistoryFields.BaseSalary)
                employee.SalarioBase = decimal.Parse(value, CultureInfo.InvariantCulture);
            else if (field == EmployeeHistoryFields.Position) employee.Cargo = value;
            else if (field == EmployeeHistoryFields.Department)
                employee.IdDepartamento = int.Parse(value, CultureInfo.InvariantCulture);
            else if (field == EmployeeHistoryFields.Status) employee.Estado = value;
        }

        private static Empleado Clone(Empleado source)
        {
            return new Empleado
            {
                IdEmpleado = source.IdEmpleado,
                Codigo = source.Codigo,
                Nombre = source.Nombre,
                Apellido = source.Apellido,
                Cedula = source.Cedula,
                Cargo = source.Cargo,
                IdDepartamento = source.IdDepartamento,
                DepartamentoNombre = source.DepartamentoNombre,
                SalarioBase = source.SalarioBase,
```

Services/EmployeeHistoryService.cs (continuacion)

```
                Estado = source.Estado,
                FechaIngreso = source.FechaIngreso,
                FechaEfectivaLaboral = source.FechaEfectivaLaboral
            };
        }
    }
}
```

Services/EmployeePortalService.cs

```
using System;
using System.Collections.Generic;
using System.IO;
using SistemaGestionNomina.Data;
using SistemaGestionNomina.Models;
using SistemaGestionNomina.Security;

namespace SistemaGestionNomina.Services
{
    public sealed class EmployeePortalService
    {
        private readonly AuthorizationService authorizationService = new AuthorizationService();
        private readonly EmployeeScopeService employeeScopeService = new EmployeeScopeService();
        private readonly EmpleadoRepository employeeRepository = new EmpleadoRepository();
        private readonly AsistenciaRepository attendanceRepository = new AsistenciaRepository();
        private readonly ComprobanteRepository payslipRepository = new ComprobanteRepository();
        private readonly AuditTrailService auditTrailService = new AuditTrailService();
        private readonly PdfExportService pdfExportService = new PdfExportService();

        public EmployeePortalDashboardViewModel GetDashboard()
        {
            authorizationService.DemandPermission(Permissions.PortalView);
            EmployeeProfileViewModel profile = GetProfileCore();
            DateTime today = DateTime.Today;
            List<Asistencia> monthAttendance = attendanceRepository.GetByEmployee(
                profile.IdEmpleado, new DateTime(today.Year, today.Month, 1), today, string.Empty);
            List<Asistencia> recent = new List<Asistencia>();
            for (int i = 0; i < monthAttendance.Count && i < 5; i++) recent.Add(monthAttendance[i]);

            List<Comprobante> payslips = payslipRepository.GetByEmployee(profile.IdEmpleado, string.Empty);
            auditTrailService.RegistrarAccion("Portal trabajador", "Consultar inicio", "Empleado=" + profile.IdEmpleado);
            return new EmployeePortalDashboardViewModel
            {
                Profile = profile,
                LatestPayslip = payslips.Count > 0 ? payslips[0] : null,
                RecentAttendance = recent,
                AttendanceSummary = SummarizeAttendance(monthAttendance)
            };
        }

        public EmployeeProfileViewModel GetMyProfile()
        {
            authorizationService.DemandPermission(Permissions.OwnProfileView);
            EmployeeProfileViewModel profile = GetProfileCore();
            auditTrailService.RegistrarAccion("Portal trabajador", "Consultar perfil", "Empleado=" + profile.IdEmpleado);
            return profile;
        }

        public List<Asistencia> GetMyAttendance(DateTime? start, DateTime? end, string status)
        {
            authorizationService.DemandPermission(Permissions.OwnAttendanceView);
            if (start.HasValue && end.HasValue && start.Value.Date > end.Value.Date)
            {
                throw new ArgumentException("La fecha inicial no puede ser posterior a la fecha final.");
            }

            string normalizedStatus = NormalizeAttendanceStatus(status);
            int employeeId = employeeScopeService.RequireCurrentEmployeeId();
            List<Asistencia> items = attendanceRepository.GetByEmployee(employeeId, start, end, normalizedStatus);
            auditTrailService.RegistrarAccion("Portal trabajador", "Consultar asistencia",
```

Services/EmployeePortalService.cs (continuacion)

```
                "Empleado=" + employeeId + ", Registros=" + items.Count);
            return items;
        }

        public List<Comprobante> GetMyPayslips(string search)
        {
            authorizationService.DemandPermission(Permissions.OwnPayslipsView);
            int employeeId = employeeScopeService.RequireCurrentEmployeeId();
            List<Comprobante> items = payslipRepository.GetByEmployee(employeeId, (search ?? string.Empty).Trim());
            auditTrailService.RegistrarAccion("Portal trabajador", "Consultar comprobantes",
                "Empleado=" + employeeId + ", Registros=" + items.Count);
            return items;
        }

        public Comprobante GetMyPayslipById(int payslipId)
        {
            authorizationService.DemandPermission(Permissions.OwnPayslipsView);
            if (payslipId <= 0) throw new ArgumentException("Seleccione un comprobante valido.");

            int employeeId = employeeScopeService.RequireCurrentEmployeeId();
            Comprobante payslip = payslipRepository.GetByIdAndEmployee(payslipId, employeeId);
            if (payslip == null)
            {
                auditTrailService.RegistrarAccion("Portal trabajador", "Acceso rechazado a comprobante",
                    "IdComprobante=" + payslipId);
                throw new UnauthorizedAccessException("El comprobante no esta disponible para su cuenta.");
            }

            auditTrailService.RegistrarAccion("Portal trabajador", "Consultar comprobante",
                "IdComprobante=" + payslipId);
            return payslip;
        }

        public string DownloadMyPayslipPdf(int payslipId)
        {
            return DownloadMyPayslipPdfCore(payslipId, null);
        }

        internal string DownloadMyPayslipPdf(int payslipId, string destinationPath)
        {
            return DownloadMyPayslipPdfCore(payslipId, destinationPath);
        }

        public string GetExistingMyPayslipPdf(int payslipId)
        {
            Comprobante payslip = GetMyPayslipById(payslipId);
            if (string.IsNullOrWhiteSpace(payslip.RutaPdf) || !File.Exists(payslip.RutaPdf))
            {
                throw new FileNotFoundException("El comprobante no tiene un PDF existente. Descarguelo primero.");
            }

            auditTrailService.RegistrarAccion("Portal trabajador", "Abrir PDF propio",
                "IdComprobante=" + payslipId);
            return Path.GetFullPath(payslip.RutaPdf);
        }

        public Comprobante GetMyPayslipForPrint(int payslipId)
        {
            Comprobante payslip = GetMyPayslipById(payslipId);
            auditTrailService.RegistrarAccion("Portal trabajador", "Imprimir comprobante propio",
```

Services/EmployeePortalService.cs (continuacion)

```
                "IdComprobante=" + payslipId);
            return payslip;
        }

        public static EmployeeAttendanceSummaryViewModel SummarizeAttendance(IEnumerable<Asistencia> attendance)
        {
            EmployeeAttendanceSummaryViewModel summary = new EmployeeAttendanceSummaryViewModel();
            if (attendance == null) return summary;

            foreach (Asistencia item in attendance)
            {
                if (string.Equals(item.Estado, "Puntual", StringComparison.OrdinalIgnoreCase)) summary.Puntuales++;
                else if (string.Equals(item.Estado, "Tardanza", StringComparison.OrdinalIgnoreCase)) summary.Tardanzas++;
                else if (string.Equals(item.Estado, "Falta", StringComparison.OrdinalIgnoreCase)) summary.Faltas++;
                else if (string.Equals(item.Estado, "Permiso", StringComparison.OrdinalIgnoreCase)) summary.Permisos++;

                summary.HorasTrabajadas += item.HorasTrabajadas;
                if (item.HorasTrabajadas > 8) summary.HorasExtra += item.HorasTrabajadas - 8;
            }

            return summary;
        }

        private EmployeeProfileViewModel GetProfileCore()
        {
            int employeeId = employeeScopeService.RequireCurrentEmployeeId();
            Empleado employee = employeeRepository.GetById(employeeId);
            employeeScopeService.DemandCurrentEmployee(employee.IdEmpleado);
            return new EmployeeProfileViewModel
            {
                IdEmpleado = employee.IdEmpleado,
                Codigo = employee.Codigo,
                NombreCompleto = employee.NombreCompleto,
                Cedula = employee.Cedula,
                Cargo = employee.Cargo,
                Departamento = employee.DepartamentoNombre,
                SalarioBase = employee.SalarioBase,
                Estado = employee.Estado,
                FechaIngreso = employee.FechaIngreso
            };
        }

        private string DownloadMyPayslipPdfCore(int payslipId, string destinationPath)
        {
            authorizationService.DemandPermission(Permissions.OwnPayslipsDownload);
            Comprobante payslip = GetMyPayslipById(payslipId);
            int employeeId = employeeScopeService.RequireCurrentEmployeeId();
            string path = destinationPath == null
                ? pdfExportService.ExportarComprobantePersonal(payslip)
                : pdfExportService.ExportarComprobantePersonal(payslip, destinationPath);
            if (string.IsNullOrWhiteSpace(path)) return string.Empty;

            if (!payslipRepository.UpdateRutaPdfForEmployee(payslipId, employeeId, path))
            {
                throw new InvalidOperationException("No se pudo asociar el PDF al comprobante.");
            }

            auditTrailService.RegistrarAccion("Portal trabajador", "Descargar PDF propio",
                "IdComprobante=" + payslipId + ", Archivo=" + Path.GetFileName(path));
            return path;
```

Services/EmployeePortalService.cs (continuacion)

```
        }

        private static string NormalizeAttendanceStatus(string status)
        {
            string normalized = (status ?? string.Empty).Trim();
            if (normalized.Length == 0 || string.Equals(normalized, "Todos", StringComparison.OrdinalIgnoreCase))
            {
                return string.Empty;
            }

            string[] allowed = { "Puntual", "Tardanza", "Falta", "Permiso" };
            for (int i = 0; i < allowed.Length; i++)
            {
                if (string.Equals(normalized, allowed[i], StringComparison.OrdinalIgnoreCase)) return allowed[i];
            }

            throw new ArgumentException("El estado de asistencia no es valido.");
        }
    }
}
```

Services/ExcelExportService.cs

```
using System;
using System.Collections.Generic;
using ClosedXML.Excel;
using SistemaGestionNomina.Helpers;
using SistemaGestionNomina.Models;
using SistemaGestionNomina.Security;

namespace SistemaGestionNomina.Services
{
    public class ExcelExportService
    {
        private readonly AuthorizationService authorizationService = new AuthorizationService();
        private readonly AuditTrailService auditTrailService = new AuditTrailService();

        public string ExportarEmpleados(List<Empleado> empleados)
        {
            authorizationService.DemandAny(Permissions.EmployeesExport, Permissions.ReportsPersonal);
            string path = BuildPath("Empleados");
            if (string.IsNullOrWhiteSpace(path)) return string.Empty;

            using (XLWorkbook workbook = new XLWorkbook())
            {
                IXLWorksheet sheet = workbook.Worksheets.Add("Empleados");
                WriteHeader(sheet, "Código", "Nombre", "Cédula", "Cargo", "Departamento", "Salario Base", "Estado");
                for (int i = 0; i < empleados.Count; i++)
                {
                    Empleado e = empleados[i];
                    int row = i + 2;
                    sheet.Cell(row, 1).Value = e.Codigo;
                    sheet.Cell(row, 2).Value = e.NombreCompleto;
                    sheet.Cell(row, 3).Value = e.Cedula;
                    sheet.Cell(row, 4).Value = e.Cargo;
                    sheet.Cell(row, 5).Value = e.DepartamentoNombre;
                    sheet.Cell(row, 6).Value = e.SalarioBase;
                    sheet.Cell(row, 7).Value = e.Estado;
                }
                Finish(sheet);
                workbook.SaveAs(path);
            }

            auditTrailService.RegistrarAccion("Exportaciones", "Excel empleados", System.IO.Path.GetFileName(path));
            return path;
        }

        public string ExportarAsistencias(List<Asistencia> asistencias)
        {
            authorizationService.DemandPermission(Permissions.AttendanceExport);
            string path = BuildPath("Asistencia");
            if (string.IsNullOrWhiteSpace(path)) return string.Empty;

            using (XLWorkbook workbook = new XLWorkbook())
            {
                IXLWorksheet sheet = workbook.Worksheets.Add("Asistencia");
                WriteHeader(sheet, "Empleado", "Fecha", "Entrada", "Salida", "Horas", "Estado");
                for (int i = 0; i < asistencias.Count; i++)
                {
                    Asistencia a = asistencias[i];
                    int row = i + 2;
                    sheet.Cell(row, 1).Value = a.EmpleadoNombre;
                    sheet.Cell(row, 2).Value = a.Fecha.ToString("dd/MM/yyyy");
```

Services/ExcelExportService.cs (continuacion)

```
                    sheet.Cell(row, 3).Value = a.HoraEntrada.HasValue ? a.HoraEntrada.Value.ToString(@"hh\:mm") : "--";
                    sheet.Cell(row, 4).Value = a.HoraSalida.HasValue ? a.HoraSalida.Value.ToString(@"hh\:mm") : "--";
                    sheet.Cell(row, 5).Value = a.HorasTrabajadas;
                    sheet.Cell(row, 6).Value = a.Estado;
                }
                Finish(sheet);
                workbook.SaveAs(path);
            }

            auditTrailService.RegistrarAccion("Exportaciones", "Excel asistencia", System.IO.Path.GetFileName(path));
            return path;
        }

        public string ExportarHistorialEmpleados(List<HistorialEmpleado> history)
        {
            authorizationService.DemandPermission(Permissions.EmployeeHistoryExport);
            string path = BuildPath("Historial_Laboral");
            if (string.IsNullOrWhiteSpace(path)) return string.Empty;
            using (XLWorkbook workbook = new XLWorkbook())
            {
                IXLWorksheet sheet = workbook.Worksheets.Add("Historial laboral");
                WriteHeader(sheet, "Código", "Empleado", "Campo", "Valor anterior", "Valor nuevo",
                    "Fecha efectiva", "Registrado", "Usuario", "Motivo", "Aplicado");
                for (int i = 0; i < history.Count; i++)
                {
                    HistorialEmpleado item = history[i];
                    int row = i + 2;
                    sheet.Cell(row, 1).Value = item.CodigoEmpleado;
                    sheet.Cell(row, 2).Value = item.EmpleadoNombre;
                    sheet.Cell(row, 3).Value = item.CampoModificado;
                    sheet.Cell(row, 4).Value = item.ValorAnterior;
                    sheet.Cell(row, 5).Value = item.ValorNuevo;
                    sheet.Cell(row, 6).Value = item.FechaEfectiva.ToString("dd/MM/yyyy");
                    sheet.Cell(row, 7).Value = item.FechaCambio.ToString("dd/MM/yyyy HH:mm");
                    sheet.Cell(row, 8).Value = item.UsuarioResponsable;
                    sheet.Cell(row, 9).Value = item.Motivo;
                    sheet.Cell(row, 10).Value = item.Aplicado ? "Sí" : "Programado";
                }
                Finish(sheet);
                workbook.SaveAs(path);
            }
            auditTrailService.RegistrarAccion("Exportaciones", "Excel historial laboral",
                System.IO.Path.GetFileName(path));
            return path;
        }

        public string ExportarAusencias(List<SolicitudAusencia> requests)
        {
            authorizationService.DemandPermission(Permissions.AbsencesExport);
            string path = BuildPath("Solicitudes_Ausencia");
            if (string.IsNullOrWhiteSpace(path)) return string.Empty;
            using (XLWorkbook workbook = new XLWorkbook())
            {
                IXLWorksheet sheet = workbook.Worksheets.Add("Ausencias");
                WriteHeader(sheet, "Código", "Empleado", "Departamento", "Tipo", "Desde", "Hasta",
                    "Estado", "Solicitante", "Resolución", "Motivo");
                for (int i = 0; i < requests.Count; i++)
                {
                    SolicitudAusencia item = requests[i];
                    int row = i + 2;
```

Services/ExcelExportService.cs (continuacion)

```
                    sheet.Cell(row, 1).Value = item.CodigoEmpleado;
                    sheet.Cell(row, 2).Value = item.EmpleadoNombre;
                    sheet.Cell(row, 3).Value = item.DepartamentoNombre;
                    sheet.Cell(row, 4).Value = item.Tipo;
                    sheet.Cell(row, 5).Value = item.FechaInicio.ToString("dd/MM/yyyy");
                    sheet.Cell(row, 6).Value = item.FechaFin.ToString("dd/MM/yyyy");
                    sheet.Cell(row, 7).Value = item.Estado;
                    sheet.Cell(row, 8).Value = item.UsuarioSolicitante;
                    sheet.Cell(row, 9).Value = item.ObservacionResolucion;
                    sheet.Cell(row, 10).Value = item.Motivo;
                }
                Finish(sheet);
                workbook.SaveAs(path);
            }
            auditTrailService.RegistrarAccion("Exportaciones", "Excel ausencias",
                System.IO.Path.GetFileName(path));
            return path;
        }

        public string ExportarNomina(Nomina nomina)
        {
            authorizationService.DemandPermission(Permissions.PayrollExport);
            string path = BuildPath("Nomina");
            if (string.IsNullOrWhiteSpace(path)) return string.Empty;

            using (XLWorkbook workbook = new XLWorkbook())
            {
                IXLWorksheet sheet = workbook.Worksheets.Add("Nómina");
                sheet.Cell(1, 1).Value = nomina.PeriodoNombre;
                sheet.Range(1, 1, 1, 9).Merge().Style.Font.Bold = true;
                WriteHeader(sheet, 3, "Código", "Empleado", "Departamento", "Sueldo Base", "Bonos", "Horas Extra", "Monto H.E.", "Deducciones", "Neto");
                for (int i = 0; i < nomina.Detalles.Count; i++)
                {
                    NominaDetalle d = nomina.Detalles[i];
                    int row = i + 4;
                    sheet.Cell(row, 1).Value = d.CodigoEmpleado;
                    sheet.Cell(row, 2).Value = d.EmpleadoNombre;
                    sheet.Cell(row, 3).Value = d.Departamento;
                    sheet.Cell(row, 4).Value = d.SueldoBase;
                    sheet.Cell(row, 5).Value = d.Bonos;
                    sheet.Cell(row, 6).Value = d.HorasExtra;
                    sheet.Cell(row, 7).Value = d.MontoHorasExtra;
                    sheet.Cell(row, 8).Value = d.TotalDeducciones;
                    sheet.Cell(row, 9).Value = d.NetoPagar;
                }
                int totalRow = nomina.Detalles.Count + 5;
                sheet.Cell(totalRow, 7).Value = "Totales";
                sheet.Cell(totalRow, 8).Value = nomina.TotalDeducciones;
                sheet.Cell(totalRow, 9).Value = nomina.TotalNeto;
                Finish(sheet);
                workbook.SaveAs(path);
            }

            auditTrailService.RegistrarAccion("Exportaciones", "Excel nómina", System.IO.Path.GetFileName(path));
            return path;
        }

        public string ExportarComprobantes(List<Comprobante> comprobantes)
        {
            authorizationService.DemandPermission(Permissions.PayslipsExport);
```

Services/ExcelExportService.cs (continuacion)

```
            string path = BuildPath("Comprobantes");
            if (string.IsNullOrWhiteSpace(path)) return string.Empty;

            using (XLWorkbook workbook = new XLWorkbook())
            {
                IXLWorksheet sheet = workbook.Worksheets.Add("Comprobantes");
                WriteHeader(sheet, "Número", "Empleado", "Periodo", "Fecha", "Ingresos", "Deducciones", "Neto");
                for (int i = 0; i < comprobantes.Count; i++)
                {
                    Comprobante c = comprobantes[i];
                    int row = i + 2;
                    sheet.Cell(row, 1).Value = c.NumeroComprobante;
                    sheet.Cell(row, 2).Value = c.EmpleadoNombre;
                    sheet.Cell(row, 3).Value = c.PeriodoNombre;
                    sheet.Cell(row, 4).Value = c.FechaGeneracion.ToString("dd/MM/yyyy HH:mm");
                    sheet.Cell(row, 5).Value = c.TotalIngresos;
                    sheet.Cell(row, 6).Value = c.TotalDeducciones;
                    sheet.Cell(row, 7).Value = c.NetoPagar;
                }
                Finish(sheet);
                workbook.SaveAs(path);
            }

            auditTrailService.RegistrarAccion("Exportaciones", "Excel comprobantes", System.IO.Path.GetFileName(path));
            return path;
        }

        public string ExportarReportes(List<ReporteGenerado> reportes)
        {
            authorizationService.DemandPermission(Permissions.ReportsExport);
            string path = BuildPath("Reportes");
            if (string.IsNullOrWhiteSpace(path)) return string.Empty;

            using (XLWorkbook workbook = new XLWorkbook())
            {
                IXLWorksheet sheet = workbook.Worksheets.Add("Reportes");
                WriteHeader(sheet, "Nombre", "Tipo", "Generado por", "Fecha", "Ruta");
                for (int i = 0; i < reportes.Count; i++)
                {
                    ReporteGenerado r = reportes[i];
                    int row = i + 2;
                    sheet.Cell(row, 1).Value = r.NombreReporte;
                    sheet.Cell(row, 2).Value = r.Tipo;
                    sheet.Cell(row, 3).Value = r.GeneradoPor;
                    sheet.Cell(row, 4).Value = r.FechaGeneracion.ToString("dd/MM/yyyy HH:mm");
                    sheet.Cell(row, 5).Value = r.RutaArchivo;
                }
                Finish(sheet);
                workbook.SaveAs(path);
            }

            auditTrailService.RegistrarAccion("Exportaciones", "Excel reportes", System.IO.Path.GetFileName(path));
            return path;
        }

        private static string BuildPath(string prefix)
        {
            // El usuario elige la ubicacion final; la aplicacion no guarda exportaciones en rutas fijas.
            return PathHelper.RequestExportPath(prefix, ".xlsx", "Libro de Excel (*.xlsx)|*.xlsx");
        }
```

Services/ExcelExportService.cs (continuacion)

```

        private static void WriteHeader(IXLWorksheet sheet, params string[] headers)
        {
            WriteHeader(sheet, 1, headers);
        }

        private static void WriteHeader(IXLWorksheet sheet, int row, params string[] headers)
        {
            for (int i = 0; i < headers.Length; i++)
            {
                sheet.Cell(row, i + 1).Value = headers[i];
                sheet.Cell(row, i + 1).Style.Font.Bold = true;
                sheet.Cell(row, i + 1).Style.Fill.BackgroundColor = XLColor.FromHtml("#A78BFA");
            }
        }

        private static void Finish(IXLWorksheet sheet)
        {
            sheet.Columns().AdjustToContents();
        }
    }
}
```

Services/PayrollLifecycleService.cs

```
using System;
using System.Collections.Generic;
using System.Data.SQLite;
using SistemaGestionNomina.Data;
using SistemaGestionNomina.Models;
using SistemaGestionNomina.Security;

namespace SistemaGestionNomina.Services
{
    public class PayrollLifecycleService
    {
        private readonly NominaVersionRepository versionRepository = new NominaVersionRepository();
        private readonly AuditRepository auditRepository = new AuditRepository();
        private readonly AuthorizationService authorizationService = new AuthorizationService();
        private readonly PayrollPeriodPolicyService periodPolicyService = new PayrollPeriodPolicyService();

        public int Confirmar(Nomina nominaCalculada, DateTime fechaInicio, DateTime fechaFin)
        {
            authorizationService.DemandPermission(Permissions.PayrollConfirm);
            ValidateCalculatedPayroll(nominaCalculada);
            periodPolicyService.VerificarFechasAbiertas(fechaInicio, fechaFin);

            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteTransaction transaction = connection.BeginTransaction())
            {
                try
                {
                    // Periodo, nomina, detalles, comprobantes y auditoria deben persistirse juntos.
                    int idNomina = CreateConfirmedPayroll(connection, transaction, nominaCalculada,
                        fechaInicio.Date, fechaFin.Date, CurrentUsername(), DateTime.Now);
                    transaction.Commit();
                    return idNomina;
                }
                catch
                {
                    transaction.Rollback();
                    throw;
                }
            }
        }

        public void Pagar(int idNomina)
        {
            authorizationService.DemandPermission(Permissions.PayrollPay);
            string user = CurrentUsername();
            DateTime now = DateTime.Now;

            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteTransaction transaction = connection.BeginTransaction())
            {
                try
                {
                    Nomina payroll = GetPayroll(connection, transaction, idNomina);
                    if (payroll == null) throw new InvalidOperationException("La nómina no existe.");
                    PayrollStateMachine.DemandTransition(payroll.Estado, PayrollStates.Paid);

                    // La condicion por estado evita registrar un pago duplicado ante acciones simultaneas.
                    using (SQLiteCommand command = new SQLiteCommand(@"UPDATE Nominas
                        SET Estado = @nuevo, FechaPago = @fecha, PagadaPor = @usuario
                        WHERE IdNomina = @id AND Estado = @anterior;", connection, transaction))
```

Services/PayrollLifecycleService.cs (continuacion)

```
                    {
                        command.Parameters.AddWithValue("@nuevo", PayrollStates.Paid);
                        command.Parameters.AddWithValue("@fecha", now.ToString("yyyy-MM-dd HH:mm:ss"));
                        command.Parameters.AddWithValue("@usuario", user);
                        command.Parameters.AddWithValue("@id", idNomina);
                        command.Parameters.AddWithValue("@anterior", PayrollStates.Confirmed);
                        if (command.ExecuteNonQuery() != 1)
                            throw new InvalidOperationException("La nómina cambió de estado antes de registrar el pago.");
                    }
                    using (SQLiteCommand command = new SQLiteCommand(@"UPDATE PeriodosNomina
                        SET Estado = @estado WHERE IdPeriodo = @id;", connection, transaction))
                    {
                        command.Parameters.AddWithValue("@estado", PayrollPeriodStates.Paid);
                        command.Parameters.AddWithValue("@id", payroll.IdPeriodo);
                        command.ExecuteNonQuery();
                    }
                    AddAudit(connection, transaction, user, "Nómina", "Pagar", "IdNomina=" + idNomina, now);
                    transaction.Commit();
                }
                catch
                {
                    transaction.Rollback();
                    throw;
                }
            }
        }

        public void Anular(int idNomina, string motivo)
        {
            authorizationService.DemandPermission(Permissions.PayrollAnnul);
            if (string.IsNullOrWhiteSpace(motivo))
                throw new InvalidOperationException("Debe indicar un motivo de anulación.");

            string user = CurrentUsername();
            DateTime now = DateTime.Now;
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteTransaction transaction = connection.BeginTransaction())
            {
                try
                {
                    Nomina payroll = GetPayroll(connection, transaction, idNomina);
                    if (payroll == null) throw new InvalidOperationException("La nómina no existe.");
                    PayrollStateMachine.DemandTransition(payroll.Estado, PayrollStates.Annulled);

                    // La anulacion conserva la nomina original para mantener la trazabilidad financiera.
                    using (SQLiteCommand command = new SQLiteCommand(@"UPDATE Nominas
                        SET Estado = @nuevo, FechaAnulacion = @fecha, AnuladaPor = @usuario,
                            MotivoAnulacion = @motivo
                        WHERE IdNomina = @id AND Estado = @anterior;", connection, transaction))
                    {
                        command.Parameters.AddWithValue("@nuevo", PayrollStates.Annulled);
                        command.Parameters.AddWithValue("@fecha", now.ToString("yyyy-MM-dd HH:mm:ss"));
                        command.Parameters.AddWithValue("@usuario", user);
                        command.Parameters.AddWithValue("@motivo", motivo.Trim());
                        command.Parameters.AddWithValue("@id", idNomina);
                        command.Parameters.AddWithValue("@anterior", payroll.Estado);
                        if (command.ExecuteNonQuery() != 1)
                            throw new InvalidOperationException("La nómina cambió de estado antes de anularse.");
                    }
                    using (SQLiteCommand command = new SQLiteCommand(@"INSERT INTO NominaVersiones
```

Services/PayrollLifecycleService.cs (continuacion)

```
                        (IdNominaOriginal, IdNominaNueva, MotivoCambio, UsuarioResponsable, FechaCambio)
                        VALUES (@original, NULL, @motivo, @usuario, @fecha);", connection, transaction))
                    {
                        command.Parameters.AddWithValue("@original", idNomina);
                        command.Parameters.AddWithValue("@motivo", motivo.Trim());
                        command.Parameters.AddWithValue("@usuario", user);
                        command.Parameters.AddWithValue("@fecha", now.ToString("yyyy-MM-dd HH:mm:ss"));
                        command.ExecuteNonQuery();
                    }
                    using (SQLiteCommand command = new SQLiteCommand(@"UPDATE PeriodosNomina
                        SET Estado = @estado, Cerrado = 0, FechaCierre = NULL, CerradoPor = NULL
                        WHERE IdPeriodo = @id;", connection, transaction))
                    {
                        command.Parameters.AddWithValue("@estado", PayrollPeriodStates.Reopened);
                        command.Parameters.AddWithValue("@id", payroll.IdPeriodo);
                        command.ExecuteNonQuery();
                    }
                    AddAudit(connection, transaction, user, "Nómina", "Anular",
                        "IdNomina=" + idNomina + ", Motivo=" + motivo.Trim(), now);
                    transaction.Commit();
                }
                catch
                {
                    transaction.Rollback();
                    throw;
                }
            }
        }

        public int Recalcular(int idNominaAnulada, DateTime fechaInicio, DateTime fechaFin,
            Nomina nominaRecalculada)
        {
            authorizationService.DemandPermission(Permissions.PayrollRecalculate);
            ValidateCalculatedPayroll(nominaRecalculada);
            periodPolicyService.VerificarFechasAbiertas(fechaInicio, fechaFin);

            string user = CurrentUsername();
            DateTime now = DateTime.Now;
            using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
            using (SQLiteTransaction transaction = connection.BeginTransaction())
            {
                try
                {
                    Nomina original = GetPayroll(connection, transaction, idNominaAnulada);
                    if (original == null) throw new InvalidOperationException("La nómina original no existe.");
                    if (!string.Equals(original.Estado, PayrollStates.Annulled, StringComparison.OrdinalIgnoreCase))
                        throw new InvalidOperationException("Solo se puede recalcular a partir de una nómina anulada.");

                    int pendingVersionId = GetPendingVersionId(connection, transaction, idNominaAnulada);
                    if (pendingVersionId <= 0)
                        throw new InvalidOperationException("No hay una versión pendiente para la nómina anulada.");

                    int newPayrollId = CreateConfirmedPayroll(connection, transaction, nominaRecalculada,
                        fechaInicio.Date, fechaFin.Date, user, now);

                    // La nueva nomina queda enlazada con la anulada como una version corregida.
                    using (SQLiteCommand command = new SQLiteCommand(@"UPDATE NominaVersiones
                        SET IdNominaNueva = @nueva
                        WHERE IdVersion = @id AND IdNominaNueva IS NULL;", connection, transaction))
                    {
```

Services/PayrollLifecycleService.cs (continuacion)

```
                        command.Parameters.AddWithValue("@nueva", newPayrollId);
                        command.Parameters.AddWithValue("@id", pendingVersionId);
                        if (command.ExecuteNonQuery() != 1)
                            throw new InvalidOperationException("No se pudo vincular la nueva versión de nómina.");
                    }

                    AddAudit(connection, transaction, user, "Nómina", "Recalcular",
                        "IdNominaOriginal=" + idNominaAnulada + ", IdNominaNueva=" + newPayrollId, now);
                    transaction.Commit();
                    return newPayrollId;
                }
                catch
                {
                    transaction.Rollback();
                    throw;
                }
            }
        }

        public List<NominaVersion> ObtenerHistorial(int idNomina)
        {
            authorizationService.DemandPermission(Permissions.PayrollHistoryView);
            return versionRepository.GetHistorialChain(idNomina);
        }

        private int CreateConfirmedPayroll(SQLiteConnection connection, SQLiteTransaction transaction,
            Nomina calculated, DateTime start, DateTime end, string user, DateTime now)
        {
            int periodId;
            using (SQLiteCommand command = new SQLiteCommand(@"INSERT INTO PeriodosNomina
                (Nombre, FechaInicio, FechaFin, Estado, Cerrado, FechaCierre, CerradoPor)
                VALUES (@nombre, @inicio, @fin, @estado, 1, @cierre, @usuario);
                SELECT last_insert_rowid();", connection, transaction))
            {
                command.Parameters.AddWithValue("@nombre", calculated.PeriodoNombre);
                command.Parameters.AddWithValue("@inicio", start.ToString("yyyy-MM-dd"));
                command.Parameters.AddWithValue("@fin", end.ToString("yyyy-MM-dd"));
                command.Parameters.AddWithValue("@estado", PayrollPeriodStates.Confirmed);
                command.Parameters.AddWithValue("@cierre", now.ToString("yyyy-MM-dd HH:mm:ss"));
                command.Parameters.AddWithValue("@usuario", user);
                periodId = Convert.ToInt32(command.ExecuteScalar());
            }

            int payrollId;
            using (SQLiteCommand command = new SQLiteCommand(@"INSERT INTO Nominas
                (IdPeriodo, FechaCalculo, TotalIngresos, TotalDeducciones, TotalNeto, Estado,
                 FechaConfirmacion, ConfirmadaPor)
                VALUES (@periodo, @fecha, @ingresos, @deducciones, @neto, @estado,
                 @confirmacion, @usuario);
                SELECT last_insert_rowid();", connection, transaction))
            {
                command.Parameters.AddWithValue("@periodo", periodId);
                command.Parameters.AddWithValue("@fecha", calculated.FechaCalculo.ToString("yyyy-MM-dd HH:mm:ss"));
                command.Parameters.AddWithValue("@ingresos", calculated.TotalIngresos);
                command.Parameters.AddWithValue("@deducciones", calculated.TotalDeducciones);
                command.Parameters.AddWithValue("@neto", calculated.TotalNeto);
                command.Parameters.AddWithValue("@estado", PayrollStates.Confirmed);
                command.Parameters.AddWithValue("@confirmacion", now.ToString("yyyy-MM-dd HH:mm:ss"));
                command.Parameters.AddWithValue("@usuario", user);
                payrollId = Convert.ToInt32(command.ExecuteScalar());
```

Services/PayrollLifecycleService.cs (continuacion)

```
            }

            for (int i = 0; i < calculated.Detalles.Count; i++)
            {
                NominaDetalle detail = calculated.Detalles[i];
                // El snapshot evita que cambios futuros del empleado alteren comprobantes historicos.
                using (SQLiteCommand command = new SQLiteCommand(@"INSERT INTO NominaDetalle
                    (IdNomina, IdEmpleado, SueldoBase, Bonos, HorasExtra, MontoHorasExtra,
                     TotalIngresos, TotalDeducciones, NetoPagar, CodigoEmpleadoSnapshot,
                     NombreEmpleadoSnapshot, CargoEmpleadoSnapshot, DepartamentoSnapshot)
                    VALUES (@nomina, @empleado, @sueldo, @bonos, @horas, @montoHoras,
                     @ingresos, @deducciones, @neto, @codigo, @nombre, @cargo, @departamento);",
                    connection, transaction))
                {
                    command.Parameters.AddWithValue("@nomina", payrollId);
                    command.Parameters.AddWithValue("@empleado", detail.IdEmpleado);
                    command.Parameters.AddWithValue("@sueldo", detail.SueldoBase);
                    command.Parameters.AddWithValue("@bonos", detail.Bonos);
                    command.Parameters.AddWithValue("@horas", detail.HorasExtra);
                    command.Parameters.AddWithValue("@montoHoras", detail.MontoHorasExtra);
                    command.Parameters.AddWithValue("@ingresos", detail.TotalIngresos);
                    command.Parameters.AddWithValue("@deducciones", detail.TotalDeducciones);
                    command.Parameters.AddWithValue("@neto", detail.NetoPagar);
                    command.Parameters.AddWithValue("@codigo", detail.CodigoEmpleado ?? string.Empty);
                    command.Parameters.AddWithValue("@nombre", detail.EmpleadoNombre ?? string.Empty);
                    command.Parameters.AddWithValue("@cargo", detail.CargoEmpleado ?? string.Empty);
                    command.Parameters.AddWithValue("@departamento", detail.Departamento ?? string.Empty);
                    command.ExecuteNonQuery();
                }
                using (SQLiteCommand command = new SQLiteCommand(@"INSERT INTO Comprobantes
                    (IdNomina, IdEmpleado, NumeroComprobante, FechaGeneracion, RutaPdf)
                    VALUES (@nomina, @empleado, @numero, @fecha, '');", connection, transaction))
                {
                    command.Parameters.AddWithValue("@nomina", payrollId);
                    command.Parameters.AddWithValue("@empleado", detail.IdEmpleado);
                    command.Parameters.AddWithValue("@numero", "COMP-" + payrollId.ToString("0000") + "-" + detail.IdEmpleado.ToString("0000"));
                    command.Parameters.AddWithValue("@fecha", now.ToString("yyyy-MM-dd HH:mm:ss"));
                    command.ExecuteNonQuery();
                }
            }

            AddAudit(connection, transaction, user, "Nómina", "Confirmar",
                "IdNomina=" + payrollId + ", Periodo=" + periodId + ", Empleados=" + calculated.Detalles.Count, now);
            AddAudit(connection, transaction, user, "Comprobantes", "Generar",
                "IdNomina=" + payrollId + ", Cantidad=" + calculated.Detalles.Count, now);
            return payrollId;
        }

        private static Nomina GetPayroll(SQLiteConnection connection, SQLiteTransaction transaction, int payrollId)
        {
            using (SQLiteCommand command = new SQLiteCommand(@"SELECT n.*, p.Nombre AS PeriodoNombre
                FROM Nominas n INNER JOIN PeriodosNomina p ON p.IdPeriodo = n.IdPeriodo
                WHERE n.IdNomina = @id LIMIT 1;", connection, transaction))
            {
                command.Parameters.AddWithValue("@id", payrollId);
                using (SQLiteDataReader reader = command.ExecuteReader())
                {
                    if (!reader.Read()) return null;
                    return new Nomina
                    {
```

Services/PayrollLifecycleService.cs (continuacion)

```
                        IdNomina = Convert.ToInt32(reader["IdNomina"]),
                        IdPeriodo = Convert.ToInt32(reader["IdPeriodo"]),
                        PeriodoNombre = Convert.ToString(reader["PeriodoNombre"]),
                        FechaCalculo = DateTime.Parse(Convert.ToString(reader["FechaCalculo"])),
                        TotalIngresos = Convert.ToDecimal(reader["TotalIngresos"]),
                        TotalDeducciones = Convert.ToDecimal(reader["TotalDeducciones"]),
                        TotalNeto = Convert.ToDecimal(reader["TotalNeto"]),
                        Estado = Convert.ToString(reader["Estado"])
                    };
                }
            }
        }

        private static int GetPendingVersionId(SQLiteConnection connection, SQLiteTransaction transaction,
            int originalPayrollId)
        {
            using (SQLiteCommand command = new SQLiteCommand(@"SELECT IdVersion FROM NominaVersiones
                WHERE IdNominaOriginal = @id AND IdNominaNueva IS NULL
                ORDER BY IdVersion DESC LIMIT 1;", connection, transaction))
            {
                command.Parameters.AddWithValue("@id", originalPayrollId);
                object value = command.ExecuteScalar();
                return value == null || value == DBNull.Value ? 0 : Convert.ToInt32(value);
            }
        }

        private void AddAudit(SQLiteConnection connection, SQLiteTransaction transaction, string user,
            string module, string action, string detail, DateTime date)
        {
            auditRepository.Add(connection, transaction, new AuditRecord
            {
                Usuario = user,
                Modulo = module,
                Accion = action,
                Detalle = detail,
                Fecha = date
            });
        }

        private static void ValidateCalculatedPayroll(Nomina payroll)
        {
            if (payroll == null || payroll.Detalles == null || payroll.Detalles.Count == 0)
                throw new InvalidOperationException("No hay una nómina calculada para confirmar.");
            PayrollStateMachine.DemandTransition(payroll.Estado, PayrollStates.Confirmed);
        }

        private static string CurrentUsername()
        {
            if (!SessionContext.IsAuthenticated || string.IsNullOrWhiteSpace(SessionContext.Username))
                throw new UnauthorizedAccessException("Debe iniciar sesión para gestionar la nómina.");
            return SessionContext.Username;
        }
    }
}
```

Services/PayrollPeriodPolicyService.cs

```
using System;
using SistemaGestionNomina.Data;
using SistemaGestionNomina.Models;

namespace SistemaGestionNomina.Services
{
    public class PayrollPeriodPolicyService
    {
        private readonly NominaRepository nominaRepository = new NominaRepository();

        public bool EstaAbierto(DateTime fecha)
        {
            return nominaRepository.GetOverlappingProtectedPeriod(fecha.Date, fecha.Date) == null;
        }

        public bool EstaAbierto(DateTime fechaInicio, DateTime fechaFin)
        {
            if (fechaInicio.Date > fechaFin.Date) return false;
            return nominaRepository.GetOverlappingProtectedPeriod(fechaInicio.Date, fechaFin.Date) == null;
        }

        public void VerificarPeriodoAbierto(int idPeriodo)
        {
            PeriodoNomina periodo = nominaRepository.GetPeriodoById(idPeriodo);
            if (periodo == null)
                throw new InvalidOperationException("El período no existe.");
            if (periodo.Cerrado)
                throw new InvalidOperationException("El período ya está cerrado. No se pueden realizar cambios.");
        }

        public void VerificarFechasAbiertas(DateTime fechaInicio, DateTime fechaFin)
        {
            if (fechaInicio.Date > fechaFin.Date)
                throw new ArgumentException("La fecha inicial no puede superar la fecha final.");
            if (!EstaAbierto(fechaInicio, fechaFin))
                throw new InvalidOperationException("El rango seleccionado se solapa con un período cerrado o confirmado.");
        }

        public void VerificarCambioLaboralPermitido(DateTime fechaEfectiva)
        {
            if (nominaRepository.HasClosedPeriodAffectedByEmployeeChange(fechaEfectiva.Date))
                throw new InvalidOperationException("La fecha efectiva del cambio afectaría una nómina cerrada.");
        }
    }
}
```

Services/PdfExportService.cs

```
using System;
using System.Collections.Generic;
using PdfSharp.Drawing;
using PdfSharp.Pdf;
using SistemaGestionNomina.Helpers;
using SistemaGestionNomina.Models;
using SistemaGestionNomina.Security;

namespace SistemaGestionNomina.Services
{
    public class PdfExportService
    {
        private readonly AuthorizationService authorizationService = new AuthorizationService();
        private readonly AuditTrailService auditTrailService = new AuditTrailService();
        private readonly EmployeeScopeService employeeScopeService = new EmployeeScopeService();

        public string ExportarEmpleados(List<Empleado> empleados)
        {
            authorizationService.DemandAny(Permissions.EmployeesExport, Permissions.ReportsPersonal);
            string path = BuildPath("Empleados");
            if (string.IsNullOrWhiteSpace(path)) return string.Empty;

            PdfDocument document = CreateDocument("Empleados");
            // Cada reporte crea un documento independiente para que pueda guardarse donde indique el usuario.
            PdfPage page = document.AddPage();
            XGraphics gfx = XGraphics.FromPdfPage(page);
            DrawTitle(gfx, "Reporte de Empleados");
            int y = 80;
            for (int i = 0; i < empleados.Count; i++)
            {
                Empleado e = empleados[i];
                DrawLine(gfx, 40, y, e.Codigo + " | " + e.NombreCompleto + " | " + e.DepartamentoNombre + " | B/. " + e.SalarioBase.ToString("0.00") + "  \
    | " + e.Estado);
                y = NextLine(document, ref page, ref gfx, y);
            }
            document.Save(path);
            auditTrailService.RegistrarAccion("Exportaciones", "PDF empleados", System.IO.Path.GetFileName(path));
            return path;
        }

        public string ExportarAsistencias(List<Asistencia> asistencias)
        {
            authorizationService.DemandPermission(Permissions.AttendanceExport);
            string path = BuildPath("Asistencia");
            if (string.IsNullOrWhiteSpace(path)) return string.Empty;

            PdfDocument document = CreateDocument("Asistencia");
            PdfPage page = document.AddPage();
            XGraphics gfx = XGraphics.FromPdfPage(page);
            DrawTitle(gfx, "Reporte de Asistencia");
            int y = 80;
            for (int i = 0; i < asistencias.Count; i++)
            {
                Asistencia a = asistencias[i];
                DrawLine(gfx, 40, y, a.Fecha.ToString("dd/MM/yyyy") + " | " + a.EmpleadoNombre + " | " + a.HorasTrabajadas.ToString("0.00") + " h | " +  \
    a.Estado);
                y = NextLine(document, ref page, ref gfx, y);
            }
            document.Save(path);
            auditTrailService.RegistrarAccion("Exportaciones", "PDF asistencia", System.IO.Path.GetFileName(path));
```

Services/PdfExportService.cs (continuacion)

```
            return path;
        }

        public string ExportarHistorialEmpleados(List<HistorialEmpleado> history)
        {
            authorizationService.DemandPermission(Permissions.EmployeeHistoryExport);
            string path = BuildPath("Historial_Laboral");
            if (string.IsNullOrWhiteSpace(path)) return string.Empty;
            PdfDocument document = CreateDocument("Historial laboral");
            PdfPage page = document.AddPage();
            XGraphics gfx = XGraphics.FromPdfPage(page);
            DrawTitle(gfx, "Historial Laboral");
            int y = 80;
            for (int i = 0; i < history.Count; i++)
            {
                HistorialEmpleado item = history[i];
                DrawLine(gfx, 40, y, item.FechaEfectiva.ToString("dd/MM/yyyy") + " | " +
                    item.CodigoEmpleado + " | " + item.CampoModificado + " | " +
                    item.ValorAnterior + " -> " + item.ValorNuevo + " | " + item.UsuarioResponsable);
                y = NextLine(document, ref page, ref gfx, y);
            }
            document.Save(path);
            auditTrailService.RegistrarAccion("Exportaciones", "PDF historial laboral",
                System.IO.Path.GetFileName(path));
            return path;
        }

        public string ExportarAusencias(List<SolicitudAusencia> requests)
        {
            authorizationService.DemandPermission(Permissions.AbsencesExport);
            string path = BuildPath("Solicitudes_Ausencia");
            if (string.IsNullOrWhiteSpace(path)) return string.Empty;
            PdfDocument document = CreateDocument("Solicitudes de ausencia");
            PdfPage page = document.AddPage();
            XGraphics gfx = XGraphics.FromPdfPage(page);
            DrawTitle(gfx, "Solicitudes de Ausencia");
            int y = 80;
            for (int i = 0; i < requests.Count; i++)
            {
                SolicitudAusencia item = requests[i];
                DrawLine(gfx, 40, y, item.CodigoEmpleado + " | " + item.EmpleadoNombre + " | " +
                    item.Tipo + " | " + item.FechaInicio.ToString("dd/MM") + " - " +
                    item.FechaFin.ToString("dd/MM/yyyy") + " | " + item.Estado);
                y = NextLine(document, ref page, ref gfx, y);
            }
            document.Save(path);
            auditTrailService.RegistrarAccion("Exportaciones", "PDF ausencias",
                System.IO.Path.GetFileName(path));
            return path;
        }

        public string ExportarNomina(Nomina nomina)
        {
            authorizationService.DemandPermission(Permissions.PayrollExport);
            string path = BuildPath("Nomina");
            if (string.IsNullOrWhiteSpace(path)) return string.Empty;

            PdfDocument document = CreateDocument("Nómina");
            PdfPage page = document.AddPage();
            XGraphics gfx = XGraphics.FromPdfPage(page);
```

Services/PdfExportService.cs (continuacion)

```
            DrawTitle(gfx, nomina.PeriodoNombre);
            int y = 80;
            for (int i = 0; i < nomina.Detalles.Count; i++)
            {
                NominaDetalle d = nomina.Detalles[i];
                DrawLine(gfx, 40, y, d.CodigoEmpleado + " | " + d.EmpleadoNombre + " | Ingresos B/. " + d.TotalIngresos.ToString("0.00") + " |  \
        Deducciones B/. " + d.TotalDeducciones.ToString("0.00") + " | Neto B/. " + d.NetoPagar.ToString("0.00"));
                y = NextLine(document, ref page, ref gfx, y);
            }
            y += 20;
            DrawLine(gfx, 40, y, "Total neto: B/. " + nomina.TotalNeto.ToString("0.00"));
            document.Save(path);
            auditTrailService.RegistrarAccion("Exportaciones", "PDF nómina", System.IO.Path.GetFileName(path));
            return path;
        }

        public string ExportarComprobante(Comprobante comprobante)
        {
            authorizationService.DemandPermission(Permissions.PayslipsExport);
            string path = PathHelper.RequestExportPath(comprobante.NumeroComprobante, ".pdf", "Documento PDF (*.pdf)|*.pdf");
            if (string.IsNullOrWhiteSpace(path)) return string.Empty;

            SaveComprobantePdf(comprobante, path);
            auditTrailService.RegistrarAccion("Exportaciones", "PDF comprobante", System.IO.Path.GetFileName(path));
            return path;
        }

        public string ExportarComprobantePersonal(Comprobante comprobante)
        {
            authorizationService.DemandPermission(Permissions.OwnPayslipsDownload);
            if (comprobante == null) throw new ArgumentNullException("comprobante");
            employeeScopeService.DemandCurrentEmployee(comprobante.IdEmpleado);
            string path = PathHelper.RequestExportPath(comprobante.NumeroComprobante, ".pdf", "Documento PDF (*.pdf)|*.pdf");
            return ExportarComprobantePersonal(comprobante, path);
        }

        internal string ExportarComprobantePersonal(Comprobante comprobante, string path)
        {
            authorizationService.DemandPermission(Permissions.OwnPayslipsDownload);
            if (comprobante == null) throw new ArgumentNullException("comprobante");
            employeeScopeService.DemandCurrentEmployee(comprobante.IdEmpleado);
            if (string.IsNullOrWhiteSpace(path)) return string.Empty;
            if (!string.Equals(System.IO.Path.GetExtension(path), ".pdf", StringComparison.OrdinalIgnoreCase))
            {
                throw new ArgumentException("La ubicacion seleccionada debe ser un archivo PDF.");
            }

            string fullPath = System.IO.Path.GetFullPath(path);
            SaveComprobantePdf(comprobante, fullPath);
            return fullPath;
        }

        public string ExportarReportes(List<ReporteGenerado> reportes)
        {
            authorizationService.DemandPermission(Permissions.ReportsExport);
            string path = BuildPath("Reportes");
            if (string.IsNullOrWhiteSpace(path)) return string.Empty;

            PdfDocument document = CreateDocument("Reportes");
            PdfPage page = document.AddPage();
```

Services/PdfExportService.cs (continuacion)

```
            XGraphics gfx = XGraphics.FromPdfPage(page);
            DrawTitle(gfx, "Reportes Generados");
            int y = 80;
            for (int i = 0; i < reportes.Count; i++)
            {
                ReporteGenerado r = reportes[i];
                DrawLine(gfx, 40, y, r.NombreReporte + " | " + r.Tipo + " | " + r.FechaGeneracion.ToString("dd/MM/yyyy HH:mm"));
                y = NextLine(document, ref page, ref gfx, y);
            }
            document.Save(path);
            auditTrailService.RegistrarAccion("Exportaciones", "PDF reportes", System.IO.Path.GetFileName(path));
            return path;
        }

        private static PdfDocument CreateDocument(string title)
        {
            PdfDocument document = new PdfDocument();
            document.Info.Title = title;
            return document;
        }

        private static void SaveComprobantePdf(Comprobante comprobante, string path)
        {
            PdfDocument document = CreateDocument("Comprobante");
            PdfPage page = document.AddPage();
            XGraphics gfx = XGraphics.FromPdfPage(page);
            DrawTitle(gfx, "ProyFinal_LPI_Eq01_NomiCore");
            DrawLine(gfx, 40, 82, "COMPROBANTE DE PAGO");
            DrawLine(gfx, 40, 120, "Numero: " + comprobante.NumeroComprobante);
            DrawLine(gfx, 40, 145, "Empleado: " + comprobante.EmpleadoNombre + " (" + comprobante.CodigoEmpleado + ")");
            DrawLine(gfx, 40, 170, "Periodo: " + comprobante.PeriodoNombre);
            DrawLine(gfx, 40, 220, "Total ingresos: B/. " + comprobante.TotalIngresos.ToString("0.00"));
            DrawLine(gfx, 40, 245, "Total deducciones: B/. " + comprobante.TotalDeducciones.ToString("0.00"));
            DrawLine(gfx, 40, 280, "Neto a pagar: B/. " + comprobante.NetoPagar.ToString("0.00"));
            DrawLine(gfx, 40, 340, "Documento generado para fines academicos.");
            document.Save(path);
        }

        private static string BuildPath(string prefix)
        {
            return PathHelper.RequestExportPath(prefix, ".pdf", "Documento PDF (*.pdf)|*.pdf");
        }

        private static void DrawTitle(XGraphics gfx, string title)
        {
            XFont font = new XFont("Segoe UI", 18, XFontStyleEx.Bold);
            gfx.DrawString(title, font, XBrushes.Black, new XRect(40, 35, 520, 30), XStringFormats.TopLeft);
        }

        private static void DrawLine(XGraphics gfx, int x, int y, string text)
        {
            XFont font = new XFont("Segoe UI", 10, XFontStyleEx.Regular);
            gfx.DrawString(text, font, XBrushes.Black, new XRect(x, y, 520, 18), XStringFormats.TopLeft);
        }

        private static int NextLine(PdfDocument document, ref PdfPage page, ref XGraphics gfx, int currentY)
        {
            int next = currentY + 22;
            if (next > 760)
            {
```

Services/PdfExportService.cs (continuacion)

```
                page = document.AddPage();
                gfx = XGraphics.FromPdfPage(page);
                return 45;
            }

            return next;
        }
    }
}
```

Services/PdfFontResolver.cs

```
using System.Collections.Generic;
using System.IO;
using PdfSharp.Fonts;

namespace SistemaGestionNomina.Services
{
    public class PdfFontResolver : IFontResolver
    {
        private const string RegularFace = "SegoeUI#Regular";
        private const string BoldFace = "SegoeUI#Bold";
        private static bool registered;

        public static void EnsureRegistered()
        {
            if (!registered && GlobalFontSettings.FontResolver == null)
            {
                GlobalFontSettings.FontResolver = new PdfFontResolver();
                registered = true;
            }
        }

        public FontResolverInfo ResolveTypeface(string familyName, bool isBold, bool isItalic)
        {
            return new FontResolverInfo(isBold ? BoldFace : RegularFace);
        }

        public byte[] GetFont(string faceName)
        {
            Dictionary<string, string[]> candidates = new Dictionary<string, string[]>();
            candidates[RegularFace] = new[] { "segoeui.ttf", "arial.ttf" };
            candidates[BoldFace] = new[] { "segoeuib.ttf", "arialbd.ttf", "arial.ttf" };

            string windowsFonts = Path.Combine(System.Environment.GetFolderPath(System.Environment.SpecialFolder.Windows), "Fonts");
            string[] files = candidates.ContainsKey(faceName) ? candidates[faceName] : candidates[RegularFace];
            for (int i = 0; i < files.Length; i++)
            {
                string path = Path.Combine(windowsFonts, files[i]);
                if (File.Exists(path))
                {
                    return File.ReadAllBytes(path);
                }
            }

            throw new FileNotFoundException("No se encontró una fuente compatible para generar el PDF.");
        }
    }
}
```

Services/RolePermissionService.cs

```
using System;
using System.Collections.Generic;
using SistemaGestionNomina.Security;

namespace SistemaGestionNomina.Services
{
    /// <summary>
    /// Servicio reservado para permisos avanzados por rol.
    /// </summary>
    public class RolePermissionService
    {
        private readonly Dictionary<string, HashSet<string>> permisosPorRol;

        public RolePermissionService()
        {
            permisosPorRol = new Dictionary<string, HashSet<string>>(StringComparer.OrdinalIgnoreCase);
            permisosPorRol[Roles.Admin] = new HashSet<string>(StringComparer.OrdinalIgnoreCase) { "*" };
            permisosPorRol[Roles.RRHH] = new HashSet<string>(StringComparer.OrdinalIgnoreCase)
            {
                Permissions.EmployeesView, Permissions.EmployeesCreate, Permissions.EmployeesEdit,
                Permissions.EmployeesDeactivate, Permissions.EmployeesExport,
                Permissions.EmployeeHistoryView, Permissions.EmployeeHistoryExport,
                Permissions.EmployeeChangesSchedule,
                Permissions.AttendanceView, Permissions.AttendanceRegister, Permissions.AttendanceImport,
                Permissions.AttendanceExport, Permissions.PayslipsView, Permissions.PayslipsExport,
                Permissions.PayslipsPrint, Permissions.PayslipsEmail,
                Permissions.ReportsView, Permissions.ReportsPersonal, Permissions.ReportsExport,
                Permissions.PayrollHistoryView,
                Permissions.AbsencesView, Permissions.AbsencesCreate, Permissions.AbsencesApprove,
                Permissions.AbsencesReject, Permissions.AbsencesCancel, Permissions.AbsencesExport
            };
            permisosPorRol[Roles.Contabilidad] = new HashSet<string>(StringComparer.OrdinalIgnoreCase)
            {
                Permissions.PayrollView, Permissions.PayrollCalculate, Permissions.PayrollConfirm,
                Permissions.PayrollExport, Permissions.PayslipsView, Permissions.PayslipsExport,
                Permissions.PayslipsPrint, Permissions.PayslipsEmail,
                Permissions.ReportsView, Permissions.ReportsFinancial, Permissions.ReportsExport,
                Permissions.PayrollPay, Permissions.PayrollAnnul, Permissions.PayrollRecalculate,
                Permissions.PayrollHistoryView, Permissions.AbsencesView
            };
            permisosPorRol[Roles.Supervisor] = new HashSet<string>(StringComparer.OrdinalIgnoreCase)
            {
                Permissions.AttendanceView, Permissions.AttendanceRegister, Permissions.AttendanceExport,
                Permissions.AbsencesView, Permissions.AbsencesApprove, Permissions.AbsencesReject
            };
            permisosPorRol[Roles.Trabajador] = new HashSet<string>(StringComparer.OrdinalIgnoreCase)
            {
                Permissions.PortalView, Permissions.OwnProfileView, Permissions.OwnAttendanceView,
                Permissions.OwnPayslipsView, Permissions.OwnPayslipsDownload, Permissions.OwnPasswordChange,
                Permissions.OwnAbsencesView, Permissions.OwnAbsencesCreate, Permissions.OwnAbsencesCancel
            };

            HashSet<string> admin = permisosPorRol[Roles.Admin];
            admin.Add(Permissions.DashboardView);
        }

        /// <summary>
        /// Indica si un rol tiene permiso para ejecutar una acción.
        /// </summary>
        public bool TienePermiso(string rol, string accion)
```

Services/RolePermissionService.cs (continuacion)

```
        {
            if (string.IsNullOrWhiteSpace(rol) || string.IsNullOrWhiteSpace(accion))
            {
                return false;
            }

            HashSet<string> permisos;
            if (!permisosPorRol.TryGetValue(rol.Trim(), out permisos))
            {
                return false;
            }

            return permisos.Contains("*") || permisos.Contains(accion.Trim());
        }

        /// <summary>
        /// Indica si un rol puede ejecutar una accion especifica en un modulo.
        /// </summary>
        public bool TienePermiso(string rol, string modulo, string accion)
        {
            if (string.IsNullOrWhiteSpace(rol))
            {
                return false;
            }

            HashSet<string> permisos;
            if (!permisosPorRol.TryGetValue(rol.Trim(), out permisos))
            {
                return false;
            }

            if (permisos.Contains("*"))
            {
                return true;
            }

            string key = (modulo ?? string.Empty).Trim().ToLowerInvariant() + "." + (accion ?? string.Empty).Trim().ToLowerInvariant();
            return permisos.Contains(key);
        }

        public bool EsRolValido(string rol)
        {
            return !string.IsNullOrWhiteSpace(rol) && permisosPorRol.ContainsKey(rol.Trim());
        }

        /// <summary>
        /// Devuelve las acciones permitidas para mostrar u ocultar opciones de interfaz.
        /// </summary>
        public List<string> ObtenerPermisos(string rol)
        {
            List<string> result = new List<string>();
            if (string.IsNullOrWhiteSpace(rol))
            {
                return result;
            }

            HashSet<string> permisos;
            if (permisosPorRol.TryGetValue(rol.Trim(), out permisos))
            {
                foreach (string permiso in permisos)
```

Services/RolePermissionService.cs (continuacion)

```
                {
                    result.Add(permiso);
                }
            }

            return result;
        }
    }
}
```

Tools/Fase2PdfSmokeHost.cs

```
using System;
using System.IO;
using System.Reflection;
using SistemaGestionNomina.Data;
using SistemaGestionNomina.Models;
using SistemaGestionNomina.Security;
using SistemaGestionNomina.Services;

internal static class Fase2PdfSmokeHost
{
    private static int Main(string[] args)
    {
        if (args.Length != 6) return 2;
        try
        {
            string dbPath = args[0];
            string pdfPath = args[1];
            int ownPayslipId = int.Parse(args[2]);
            int foreignPayslipId = int.Parse(args[3]);
            int userId = int.Parse(args[4]);
            string username = args[5];
            Environment.SetEnvironmentVariable("NOMINA_DB_PATH", dbPath);
            DatabaseInitializer.Initialize();
            SessionContext.Begin(new Usuario
            {
                IdUsuario = userId,
                NombreUsuario = username,
                Rol = Roles.Trabajador,
                Estado = "Activo",
                IdEmpleado = 1
            });

            EmployeePortalService service = new EmployeePortalService();
            MethodInfo download = null;
            MethodInfo[] methods = typeof(EmployeePortalService).GetMethods(BindingFlags.Instance | BindingFlags.NonPublic);
            for (int i = 0; i < methods.Length; i++)
            {
                if (methods[i].Name == "DownloadMyPayslipPdf" && methods[i].GetParameters().Length == 2)
                {
                    download = methods[i];
                    break;
                }
            }

            if (download == null) throw new InvalidOperationException("No se encontro la descarga interna de prueba.");
            string generated = Convert.ToString(download.Invoke(service, new object[] { ownPayslipId, pdfPath }));
            if (!File.Exists(generated)) throw new InvalidOperationException("No se genero el PDF personal.");
            byte[] bytes = File.ReadAllBytes(generated);
            if (bytes.Length < 4 || bytes[0] != 37 || bytes[1] != 80 || bytes[2] != 68 || bytes[3] != 70)
            {
                throw new InvalidOperationException("El archivo generado no tiene cabecera PDF.");
            }

            bool rejected = false;
            try
            {
                download.Invoke(service, new object[] { foreignPayslipId, Path.Combine(Path.GetDirectoryName(pdfPath), "ajeno.pdf") });
            }
            catch (TargetInvocationException ex)
            {
```

Tools/Fase2PdfSmokeHost.cs (continuacion)

```
                rejected = ex.InnerException is UnauthorizedAccessException;
            }

            if (!rejected) throw new InvalidOperationException("La descarga permitio un comprobante ajeno.");
            SessionContext.Clear();
            Console.WriteLine("PDF personal y rechazo de archivo ajeno OK");
            return 0;
        }
        catch (Exception ex)
        {
            Console.Error.WriteLine(ex);
            return 1;
        }
    }
}
```

Tools/Fase3NominaSmokeHost.cs

```
using System;
using System.Data.SQLite;
using SistemaGestionNomina.Data;
using SistemaGestionNomina.Models;
using SistemaGestionNomina.Security;
using SistemaGestionNomina.Services;

internal static class Fase3NominaSmokeHost
{
    private static int Main(string[] args)
    {
        try
        {
            if (args.Length != 1) throw new ArgumentException("Debe indicar la base temporal.");
            Environment.SetEnvironmentVariable("NOMINA_DB_PATH", args[0]);
            DatabaseInitializer.Initialize();
            DatabaseInitializer.Initialize();
            Assert(ScalarInt("PRAGMA user_version;") == DatabaseMigrationRunner.LatestVersion,
                "Las migraciones no alcanzaron la última versión.");
            Assert(ScalarInt("SELECT COUNT(1) FROM MigracionesLog WHERE Version IN (4,5,6);") == 3,
                "No se registraron las migraciones 4, 5 y 6.");

            SessionContext.Begin(new Usuario
            {
                IdUsuario = 1,
                NombreUsuario = "admin_smoke_f3",
                Rol = Roles.Admin,
                Estado = "Activo"
            });

            DateTime januaryStart = new DateTime(2034, 1, 1);
            DateTime januaryEnd = new DateTime(2034, 1, 15);
            AsistenciaService attendanceService = new AsistenciaService();
            attendanceService.Register(new Asistencia
            {
                IdEmpleado = 1,
                Fecha = new DateTime(2034, 1, 2),
                HoraEntrada = new TimeSpan(8, 0, 0),
                HoraSalida = new TimeSpan(17, 0, 0),
                Estado = "Puntual"
            });

            NominaService payrollService = new NominaService();
            Nomina january = payrollService.CalcularNomina(januaryStart, januaryEnd, null);
            int januaryId = payrollService.ConfirmarPago(january, januaryStart, januaryEnd);
            Assert(ScalarInt("SELECT COUNT(1) FROM PeriodosNomina WHERE Cerrado=1 AND FechaInicio='2034-01-01' AND FechaFin='2034-01-15';") == 1,
                "La confirmación no cerró el período.");

            ExpectFailure(delegate
            {
                attendanceService.Register(new Asistencia
                {
                    IdEmpleado = 1, Fecha = new DateTime(2034, 1, 3),
                    HoraEntrada = new TimeSpan(8, 0, 0), HoraSalida = new TimeSpan(17, 0, 0), Estado = "Puntual"
                });
            }, "El servicio permitió asistencia en período cerrado.");
            ExpectSqlFailure("INSERT INTO Asistencias (IdEmpleado,Fecha,HorasTrabajadas,Estado) VALUES (1,'2034-01-04',0,'Falta');",
                "El trigger permitió insertar asistencia en período cerrado.");
            ExpectSqlFailure("UPDATE Asistencias SET Estado='Tardanza' WHERE IdEmpleado=1 AND Fecha='2034-01-02';",
                "El trigger permitió actualizar asistencia en período cerrado.");
```

Tools/Fase3NominaSmokeHost.cs (continuacion)

```
            ExpectFailure(delegate { payrollService.CalcularNomina(new DateTime(2034, 1, 5), new DateTime(2034, 1, 20), null); },
                "Se permitió calcular un rango solapado.");
            ExpectSqlFailure(@"INSERT INTO PeriodosNomina
                (Nombre,FechaInicio,FechaFin,Estado,Cerrado) VALUES ('Solapado','2034-01-10','2034-01-20','Confirmado',1);",
                "SQLite permitió un período protegido solapado.");

            payrollService.PagarNomina(januaryId);
            Assert(ScalarString("SELECT Estado FROM Nominas WHERE IdNomina=" + januaryId + ";") == PayrollStates.Paid,
                "La nómina confirmada no pasó a Pagada.");
            ExpectFailure(delegate { payrollService.PagarNomina(januaryId); }, "Se pagó dos veces la misma nómina.");
            payrollService.AnularNomina(januaryId, "Corrección integral de smoke test");
            Assert(ScalarString("SELECT Estado FROM Nominas WHERE IdNomina=" + januaryId + ";") == PayrollStates.Annulled,
                "La nómina pagada no se anuló con trazabilidad.");
            Assert(ScalarInt("SELECT COUNT(1) FROM NominaDetalle WHERE IdNomina=" + januaryId + ";") > 0 &&
                ScalarInt("SELECT COUNT(1) FROM Comprobantes WHERE IdNomina=" + januaryId + ";") > 0,
                "La anulación eliminó detalles o comprobantes.");
            ExpectFailure(delegate { payrollService.PagarNomina(januaryId); }, "Se permitió pagar una nómina anulada.");

            Nomina januaryRecalculated = payrollService.CalcularNomina(januaryStart, januaryEnd, null);
            int januaryNewId = payrollService.RecalcularNomina(januaryId, januaryStart, januaryEnd, januaryRecalculated);
            Assert(ScalarInt("SELECT COUNT(1) FROM NominaVersiones WHERE IdNominaOriginal=" + januaryId +
                " AND IdNominaNueva=" + januaryNewId + ";") == 1, "El recálculo no vinculó la versión nueva.");

            DateTime februaryStart = new DateTime(2034, 2, 1);
            DateTime februaryEnd = new DateTime(2034, 2, 15);
            Nomina february = payrollService.CalcularNomina(februaryStart, februaryEnd, null);
            int februaryId = payrollService.ConfirmarPago(february, februaryStart, februaryEnd);
            payrollService.AnularNomina(februaryId, "Preparar prueba de rollback");
            Nomina februaryRecalculated = payrollService.CalcularNomina(februaryStart, februaryEnd, null);
            Execute(@"CREATE TRIGGER Smoke_Fallar_Version BEFORE UPDATE ON NominaVersiones
                BEGIN SELECT RAISE(ABORT, 'Fallo controlado de version'); END;");
            int periodCount = ScalarInt("SELECT COUNT(1) FROM PeriodosNomina;");
            int payrollCount = ScalarInt("SELECT COUNT(1) FROM Nominas;");
            int detailCount = ScalarInt("SELECT COUNT(1) FROM NominaDetalle;");
            int payslipCount = ScalarInt("SELECT COUNT(1) FROM Comprobantes;");
            int auditCount = ScalarInt("SELECT COUNT(1) FROM Auditoria;");
            ExpectFailure(delegate
            {
                payrollService.RecalcularNomina(februaryId, februaryStart, februaryEnd, februaryRecalculated);
            }, "El recálculo no propagó el fallo controlado.");
            Execute("DROP TRIGGER Smoke_Fallar_Version;");
            Assert(periodCount == ScalarInt("SELECT COUNT(1) FROM PeriodosNomina;") &&
                payrollCount == ScalarInt("SELECT COUNT(1) FROM Nominas;") &&
                detailCount == ScalarInt("SELECT COUNT(1) FROM NominaDetalle;") &&
                payslipCount == ScalarInt("SELECT COUNT(1) FROM Comprobantes;") &&
                auditCount == ScalarInt("SELECT COUNT(1) FROM Auditoria;"),
                "El recálculo fallido dejó datos parciales.");

            EmpleadoService employeeService = new EmpleadoService();
            Empleado blocked = employeeService.GetById(1);
            blocked.SalarioBase += 10m;
            ExpectFailure(delegate
            {
                employeeService.Save(blocked, new EmployeeChangeContext
                {
                    EffectiveDate = new DateTime(2034, 1, 10), Reason = "Cambio que afectaría período cerrado"
                });
            }, "Se permitió un cambio laboral que afecta nómina cerrada.");

            Empleado scheduled = employeeService.GetById(1);
```

Tools/Fase3NominaSmokeHost.cs (continuacion)

```
            decimal originalSalary = scheduled.SalarioBase;
            string originalDepartment = scheduled.DepartamentoNombre;
            scheduled.SalarioBase += 175m;
            scheduled.Cargo = scheduled.Cargo + " II";
            scheduled.IdDepartamento = 2;
            scheduled.Estado = "Inactivo";
            EmployeeSaveResult saveResult = employeeService.Save(scheduled, new EmployeeChangeContext
            {
                EffectiveDate = new DateTime(2034, 1, 20), Reason = "Prueba de cambios laborales programados"
            });
            Assert(saveResult.HistoryRecords == 4 && saveResult.HasScheduledChanges,
                "No se registraron los cuatro cambios programados.");
            Assert(employeeService.GetById(1).SalarioBase == originalSalary,
                "Un cambio futuro se aplicó antes de su fecha.");
            new EmployeeHistoryService().ApplyDueChanges(new DateTime(2034, 1, 20));
            Assert(employeeService.GetById(1).SalarioBase == originalSalary + 175m &&
                employeeService.GetById(1).Estado == "Inactivo", "Los cambios vencidos no se aplicaron.");
            Assert(ScalarInt("SELECT COUNT(1) FROM HistorialEmpleado WHERE IdEmpleado=1;") == 4,
                "El historial laboral no conservó todos los campos.");
            Assert(ScalarDecimal("SELECT SueldoBase FROM NominaDetalle WHERE IdNomina=" + januaryNewId + " AND IdEmpleado=1;") == originalSalary &&
                ScalarString("SELECT DepartamentoSnapshot FROM NominaDetalle WHERE IdNomina=" + januaryNewId + " AND IdEmpleado=1;") == originalDepartment,
                "Los snapshots históricos cambiaron después de modificar el empleado.");
            Assert(ScalarInt("SELECT COUNT(1) FROM Auditoria WHERE Usuario='admin_smoke_f3';") > 0,
                "La auditoría no registró al usuario real.");

            SessionContext.Clear();
            SQLiteConnection.ClearAllPools();
            Console.WriteLine("Migraciones, periodos, estados, rollback, historial y snapshots OK");
            return 0;
        }
        catch (Exception ex)
        {
            Console.Error.WriteLine(ex.ToString());
            try { SessionContext.Clear(); SQLiteConnection.ClearAllPools(); } catch { }
            return 1;
        }
    }

    private static void Assert(bool condition, string message)
    {
        if (!condition) throw new InvalidOperationException(message);
    }

    private static void ExpectFailure(Action action, string message)
    {
        try { action(); }
        catch { return; }
        throw new InvalidOperationException(message);
    }

    private static void ExpectSqlFailure(string sql, string message)
    {
        try { Execute(sql); }
        catch (SQLiteException) { return; }
        throw new InvalidOperationException(message);
    }

    private static void Execute(string sql)
    {
        using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
```

Tools/Fase3NominaSmokeHost.cs (continuacion)

```
        using (SQLiteCommand command = new SQLiteCommand(sql, connection)) command.ExecuteNonQuery();
    }

    private static object Scalar(string sql)
    {
        using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
        using (SQLiteCommand command = new SQLiteCommand(sql, connection)) return command.ExecuteScalar();
    }

    private static int ScalarInt(string sql) { return Convert.ToInt32(Scalar(sql)); }
    private static decimal ScalarDecimal(string sql) { return Convert.ToDecimal(Scalar(sql)); }
    private static string ScalarString(string sql) { return Convert.ToString(Scalar(sql)); }
}
```

Tools/Fase4RrhhAvanzadoSmokeHost.cs

```
using System;
using System.Collections.Generic;
using System.Data.SQLite;
using SistemaGestionNomina.Data;
using SistemaGestionNomina.Models;
using SistemaGestionNomina.Security;
using SistemaGestionNomina.Services;

internal static class Fase4RrhhAvanzadoSmokeHost
{
    private static int Main(string[] args)
    {
        try
        {
            if (args.Length != 1) throw new ArgumentException("Debe indicar la base temporal.");
            Environment.SetEnvironmentVariable("NOMINA_DB_PATH", args[0]);
            DatabaseInitializer.Initialize();
            Assert(ScalarInt("PRAGMA user_version;") == 6, "La base temporal no llegó a la migración v6.");
            Assert(ScalarInt("SELECT COUNT(1) FROM pragma_table_info('Asistencias') WHERE name='IdSolicitudAusencia';") == 1,
                "Falta la relación entre asistencia y solicitud.");

            RolePermissionService roles = new RolePermissionService();
            Assert(roles.TienePermiso(Roles.RRHH, Permissions.AbsencesApprove), "RRHH no recibió gestión de ausencias.");
            Assert(roles.TienePermiso(Roles.Supervisor, Permissions.AbsencesApprove), "Supervisor no recibió aprobación departamental.");
            Assert(!roles.TienePermiso(Roles.Contabilidad, Permissions.AbsencesApprove) &&
                roles.TienePermiso(Roles.Contabilidad, Permissions.AbsencesView), "Contabilidad tiene una matriz incorrecta.");
            Assert(roles.TienePermiso(Roles.Trabajador, Permissions.OwnAbsencesCreate) &&
                !roles.TienePermiso(Roles.Trabajador, Permissions.AbsencesView), "Trabajador recibió alcance administrativo.");

            DateTime monday = NextMonday(new DateTime(2035, 3, 1));
            AbsenceRequestService service = new AbsenceRequestService();
            BeginSession(40, "trabajador_f4", Roles.Trabajador, 1);
            int ownId = service.CreateOwn(new OwnAbsenceRequest
            {
                Type = AbsenceTypes.PaidLeave,
                StartDate = monday,
                EndDate = monday.AddDays(1),
                Reason = "Gestión personal de prueba"
            });
            List<SolicitudAusencia> ownItems = service.Search(new AbsenceQuery());
            Assert(ownItems.Count == 1 && ownItems[0].IdEmpleado == 1, "El portal no aisló solicitudes propias.");

            BeginSession(1, "admin_f4", Roles.Admin, null);
            int employeeTwoId = service.CreateForEmployee(NewRequest(2, AbsenceTypes.Vacation,
                monday, monday.AddDays(4), "Vacaciones de prueba"));
            BeginSession(40, "trabajador_f4", Roles.Trabajador, 1);
            ExpectUnauthorized(delegate { service.GetById(employeeTwoId); }, "El trabajador consultó una solicitud ajena.");
            service.Cancel(ownId, "Cancelación personal de prueba");
            Assert(ScalarString("SELECT Estado FROM SolicitudesAusencia WHERE IdSolicitud=" + ownId + ";") == AbsenceStates.Cancelled,
                "El trabajador no pudo cancelar su solicitud pendiente.");

            BeginSession(1, "admin_f4", Roles.Admin, null);
            service.Approve(employeeTwoId, "Aprobada por RRHH de prueba");
            Assert(ScalarInt("SELECT COUNT(1) FROM Asistencias WHERE IdSolicitudAusencia=" + employeeTwoId + ";") == 5,
                "La aprobación no creó exactamente cinco días laborables.");
            Assert(ScalarInt("SELECT COUNT(1) FROM Asistencias WHERE IdSolicitudAusencia=" + employeeTwoId + " AND Estado='Permiso';") == 5,
                "Vacaciones no se reflejaron como Permiso.");

            int overlapId = service.CreateForEmployee(NewRequest(2, AbsenceTypes.MedicalLeave,
                monday.AddDays(2), monday.AddDays(4), "Solicitud solapada"));
```

Tools/Fase4RrhhAvanzadoSmokeHost.cs (continuacion)

```
            ExpectFailure(delegate { service.Approve(overlapId, "No debe aprobar"); },
                "Se aprobó una ausencia solapada.");
            Assert(ScalarString("SELECT Estado FROM SolicitudesAusencia WHERE IdSolicitud=" + overlapId + ";") == AbsenceStates.Pending,
                "La aprobación fallida cambió el estado de la solicitud.");

            service.Cancel(employeeTwoId, "Cancelar ausencia aprobada");
            Assert(ScalarInt("SELECT COUNT(1) FROM Asistencias WHERE IdSolicitudAusencia=" + employeeTwoId + ";") == 0,
                "La cancelación no retiró las asistencias generadas.");

            DateTime conflictDate = monday.AddDays(14);
            Execute("INSERT INTO Asistencias (IdEmpleado,Fecha,HoraEntrada,HoraSalida,HorasTrabajadas,Estado) VALUES (2,'" +
                conflictDate.ToString("yyyy-MM-dd") + "','08:00','17:00',9,'Puntual');");
            int conflictId = service.CreateForEmployee(NewRequest(2, AbsenceTypes.UnpaidLeave,
                conflictDate, conflictDate, "Permiso con asistencia existente"));
            ExpectFailure(delegate { service.Approve(conflictId, "Debe fallar sin sobrescribir"); },
                "Se sobrescribió una asistencia existente.");
            Assert(ScalarInt("SELECT COUNT(1) FROM Asistencias WHERE IdEmpleado=2 AND Fecha='" +
                conflictDate.ToString("yyyy-MM-dd") + "';") == 1 &&
                ScalarString("SELECT Estado FROM SolicitudesAusencia WHERE IdSolicitud=" + conflictId + ";") == AbsenceStates.Pending,
                "El conflicto de asistencia dejó una operación parcial.");

            DateTime suspensionDate = monday.AddDays(21);
            int suspensionId = service.CreateForEmployee(NewRequest(3, AbsenceTypes.Suspension,
                suspensionDate, suspensionDate, "Suspensión académica"));
            service.Approve(suspensionId, "Aprobada");
            Assert(ScalarString("SELECT Estado FROM Asistencias WHERE IdSolicitudAusencia=" + suspensionId + ";") == "Falta",
                "La suspensión no se reflejó como Falta.");

            DateTime supervisorDate = monday.AddDays(28);
            int departmentRequestId = service.CreateForEmployee(NewRequest(4, AbsenceTypes.License,
                supervisorDate, supervisorDate.AddDays(1), "Solicitud de Operaciones"));
            int foreignDepartmentRequestId = service.CreateForEmployee(NewRequest(2, AbsenceTypes.PaidLeave,
                supervisorDate, supervisorDate, "Solicitud de Ventas"));
            BeginSession(50, "supervisor_f4", Roles.Supervisor, 4);
            service.Approve(departmentRequestId, "Aprobación departamental");
            ExpectUnauthorized(delegate { service.Reject(foreignDepartmentRequestId, "Fuera de alcance"); },
                "El supervisor resolvió una solicitud de otro departamento.");
            List<SolicitudAusencia> scoped = service.Search(new AbsenceQuery());
            for (int i = 0; i < scoped.Count; i++)
                Assert(scoped[i].DepartamentoNombre == "Operaciones", "La consulta del supervisor filtró otro departamento.");

            BeginSession(60, "contabilidad_f4", Roles.Contabilidad, null);
            Assert(service.Search(new AbsenceQuery()).Count > 0, "Contabilidad no pudo consultar ausencias.");
            ExpectUnauthorized(delegate { service.Approve(foreignDepartmentRequestId, "No autorizado"); },
                "Contabilidad aprobó una solicitud.");

            BeginSession(1, "admin_f4", Roles.Admin, null);
            DateTime pendingClosedDate = monday.AddDays(70);
            int pendingClosedId = service.CreateForEmployee(NewRequest(3, AbsenceTypes.JustifiedAbsence,
                pendingClosedDate, pendingClosedDate, "Pendiente antes del cierre"));
            InsertClosedPeriod(pendingClosedDate, pendingClosedDate, "Cierre posterior");
            ExpectFailure(delegate { service.Reject(pendingClosedId, "No puede alterar período cerrado"); },
                "Se modificó una ausencia después de cerrar el período.");
            ExpectSqlFailure("UPDATE SolicitudesAusencia SET Estado='Rechazada' WHERE IdSolicitud=" + pendingClosedId + ";",
                "SQLite permitió modificar una ausencia de período cerrado.");

            DateTime closedStart = monday.AddDays(84);
            InsertClosedPeriod(closedStart, closedStart.AddDays(4), "Período cerrado de ausencia");
            ExpectFailure(delegate
            {
```

Tools/Fase4RrhhAvanzadoSmokeHost.cs (continuacion)

```
                service.CreateForEmployee(NewRequest(2, AbsenceTypes.Vacation,
                    closedStart, closedStart.AddDays(1), "Debe bloquearse"));
            }, "Se creó una solicitud dentro de un período cerrado.");
            ExpectSqlFailure(@"INSERT INTO SolicitudesAusencia
                (IdEmpleado,Tipo,FechaInicio,FechaFin,Estado,Motivo,UsuarioSolicitante,FechaCreacion)
                VALUES (2,'Vacaciones','" + closedStart.ToString("yyyy-MM-dd") + "','" +
                closedStart.AddDays(1).ToString("yyyy-MM-dd") + "','Pendiente','Directa','sql','2035-01-01');",
                "SQLite permitió insertar una ausencia dentro de un período cerrado.");

            Assert(ScalarInt("SELECT COUNT(1) FROM Auditoria WHERE Modulo='Ausencias' AND Usuario IN ('admin_f4','trabajador_f4','supervisor_f4');") > 0,
                "La auditoría no registró a los usuarios reales de ausencias.");
            SessionContext.Clear();
            SQLiteConnection.ClearAllPools();
            Console.WriteLine("Permisos, alcance, ausencias, asistencia, cancelación y periodos cerrados OK");
            return 0;
        }
        catch (Exception ex)
        {
            Console.Error.WriteLine(ex.ToString());
            try { SessionContext.Clear(); SQLiteConnection.ClearAllPools(); } catch { }
            return 1;
        }
    }

    private static SolicitudAusencia NewRequest(int employeeId, string type, DateTime start, DateTime end, string reason)
    {
        return new SolicitudAusencia { IdEmpleado = employeeId, Tipo = type, FechaInicio = start, FechaFin = end, Motivo = reason };
    }

    private static void BeginSession(int id, string username, string role, int? employeeId)
    {
        SessionContext.Begin(new Usuario
        {
            IdUsuario = id, NombreUsuario = username, Rol = role, Estado = "Activo", IdEmpleado = employeeId
        });
    }

    private static DateTime NextMonday(DateTime date)
    {
        DateTime result = date.Date;
        while (result.DayOfWeek != DayOfWeek.Monday) result = result.AddDays(1);
        return result;
    }

    private static void InsertClosedPeriod(DateTime start, DateTime end, string name)
    {
        using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
        using (SQLiteCommand command = new SQLiteCommand(@"INSERT INTO PeriodosNomina
            (Nombre,FechaInicio,FechaFin,Estado,Cerrado,FechaCierre,CerradoPor)
            VALUES (@nombre,@inicio,@fin,'Confirmado',1,@cierre,'admin_f4');", connection))
        {
            command.Parameters.AddWithValue("@nombre", name);
            command.Parameters.AddWithValue("@inicio", start.ToString("yyyy-MM-dd"));
            command.Parameters.AddWithValue("@fin", end.ToString("yyyy-MM-dd"));
            command.Parameters.AddWithValue("@cierre", DateTime.Now.ToString("yyyy-MM-dd HH:mm:ss"));
            command.ExecuteNonQuery();
        }
    }

    private static void Assert(bool condition, string message) { if (!condition) throw new InvalidOperationException(message); }
```

Tools/Fase4RrhhAvanzadoSmokeHost.cs (continuacion)

```
    private static void ExpectFailure(Action action, string message) { try { action(); } catch { return; } throw new InvalidOperationException(message); }
    private static void ExpectUnauthorized(Action action, string message)
    {
        try { action(); }
        catch (UnauthorizedAccessException) { return; }
        throw new InvalidOperationException(message);
    }
    private static void ExpectSqlFailure(string sql, string message)
    {
        try { Execute(sql); } catch (SQLiteException) { return; }
        throw new InvalidOperationException(message);
    }
    private static void Execute(string sql)
    {
        using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
        using (SQLiteCommand command = new SQLiteCommand(sql, connection)) command.ExecuteNonQuery();
    }
    private static object Scalar(string sql)
    {
        using (SQLiteConnection connection = SQLiteConnectionFactory.CreateConnection())
        using (SQLiteCommand command = new SQLiteCommand(sql, connection)) return command.ExecuteScalar();
    }
    private static int ScalarInt(string sql) { return Convert.ToInt32(Scalar(sql)); }
    private static string ScalarString(string sql) { return Convert.ToString(Scalar(sql)); }
}
```

UI/Controls/RoundedButton.cs

```
using System.Drawing;
using System.Drawing.Drawing2D;
using System.Windows.Forms;
using SistemaGestionNomina.Helpers;

namespace SistemaGestionNomina.UI.Controls
{
    public class RoundedButton : Button
    {
        public int Radius { get; set; }
        public Color BorderColor { get; set; }

        public RoundedButton()
        {
            Radius = 8;
            BorderColor = ThemeHelper.Border;
            FlatStyle = FlatStyle.Flat;
            FlatAppearance.BorderSize = 0;
            BackColor = ThemeHelper.Accent;
            ForeColor = Color.FromArgb(32, 18, 67);
            Font = ThemeHelper.BodyFont(9.5F, FontStyle.Bold);
            Cursor = Cursors.Hand;
            Height = 38;
        }

        protected override void OnPaint(PaintEventArgs pevent)
        {
            pevent.Graphics.SmoothingMode = SmoothingMode.AntiAlias;
            Rectangle rect = new Rectangle(0, 0, Width - 1, Height - 1);
            using (GraphicsPath path = CreateRoundPath(rect, Radius))
            using (SolidBrush brush = new SolidBrush(BackColor))
            using (Pen pen = new Pen(BorderColor, 1))
            using (SolidBrush textBrush = new SolidBrush(ForeColor))
            {
                pevent.Graphics.FillPath(brush, path);
                pevent.Graphics.DrawPath(pen, path);
                TextRenderer.DrawText(pevent.Graphics, Text, Font, rect, ForeColor,
                    TextFormatFlags.HorizontalCenter | TextFormatFlags.VerticalCenter | TextFormatFlags.EndEllipsis);
            }
        }

        private static GraphicsPath CreateRoundPath(Rectangle bounds, int radius)
        {
            int diameter = radius * 2;
            GraphicsPath path = new GraphicsPath();
            path.AddArc(bounds.X, bounds.Y, diameter, diameter, 180, 90);
            path.AddArc(bounds.Right - diameter, bounds.Y, diameter, diameter, 270, 90);
            path.AddArc(bounds.Right - diameter, bounds.Bottom - diameter, diameter, diameter, 0, 90);
            path.AddArc(bounds.X, bounds.Bottom - diameter, diameter, diameter, 90, 90);
            path.CloseFigure();
            return path;
        }
    }
}
```

UI/Controls/RoundedPanel.cs

```
using System.Drawing;
using System.Drawing.Drawing2D;
using System.Windows.Forms;
using SistemaGestionNomina.Helpers;

namespace SistemaGestionNomina.UI.Controls
{
    public class RoundedPanel : Panel
    {
        public int Radius { get; set; }
        public Color BorderColor { get; set; }

        public RoundedPanel()
        {
            Radius = 12;
            BorderColor = ThemeHelper.Border;
            BackColor = ThemeHelper.Card;
            DoubleBuffered = true;
            Padding = new Padding(16);
        }

        protected override void OnPaint(PaintEventArgs e)
        {
            base.OnPaint(e);
            e.Graphics.SmoothingMode = SmoothingMode.AntiAlias;
            using (GraphicsPath path = CreateRoundPath(ClientRectangle, Radius))
            using (Pen pen = new Pen(BorderColor, 1))
            {
                Region = new Region(path);
                e.Graphics.DrawPath(pen, path);
            }
        }

        private static GraphicsPath CreateRoundPath(Rectangle bounds, int radius)
        {
            int diameter = radius * 2;
            Rectangle rect = new Rectangle(bounds.X, bounds.Y, bounds.Width - 1, bounds.Height - 1);
            GraphicsPath path = new GraphicsPath();
            path.AddArc(rect.X, rect.Y, diameter, diameter, 180, 90);
            path.AddArc(rect.Right - diameter, rect.Y, diameter, diameter, 270, 90);
            path.AddArc(rect.Right - diameter, rect.Bottom - diameter, diameter, diameter, 0, 90);
            path.AddArc(rect.X, rect.Bottom - diameter, diameter, diameter, 90, 90);
            path.CloseFigure();
            return path;
        }
    }
}
```

UI/FrmAcercaDe.cs

```
using System;
using System.Windows.Forms;
using SistemaGestionNomina.Helpers;

namespace SistemaGestionNomina.UI
{
    public partial class FrmAcercaDe : Form
    {
        public FrmAcercaDe()
        {
            InitializeComponent();
            ControlStyleHelper.ApplyModernForm(this);
        }

        private void btnCerrar_Click(object sender, EventArgs e)
        {
            Close();
        }

        private void lblIntegrantes_Click(object sender, EventArgs e)
        {

        }
    }
}
```

UI/FrmAcercaDe.Designer.cs

```
namespace SistemaGestionNomina.UI
{
    partial class FrmAcercaDe
    {
        private System.ComponentModel.IContainer components = null;
        private System.Windows.Forms.Panel panelInfo;
        private System.Windows.Forms.Label lblTitulo;
        private System.Windows.Forms.Label lblProyecto;
        private System.Windows.Forms.Label lblCurso;
        private System.Windows.Forms.Label lblIntegrantes;
        private System.Windows.Forms.Label lblDescripcion;
        private System.Windows.Forms.Button btnCerrar;

        protected override void Dispose(bool disposing)
        {
            if (disposing && (components != null)) components.Dispose();
            base.Dispose(disposing);
        }

        private void InitializeComponent()
        {
            this.panelInfo = new System.Windows.Forms.Panel();
            this.lblTitulo = new System.Windows.Forms.Label();
            this.lblProyecto = new System.Windows.Forms.Label();
            this.lblCurso = new System.Windows.Forms.Label();
            this.lblIntegrantes = new System.Windows.Forms.Label();
            this.lblDescripcion = new System.Windows.Forms.Label();
            this.btnCerrar = new System.Windows.Forms.Button();
            this.panelInfo.SuspendLayout();
            this.SuspendLayout();
            //
            // panelInfo
            //
            this.panelInfo.BackColor = System.Drawing.Color.Black;
            this.panelInfo.BorderStyle = System.Windows.Forms.BorderStyle.FixedSingle;
            this.panelInfo.Controls.Add(this.lblTitulo);
            this.panelInfo.Controls.Add(this.lblProyecto);
            this.panelInfo.Controls.Add(this.lblCurso);
            this.panelInfo.Controls.Add(this.lblIntegrantes);
            this.panelInfo.Controls.Add(this.lblDescripcion);
            this.panelInfo.Controls.Add(this.btnCerrar);
            this.panelInfo.Location = new System.Drawing.Point(120, 108);
            this.panelInfo.Name = "panelInfo";
            this.panelInfo.Size = new System.Drawing.Size(780, 460);
            this.panelInfo.TabIndex = 0;
            //
            // lblTitulo
            //
            this.lblTitulo.Font = new System.Drawing.Font("Segoe UI", 22F, System.Drawing.FontStyle.Bold);
            this.lblTitulo.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(167)))), ((int)(((byte)(139)))), ((int)(((byte)(250)))));
            this.lblTitulo.Location = new System.Drawing.Point(34, 28);
            this.lblTitulo.Name = "lblTitulo";
            this.lblTitulo.Size = new System.Drawing.Size(650, 44);
            this.lblTitulo.TabIndex = 0;
            this.lblTitulo.Text = "ProyFinal_LPI_Eq01_NomiCore";
            //
            // lblProyecto
            //
            this.lblProyecto.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(229)))), ((int)(((byte)(228)))), ((int)(((byte)(245)))));
            this.lblProyecto.Location = new System.Drawing.Point(36, 104);
```

UI/FrmAcercaDe.Designer.cs (continuacion)

```
            this.lblProyecto.Name = "lblProyecto";
            this.lblProyecto.Size = new System.Drawing.Size(680, 90);
            this.lblProyecto.TabIndex = 1;
            this.lblProyecto.Text = "Proyecto N° 2\r\nTema H: Manejo de los campos en las clases + PrintDocument\r\nAplica" +
    "ción: ProyFinal_LPI_Eq01_NomiCore\r\nProblema: control de empleados, a" +
    "sistencia, nómina y comprobantes";
            // 
            // lblCurso
            // 
            this.lblCurso.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(170)))), ((int)(((byte)(170)))), ((int)(((byte)(185)))));
            this.lblCurso.Location = new System.Drawing.Point(36, 208);
            this.lblCurso.Name = "lblCurso";
            this.lblCurso.Size = new System.Drawing.Size(680, 82);
            this.lblCurso.TabIndex = 2;
            this.lblCurso.Text = "Universidad Tecnológica de Panamá\r\nFacultad de Ingeniería de Sistemas Computacion" +
    "ales\r\nLenguaje de Programación I\r\nProfesora: Anna Araba de Ruiz | Grupo: 1SF121";
            // 
            // lblIntegrantes
            // 
            this.lblIntegrantes.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(229)))), ((int)(((byte)(228)))), ((int)(((byte)(245)))));
            this.lblIntegrantes.Location = new System.Drawing.Point(36, 304);
            this.lblIntegrantes.Name = "lblIntegrantes";
            this.lblIntegrantes.Size = new System.Drawing.Size(680, 50);
            this.lblIntegrantes.TabIndex = 3;
            this.lblIntegrantes.Text = "Equipo: Eq01\r\nIntegrantes: Jonathan Romero, Jose Martinez, Karen Wen, Davis Batis" +
    "ta";
            this.lblIntegrantes.Click += new System.EventHandler(this.lblIntegrantes_Click);
            // 
            // lblDescripcion
            // 
            this.lblDescripcion.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(170)))), ((int)(((byte)(170)))), ((int)(((byte)(185)))));
            this.lblDescripcion.Location = new System.Drawing.Point(36, 366);
            this.lblDescripcion.Name = "lblDescripcion";
            this.lblDescripcion.Size = new System.Drawing.Size(520, 72);
            this.lblDescripcion.TabIndex = 4;
            this.lblDescripcion.Text = "Windows Forms App (.NET Framework 4.8)\r\nVisual Studio 2022 | Versión académica 1." +
    "0 | Año 2026\r\nIncluye SQLite, servicios, repositorios, exportación e impresión.";
            // 
            // btnCerrar
            // 
            this.btnCerrar.BackColor = System.Drawing.Color.FromArgb(((int)(((byte)(124)))), ((int)(((byte)(58)))), ((int)(((byte)(237)))));
            this.btnCerrar.FlatAppearance.BorderSize = 0;
            this.btnCerrar.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
            this.btnCerrar.Font = new System.Drawing.Font("Segoe UI", 9F, System.Drawing.FontStyle.Bold);
            this.btnCerrar.ForeColor = System.Drawing.Color.White;
            this.btnCerrar.Location = new System.Drawing.Point(600, 386);
            this.btnCerrar.Name = "btnCerrar";
            this.btnCerrar.Size = new System.Drawing.Size(116, 38);
            this.btnCerrar.TabIndex = 5;
            this.btnCerrar.Text = "Cerrar";
            this.btnCerrar.UseVisualStyleBackColor = false;
            this.btnCerrar.Click += new System.EventHandler(this.btnCerrar_Click);
            // 
            // FrmAcercaDe
            // 
            this.AutoScaleDimensions = new System.Drawing.SizeF(8F, 20F);
            this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font;
            this.BackColor = System.Drawing.Color.FromArgb(((int)(((byte)(13)))), ((int)(((byte)(14)))), ((int)(((byte)(20)))));
            this.ClientSize = new System.Drawing.Size(1020, 742);
            this.Controls.Add(this.panelInfo);
```

UI/FrmAcercaDe.Designer.cs (continuacion)

```
            this.Font = new System.Drawing.Font("Segoe UI", 9F);
            this.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(229)))), ((int)(((byte)(228)))), ((int)(((byte)(245)))));
            this.Name = "FrmAcercaDe";
            this.Text = "Acerca de";
            this.panelInfo.ResumeLayout(false);
            this.ResumeLayout(false);

        }
    }
}
```

UI/FrmAnularNomina.cs

```
using System;
using System.Windows.Forms;
using SistemaGestionNomina.Models;

namespace SistemaGestionNomina.UI
{
    public partial class FrmAnularNomina : Form
    {
        private TextBox txtMotivo;
        private Label lblAdvertencia;
        private Label lblResumen;
        private Button btnConfirmar;
        private Button btnCancelar;

        public string Motivo { get; private set; }

        public FrmAnularNomina(Nomina nomina)
        {
            InitializeComponent();
            lblResumen.Text = "Nómina: " + (nomina.PeriodoNombre ?? "Sin nombre") +
                "\nEstado actual: " + (nomina.Estado ?? "Desconocido") +
                "\nTotal neto: B/. " + nomina.TotalNeto.ToString("0.00") +
                "\nEmpleados: " + nomina.Detalles.Count;
        }

        private void InitializeComponent()
        {
            this.lblResumen = new Label();
            this.lblAdvertencia = new Label();
            this.txtMotivo = new TextBox();
            this.btnConfirmar = new Button();
            this.btnCancelar = new Button();
            this.SuspendLayout();

            this.lblResumen.AutoSize = true;
            this.lblResumen.Font = new System.Drawing.Font("Segoe UI", 10F);
            this.lblResumen.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245);
            this.lblResumen.Location = new System.Drawing.Point(16, 16);
            this.lblResumen.Size = new System.Drawing.Size(50, 19);
            this.lblResumen.Text = "Resumen";

            this.lblAdvertencia.AutoSize = true;
            this.lblAdvertencia.Font = new System.Drawing.Font("Segoe UI", 9F, System.Drawing.FontStyle.Bold);
            this.lblAdvertencia.ForeColor = System.Drawing.Color.FromArgb(239, 68, 68);
            this.lblAdvertencia.Location = new System.Drawing.Point(16, 96);
            this.lblAdvertencia.Size = new System.Drawing.Size(80, 15);
            this.lblAdvertencia.Text = "La nómina anterior seguirá existiendo como registro histórico.";

            this.txtMotivo.Location = new System.Drawing.Point(16, 124);
            this.txtMotivo.Multiline = true;
            this.txtMotivo.Size = new System.Drawing.Size(400, 80);
            this.txtMotivo.BackColor = System.Drawing.Color.FromArgb(24, 25, 34);
            this.txtMotivo.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245);
            this.txtMotivo.BorderStyle = BorderStyle.FixedSingle;

            this.btnConfirmar.BackColor = System.Drawing.Color.FromArgb(239, 68, 68);
            this.btnConfirmar.FlatStyle = FlatStyle.Flat;
            this.btnConfirmar.Font = new System.Drawing.Font("Segoe UI", 10F, System.Drawing.FontStyle.Bold);
            this.btnConfirmar.ForeColor = System.Drawing.Color.White;
            this.btnConfirmar.Location = new System.Drawing.Point(220, 220);
```

UI/FrmAnularNomina.cs (continuacion)

```
            this.btnConfirmar.Size = new System.Drawing.Size(100, 36);
            this.btnConfirmar.Text = "Anular";
            this.btnConfirmar.UseVisualStyleBackColor = false;
            this.btnConfirmar.Click += new EventHandler(this.btnConfirmar_Click);

            this.btnCancelar.BackColor = System.Drawing.Color.Black;
            this.btnCancelar.FlatStyle = FlatStyle.Flat;
            this.btnCancelar.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245);
            this.btnCancelar.Location = new System.Drawing.Point(336, 220);
            this.btnCancelar.Size = new System.Drawing.Size(80, 36);
            this.btnCancelar.Text = "Cancelar";
            this.btnCancelar.UseVisualStyleBackColor = false;
            this.btnCancelar.Click += new EventHandler(this.btnCancelar_Click);

            this.AutoScaleDimensions = new System.Drawing.SizeF(7F, 15F);
            this.AutoScaleMode = AutoScaleMode.Font;
            this.BackColor = System.Drawing.Color.FromArgb(13, 14, 20);
            this.ClientSize = new System.Drawing.Size(434, 271);
            this.Controls.Add(this.lblResumen);
            this.Controls.Add(this.lblAdvertencia);
            this.Controls.Add(this.txtMotivo);
            this.Controls.Add(this.btnConfirmar);
            this.Controls.Add(this.btnCancelar);
            this.Font = new System.Drawing.Font("Segoe UI", 9F);
            this.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245);
            this.FormBorderStyle = FormBorderStyle.FixedDialog;
            this.MaximizeBox = false;
            this.MinimizeBox = false;
            this.StartPosition = FormStartPosition.CenterParent;
            this.Text = "Anular nómina";
            this.ResumeLayout(false);
            this.PerformLayout();
        }

        private void btnConfirmar_Click(object sender, EventArgs e)
        {
            if (string.IsNullOrWhiteSpace(txtMotivo.Text))
            {
                MessageBox.Show("Debe indicar el motivo de la anulación.", "Anular",
                    MessageBoxButtons.OK, MessageBoxIcon.Warning);
                return;
            }

            Motivo = txtMotivo.Text.Trim();
            DialogResult = DialogResult.OK;
            Close();
        }

        private void btnCancelar_Click(object sender, EventArgs e)
        {
            DialogResult = DialogResult.Cancel;
            Close();
        }
    }
}
```

UI/FrmAsistencia.cs

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Windows.Forms;
using SistemaGestionNomina.Helpers;
using SistemaGestionNomina.Models;
using SistemaGestionNomina.Services;
using SistemaGestionNomina.Security;

namespace SistemaGestionNomina.UI
{
    public partial class FrmAsistencia : Form
    {
        private readonly EmpleadoService empleadoService = new EmpleadoService();
        private readonly AsistenciaService asistenciaService = new AsistenciaService();
        private readonly ExcelExportService excelExportService = new ExcelExportService();
        private readonly PdfExportService pdfExportService = new PdfExportService();
        private readonly AttendanceDeviceImportService attendanceDeviceImportService = new AttendanceDeviceImportService();
        private readonly AuthorizationService authorizationService = new AuthorizationService();
        private List<Asistencia> currentItems = new List<Asistencia>();

        public FrmAsistencia()
        {
            InitializeComponent();
            ControlStyleHelper.ApplyModernForm(this);
        }

        private void FrmAsistencia_Load(object sender, EventArgs e)
        {
            if (LicenseManager.UsageMode == LicenseUsageMode.Designtime) return;
            btnRegistrar.Visible = authorizationService.HasPermission(Permissions.AttendanceRegister);
            btnCargarReloj.Visible = authorizationService.HasPermission(Permissions.AttendanceImport);
            btnExportarExcel.Visible = authorizationService.HasPermission(Permissions.AttendanceExport);
            btnExportarPdf.Visible = authorizationService.HasPermission(Permissions.AttendanceExport);
            LoadEmployees();
            LoadAttendance();
        }

        private void LoadEmployees()
        {
            List<Empleado> employees = empleadoService.GetActive(null);
            cmbEmpleado.DataSource = new List<Empleado>(employees);
            cmbEmpleado.DisplayMember = "NombreCompleto";
            cmbEmpleado.ValueMember = "IdEmpleado";

            List<Empleado> filters = new List<Empleado>();
            filters.Add(new Empleado { IdEmpleado = 0, Nombre = "Todos", Apellido = "los empleados" });
            filters.AddRange(employees);
            cmbFiltroEmpleado.DataSource = filters;
            cmbFiltroEmpleado.DisplayMember = "NombreCompleto";
            cmbFiltroEmpleado.ValueMember = "IdEmpleado";
        }

        private void LoadAttendance()
        {
            if (dtpFiltroInicio.Value.Date > dtpFiltroFin.Value.Date)
            {
                MessageBox.Show("La fecha inicial no puede ser mayor que la fecha final.", "Asistencia", MessageBoxButtons.OK, MessageBoxIcon.Warning);
                return;
            }
```

UI/FrmAsistencia.cs (continuacion)

```
            int? employeeId = null;
            if (cmbFiltroEmpleado.SelectedValue is int && (int)cmbFiltroEmpleado.SelectedValue > 0)
            {
                employeeId = (int)cmbFiltroEmpleado.SelectedValue;
            }

            string status = cmbFiltroEstado.SelectedItem == null || cmbFiltroEstado.SelectedItem.ToString() == "Todos"
                ? string.Empty
                : cmbFiltroEstado.SelectedItem.ToString();

            currentItems = asistenciaService.GetAll(dtpFiltroInicio.Value.Date, dtpFiltroFin.Value.Date, employeeId, status);
            dgvAsistencia.Rows.Clear();
            foreach (Asistencia a in currentItems)
            {
                dgvAsistencia.Rows.Add(a.EmpleadoNombre, a.Fecha.ToString("dd/MM/yyyy"),
                    a.HoraEntrada.HasValue ? a.HoraEntrada.Value.ToString(@"hh\:mm") : "--",
                    a.HoraSalida.HasValue ? a.HoraSalida.Value.ToString(@"hh\:mm") : "--",
                    a.HorasTrabajadas.ToString("0.00"), a.Estado);
            }

            UpdateSummaryCards();
        }

        private void UpdateSummaryCards()
        {
            int presentes = 0;
            int ausentes = 0;
            int tardanzas = 0;

            foreach (Asistencia asistencia in currentItems)
            {
                if (asistencia.Estado == "Puntual")
                {
                    presentes++;
                }
                else if (asistencia.Estado == "Tardanza")
                {
                    presentes++;
                    tardanzas++;
                }
                else if (asistencia.Estado == "Falta")
                {
                    ausentes++;
                }
            }

            int total = currentItems.Count;
            decimal porcentaje = total == 0 ? 0 : Math.Round((presentes * 100m) / total, 0);
            lblPresentesValor.Text = presentes.ToString();
            lblAusentesValor.Text = ausentes.ToString();
            lblTardanzasValor.Text = tardanzas.ToString();
            lblPorcentajeValor.Text = porcentaje.ToString("0") + "%";
        }

        private void btnRegistrar_Click(object sender, EventArgs e)
        {
            if (cmbEmpleado.SelectedValue == null)
            {
                MessageBox.Show("Seleccione un empleado.", "Asistencia", MessageBoxButtons.OK, MessageBoxIcon.Warning);
```

UI/FrmAsistencia.cs (continuacion)

```
                return;
            }

            if (cmbEstado.SelectedItem == null)
            {
                MessageBox.Show("Seleccione el estado de asistencia.", "Asistencia", MessageBoxButtons.OK, MessageBoxIcon.Warning);
                return;
            }

            try
            {
                Asistencia asistencia = new Asistencia();
                asistencia.IdEmpleado = Convert.ToInt32(cmbEmpleado.SelectedValue);
                asistencia.Fecha = dtpFecha.Value.Date;
                asistencia.HoraEntrada = dtpHoraEntrada.Value.TimeOfDay;
                asistencia.HoraSalida = dtpHoraSalida.Value.TimeOfDay;
                asistencia.Estado = cmbEstado.SelectedItem.ToString();
                asistenciaService.Register(asistencia);
                MessageBox.Show("Asistencia registrada correctamente.", "Asistencia", MessageBoxButtons.OK, MessageBoxIcon.Information);
                LoadAttendance();
            }
            catch (Exception ex)
            {
                MessageBox.Show(ex.Message, "Asistencia", MessageBoxButtons.OK, MessageBoxIcon.Warning);
            }
        }

        private void btnFiltrar_Click(object sender, EventArgs e)
        {
            LoadAttendance();
        }

        private void btnExportarExcel_Click(object sender, EventArgs e)
        {
            if (currentItems.Count == 0)
            {
                MessageBox.Show("No hay asistencias para exportar.", "Asistencia", MessageBoxButtons.OK, MessageBoxIcon.Information);
                return;
            }

            UiFactory.ShowExported(excelExportService.ExportarAsistencias(currentItems));
        }

        private void btnExportarPdf_Click(object sender, EventArgs e)
        {
            if (currentItems.Count == 0)
            {
                MessageBox.Show("No hay asistencias para exportar.", "Asistencia", MessageBoxButtons.OK, MessageBoxIcon.Information);
                return;
            }

            UiFactory.ShowExported(pdfExportService.ExportarAsistencias(currentItems));
        }

        private void btnCargarReloj_Click(object sender, EventArgs e)
        {
            using (OpenFileDialog dialog = new OpenFileDialog())
            {
                dialog.Filter = "Archivos CSV (*.csv)|*.csv|Todos los archivos (*.*)|*.*";
                dialog.Title = "Importar asistencia desde archivo";
```

UI/FrmAsistencia.cs (continuacion)

```
                if (dialog.ShowDialog(this) != DialogResult.OK)
                {
                    return;
                }

                try
                {
                    int imported = attendanceDeviceImportService.ImportarDesdeArchivo(dialog.FileName);
                    MessageBox.Show("Importación completada. Registros procesados: " + imported,
                        "Asistencia", MessageBoxButtons.OK, MessageBoxIcon.Information);
                    LoadAttendance();
                }
                catch (Exception ex)
                {
                    MessageBox.Show(ex.Message, "Importar asistencia", MessageBoxButtons.OK, MessageBoxIcon.Warning);
                }
            }
        }
    }
}
```

UI/FrmAsistencia.Designer.cs

```
namespace SistemaGestionNomina.UI
{
    partial class FrmAsistencia
    {
        private System.ComponentModel.IContainer components = null;
        private System.Windows.Forms.Label lblTitulo;
        private System.Windows.Forms.Panel panelRegistro;
        private System.Windows.Forms.ComboBox cmbEmpleado;
        private System.Windows.Forms.Label lblEmpleado;
        private System.Windows.Forms.Label lblFecha;
        private System.Windows.Forms.Label lblEntrada;
        private System.Windows.Forms.Label lblSalida;
        private System.Windows.Forms.Label lblEstado;
        private System.Windows.Forms.DateTimePicker dtpFecha;
        private System.Windows.Forms.DateTimePicker dtpHoraEntrada;
        private System.Windows.Forms.DateTimePicker dtpHoraSalida;
        private System.Windows.Forms.ComboBox cmbEstado;
        private System.Windows.Forms.Button btnRegistrar;
        private System.Windows.Forms.Button btnCargarReloj;
        private System.Windows.Forms.Panel panelFiltros;
        private System.Windows.Forms.DateTimePicker dtpFiltroInicio;
        private System.Windows.Forms.DateTimePicker dtpFiltroFin;
        private System.Windows.Forms.ComboBox cmbFiltroEmpleado;
        private System.Windows.Forms.ComboBox cmbFiltroEstado;
        private System.Windows.Forms.Button btnFiltrar;
        private System.Windows.Forms.Button btnExportarExcel;
        private System.Windows.Forms.Button btnExportarPdf;
        private System.Windows.Forms.Panel cardPresentes;
        private System.Windows.Forms.Panel cardAusentes;
        private System.Windows.Forms.Panel cardTardanzas;
        private System.Windows.Forms.Panel cardPorcentaje;
        private System.Windows.Forms.Label lblPresentesTitulo;
        private System.Windows.Forms.Label lblAusentesTitulo;
        private System.Windows.Forms.Label lblTardanzasTitulo;
        private System.Windows.Forms.Label lblPorcentajeTitulo;
        private System.Windows.Forms.Label lblPresentesValor;
        private System.Windows.Forms.Label lblAusentesValor;
        private System.Windows.Forms.Label lblTardanzasValor;
        private System.Windows.Forms.Label lblPorcentajeValor;
        private System.Windows.Forms.DataGridView dgvAsistencia;

        protected override void Dispose(bool disposing)
        {
            if (disposing && (components != null)) components.Dispose();
            base.Dispose(disposing);
        }

        private void InitializeComponent()
        {
            this.lblTitulo = new System.Windows.Forms.Label();
            this.panelRegistro = new System.Windows.Forms.Panel();
            this.lblEmpleado = new System.Windows.Forms.Label();
            this.lblFecha = new System.Windows.Forms.Label();
            this.lblEntrada = new System.Windows.Forms.Label();
            this.lblSalida = new System.Windows.Forms.Label();
            this.lblEstado = new System.Windows.Forms.Label();
            this.cmbEmpleado = new System.Windows.Forms.ComboBox();
            this.dtpFecha = new System.Windows.Forms.DateTimePicker();
            this.dtpHoraEntrada = new System.Windows.Forms.DateTimePicker();
            this.dtpHoraSalida = new System.Windows.Forms.DateTimePicker();
```

UI/FrmAsistencia.Designer.cs (continuacion)

```
            this.cmbEstado = new System.Windows.Forms.ComboBox();
            this.btnRegistrar = new System.Windows.Forms.Button();
            this.btnCargarReloj = new System.Windows.Forms.Button();
            this.panelFiltros = new System.Windows.Forms.Panel();
            this.dtpFiltroInicio = new System.Windows.Forms.DateTimePicker();
            this.dtpFiltroFin = new System.Windows.Forms.DateTimePicker();
            this.cmbFiltroEmpleado = new System.Windows.Forms.ComboBox();
            this.cmbFiltroEstado = new System.Windows.Forms.ComboBox();
            this.btnFiltrar = new System.Windows.Forms.Button();
            this.btnExportarExcel = new System.Windows.Forms.Button();
            this.btnExportarPdf = new System.Windows.Forms.Button();
            this.cardPresentes = new System.Windows.Forms.Panel();
            this.lblPresentesTitulo = new System.Windows.Forms.Label();
            this.lblPresentesValor = new System.Windows.Forms.Label();
            this.cardAusentes = new System.Windows.Forms.Panel();
            this.lblAusentesTitulo = new System.Windows.Forms.Label();
            this.lblAusentesValor = new System.Windows.Forms.Label();
            this.cardTardanzas = new System.Windows.Forms.Panel();
            this.lblTardanzasTitulo = new System.Windows.Forms.Label();
            this.lblTardanzasValor = new System.Windows.Forms.Label();
            this.cardPorcentaje = new System.Windows.Forms.Panel();
            this.lblPorcentajeTitulo = new System.Windows.Forms.Label();
            this.lblPorcentajeValor = new System.Windows.Forms.Label();
            this.dgvAsistencia = new System.Windows.Forms.DataGridView();
            this.dataGridViewTextBoxColumn1 = new System.Windows.Forms.DataGridViewTextBoxColumn();
            this.dataGridViewTextBoxColumn2 = new System.Windows.Forms.DataGridViewTextBoxColumn();
            this.dataGridViewTextBoxColumn3 = new System.Windows.Forms.DataGridViewTextBoxColumn();
            this.dataGridViewTextBoxColumn4 = new System.Windows.Forms.DataGridViewTextBoxColumn();
            this.dataGridViewTextBoxColumn5 = new System.Windows.Forms.DataGridViewTextBoxColumn();
            this.dataGridViewTextBoxColumn6 = new System.Windows.Forms.DataGridViewTextBoxColumn();
            this.panelRegistro.SuspendLayout();
            this.panelFiltros.SuspendLayout();
            this.cardPresentes.SuspendLayout();
            this.cardAusentes.SuspendLayout();
            this.cardTardanzas.SuspendLayout();
            this.cardPorcentaje.SuspendLayout();
            ((System.ComponentModel.ISupportInitialize)(this.dgvAsistencia)).BeginInit();
            this.SuspendLayout();
            //
            // lblTitulo
            //
            this.lblTitulo.Font = new System.Drawing.Font("Segoe UI", 22F, System.Drawing.FontStyle.Bold);
            this.lblTitulo.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(229)))), ((int)(((byte)(228)))), ((int)(((byte)(245)))));
            this.lblTitulo.Location = new System.Drawing.Point(34, 24);
            this.lblTitulo.Name = "lblTitulo";
            this.lblTitulo.Size = new System.Drawing.Size(500, 42);
            this.lblTitulo.TabIndex = 0;
            this.lblTitulo.Text = "Control de Asistencia";
            //
            // panelRegistro
            //
            this.panelRegistro.BackColor = System.Drawing.Color.FromArgb(((int)(((byte)(17)))), ((int)(((byte)(18)))), ((int)(((byte)(26)))));
            this.panelRegistro.BorderStyle = System.Windows.Forms.BorderStyle.FixedSingle;
            this.panelRegistro.Controls.Add(this.lblEmpleado);
            this.panelRegistro.Controls.Add(this.lblFecha);
            this.panelRegistro.Controls.Add(this.lblEntrada);
            this.panelRegistro.Controls.Add(this.lblSalida);
            this.panelRegistro.Controls.Add(this.lblEstado);
            this.panelRegistro.Controls.Add(this.cmbEmpleado);
            this.panelRegistro.Controls.Add(this.dtpFecha);
```

UI/FrmAsistencia.Designer.cs (continuacion)

```
            this.panelRegistro.Controls.Add(this.dtpHoraEntrada);
            this.panelRegistro.Controls.Add(this.dtpHoraSalida);
            this.panelRegistro.Controls.Add(this.cmbEstado);
            this.panelRegistro.Controls.Add(this.btnRegistrar);
            this.panelRegistro.Controls.Add(this.btnCargarReloj);
            this.panelRegistro.Location = new System.Drawing.Point(36, 92);
            this.panelRegistro.Name = "panelRegistro";
            this.panelRegistro.Size = new System.Drawing.Size(948, 92);
            this.panelRegistro.TabIndex = 1;
            //
            // lblEmpleado
            //
            this.lblEmpleado.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(170)))), ((int)(((byte)(170)))), ((int)(((byte)(185)))));
            this.lblEmpleado.Location = new System.Drawing.Point(18, 22);
            this.lblEmpleado.Name = "lblEmpleado";
            this.lblEmpleado.Size = new System.Drawing.Size(120, 18);
            this.lblEmpleado.TabIndex = 0;
            this.lblEmpleado.Text = "Empleado";
            //
            // lblFecha
            //
            this.lblFecha.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(170)))), ((int)(((byte)(170)))), ((int)(((byte)(185)))));
            this.lblFecha.Location = new System.Drawing.Point(270, 22);
            this.lblFecha.Name = "lblFecha";
            this.lblFecha.Size = new System.Drawing.Size(80, 18);
            this.lblFecha.TabIndex = 1;
            this.lblFecha.Text = "Fecha";
            //
            // lblEntrada
            //
            this.lblEntrada.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(170)))), ((int)(((byte)(170)))), ((int)(((byte)(185)))));
            this.lblEntrada.Location = new System.Drawing.Point(392, 22);
            this.lblEntrada.Name = "lblEntrada";
            this.lblEntrada.Size = new System.Drawing.Size(90, 18);
            this.lblEntrada.TabIndex = 2;
            this.lblEntrada.Text = "Entrada";
            //
            // lblSalida
            //
            this.lblSalida.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(170)))), ((int)(((byte)(170)))), ((int)(((byte)(185)))));
            this.lblSalida.Location = new System.Drawing.Point(504, 22);
            this.lblSalida.Name = "lblSalida";
            this.lblSalida.Size = new System.Drawing.Size(90, 18);
            this.lblSalida.TabIndex = 3;
            this.lblSalida.Text = "Salida";
            //
            // lblEstado
            //
            this.lblEstado.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(170)))), ((int)(((byte)(170)))), ((int)(((byte)(185)))));
            this.lblEstado.Location = new System.Drawing.Point(616, 22);
            this.lblEstado.Name = "lblEstado";
            this.lblEstado.Size = new System.Drawing.Size(90, 18);
            this.lblEstado.TabIndex = 4;
            this.lblEstado.Text = "Estado";
            //
            // cmbEmpleado
            //
            this.cmbEmpleado.DropDownStyle = System.Windows.Forms.ComboBoxStyle.DropDownList;
            this.cmbEmpleado.Location = new System.Drawing.Point(18, 44);
            this.cmbEmpleado.Name = "cmbEmpleado";
```

UI/FrmAsistencia.Designer.cs (continuacion)

```
            this.cmbEmpleado.Size = new System.Drawing.Size(240, 28);
            this.cmbEmpleado.TabIndex = 0;
            //
            // dtpFecha
            //
            this.dtpFecha.Format = System.Windows.Forms.DateTimePickerFormat.Short;
            this.dtpFecha.Location = new System.Drawing.Point(270, 44);
            this.dtpFecha.Name = "dtpFecha";
            this.dtpFecha.Size = new System.Drawing.Size(110, 27);
            this.dtpFecha.TabIndex = 1;
            //
            // dtpHoraEntrada
            //
            this.dtpHoraEntrada.Format = System.Windows.Forms.DateTimePickerFormat.Time;
            this.dtpHoraEntrada.Location = new System.Drawing.Point(392, 44);
            this.dtpHoraEntrada.Name = "dtpHoraEntrada";
            this.dtpHoraEntrada.ShowUpDown = true;
            this.dtpHoraEntrada.Size = new System.Drawing.Size(100, 27);
            this.dtpHoraEntrada.TabIndex = 2;
            //
            // dtpHoraSalida
            //
            this.dtpHoraSalida.Format = System.Windows.Forms.DateTimePickerFormat.Time;
            this.dtpHoraSalida.Location = new System.Drawing.Point(504, 44);
            this.dtpHoraSalida.Name = "dtpHoraSalida";
            this.dtpHoraSalida.ShowUpDown = true;
            this.dtpHoraSalida.Size = new System.Drawing.Size(100, 27);
            this.dtpHoraSalida.TabIndex = 3;
            //
            // cmbEstado
            //
            this.cmbEstado.DropDownStyle = System.Windows.Forms.ComboBoxStyle.DropDownList;
            this.cmbEstado.Items.AddRange(new object[] {
            "Puntual",
            "Tardanza",
            "Falta",
            "Permiso"});
            this.cmbEstado.Location = new System.Drawing.Point(616, 44);
            this.cmbEstado.Name = "cmbEstado";
            this.cmbEstado.Size = new System.Drawing.Size(120, 28);
            this.cmbEstado.TabIndex = 4;
            //
            // btnRegistrar
            //
            this.btnRegistrar.BackColor = System.Drawing.Color.FromArgb(((int)(((byte)(167)))), ((int)(((byte)(139)))), ((int)(((byte)(250)))));
            this.btnRegistrar.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
            this.btnRegistrar.Location = new System.Drawing.Point(750, 38);
            this.btnRegistrar.Name = "btnRegistrar";
            this.btnRegistrar.Size = new System.Drawing.Size(90, 30);
            this.btnRegistrar.TabIndex = 5;
            this.btnRegistrar.Text = "Registrar";
            this.btnRegistrar.UseVisualStyleBackColor = false;
            this.btnRegistrar.Click += new System.EventHandler(this.btnRegistrar_Click);
            //
            // btnCargarReloj
            //
            this.btnCargarReloj.BackColor = System.Drawing.Color.Black;
            this.btnCargarReloj.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
            this.btnCargarReloj.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(229)))), ((int)(((byte)(228)))), ((int)(((byte)(245)))));
            this.btnCargarReloj.Location = new System.Drawing.Point(846, 38);
```

UI/FrmAsistencia.Designer.cs (continuacion)

```
            this.btnCargarReloj.Name = "btnCargarReloj";
            this.btnCargarReloj.Size = new System.Drawing.Size(86, 30);
            this.btnCargarReloj.TabIndex = 6;
            this.btnCargarReloj.Text = "Reloj";
            this.btnCargarReloj.UseVisualStyleBackColor = false;
            this.btnCargarReloj.Click += new System.EventHandler(this.btnCargarReloj_Click);
            //
            // panelFiltros
            //
            this.panelFiltros.BackColor = System.Drawing.Color.FromArgb(((int)(((byte)(17)))), ((int)(((byte)(18)))), ((int)(((byte)(26)))));
            this.panelFiltros.BorderStyle = System.Windows.Forms.BorderStyle.FixedSingle;
            this.panelFiltros.Controls.Add(this.dtpFiltroInicio);
            this.panelFiltros.Controls.Add(this.dtpFiltroFin);
            this.panelFiltros.Controls.Add(this.cmbFiltroEmpleado);
            this.panelFiltros.Controls.Add(this.cmbFiltroEstado);
            this.panelFiltros.Controls.Add(this.btnFiltrar);
            this.panelFiltros.Controls.Add(this.btnExportarExcel);
            this.panelFiltros.Controls.Add(this.btnExportarPdf);
            this.panelFiltros.Location = new System.Drawing.Point(36, 206);
            this.panelFiltros.Name = "panelFiltros";
            this.panelFiltros.Size = new System.Drawing.Size(948, 80);
            this.panelFiltros.TabIndex = 2;
            //
            // dtpFiltroInicio
            //
            this.dtpFiltroInicio.Format = System.Windows.Forms.DateTimePickerFormat.Short;
            this.dtpFiltroInicio.Location = new System.Drawing.Point(18, 28);
            this.dtpFiltroInicio.Name = "dtpFiltroInicio";
            this.dtpFiltroInicio.Size = new System.Drawing.Size(110, 27);
            this.dtpFiltroInicio.TabIndex = 0;
            //
            // dtpFiltroFin
            //
            this.dtpFiltroFin.Format = System.Windows.Forms.DateTimePickerFormat.Short;
            this.dtpFiltroFin.Location = new System.Drawing.Point(140, 28);
            this.dtpFiltroFin.Name = "dtpFiltroFin";
            this.dtpFiltroFin.Size = new System.Drawing.Size(110, 27);
            this.dtpFiltroFin.TabIndex = 1;
            //
            // cmbFiltroEmpleado
            //
            this.cmbFiltroEmpleado.DropDownStyle = System.Windows.Forms.ComboBoxStyle.DropDownList;
            this.cmbFiltroEmpleado.Location = new System.Drawing.Point(262, 28);
            this.cmbFiltroEmpleado.Name = "cmbFiltroEmpleado";
            this.cmbFiltroEmpleado.Size = new System.Drawing.Size(220, 28);
            this.cmbFiltroEmpleado.TabIndex = 2;
            //
            // cmbFiltroEstado
            //
            this.cmbFiltroEstado.DropDownStyle = System.Windows.Forms.ComboBoxStyle.DropDownList;
            this.cmbFiltroEstado.Items.AddRange(new object[] {
            "Todos",
            "Puntual",
            "Tardanza",
            "Falta",
            "Permiso"});
            this.cmbFiltroEstado.Location = new System.Drawing.Point(494, 28);
            this.cmbFiltroEstado.Name = "cmbFiltroEstado";
            this.cmbFiltroEstado.Size = new System.Drawing.Size(120, 28);
            this.cmbFiltroEstado.TabIndex = 3;
```

UI/FrmAsistencia.Designer.cs (continuacion)

```
            //
            // btnFiltrar
            //
            this.btnFiltrar.BackColor = System.Drawing.Color.FromArgb(((int)(((byte)(167)))), ((int)(((byte)(139)))), ((int)(((byte)(250)))));
            this.btnFiltrar.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
            this.btnFiltrar.Location = new System.Drawing.Point(630, 24);
            this.btnFiltrar.Name = "btnFiltrar";
            this.btnFiltrar.Size = new System.Drawing.Size(80, 30);
            this.btnFiltrar.TabIndex = 4;
            this.btnFiltrar.Text = "Filtrar";
            this.btnFiltrar.UseVisualStyleBackColor = false;
            this.btnFiltrar.Click += new System.EventHandler(this.btnFiltrar_Click);
            //
            // btnExportarExcel
            //
            this.btnExportarExcel.BackColor = System.Drawing.Color.Black;
            this.btnExportarExcel.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
            this.btnExportarExcel.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(229)))), ((int)(((byte)(228)))), ((int)(((byte)(245)))));
            this.btnExportarExcel.Location = new System.Drawing.Point(724, 24);
            this.btnExportarExcel.Name = "btnExportarExcel";
            this.btnExportarExcel.Size = new System.Drawing.Size(80, 30);
            this.btnExportarExcel.TabIndex = 5;
            this.btnExportarExcel.Text = "Excel";
            this.btnExportarExcel.UseVisualStyleBackColor = false;
            this.btnExportarExcel.Click += new System.EventHandler(this.btnExportarExcel_Click);
            //
            // btnExportarPdf
            //
            this.btnExportarPdf.BackColor = System.Drawing.Color.Black;
            this.btnExportarPdf.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
            this.btnExportarPdf.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(229)))), ((int)(((byte)(228)))), ((int)(((byte)(245)))));
            this.btnExportarPdf.Location = new System.Drawing.Point(816, 24);
            this.btnExportarPdf.Name = "btnExportarPdf";
            this.btnExportarPdf.Size = new System.Drawing.Size(80, 30);
            this.btnExportarPdf.TabIndex = 6;
            this.btnExportarPdf.Text = "PDF";
            this.btnExportarPdf.UseVisualStyleBackColor = false;
            this.btnExportarPdf.Click += new System.EventHandler(this.btnExportarPdf_Click);
            //
            // cardPresentes
            //
            this.cardPresentes.BackColor = System.Drawing.Color.Black;
            this.cardPresentes.BorderStyle = System.Windows.Forms.BorderStyle.FixedSingle;
            this.cardPresentes.Controls.Add(this.lblPresentesTitulo);
            this.cardPresentes.Controls.Add(this.lblPresentesValor);
            this.cardPresentes.Location = new System.Drawing.Point(36, 304);
            this.cardPresentes.Name = "cardPresentes";
            this.cardPresentes.Size = new System.Drawing.Size(222, 64);
            this.cardPresentes.TabIndex = 3;
            //
            // lblPresentesTitulo
            //
            this.lblPresentesTitulo.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(170)))), ((int)(((byte)(170)))), ((int)(((byte)(185)))));
            this.lblPresentesTitulo.Location = new System.Drawing.Point(16, 10);
            this.lblPresentesTitulo.Name = "lblPresentesTitulo";
            this.lblPresentesTitulo.Size = new System.Drawing.Size(160, 18);
            this.lblPresentesTitulo.TabIndex = 0;
            this.lblPresentesTitulo.Text = "Presentes";
            //
            // lblPresentesValor
```

UI/FrmAsistencia.Designer.cs (continuacion)

```
            //
            this.lblPresentesValor.Font = new System.Drawing.Font("Segoe UI", 16F, System.Drawing.FontStyle.Bold);
            this.lblPresentesValor.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(34)))), ((int)(((byte)(197)))), ((int)(((byte)(94)))));
            this.lblPresentesValor.Location = new System.Drawing.Point(16, 28);
            this.lblPresentesValor.Name = "lblPresentesValor";
            this.lblPresentesValor.Size = new System.Drawing.Size(170, 35);
            this.lblPresentesValor.TabIndex = 1;
            this.lblPresentesValor.Text = "0";
            //
            // cardAusentes
            //
            this.cardAusentes.BackColor = System.Drawing.Color.Black;
            this.cardAusentes.BorderStyle = System.Windows.Forms.BorderStyle.FixedSingle;
            this.cardAusentes.Controls.Add(this.lblAusentesTitulo);
            this.cardAusentes.Controls.Add(this.lblAusentesValor);
            this.cardAusentes.Location = new System.Drawing.Point(278, 304);
            this.cardAusentes.Name = "cardAusentes";
            this.cardAusentes.Size = new System.Drawing.Size(222, 64);
            this.cardAusentes.TabIndex = 4;
            //
            // lblAusentesTitulo
            //
            this.lblAusentesTitulo.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(170)))), ((int)(((byte)(170)))), ((int)(((byte)(185)))));
            this.lblAusentesTitulo.Location = new System.Drawing.Point(16, 10);
            this.lblAusentesTitulo.Name = "lblAusentesTitulo";
            this.lblAusentesTitulo.Size = new System.Drawing.Size(160, 18);
            this.lblAusentesTitulo.TabIndex = 0;
            this.lblAusentesTitulo.Text = "Ausentes";
            //
            // lblAusentesValor
            //
            this.lblAusentesValor.Font = new System.Drawing.Font("Segoe UI", 16F, System.Drawing.FontStyle.Bold);
            this.lblAusentesValor.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(239)))), ((int)(((byte)(68)))), ((int)(((byte)(68)))));
            this.lblAusentesValor.Location = new System.Drawing.Point(16, 28);
            this.lblAusentesValor.Name = "lblAusentesValor";
            this.lblAusentesValor.Size = new System.Drawing.Size(170, 34);
            this.lblAusentesValor.TabIndex = 1;
            this.lblAusentesValor.Text = "0";
            //
            // cardTardanzas
            //
            this.cardTardanzas.BackColor = System.Drawing.Color.Black;
            this.cardTardanzas.BorderStyle = System.Windows.Forms.BorderStyle.FixedSingle;
            this.cardTardanzas.Controls.Add(this.lblTardanzasTitulo);
            this.cardTardanzas.Controls.Add(this.lblTardanzasValor);
            this.cardTardanzas.Location = new System.Drawing.Point(520, 304);
            this.cardTardanzas.Name = "cardTardanzas";
            this.cardTardanzas.Size = new System.Drawing.Size(222, 64);
            this.cardTardanzas.TabIndex = 5;
            //
            // lblTardanzasTitulo
            //
            this.lblTardanzasTitulo.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(170)))), ((int)(((byte)(170)))), ((int)(((byte)(185)))));
            this.lblTardanzasTitulo.Location = new System.Drawing.Point(16, 10);
            this.lblTardanzasTitulo.Name = "lblTardanzasTitulo";
            this.lblTardanzasTitulo.Size = new System.Drawing.Size(160, 18);
            this.lblTardanzasTitulo.TabIndex = 0;
            this.lblTardanzasTitulo.Text = "Tardanzas";
            //
            // lblTardanzasValor
```

UI/FrmAsistencia.Designer.cs (continuacion)

```
            //
            this.lblTardanzasValor.Font = new System.Drawing.Font("Segoe UI", 16F, System.Drawing.FontStyle.Bold);
            this.lblTardanzasValor.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(251)))), ((int)(((byte)(191)))), ((int)(((byte)(36)))));
            this.lblTardanzasValor.Location = new System.Drawing.Point(16, 28);
            this.lblTardanzasValor.Name = "lblTardanzasValor";
            this.lblTardanzasValor.Size = new System.Drawing.Size(170, 34);
            this.lblTardanzasValor.TabIndex = 1;
            this.lblTardanzasValor.Text = "0";
            //
            // cardPorcentaje
            //
            this.cardPorcentaje.BackColor = System.Drawing.Color.Black;
            this.cardPorcentaje.BorderStyle = System.Windows.Forms.BorderStyle.FixedSingle;
            this.cardPorcentaje.Controls.Add(this.lblPorcentajeTitulo);
            this.cardPorcentaje.Controls.Add(this.lblPorcentajeValor);
            this.cardPorcentaje.Location = new System.Drawing.Point(762, 304);
            this.cardPorcentaje.Name = "cardPorcentaje";
            this.cardPorcentaje.Size = new System.Drawing.Size(222, 64);
            this.cardPorcentaje.TabIndex = 6;
            //
            // lblPorcentajeTitulo
            //
            this.lblPorcentajeTitulo.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(170)))), ((int)(((byte)(170)))), ((int)(((byte)(185)))));
            this.lblPorcentajeTitulo.Location = new System.Drawing.Point(16, 10);
            this.lblPorcentajeTitulo.Name = "lblPorcentajeTitulo";
            this.lblPorcentajeTitulo.Size = new System.Drawing.Size(160, 18);
            this.lblPorcentajeTitulo.TabIndex = 0;
            this.lblPorcentajeTitulo.Text = "Asistencia";
            //
            // lblPorcentajeValor
            //
            this.lblPorcentajeValor.Font = new System.Drawing.Font("Segoe UI", 16F, System.Drawing.FontStyle.Bold);
            this.lblPorcentajeValor.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(139)))), ((int)(((byte)(92)))), ((int)(((byte)(246)))));
            this.lblPorcentajeValor.Location = new System.Drawing.Point(16, 28);
            this.lblPorcentajeValor.Name = "lblPorcentajeValor";
            this.lblPorcentajeValor.Size = new System.Drawing.Size(170, 34);
            this.lblPorcentajeValor.TabIndex = 1;
            this.lblPorcentajeValor.Text = "0%";
            //
            // dgvAsistencia
            //
            this.dgvAsistencia.AllowUserToAddRows = false;
            this.dgvAsistencia.AllowUserToDeleteRows = false;
            this.dgvAsistencia.BackgroundColor = System.Drawing.Color.Black;
            this.dgvAsistencia.ColumnHeadersHeightSizeMode = System.Windows.Forms.DataGridViewColumnHeadersHeightSizeMode.AutoSize;
            this.dgvAsistencia.Columns.AddRange(new System.Windows.Forms.DataGridViewColumn[] {
            this.dataGridViewTextBoxColumn1,
            this.dataGridViewTextBoxColumn2,
            this.dataGridViewTextBoxColumn3,
            this.dataGridViewTextBoxColumn4,
            this.dataGridViewTextBoxColumn5,
            this.dataGridViewTextBoxColumn6});
            this.dgvAsistencia.Location = new System.Drawing.Point(36, 392);
            this.dgvAsistencia.Name = "dgvAsistencia";
            this.dgvAsistencia.ReadOnly = true;
            this.dgvAsistencia.RowHeadersVisible = false;
            this.dgvAsistencia.RowHeadersWidth = 51;
            this.dgvAsistencia.SelectionMode = System.Windows.Forms.DataGridViewSelectionMode.FullRowSelect;
            this.dgvAsistencia.Size = new System.Drawing.Size(948, 286);
            this.dgvAsistencia.TabIndex = 7;
```

UI/FrmAsistencia.Designer.cs (continuacion)

```
            //
            // dataGridViewTextBoxColumn1
            //
            this.dataGridViewTextBoxColumn1.HeaderText = "Empleado";
            this.dataGridViewTextBoxColumn1.MinimumWidth = 6;
            this.dataGridViewTextBoxColumn1.Name = "dataGridViewTextBoxColumn1";
            this.dataGridViewTextBoxColumn1.ReadOnly = true;
            this.dataGridViewTextBoxColumn1.Width = 125;
            //
            // dataGridViewTextBoxColumn2
            //
            this.dataGridViewTextBoxColumn2.HeaderText = "Fecha";
            this.dataGridViewTextBoxColumn2.MinimumWidth = 6;
            this.dataGridViewTextBoxColumn2.Name = "dataGridViewTextBoxColumn2";
            this.dataGridViewTextBoxColumn2.ReadOnly = true;
            this.dataGridViewTextBoxColumn2.Width = 125;
            //
            // dataGridViewTextBoxColumn3
            //
            this.dataGridViewTextBoxColumn3.HeaderText = "Entrada";
            this.dataGridViewTextBoxColumn3.MinimumWidth = 6;
            this.dataGridViewTextBoxColumn3.Name = "dataGridViewTextBoxColumn3";
            this.dataGridViewTextBoxColumn3.ReadOnly = true;
            this.dataGridViewTextBoxColumn3.Width = 125;
            //
            // dataGridViewTextBoxColumn4
            //
            this.dataGridViewTextBoxColumn4.HeaderText = "Salida";
            this.dataGridViewTextBoxColumn4.MinimumWidth = 6;
            this.dataGridViewTextBoxColumn4.Name = "dataGridViewTextBoxColumn4";
            this.dataGridViewTextBoxColumn4.ReadOnly = true;
            this.dataGridViewTextBoxColumn4.Width = 125;
            //
            // dataGridViewTextBoxColumn5
            //
            this.dataGridViewTextBoxColumn5.HeaderText = "Horas";
            this.dataGridViewTextBoxColumn5.MinimumWidth = 6;
            this.dataGridViewTextBoxColumn5.Name = "dataGridViewTextBoxColumn5";
            this.dataGridViewTextBoxColumn5.ReadOnly = true;
            this.dataGridViewTextBoxColumn5.Width = 125;
            //
            // dataGridViewTextBoxColumn6
            //
            this.dataGridViewTextBoxColumn6.HeaderText = "Estado";
            this.dataGridViewTextBoxColumn6.MinimumWidth = 6;
            this.dataGridViewTextBoxColumn6.Name = "dataGridViewTextBoxColumn6";
            this.dataGridViewTextBoxColumn6.ReadOnly = true;
            this.dataGridViewTextBoxColumn6.Width = 125;
            //
            // FrmAsistencia
            //
            this.AutoScaleDimensions = new System.Drawing.SizeF(8F, 20F);
            this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font;
            this.BackColor = System.Drawing.Color.FromArgb(((int)(((byte)(13)))), ((int)(((byte)(14)))), ((int)(((byte)(20)))));
            this.ClientSize = new System.Drawing.Size(1020, 742);
            this.Controls.Add(this.dgvAsistencia);
            this.Controls.Add(this.cardPorcentaje);
            this.Controls.Add(this.cardTardanzas);
            this.Controls.Add(this.cardAusentes);
            this.Controls.Add(this.cardPresentes);
```

UI/FrmAsistencia.Designer.cs (continuacion)

```
            this.Controls.Add(this.panelFiltros);
            this.Controls.Add(this.panelRegistro);
            this.Controls.Add(this.lblTitulo);
            this.Font = new System.Drawing.Font("Segoe UI", 9F);
            this.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(229)))), ((int)(((byte)(228)))), ((int)(((byte)(245)))));
            this.Name = "FrmAsistencia";
            this.Text = "Asistencia";
            this.Load += new System.EventHandler(this.FrmAsistencia_Load);
            this.panelRegistro.ResumeLayout(false);
            this.panelFiltros.ResumeLayout(false);
            this.cardPresentes.ResumeLayout(false);
            this.cardAusentes.ResumeLayout(false);
            this.cardTardanzas.ResumeLayout(false);
            this.cardPorcentaje.ResumeLayout(false);
            ((System.ComponentModel.ISupportInitialize)(this.dgvAsistencia)).EndInit();
            this.ResumeLayout(false);

        }

        private System.Windows.Forms.DataGridViewTextBoxColumn dataGridViewTextBoxColumn1;
        private System.Windows.Forms.DataGridViewTextBoxColumn dataGridViewTextBoxColumn2;
        private System.Windows.Forms.DataGridViewTextBoxColumn dataGridViewTextBoxColumn3;
        private System.Windows.Forms.DataGridViewTextBoxColumn dataGridViewTextBoxColumn4;
        private System.Windows.Forms.DataGridViewTextBoxColumn dataGridViewTextBoxColumn5;
        private System.Windows.Forms.DataGridViewTextBoxColumn dataGridViewTextBoxColumn6;
    }
}
```

UI/FrmAuditoria.cs

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Windows.Forms;
using SistemaGestionNomina.Helpers;
using SistemaGestionNomina.Models;
using SistemaGestionNomina.Security;
using SistemaGestionNomina.Services;

namespace SistemaGestionNomina.UI
{
    public partial class FrmAuditoria : Form
    {
        private const int PageSize = 100;
        private readonly AuditTrailService auditTrailService = new AuditTrailService();
        private readonly AuthorizationService authorizationService = new AuthorizationService();
        private int pageIndex;

        public FrmAuditoria()
        {
            InitializeComponent();
            ControlStyleHelper.ApplyModernForm(this);
        }

        private void FrmAuditoria_Load(object sender, EventArgs e)
        {
            if (LicenseManager.UsageMode == LicenseUsageMode.Designtime) return;
            try
            {
                authorizationService.DemandPermission(Permissions.AuditView);
                LoadFilters();
                LoadRecords();
            }
            catch (Exception ex)
            {
                MessageBox.Show(ex.Message, "Auditoría", MessageBoxButtons.OK, MessageBoxIcon.Warning);
            }
        }

        private void LoadFilters()
        {
            cmbModulo.Items.Clear();
            cmbModulo.Items.Add("Todos");
            foreach (string module in auditTrailService.ObtenerModulos()) cmbModulo.Items.Add(module);
            cmbModulo.SelectedIndex = 0;

            cmbAccion.Items.Clear();
            cmbAccion.Items.Add("Todos");
            foreach (string action in auditTrailService.ObtenerAcciones()) cmbAccion.Items.Add(action);
            cmbAccion.SelectedIndex = 0;
        }

        private void LoadRecords()
        {
            AuditQuery query = new AuditQuery
            {
                Usuario = txtUsuario.Text.Trim(),
                Modulo = SelectedFilter(cmbModulo),
                Accion = SelectedFilter(cmbAccion),
                Detalle = txtDetalle.Text.Trim(),
```

UI/FrmAuditoria.cs (continuacion)

```
                FechaDesde = dtpDesde.Checked ? dtpDesde.Value.Date : (DateTime?)null,
                FechaHasta = dtpHasta.Checked ? dtpHasta.Value.Date : (DateTime?)null,
                Offset = pageIndex * PageSize,
                Limit = PageSize
            };

            if (query.FechaDesde.HasValue && query.FechaHasta.HasValue &&
                query.FechaDesde.Value > query.FechaHasta.Value)
            {
                throw new ArgumentException("La fecha inicial no puede ser mayor que la fecha final.");
            }

            List<AuditRecord> records = auditTrailService.Buscar(query);
            dgvAuditoria.Rows.Clear();
            foreach (AuditRecord record in records)
            {
                dgvAuditoria.Rows.Add(record.Fecha.ToString("dd/MM/yyyy HH:mm:ss"), record.Usuario,
                    record.Modulo, record.Accion, record.Detalle);
            }

            lblPagina.Text = "Página " + (pageIndex + 1) + " · " + records.Count + " registros";
            btnAnterior.Enabled = pageIndex > 0;
            btnSiguiente.Enabled = records.Count == PageSize;
        }

        private static string SelectedFilter(ComboBox combo)
        {
            return combo.SelectedItem == null || combo.SelectedItem.ToString() == "Todos"
                ? string.Empty : combo.SelectedItem.ToString();
        }

        private void btnBuscar_Click(object sender, EventArgs e)
        {
            pageIndex = 0;
            TryLoadRecords();
        }

        private void btnLimpiar_Click(object sender, EventArgs e)
        {
            txtUsuario.Clear();
            txtDetalle.Clear();
            cmbModulo.SelectedIndex = 0;
            cmbAccion.SelectedIndex = 0;
            dtpDesde.Checked = false;
            dtpHasta.Checked = false;
            pageIndex = 0;
            TryLoadRecords();
        }

        private void btnAnterior_Click(object sender, EventArgs e)
        {
            if (pageIndex > 0) pageIndex--;
            TryLoadRecords();
        }

        private void btnSiguiente_Click(object sender, EventArgs e)
        {
            pageIndex++;
            TryLoadRecords();
        }
```

UI/FrmAuditoria.cs (continuacion)

```

        private void TryLoadRecords()
        {
            try
            {
                LoadRecords();
            }
            catch (Exception ex)
            {
                MessageBox.Show(ex.Message, "Auditoría", MessageBoxButtons.OK, MessageBoxIcon.Warning);
            }
        }
    }
}
```

UI/FrmAuditoria.Designer.cs

```
namespace SistemaGestionNomina.UI
{
    partial class FrmAuditoria
    {
        private System.ComponentModel.IContainer components = null;
        private System.Windows.Forms.Label lblTitulo;
        private System.Windows.Forms.Panel panelFiltros;
        private System.Windows.Forms.Label lblUsuario;
        private System.Windows.Forms.TextBox txtUsuario;
        private System.Windows.Forms.Label lblModulo;
        private System.Windows.Forms.ComboBox cmbModulo;
        private System.Windows.Forms.Label lblAccion;
        private System.Windows.Forms.ComboBox cmbAccion;
        private System.Windows.Forms.Label lblDetalle;
        private System.Windows.Forms.TextBox txtDetalle;
        private System.Windows.Forms.Label lblDesde;
        private System.Windows.Forms.DateTimePicker dtpDesde;
        private System.Windows.Forms.Label lblHasta;
        private System.Windows.Forms.DateTimePicker dtpHasta;
        private System.Windows.Forms.Button btnBuscar;
        private System.Windows.Forms.Button btnLimpiar;
        private System.Windows.Forms.DataGridView dgvAuditoria;
        private System.Windows.Forms.DataGridViewTextBoxColumn colFecha;
        private System.Windows.Forms.DataGridViewTextBoxColumn colUsuario;
        private System.Windows.Forms.DataGridViewTextBoxColumn colModulo;
        private System.Windows.Forms.DataGridViewTextBoxColumn colAccion;
        private System.Windows.Forms.DataGridViewTextBoxColumn colDetalle;
        private System.Windows.Forms.Button btnAnterior;
        private System.Windows.Forms.Button btnSiguiente;
        private System.Windows.Forms.Label lblPagina;

        protected override void Dispose(bool disposing)
        {
            if (disposing && (components != null)) components.Dispose();
            base.Dispose(disposing);
        }

        private void InitializeComponent()
        {
            System.Windows.Forms.DataGridViewCellStyle headerStyle = new System.Windows.Forms.DataGridViewCellStyle();
            System.Windows.Forms.DataGridViewCellStyle cellStyle = new System.Windows.Forms.DataGridViewCellStyle();
            this.lblTitulo = new System.Windows.Forms.Label();
            this.panelFiltros = new System.Windows.Forms.Panel();
            this.lblUsuario = new System.Windows.Forms.Label();
            this.txtUsuario = new System.Windows.Forms.TextBox();
            this.lblModulo = new System.Windows.Forms.Label();
            this.cmbModulo = new System.Windows.Forms.ComboBox();
            this.lblAccion = new System.Windows.Forms.Label();
            this.cmbAccion = new System.Windows.Forms.ComboBox();
            this.lblDetalle = new System.Windows.Forms.Label();
            this.txtDetalle = new System.Windows.Forms.TextBox();
            this.lblDesde = new System.Windows.Forms.Label();
            this.dtpDesde = new System.Windows.Forms.DateTimePicker();
            this.lblHasta = new System.Windows.Forms.Label();
            this.dtpHasta = new System.Windows.Forms.DateTimePicker();
            this.btnBuscar = new System.Windows.Forms.Button();
            this.btnLimpiar = new System.Windows.Forms.Button();
            this.dgvAuditoria = new System.Windows.Forms.DataGridView();
            this.colFecha = new System.Windows.Forms.DataGridViewTextBoxColumn();
            this.colUsuario = new System.Windows.Forms.DataGridViewTextBoxColumn();
```

UI/FrmAuditoria.Designer.cs (continuacion)

```
            this.colModulo = new System.Windows.Forms.DataGridViewTextBoxColumn();
            this.colAccion = new System.Windows.Forms.DataGridViewTextBoxColumn();
            this.colDetalle = new System.Windows.Forms.DataGridViewTextBoxColumn();
            this.btnAnterior = new System.Windows.Forms.Button();
            this.btnSiguiente = new System.Windows.Forms.Button();
            this.lblPagina = new System.Windows.Forms.Label();
            this.panelFiltros.SuspendLayout();
            ((System.ComponentModel.ISupportInitialize)(this.dgvAuditoria)).BeginInit();
            this.SuspendLayout();
            // lblTitulo
            this.lblTitulo.AutoSize = true;
            this.lblTitulo.Font = new System.Drawing.Font("Segoe UI", 22F, System.Drawing.FontStyle.Bold);
            this.lblTitulo.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245);
            this.lblTitulo.Location = new System.Drawing.Point(28, 20);
            this.lblTitulo.Name = "lblTitulo";
            this.lblTitulo.Size = new System.Drawing.Size(329, 50);
            this.lblTitulo.Text = "Auditoría del sistema";
            // panelFiltros
            this.panelFiltros.Anchor = ((System.Windows.Forms.AnchorStyles)(((System.Windows.Forms.AnchorStyles.Top |  \
    System.Windows.Forms.AnchorStyles.Left) | System.Windows.Forms.AnchorStyles.Right)));
            this.panelFiltros.BackColor = System.Drawing.Color.FromArgb(24, 25, 34);
            this.panelFiltros.Controls.Add(this.lblUsuario);
            this.panelFiltros.Controls.Add(this.txtUsuario);
            this.panelFiltros.Controls.Add(this.lblModulo);
            this.panelFiltros.Controls.Add(this.cmbModulo);
            this.panelFiltros.Controls.Add(this.lblAccion);
            this.panelFiltros.Controls.Add(this.cmbAccion);
            this.panelFiltros.Controls.Add(this.lblDetalle);
            this.panelFiltros.Controls.Add(this.txtDetalle);
            this.panelFiltros.Controls.Add(this.lblDesde);
            this.panelFiltros.Controls.Add(this.dtpDesde);
            this.panelFiltros.Controls.Add(this.lblHasta);
            this.panelFiltros.Controls.Add(this.dtpHasta);
            this.panelFiltros.Controls.Add(this.btnBuscar);
            this.panelFiltros.Controls.Add(this.btnLimpiar);
            this.panelFiltros.Location = new System.Drawing.Point(32, 82);
            this.panelFiltros.Name = "panelFiltros";
            this.panelFiltros.Size = new System.Drawing.Size(956, 136);
            // labels
            this.lblUsuario.AutoSize = true; this.lblUsuario.Location = new System.Drawing.Point(18, 14); this.lblUsuario.Text = "Usuario";
            this.lblModulo.AutoSize = true; this.lblModulo.Location = new System.Drawing.Point(206, 14); this.lblModulo.Text = "Módulo";
            this.lblAccion.AutoSize = true; this.lblAccion.Location = new System.Drawing.Point(394, 14); this.lblAccion.Text = "Acción";
            this.lblDetalle.AutoSize = true; this.lblDetalle.Location = new System.Drawing.Point(582, 14); this.lblDetalle.Text = "Detalle";
            this.lblDesde.AutoSize = true; this.lblDesde.Location = new System.Drawing.Point(18, 75); this.lblDesde.Text = "Desde";
            this.lblHasta.AutoSize = true; this.lblHasta.Location = new System.Drawing.Point(238, 75); this.lblHasta.Text = "Hasta";
            // inputs
            this.txtUsuario.Location = new System.Drawing.Point(18, 36); this.txtUsuario.Size = new System.Drawing.Size(170, 27);
            this.cmbModulo.DropDownStyle = System.Windows.Forms.ComboBoxStyle.DropDownList; this.cmbModulo.Location = new System.Drawing.Point(206, 36);  \
    this.cmbModulo.Size = new System.Drawing.Size(170, 28);
            this.cmbAccion.DropDownStyle = System.Windows.Forms.ComboBoxStyle.DropDownList; this.cmbAccion.Location = new System.Drawing.Point(394, 36);  \
    this.cmbAccion.Size = new System.Drawing.Size(170, 28);
            this.txtDetalle.Location = new System.Drawing.Point(582, 36); this.txtDetalle.Size = new System.Drawing.Size(350, 27);
            this.dtpDesde.Format = System.Windows.Forms.DateTimePickerFormat.Short; this.dtpDesde.Location = new System.Drawing.Point(18, 96);  \
    this.dtpDesde.ShowCheckBox = true; this.dtpDesde.Size = new System.Drawing.Size(202, 27);
            this.dtpHasta.Format = System.Windows.Forms.DateTimePickerFormat.Short; this.dtpHasta.Location = new System.Drawing.Point(238, 96);  \
    this.dtpHasta.ShowCheckBox = true; this.dtpHasta.Size = new System.Drawing.Size(202, 27);
            // buttons
            this.btnBuscar.BackColor = System.Drawing.Color.FromArgb(124, 58, 237); this.btnBuscar.FlatStyle = System.Windows.Forms.FlatStyle.Flat;  \
    this.btnBuscar.ForeColor = System.Drawing.Color.White; this.btnBuscar.Location = new System.Drawing.Point(674, 88); this.btnBuscar.Size = new  \
    System.Drawing.Size(122, 36); this.btnBuscar.Text = "Buscar"; this.btnBuscar.UseVisualStyleBackColor = false; this.btnBuscar.Click += new  \
```

```
System.EventHandler(this.btnBuscar_Click);
        this.btnLimpiar.BackColor = System.Drawing.Color.FromArgb(42, 43, 54); this.btnLimpiar.FlatStyle = System.Windows.Forms.FlatStyle.Flat;  \
this.btnLimpiar.ForeColor = System.Drawing.Color.White; this.btnLimpiar.Location = new System.Drawing.Point(810, 88); this.btnLimpiar.Size = new  \
System.Drawing.Size(122, 36); this.btnLimpiar.Text = "Limpiar"; this.btnLimpiar.UseVisualStyleBackColor = false; this.btnLimpiar.Click += new  \
System.EventHandler(this.btnLimpiar_Click);
        // dgvAuditoria
        this.dgvAuditoria.AllowUserToAddRows = false;
        this.dgvAuditoria.AllowUserToDeleteRows = false;
        this.dgvAuditoria.AllowUserToResizeRows = false;
        this.dgvAuditoria.Anchor = ((System.Windows.Forms.AnchorStyles)((((System.Windows.Forms.AnchorStyles.Top |  \
System.Windows.Forms.AnchorStyles.Bottom) | System.Windows.Forms.AnchorStyles.Left) | System.Windows.Forms.AnchorStyles.Right)));
        this.dgvAuditoria.AutoSizeColumnsMode = System.Windows.Forms.DataGridViewAutoSizeColumnsMode.Fill;
        this.dgvAuditoria.BackgroundColor = System.Drawing.Color.FromArgb(18, 19, 27);
        this.dgvAuditoria.BorderStyle = System.Windows.Forms.BorderStyle.None;
        headerStyle.BackColor = System.Drawing.Color.FromArgb(42, 43, 54); headerStyle.ForeColor = System.Drawing.Color.White;  \
headerStyle.SelectionBackColor = System.Drawing.Color.FromArgb(42, 43, 54); headerStyle.Font = new System.Drawing.Font("Segoe UI", 9F,  \
System.Drawing.FontStyle.Bold);
        this.dgvAuditoria.ColumnHeadersDefaultCellStyle = headerStyle;
        this.dgvAuditoria.ColumnHeadersHeight = 40;
        this.dgvAuditoria.Columns.AddRange(new System.Windows.Forms.DataGridViewColumn[] { this.colFecha, this.colUsuario, this.colModulo,  \
this.colAccion, this.colDetalle });
        cellStyle.BackColor = System.Drawing.Color.FromArgb(24, 25, 34); cellStyle.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245);  \
cellStyle.SelectionBackColor = System.Drawing.Color.FromArgb(76, 59, 118); cellStyle.SelectionForeColor = System.Drawing.Color.White;
        this.dgvAuditoria.DefaultCellStyle = cellStyle;
        this.dgvAuditoria.EnableHeadersVisualStyles = false;
        this.dgvAuditoria.Location = new System.Drawing.Point(32, 236);
        this.dgvAuditoria.MultiSelect = false;
        this.dgvAuditoria.ReadOnly = true;
        this.dgvAuditoria.RowHeadersVisible = false;
        this.dgvAuditoria.RowTemplate.Height = 32;
        this.dgvAuditoria.SelectionMode = System.Windows.Forms.DataGridViewSelectionMode.FullRowSelect;
        this.dgvAuditoria.Size = new System.Drawing.Size(956, 418);
        this.colFecha.HeaderText = "Fecha"; this.colFecha.FillWeight = 95F; this.colFecha.ReadOnly = true;
        this.colUsuario.HeaderText = "Usuario"; this.colUsuario.FillWeight = 75F; this.colUsuario.ReadOnly = true;
        this.colModulo.HeaderText = "Módulo"; this.colModulo.FillWeight = 75F; this.colModulo.ReadOnly = true;
        this.colAccion.HeaderText = "Acción"; this.colAccion.FillWeight = 95F; this.colAccion.ReadOnly = true;
        this.colDetalle.HeaderText = "Detalle"; this.colDetalle.FillWeight = 190F; this.colDetalle.ReadOnly = true;
        // pagination
        this.btnAnterior.Anchor = ((System.Windows.Forms.AnchorStyles)((System.Windows.Forms.AnchorStyles.Bottom |  \
System.Windows.Forms.AnchorStyles.Right))); this.btnAnterior.Location = new System.Drawing.Point(764, 670); this.btnAnterior.Size = new  \
System.Drawing.Size(100, 34); this.btnAnterior.Text = "Anterior"; this.btnAnterior.Click += new System.EventHandler(this.btnAnterior_Click);
        this.btnSiguiente.Anchor = ((System.Windows.Forms.AnchorStyles)((System.Windows.Forms.AnchorStyles.Bottom |  \
System.Windows.Forms.AnchorStyles.Right))); this.btnSiguiente.Location = new System.Drawing.Point(888, 670); this.btnSiguiente.Size = new  \
System.Drawing.Size(100, 34); this.btnSiguiente.Text = "Siguiente"; this.btnSiguiente.Click += new System.EventHandler(this.btnSiguiente_Click);
        this.lblPagina.Anchor = ((System.Windows.Forms.AnchorStyles)((System.Windows.Forms.AnchorStyles.Bottom |  \
System.Windows.Forms.AnchorStyles.Left))); this.lblPagina.Location = new System.Drawing.Point(32, 675); this.lblPagina.Size = new  \
System.Drawing.Size(300, 24); this.lblPagina.Text = "Página 1";
        // FrmAuditoria
        this.AutoScaleDimensions = new System.Drawing.SizeF(8F, 20F);
        this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font;
        this.BackColor = System.Drawing.Color.FromArgb(13, 14, 20);
        this.ClientSize = new System.Drawing.Size(1020, 720);
        this.Controls.Add(this.lblPagina);
        this.Controls.Add(this.btnAnterior);
        this.Controls.Add(this.btnSiguiente);
        this.Controls.Add(this.dgvAuditoria);
        this.Controls.Add(this.panelFiltros);
        this.Controls.Add(this.lblTitulo);
        this.Font = new System.Drawing.Font("Segoe UI", 9F);
        this.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245);
```

UI/FrmAuditoria.Designer.cs (continuacion)

```
            this.Name = "FrmAuditoria";
            this.Text = "Auditoría";
            this.Load += new System.EventHandler(this.FrmAuditoria_Load);
            this.panelFiltros.ResumeLayout(false);
            this.panelFiltros.PerformLayout();
            ((System.ComponentModel.ISupportInitialize)(this.dgvAuditoria)).EndInit();
            this.ResumeLayout(false);
            this.PerformLayout();
        }
    }
}
```

UI/FrmCambiarPassword.cs

```
using System.Windows.Forms;
using SistemaGestionNomina.Helpers;
using SistemaGestionNomina.Services;

namespace SistemaGestionNomina.UI
{
    public partial class FrmCambiarPassword : Form
    {
        private readonly AccountService accountService = new AccountService();

        public FrmCambiarPassword()
        {
            InitializeComponent();
            ControlStyleHelper.ApplyModernForm(this);
        }

        private void ClearFields()
        {
            txtActual.Clear();
            txtNueva.Clear();
            txtConfirmar.Clear();
            txtActual.Focus();
        }

        private void btnGuardar_Click(object sender, System.EventArgs e)
        {
            try
            {
                accountService.ChangeOwnPassword(txtActual.Text, txtNueva.Text, txtConfirmar.Text);
                MessageBox.Show("La contrasena se actualizo correctamente.", "Cuenta",
                    MessageBoxButtons.OK, MessageBoxIcon.Information);
                ClearFields();
            }
            catch (System.Exception ex)
            {
                MessageBox.Show(ex.Message, "Cambiar contrasena", MessageBoxButtons.OK, MessageBoxIcon.Warning);
            }
        }
    }
}
```

UI/FrmCambiarPassword.Designer.cs

```
namespace SistemaGestionNomina.UI
{
    partial class FrmCambiarPassword
    {
        private System.ComponentModel.IContainer components = null;
        private System.Windows.Forms.Label lblTitulo;
        private System.Windows.Forms.Label lblSubtitulo;
        private System.Windows.Forms.Panel panelCuenta;
        private System.Windows.Forms.Label lblActual;
        private System.Windows.Forms.TextBox txtActual;
        private System.Windows.Forms.Label lblNueva;
        private System.Windows.Forms.TextBox txtNueva;
        private System.Windows.Forms.Label lblConfirmar;
        private System.Windows.Forms.TextBox txtConfirmar;
        private System.Windows.Forms.Label lblPolitica;
        private System.Windows.Forms.Button btnGuardar;

        protected override void Dispose(bool disposing) { if (disposing && components != null) components.Dispose(); base.Dispose(disposing); }

        private void InitializeComponent()
        {
            this.lblTitulo = new System.Windows.Forms.Label(); this.lblSubtitulo = new System.Windows.Forms.Label(); this.panelCuenta = new  \
    System.Windows.Forms.Panel(); this.lblActual = new System.Windows.Forms.Label(); this.txtActual = new System.Windows.Forms.TextBox(); this.lblNueva =  \
    new System.Windows.Forms.Label(); this.txtNueva = new System.Windows.Forms.TextBox(); this.lblConfirmar = new System.Windows.Forms.Label();  \
    this.txtConfirmar = new System.Windows.Forms.TextBox(); this.lblPolitica = new System.Windows.Forms.Label(); this.btnGuardar = new  \
    System.Windows.Forms.Button(); this.panelCuenta.SuspendLayout(); this.SuspendLayout();
            this.lblTitulo.AutoSize = true; this.lblTitulo.Font = new System.Drawing.Font("Segoe UI", 23F, System.Drawing.FontStyle.Bold);  \
    this.lblTitulo.Location = new System.Drawing.Point(30, 24); this.lblTitulo.Text = "Cambiar contrasena";
            this.lblSubtitulo.AutoSize = true; this.lblSubtitulo.ForeColor = System.Drawing.Color.FromArgb(161, 161, 175); this.lblSubtitulo.Location =  \
    new System.Drawing.Point(34, 78); this.lblSubtitulo.Text = "Actualiza la clave utilizada para iniciar sesion.";
            this.panelCuenta.BackColor = System.Drawing.Color.FromArgb(24, 25, 34); this.panelCuenta.Controls.Add(this.lblActual);  \
    this.panelCuenta.Controls.Add(this.txtActual); this.panelCuenta.Controls.Add(this.lblNueva); this.panelCuenta.Controls.Add(this.txtNueva);  \
    this.panelCuenta.Controls.Add(this.lblConfirmar); this.panelCuenta.Controls.Add(this.txtConfirmar); this.panelCuenta.Controls.Add(this.lblPolitica);  \
    this.panelCuenta.Controls.Add(this.btnGuardar); this.panelCuenta.Location = new System.Drawing.Point(38, 126); this.panelCuenta.Name = "panelCuenta";  \
    this.panelCuenta.Size = new System.Drawing.Size(560, 444);
            this.lblActual.AutoSize = true; this.lblActual.Location = new System.Drawing.Point(28, 28); this.lblActual.Text = "Contrasena actual";
            this.txtActual.Location = new System.Drawing.Point(28, 56); this.txtActual.Name = "txtActual"; this.txtActual.Size = new  \
    System.Drawing.Size(500, 27); this.txtActual.UseSystemPasswordChar = true;
            this.lblNueva.AutoSize = true; this.lblNueva.Location = new System.Drawing.Point(28, 110); this.lblNueva.Text = "Nueva contrasena";
            this.txtNueva.Location = new System.Drawing.Point(28, 138); this.txtNueva.Name = "txtNueva"; this.txtNueva.Size = new  \
    System.Drawing.Size(500, 27); this.txtNueva.UseSystemPasswordChar = true;
            this.lblConfirmar.AutoSize = true; this.lblConfirmar.Location = new System.Drawing.Point(28, 192); this.lblConfirmar.Text = "Confirmar nueva  \
    contrasena";
            this.txtConfirmar.Location = new System.Drawing.Point(28, 220); this.txtConfirmar.Name = "txtConfirmar"; this.txtConfirmar.Size = new  \
    System.Drawing.Size(500, 27); this.txtConfirmar.UseSystemPasswordChar = true;
            this.lblPolitica.ForeColor = System.Drawing.Color.FromArgb(161, 161, 175); this.lblPolitica.Location = new System.Drawing.Point(28, 274);  \
    this.lblPolitica.Name = "lblPolitica"; this.lblPolitica.Size = new System.Drawing.Size(500, 52); this.lblPolitica.Text = "Minimo 8 caracteres, con  \
    mayuscula, minuscula y numero.";
            this.btnGuardar.Location = new System.Drawing.Point(28, 354); this.btnGuardar.Name = "btnGuardar"; this.btnGuardar.Size = new  \
    System.Drawing.Size(500, 44); this.btnGuardar.Text = "Guardar nueva contrasena"; this.btnGuardar.Click += new  \
    System.EventHandler(this.btnGuardar_Click);
            this.AcceptButton = this.btnGuardar; this.AutoScaleDimensions = new System.Drawing.SizeF(8F, 20F); this.AutoScaleMode =  \
    System.Windows.Forms.AutoScaleMode.Font; this.BackColor = System.Drawing.Color.FromArgb(13, 14, 20); this.ClientSize = new System.Drawing.Size(1020,  \
    742); this.Controls.Add(this.panelCuenta); this.Controls.Add(this.lblSubtitulo); this.Controls.Add(this.lblTitulo); this.Font = new  \
    System.Drawing.Font("Segoe UI", 9F); this.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245); this.Name = "FrmCambiarPassword"; this.Text =  \
    "Cambiar contrasena"; this.panelCuenta.ResumeLayout(false); this.panelCuenta.PerformLayout(); this.ResumeLayout(false); this.PerformLayout();
        }
    }
}
```

UI/FrmCambioLaboral.cs

```
using System;
using System.Windows.Forms;
using SistemaGestionNomina.Helpers;
using SistemaGestionNomina.Models;

namespace SistemaGestionNomina.UI
{
    public partial class FrmCambioLaboral : Form
    {
        public EmployeeChangeContext ChangeContext { get; private set; }

        public FrmCambioLaboral() : this(null) { }

        public FrmCambioLaboral(Empleado employee)
        {
            InitializeComponent();
            ControlStyleHelper.ApplyModernForm(this);
            if (employee != null)
                lblEmpleado.Text = employee.Codigo + " - " + employee.NombreCompleto;
        }

        private void btnAceptar_Click(object sender, EventArgs e)
        {
            if (string.IsNullOrWhiteSpace(txtMotivo.Text))
            {
                MessageBox.Show("Indique el motivo del cambio laboral.", "Cambio laboral",
                    MessageBoxButtons.OK, MessageBoxIcon.Warning);
                txtMotivo.Focus();
                return;
            }
            ChangeContext = new EmployeeChangeContext
            {
                EffectiveDate = dtpFechaEfectiva.Value.Date,
                Reason = txtMotivo.Text.Trim()
            };
            DialogResult = DialogResult.OK;
            Close();
        }

        private void btnCancelar_Click(object sender, EventArgs e)
        {
            DialogResult = DialogResult.Cancel;
            Close();
        }
    }
}
```

UI/FrmCambioLaboral.Designer.cs

```
namespace SistemaGestionNomina.UI
{
    partial class FrmCambioLaboral
    {
        private System.ComponentModel.IContainer components = null;
        private System.Windows.Forms.Label lblTitulo;
        private System.Windows.Forms.Label lblEmpleado;
        private System.Windows.Forms.Label lblFecha;
        private System.Windows.Forms.DateTimePicker dtpFechaEfectiva;
        private System.Windows.Forms.Label lblMotivo;
        private System.Windows.Forms.TextBox txtMotivo;
        private System.Windows.Forms.Button btnAceptar;
        private System.Windows.Forms.Button btnCancelar;

        protected override void Dispose(bool disposing)
        {
            if (disposing && (components != null)) components.Dispose();
            base.Dispose(disposing);
        }

        private void InitializeComponent()
        {
            this.lblTitulo = new System.Windows.Forms.Label();
            this.lblEmpleado = new System.Windows.Forms.Label();
            this.lblFecha = new System.Windows.Forms.Label();
            this.dtpFechaEfectiva = new System.Windows.Forms.DateTimePicker();
            this.lblMotivo = new System.Windows.Forms.Label();
            this.txtMotivo = new System.Windows.Forms.TextBox();
            this.btnAceptar = new System.Windows.Forms.Button();
            this.btnCancelar = new System.Windows.Forms.Button();
            this.SuspendLayout();
            this.BackColor = System.Drawing.Color.FromArgb(18, 18, 27);
            this.ClientSize = new System.Drawing.Size(520, 330);
            this.Font = new System.Drawing.Font("Segoe UI", 9F);
            this.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245);
            this.FormBorderStyle = System.Windows.Forms.FormBorderStyle.FixedDialog;
            this.MaximizeBox = false;
            this.MinimizeBox = false;
            this.Name = "FrmCambioLaboral";
            this.StartPosition = System.Windows.Forms.FormStartPosition.CenterParent;
            this.Text = "Cambio laboral";
            this.lblTitulo.AutoSize = true;
            this.lblTitulo.Font = new System.Drawing.Font("Segoe UI", 18F, System.Drawing.FontStyle.Bold);
            this.lblTitulo.Location = new System.Drawing.Point(28, 24);
            this.lblTitulo.Text = "Fecha efectiva del cambio";
            this.lblEmpleado.AutoSize = true;
            this.lblEmpleado.ForeColor = System.Drawing.Color.FromArgb(167, 139, 250);
            this.lblEmpleado.Location = new System.Drawing.Point(31, 72);
            this.lblEmpleado.Text = "Empleado seleccionado";
            this.lblFecha.AutoSize = true;
            this.lblFecha.Location = new System.Drawing.Point(31, 112);
            this.lblFecha.Text = "Fecha efectiva";
            this.dtpFechaEfectiva.Location = new System.Drawing.Point(34, 134);
            this.dtpFechaEfectiva.Size = new System.Drawing.Size(210, 23);
            this.lblMotivo.AutoSize = true;
            this.lblMotivo.Location = new System.Drawing.Point(31, 176);
            this.lblMotivo.Text = "Motivo obligatorio";
            this.txtMotivo.Location = new System.Drawing.Point(34, 198);
            this.txtMotivo.Multiline = true;
            this.txtMotivo.Size = new System.Drawing.Size(450, 68);
```

UI/FrmCambioLaboral.Designer.cs (continuacion)

```
            this.txtMotivo.MaxLength = 300;
            this.btnAceptar.BackColor = System.Drawing.Color.FromArgb(124, 58, 237);
            this.btnAceptar.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
            this.btnAceptar.Location = new System.Drawing.Point(294, 282);
            this.btnAceptar.Size = new System.Drawing.Size(92, 32);
            this.btnAceptar.Text = "Aceptar";
            this.btnAceptar.UseVisualStyleBackColor = false;
            this.btnAceptar.Click += new System.EventHandler(this.btnAceptar_Click);
            this.btnCancelar.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
            this.btnCancelar.Location = new System.Drawing.Point(392, 282);
            this.btnCancelar.Size = new System.Drawing.Size(92, 32);
            this.btnCancelar.Text = "Cancelar";
            this.btnCancelar.Click += new System.EventHandler(this.btnCancelar_Click);
            this.Controls.Add(this.lblTitulo);
            this.Controls.Add(this.lblEmpleado);
            this.Controls.Add(this.lblFecha);
            this.Controls.Add(this.dtpFechaEfectiva);
            this.Controls.Add(this.lblMotivo);
            this.Controls.Add(this.txtMotivo);
            this.Controls.Add(this.btnAceptar);
            this.Controls.Add(this.btnCancelar);
            this.ResumeLayout(false);
            this.PerformLayout();
        }
    }
}
```

UI/FrmComprobantes.cs

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Drawing;
using System.Drawing.Printing;
using System.IO;
using System.Windows.Forms;
using SistemaGestionNomina.Helpers;
using SistemaGestionNomina.Models;
using SistemaGestionNomina.Services;
using SistemaGestionNomina.Security;

namespace SistemaGestionNomina.UI
{
    public partial class FrmComprobantes : Form
    {
        private readonly ComprobanteService comprobanteService = new ComprobanteService();
        private readonly ExcelExportService excelExportService = new ExcelExportService();
        private readonly PdfExportService pdfExportService = new PdfExportService();
        private readonly EmailService emailService = new EmailService();
        private readonly AuthorizationService authorizationService = new AuthorizationService();
        private readonly PrintDocument printDocument = new PrintDocument();
        private Comprobante selectedComprobante;
        private List<Comprobante> currentItems = new List<Comprobante>();

        public FrmComprobantes()
        {
            InitializeComponent();
            ControlStyleHelper.ApplyModernForm(this);
            printDocument.PrintPage += PrintDocument_PrintPage;
        }

        private void FrmComprobantes_Load(object sender, EventArgs e)
        {
            if (LicenseManager.UsageMode != LicenseUsageMode.Designtime)
            {
                btnExportarExcel.Visible = authorizationService.HasPermission(Permissions.PayslipsExport);
                btnExportarPdf.Visible = authorizationService.HasPermission(Permissions.PayslipsExport);
                btnImprimir.Visible = authorizationService.HasPermission(Permissions.PayslipsPrint);
                btnEnviarEmail.Visible = authorizationService.HasPermission(Permissions.PayslipsEmail);
                LoadReceipts();
            }
        }

        private void LoadReceipts()
        {
            currentItems = comprobanteService.GetAll(txtBuscar.Text);
            dgvComprobantes.Rows.Clear();
            foreach (Comprobante c in currentItems)
            {
                dgvComprobantes.Rows.Add(c.IdComprobante, c.NumeroComprobante, c.EmpleadoNombre, c.PeriodoNombre,
                    "B/. " + c.NetoPagar.ToString("0.00"));
            }
        }

        private void txtBuscar_TextChanged(object sender, EventArgs e)
        {
            if (LicenseManager.UsageMode != LicenseUsageMode.Designtime)
            {
                LoadReceipts();
```

UI/FrmComprobantes.cs (continuacion)

```
            }
        }

        private void dgvComprobantes_SelectionChanged(object sender, EventArgs e)
        {
            if (dgvComprobantes.CurrentRow == null || dgvComprobantes.CurrentRow.Cells["IdComprobante"].Value == null)
            {
                return;
            }

            int id = Convert.ToInt32(dgvComprobantes.CurrentRow.Cells["IdComprobante"].Value);
            selectedComprobante = comprobanteService.GetById(id);
            RenderPreview();
        }

        private void RenderPreview()
        {
            if (selectedComprobante == null) return;
            lblVistaPrevia.Text =
                "SISTEMA DE GESTIÓN DE NÓMINA\n" +
                "COMPROBANTE DE PAGO\n\n" +
                "-----------------------------------------------\n" +
                "Documento:      " + selectedComprobante.NumeroComprobante + "\n" +
                "Fecha emisión:  " + selectedComprobante.FechaGeneracion.ToString("dd/MM/yyyy HH:mm") + "\n" +
                "Periodo:        " + selectedComprobante.PeriodoNombre + "\n" +
                "-----------------------------------------------\n" +
                "Empleado:       " + selectedComprobante.EmpleadoNombre + "\n" +
                "Código:         " + selectedComprobante.CodigoEmpleado + "\n" +
                "-----------------------------------------------\n" +
                "Ingresos:       B/. " + selectedComprobante.TotalIngresos.ToString("0.00") + "\n" +
                "Deducciones:    B/. " + selectedComprobante.TotalDeducciones.ToString("0.00") + "\n" +
                "-----------------------------------------------\n" +
                "NETO A PAGAR:   B/. " + selectedComprobante.NetoPagar.ToString("0.00") + "\n" +
                "-----------------------------------------------\n\n" +
                "Firma autorizada: __________________________";
        }

        private void btnExportarPdf_Click(object sender, EventArgs e)
        {
            if (!EnsureSelected()) return;
            string path = pdfExportService.ExportarComprobante(selectedComprobante);
            if (string.IsNullOrWhiteSpace(path)) return;

            comprobanteService.SaveRutaPdf(selectedComprobante.IdComprobante, path);
            UiFactory.ShowExported(path);
        }

        private void btnExportarExcel_Click(object sender, EventArgs e)
        {
            if (currentItems.Count == 0)
            {
                MessageBox.Show("No hay comprobantes para exportar.", "Comprobantes", MessageBoxButtons.OK, MessageBoxIcon.Information);
                return;
            }

            UiFactory.ShowExported(excelExportService.ExportarComprobantes(currentItems));
        }

        private void btnImprimir_Click(object sender, EventArgs e)
        {
```

UI/FrmComprobantes.cs (continuacion)

```
            authorizationService.DemandPermission(Permissions.PayslipsPrint);
            if (!EnsureSelected()) return;

            try
            {
                using (PrintPreviewDialog preview = new PrintPreviewDialog())
                {
                    preview.Document = printDocument;
                    preview.Width = 900;
                    preview.Height = 700;
                    preview.StartPosition = FormStartPosition.CenterParent;
                    preview.ShowDialog(this);
                }
            }
            catch (Exception ex)
            {
                MessageBox.Show("No se pudo abrir la vista previa de impresión.\n\n" + ex.Message,
                    "Imprimir", MessageBoxButtons.OK, MessageBoxIcon.Warning);
            }
        }

        private void PrintDocument_PrintPage(object sender, PrintPageEventArgs e)
        {
            if (selectedComprobante == null)
            {
                e.HasMorePages = false;
                return;
            }

            ComprobantePrintRenderer.Draw(e.Graphics, e.MarginBounds, selectedComprobante);
            e.HasMorePages = false;
        }

        private void btnEnviarEmail_Click(object sender, EventArgs e)
        {
            if (!EnsureSelected()) return;

            string email = PromptEmail();
            if (string.IsNullOrWhiteSpace(email))
            {
                return;
            }

            try
            {
                string pdfPath = selectedComprobante.RutaPdf;
                if (string.IsNullOrWhiteSpace(pdfPath) || !File.Exists(pdfPath))
                {
                    pdfPath = pdfExportService.ExportarComprobante(selectedComprobante);
                    if (string.IsNullOrWhiteSpace(pdfPath)) return;

                    comprobanteService.SaveRutaPdf(selectedComprobante.IdComprobante, pdfPath);
                    selectedComprobante.RutaPdf = pdfPath;
                }

                string emlPath = emailService.CrearBorradorComprobante(email, pdfPath,
                    "Comprobante de pago " + selectedComprobante.NumeroComprobante);
                UiFactory.ShowExported(emlPath);
            }
            catch (Exception ex)
```

UI/FrmComprobantes.cs (continuacion)

```
            {
                MessageBox.Show(ex.Message, "Enviar email", MessageBoxButtons.OK, MessageBoxIcon.Warning);
            }
        }

        private bool EnsureSelected()
        {
            if (selectedComprobante != null) return true;
            MessageBox.Show("Seleccione un comprobante.", "Comprobantes", MessageBoxButtons.OK, MessageBoxIcon.Information);
            return false;
        }

        private string PromptEmail()
        {
            using (Form prompt = new Form())
            using (Label label = new Label())
            using (TextBox input = new TextBox())
            using (Button okButton = new Button())
            using (Button cancelButton = new Button())
            {
                prompt.Text = "Enviar comprobante";
                prompt.StartPosition = FormStartPosition.CenterParent;
                prompt.FormBorderStyle = FormBorderStyle.FixedDialog;
                prompt.MinimizeBox = false;
                prompt.MaximizeBox = false;
                prompt.ClientSize = new Size(420, 140);

                label.Text = "Correo del destinatario:";
                label.Location = new Point(18, 18);
                label.Size = new Size(360, 24);

                input.Location = new Point(18, 48);
                input.Size = new Size(380, 24);

                okButton.Text = "Crear email";
                okButton.DialogResult = DialogResult.OK;
                okButton.Location = new Point(210, 92);
                okButton.Size = new Size(92, 30);

                cancelButton.Text = "Cancelar";
                cancelButton.DialogResult = DialogResult.Cancel;
                cancelButton.Location = new Point(306, 92);
                cancelButton.Size = new Size(92, 30);

                prompt.Controls.Add(label);
                prompt.Controls.Add(input);
                prompt.Controls.Add(okButton);
                prompt.Controls.Add(cancelButton);
                prompt.AcceptButton = okButton;
                prompt.CancelButton = cancelButton;

                return prompt.ShowDialog(this) == DialogResult.OK ? input.Text.Trim() : string.Empty;
            }
        }
    }
}
```

UI/FrmComprobantes.Designer.cs

```
namespace SistemaGestionNomina.UI
{
    partial class FrmComprobantes
    {
        private System.ComponentModel.IContainer components = null;
        private System.Windows.Forms.Label lblTitulo;
        private System.Windows.Forms.TextBox txtBuscar;
        private System.Windows.Forms.DataGridView dgvComprobantes;
        private System.Windows.Forms.Panel panelVistaPrevia;
        private System.Windows.Forms.Label lblVistaPrevia;
        private System.Windows.Forms.Button btnExportarPdf;
        private System.Windows.Forms.Button btnExportarExcel;
        private System.Windows.Forms.Button btnImprimir;
        private System.Windows.Forms.Button btnEnviarEmail;

        protected override void Dispose(bool disposing)
        {
            if (disposing && (components != null)) components.Dispose();
            base.Dispose(disposing);
        }

        private void InitializeComponent()
        {
            this.lblTitulo = new System.Windows.Forms.Label();
            this.txtBuscar = new System.Windows.Forms.TextBox();
            this.dgvComprobantes = new System.Windows.Forms.DataGridView();
            this.panelVistaPrevia = new System.Windows.Forms.Panel();
            this.lblVistaPrevia = new System.Windows.Forms.Label();
            this.btnExportarPdf = new System.Windows.Forms.Button();
            this.btnExportarExcel = new System.Windows.Forms.Button();
            this.btnImprimir = new System.Windows.Forms.Button();
            this.btnEnviarEmail = new System.Windows.Forms.Button();
            ((System.ComponentModel.ISupportInitialize)(this.dgvComprobantes)).BeginInit();
            this.panelVistaPrevia.SuspendLayout();
            this.SuspendLayout();
            //
            // lblTitulo
            //
            this.lblTitulo.Font = new System.Drawing.Font("Segoe UI", 22F, System.Drawing.FontStyle.Bold);
            this.lblTitulo.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245);
            this.lblTitulo.Location = new System.Drawing.Point(34, 24);
            this.lblTitulo.Name = "lblTitulo";
            this.lblTitulo.Size = new System.Drawing.Size(420, 42);
            this.lblTitulo.TabIndex = 0;
            this.lblTitulo.Text = "Comprobantes de pago";
            //
            // txtBuscar
            //
            this.txtBuscar.BackColor = System.Drawing.Color.FromArgb(24, 25, 34);
            this.txtBuscar.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245);
            this.txtBuscar.Location = new System.Drawing.Point(36, 92);
            this.txtBuscar.Name = "txtBuscar";
            this.txtBuscar.Size = new System.Drawing.Size(320, 23);
            this.txtBuscar.TabIndex = 1;
            this.txtBuscar.TextChanged += new System.EventHandler(this.txtBuscar_TextChanged);
            //
            // dgvComprobantes
            //
            this.dgvComprobantes.AllowUserToAddRows = false;
            this.dgvComprobantes.AllowUserToDeleteRows = false;
```

UI/FrmComprobantes.Designer.cs (continuacion)

```
            this.dgvComprobantes.BackgroundColor = System.Drawing.Color.Black;
            this.dgvComprobantes.ColumnHeadersHeightSizeMode = System.Windows.Forms.DataGridViewColumnHeadersHeightSizeMode.AutoSize;
            this.dgvComprobantes.Location = new System.Drawing.Point(36, 144);
            this.dgvComprobantes.Name = "dgvComprobantes";
            this.dgvComprobantes.ReadOnly = true;
            this.dgvComprobantes.RowHeadersVisible = false;
            this.dgvComprobantes.SelectionMode = System.Windows.Forms.DataGridViewSelectionMode.FullRowSelect;
            this.dgvComprobantes.Size = new System.Drawing.Size(480, 460);
            this.dgvComprobantes.TabIndex = 2;
            this.dgvComprobantes.Columns.Add("IdComprobante", "Id");
            this.dgvComprobantes.Columns["IdComprobante"].Visible = false;
            this.dgvComprobantes.Columns.Add("Numero", "Número");
            this.dgvComprobantes.Columns.Add("Empleado", "Empleado");
            this.dgvComprobantes.Columns.Add("Periodo", "Periodo");
            this.dgvComprobantes.Columns.Add("Neto", "Neto");
            this.dgvComprobantes.SelectionChanged += new System.EventHandler(this.dgvComprobantes_SelectionChanged);
            //
            // panelVistaPrevia
            //
            this.panelVistaPrevia.BackColor = System.Drawing.Color.FromArgb(243, 244, 246);
            this.panelVistaPrevia.BorderStyle = System.Windows.Forms.BorderStyle.FixedSingle;
            this.panelVistaPrevia.Controls.Add(this.lblVistaPrevia);
            this.panelVistaPrevia.Location = new System.Drawing.Point(548, 144);
            this.panelVistaPrevia.Name = "panelVistaPrevia";
            this.panelVistaPrevia.Size = new System.Drawing.Size(436, 460);
            this.panelVistaPrevia.TabIndex = 3;
            //
            // lblVistaPrevia
            //
            this.lblVistaPrevia.Font = new System.Drawing.Font("Consolas", 10F);
            this.lblVistaPrevia.ForeColor = System.Drawing.Color.FromArgb(17, 24, 39);
            this.lblVistaPrevia.Location = new System.Drawing.Point(22, 22);
            this.lblVistaPrevia.Name = "lblVistaPrevia";
            this.lblVistaPrevia.Size = new System.Drawing.Size(390, 410);
            this.lblVistaPrevia.TabIndex = 0;
            this.lblVistaPrevia.Text = "Seleccione un comprobante para ver el detalle.";
            //
            // buttons
            //
            this.btnExportarPdf.BackColor = System.Drawing.Color.Black;
            this.btnExportarPdf.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
            this.btnExportarPdf.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245);
            this.btnExportarPdf.Location = new System.Drawing.Point(568, 86);
            this.btnExportarPdf.Name = "btnExportarPdf";
            this.btnExportarPdf.Size = new System.Drawing.Size(100, 34);
            this.btnExportarPdf.TabIndex = 4;
            this.btnExportarPdf.Text = "PDF";
            this.btnExportarPdf.UseVisualStyleBackColor = false;
            this.btnExportarPdf.Click += new System.EventHandler(this.btnExportarPdf_Click);
            this.btnExportarExcel.BackColor = System.Drawing.Color.Black;
            this.btnExportarExcel.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
            this.btnExportarExcel.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245);
            this.btnExportarExcel.Location = new System.Drawing.Point(678, 86);
            this.btnExportarExcel.Name = "btnExportarExcel";
            this.btnExportarExcel.Size = new System.Drawing.Size(110, 34);
            this.btnExportarExcel.TabIndex = 5;
            this.btnExportarExcel.Text = "Excel";
            this.btnExportarExcel.UseVisualStyleBackColor = false;
            this.btnExportarExcel.Click += new System.EventHandler(this.btnExportarExcel_Click);
            this.btnImprimir.BackColor = System.Drawing.Color.Black;
```

UI/FrmComprobantes.Designer.cs (continuacion)

```
            this.btnImprimir.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
            this.btnImprimir.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245);
            this.btnImprimir.Location = new System.Drawing.Point(798, 86);
            this.btnImprimir.Name = "btnImprimir";
            this.btnImprimir.Size = new System.Drawing.Size(80, 34);
            this.btnImprimir.TabIndex = 6;
            this.btnImprimir.Text = "Imprimir";
            this.btnImprimir.UseVisualStyleBackColor = false;
            this.btnImprimir.Click += new System.EventHandler(this.btnImprimir_Click);
            this.btnEnviarEmail.BackColor = System.Drawing.Color.FromArgb(167, 139, 250);
            this.btnEnviarEmail.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
            this.btnEnviarEmail.Location = new System.Drawing.Point(888, 86);
            this.btnEnviarEmail.Name = "btnEnviarEmail";
            this.btnEnviarEmail.Size = new System.Drawing.Size(96, 34);
            this.btnEnviarEmail.TabIndex = 7;
            this.btnEnviarEmail.Text = "Enviar Email";
            this.btnEnviarEmail.UseVisualStyleBackColor = false;
            this.btnEnviarEmail.Click += new System.EventHandler(this.btnEnviarEmail_Click);
            //
            // FrmComprobantes
            //
            this.AutoScaleDimensions = new System.Drawing.SizeF(7F, 15F);
            this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font;
            this.BackColor = System.Drawing.Color.FromArgb(13, 14, 20);
            this.ClientSize = new System.Drawing.Size(1020, 742);
            this.Controls.Add(this.btnEnviarEmail);
            this.Controls.Add(this.btnImprimir);
            this.Controls.Add(this.btnExportarExcel);
            this.Controls.Add(this.btnExportarPdf);
            this.Controls.Add(this.panelVistaPrevia);
            this.Controls.Add(this.dgvComprobantes);
            this.Controls.Add(this.txtBuscar);
            this.Controls.Add(this.lblTitulo);
            this.Font = new System.Drawing.Font("Segoe UI", 9F);
            this.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245);
            this.Name = "FrmComprobantes";
            this.Text = "Comprobantes";
            this.Load += new System.EventHandler(this.FrmComprobantes_Load);
            ((System.ComponentModel.ISupportInitialize)(this.dgvComprobantes)).EndInit();
            this.panelVistaPrevia.ResumeLayout(false);
            this.ResumeLayout(false);
            this.PerformLayout();
        }
    }
}
```

UI/FrmConfiguracion.cs

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Windows.Forms;
using SistemaGestionNomina.Helpers;
using SistemaGestionNomina.Services;

namespace SistemaGestionNomina.UI
{
    public partial class FrmConfiguracion : Form
    {
        private readonly ConfiguracionService configuracionService = new ConfiguracionService();
        private readonly BackupService backupService = new BackupService();

        public FrmConfiguracion()
        {
            InitializeComponent();
            ControlStyleHelper.ApplyModernForm(this);
        }

        private void FrmConfiguracion_Load(object sender, EventArgs e)
        {
            if (LicenseManager.UsageMode != LicenseUsageMode.Designtime)
            {
                LoadSettings();
            }
        }

        private void LoadSettings()
        {
            nudSeguroSocial.Value = Clamp(configuracionService.GetValue("SeguroSocial", 9.75m), nudSeguroSocial);
            nudISR.Value = Clamp(configuracionService.GetValue("ISR", 10m), nudISR);
            nudSeguroEducativo.Value = Clamp(configuracionService.GetValue("SeguroEducativo", 1.25m), nudSeguroEducativo);
            nudRecargoHoraExtra.Value = Clamp(configuracionService.GetValue("RecargoHoraExtra", 1.25m), nudRecargoHoraExtra);
            nudHorasMensualesBase.Value = Clamp(configuracionService.GetValue("HorasMensualesBase", 160m), nudHorasMensualesBase);
        }

        private void btnGuardar_Click(object sender, EventArgs e)
        {
            Dictionary<string, decimal> values = new Dictionary<string, decimal>();
            values["SeguroSocial"] = nudSeguroSocial.Value;
            values["ISR"] = nudISR.Value;
            values["SeguroEducativo"] = nudSeguroEducativo.Value;
            values["RecargoHoraExtra"] = nudRecargoHoraExtra.Value;
            values["HorasMensualesBase"] = nudHorasMensualesBase.Value;
            configuracionService.SaveDefaults(values);
            MessageBox.Show("Configuración guardada correctamente.", "Configuración", MessageBoxButtons.OK, MessageBoxIcon.Information);
        }

        private void btnDescartar_Click(object sender, EventArgs e)
        {
            LoadSettings();
        }

        private void btnBackup_Click(object sender, EventArgs e)
        {
            using (FolderBrowserDialog dialog = new FolderBrowserDialog())
            {
                dialog.Description = "Seleccione la carpeta donde se guardará el backup.";
                if (dialog.ShowDialog(this) != DialogResult.OK)
```

UI/FrmConfiguracion.cs (continuacion)

```
                {
                    return;
                }

                try
                {
                    string path = backupService.CrearBackup(dialog.SelectedPath);
                    bool verified = backupService.VerificarBackup(path);
                    MessageBox.Show("Backup generado correctamente:\n" + path + "\n\nVerificación SHA-256: " +
                        (verified ? "correcta" : "no disponible"), "Backup", MessageBoxButtons.OK, MessageBoxIcon.Information);
                }
                catch (Exception ex)
                {
                    MessageBox.Show(ex.Message, "Backup", MessageBoxButtons.OK, MessageBoxIcon.Warning);
                }
            }
        }

        private static decimal Clamp(decimal value, NumericUpDown control)
        {
            if (value < control.Minimum) return control.Minimum;
            if (value > control.Maximum) return control.Maximum;
            return value;
        }
    }
}
```

UI/FrmConfiguracion.Designer.cs

```
namespace SistemaGestionNomina.UI
{
    partial class FrmConfiguracion
    {
        private System.ComponentModel.IContainer components = null;
        private System.Windows.Forms.Label lblTitulo;
        private System.Windows.Forms.Panel panelConfiguracion;
        private System.Windows.Forms.NumericUpDown nudSeguroSocial;
        private System.Windows.Forms.NumericUpDown nudISR;
        private System.Windows.Forms.NumericUpDown nudSeguroEducativo;
        private System.Windows.Forms.NumericUpDown nudRecargoHoraExtra;
        private System.Windows.Forms.NumericUpDown nudHorasMensualesBase;
        private System.Windows.Forms.Label lblSeguroSocial;
        private System.Windows.Forms.Label lblISR;
        private System.Windows.Forms.Label lblSeguroEducativo;
        private System.Windows.Forms.Label lblRecargoHoraExtra;
        private System.Windows.Forms.Label lblHorasMensualesBase;
        private System.Windows.Forms.Label lblDescripcion;
        private System.Windows.Forms.Button btnGuardar;
        private System.Windows.Forms.Button btnDescartar;
        private System.Windows.Forms.Button btnBackup;

        protected override void Dispose(bool disposing)
        {
            if (disposing && (components != null)) components.Dispose();
            base.Dispose(disposing);
        }

        private void InitializeComponent()
        {
            this.lblTitulo = new System.Windows.Forms.Label();
            this.panelConfiguracion = new System.Windows.Forms.Panel();
            this.nudSeguroSocial = new System.Windows.Forms.NumericUpDown();
            this.nudISR = new System.Windows.Forms.NumericUpDown();
            this.nudSeguroEducativo = new System.Windows.Forms.NumericUpDown();
            this.nudRecargoHoraExtra = new System.Windows.Forms.NumericUpDown();
            this.nudHorasMensualesBase = new System.Windows.Forms.NumericUpDown();
            this.lblSeguroSocial = new System.Windows.Forms.Label();
            this.lblISR = new System.Windows.Forms.Label();
            this.lblSeguroEducativo = new System.Windows.Forms.Label();
            this.lblRecargoHoraExtra = new System.Windows.Forms.Label();
            this.lblHorasMensualesBase = new System.Windows.Forms.Label();
            this.lblDescripcion = new System.Windows.Forms.Label();
            this.btnGuardar = new System.Windows.Forms.Button();
            this.btnDescartar = new System.Windows.Forms.Button();
            this.btnBackup = new System.Windows.Forms.Button();
            this.panelConfiguracion.SuspendLayout();
            ((System.ComponentModel.ISupportInitialize)(this.nudSeguroSocial)).BeginInit();
            ((System.ComponentModel.ISupportInitialize)(this.nudISR)).BeginInit();
            ((System.ComponentModel.ISupportInitialize)(this.nudSeguroEducativo)).BeginInit();
            ((System.ComponentModel.ISupportInitialize)(this.nudRecargoHoraExtra)).BeginInit();
            ((System.ComponentModel.ISupportInitialize)(this.nudHorasMensualesBase)).BeginInit();
            this.SuspendLayout();
            //
            // lblTitulo
            //
            this.lblTitulo.Font = new System.Drawing.Font("Segoe UI", 22F, System.Drawing.FontStyle.Bold);
            this.lblTitulo.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245);
            this.lblTitulo.Location = new System.Drawing.Point(34, 24);
            this.lblTitulo.Name = "lblTitulo";
```

UI/FrmConfiguracion.Designer.cs (continuacion)

```
            this.lblTitulo.Size = new System.Drawing.Size(520, 42);
            this.lblTitulo.TabIndex = 0;
            this.lblTitulo.Text = "Configuración del Sistema";
            //
            // panelConfiguracion
            //
            this.panelConfiguracion.BackColor = System.Drawing.Color.Black;
            this.panelConfiguracion.BorderStyle = System.Windows.Forms.BorderStyle.FixedSingle;
            this.panelConfiguracion.Controls.Add(this.lblSeguroSocial);
            this.panelConfiguracion.Controls.Add(this.lblISR);
            this.panelConfiguracion.Controls.Add(this.lblSeguroEducativo);
            this.panelConfiguracion.Controls.Add(this.lblRecargoHoraExtra);
            this.panelConfiguracion.Controls.Add(this.lblHorasMensualesBase);
            this.panelConfiguracion.Controls.Add(this.lblDescripcion);
            this.panelConfiguracion.Controls.Add(this.nudSeguroSocial);
            this.panelConfiguracion.Controls.Add(this.nudISR);
            this.panelConfiguracion.Controls.Add(this.nudSeguroEducativo);
            this.panelConfiguracion.Controls.Add(this.nudRecargoHoraExtra);
            this.panelConfiguracion.Controls.Add(this.nudHorasMensualesBase);
            this.panelConfiguracion.Controls.Add(this.btnGuardar);
            this.panelConfiguracion.Controls.Add(this.btnDescartar);
            this.panelConfiguracion.Controls.Add(this.btnBackup);
            this.panelConfiguracion.Location = new System.Drawing.Point(210, 116);
            this.panelConfiguracion.Name = "panelConfiguracion";
            this.panelConfiguracion.Size = new System.Drawing.Size(600, 390);
            this.panelConfiguracion.TabIndex = 1;
            //
            // numeric controls
            //
            this.nudSeguroSocial.DecimalPlaces = 2;
            this.nudSeguroSocial.Location = new System.Drawing.Point(42, 54);
            this.nudSeguroSocial.Maximum = new decimal(new int[] { 100, 0, 0, 0 });
            this.nudSeguroSocial.Name = "nudSeguroSocial";
            this.nudSeguroSocial.Size = new System.Drawing.Size(210, 23);
            this.nudSeguroSocial.TabIndex = 0;
            this.nudISR.DecimalPlaces = 2;
            this.nudISR.Location = new System.Drawing.Point(320, 54);
            this.nudISR.Maximum = new decimal(new int[] { 100, 0, 0, 0 });
            this.nudISR.Name = "nudISR";
            this.nudISR.Size = new System.Drawing.Size(210, 23);
            this.nudISR.TabIndex = 1;
            this.nudSeguroEducativo.DecimalPlaces = 2;
            this.nudSeguroEducativo.Location = new System.Drawing.Point(42, 132);
            this.nudSeguroEducativo.Maximum = new decimal(new int[] { 100, 0, 0, 0 });
            this.nudSeguroEducativo.Name = "nudSeguroEducativo";
            this.nudSeguroEducativo.Size = new System.Drawing.Size(210, 23);
            this.nudSeguroEducativo.TabIndex = 2;
            this.nudRecargoHoraExtra.DecimalPlaces = 2;
            this.nudRecargoHoraExtra.Location = new System.Drawing.Point(320, 132);
            this.nudRecargoHoraExtra.Maximum = new decimal(new int[] { 10, 0, 0, 0 });
            this.nudRecargoHoraExtra.Name = "nudRecargoHoraExtra";
            this.nudRecargoHoraExtra.Size = new System.Drawing.Size(210, 23);
            this.nudRecargoHoraExtra.TabIndex = 3;
            this.nudHorasMensualesBase.Location = new System.Drawing.Point(42, 210);
            this.nudHorasMensualesBase.Maximum = new decimal(new int[] { 400, 0, 0, 0 });
            this.nudHorasMensualesBase.Minimum = new decimal(new int[] { 1, 0, 0, 0 });
            this.nudHorasMensualesBase.Name = "nudHorasMensualesBase";
            this.nudHorasMensualesBase.Size = new System.Drawing.Size(210, 23);
            this.nudHorasMensualesBase.TabIndex = 4;
            this.nudHorasMensualesBase.Value = new decimal(new int[] { 160, 0, 0, 0 });
```

UI/FrmConfiguracion.Designer.cs (continuacion)

```
            //
            // labels
            //
            this.lblDescripcion.ForeColor = System.Drawing.Color.FromArgb(170, 170, 185);
            this.lblDescripcion.Location = new System.Drawing.Point(42, 18);
            this.lblDescripcion.Name = "lblDescripcion";
            this.lblDescripcion.Size = new System.Drawing.Size(500, 24);
            this.lblDescripcion.TabIndex = 7;
            this.lblDescripcion.Text = "Valores académicos configurables para el cálculo de nómina.";
            this.lblSeguroSocial.Font = new System.Drawing.Font("Segoe UI", 9F, System.Drawing.FontStyle.Bold);
            this.lblSeguroSocial.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245);
            this.lblSeguroSocial.Location = new System.Drawing.Point(42, 34);
            this.lblSeguroSocial.Name = "lblSeguroSocial";
            this.lblSeguroSocial.Size = new System.Drawing.Size(210, 18);
            this.lblSeguroSocial.TabIndex = 8;
            this.lblSeguroSocial.Text = "Seguro Social (%)";
            this.lblISR.Font = new System.Drawing.Font("Segoe UI", 9F, System.Drawing.FontStyle.Bold);
            this.lblISR.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245);
            this.lblISR.Location = new System.Drawing.Point(320, 34);
            this.lblISR.Name = "lblISR";
            this.lblISR.Size = new System.Drawing.Size(210, 18);
            this.lblISR.TabIndex = 9;
            this.lblISR.Text = "ISR (%)";
            this.lblSeguroEducativo.Font = new System.Drawing.Font("Segoe UI", 9F, System.Drawing.FontStyle.Bold);
            this.lblSeguroEducativo.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245);
            this.lblSeguroEducativo.Location = new System.Drawing.Point(42, 112);
            this.lblSeguroEducativo.Name = "lblSeguroEducativo";
            this.lblSeguroEducativo.Size = new System.Drawing.Size(210, 18);
            this.lblSeguroEducativo.TabIndex = 10;
            this.lblSeguroEducativo.Text = "Seguro Educativo (%)";
            this.lblRecargoHoraExtra.Font = new System.Drawing.Font("Segoe UI", 9F, System.Drawing.FontStyle.Bold);
            this.lblRecargoHoraExtra.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245);
            this.lblRecargoHoraExtra.Location = new System.Drawing.Point(320, 112);
            this.lblRecargoHoraExtra.Name = "lblRecargoHoraExtra";
            this.lblRecargoHoraExtra.Size = new System.Drawing.Size(210, 18);
            this.lblRecargoHoraExtra.TabIndex = 11;
            this.lblRecargoHoraExtra.Text = "Recargo de horas extra";
            this.lblHorasMensualesBase.Font = new System.Drawing.Font("Segoe UI", 9F, System.Drawing.FontStyle.Bold);
            this.lblHorasMensualesBase.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245);
            this.lblHorasMensualesBase.Location = new System.Drawing.Point(42, 190);
            this.lblHorasMensualesBase.Name = "lblHorasMensualesBase";
            this.lblHorasMensualesBase.Size = new System.Drawing.Size(210, 18);
            this.lblHorasMensualesBase.TabIndex = 12;
            this.lblHorasMensualesBase.Text = "Horas mensuales base";
            //
            // buttons
            //
            this.btnGuardar.BackColor = System.Drawing.Color.FromArgb(167, 139, 250);
            this.btnGuardar.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
            this.btnGuardar.Location = new System.Drawing.Point(320, 304);
            this.btnGuardar.Name = "btnGuardar";
            this.btnGuardar.Size = new System.Drawing.Size(130, 34);
            this.btnGuardar.TabIndex = 5;
            this.btnGuardar.Text = "Guardar cambios";
            this.btnGuardar.UseVisualStyleBackColor = false;
            this.btnGuardar.Click += new System.EventHandler(this.btnGuardar_Click);
            this.btnDescartar.BackColor = System.Drawing.Color.Black;
            this.btnDescartar.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
            this.btnDescartar.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245);
            this.btnDescartar.Location = new System.Drawing.Point(462, 304);
```

UI/FrmConfiguracion.Designer.cs (continuacion)

```
            this.btnDescartar.Name = "btnDescartar";
            this.btnDescartar.Size = new System.Drawing.Size(92, 34);
            this.btnDescartar.TabIndex = 6;
            this.btnDescartar.Text = "Descartar";
            this.btnDescartar.UseVisualStyleBackColor = false;
            this.btnDescartar.Click += new System.EventHandler(this.btnDescartar_Click);
            this.btnBackup.BackColor = System.Drawing.Color.Black;
            this.btnBackup.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
            this.btnBackup.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245);
            this.btnBackup.Location = new System.Drawing.Point(42, 304);
            this.btnBackup.Name = "btnBackup";
            this.btnBackup.Size = new System.Drawing.Size(150, 34);
            this.btnBackup.TabIndex = 7;
            this.btnBackup.Text = "Crear backup";
            this.btnBackup.UseVisualStyleBackColor = false;
            this.btnBackup.Click += new System.EventHandler(this.btnBackup_Click);
            //
            // FrmConfiguracion
            //
            this.AutoScaleDimensions = new System.Drawing.SizeF(7F, 15F);
            this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font;
            this.BackColor = System.Drawing.Color.FromArgb(13, 14, 20);
            this.ClientSize = new System.Drawing.Size(1020, 742);
            this.Controls.Add(this.panelConfiguracion);
            this.Controls.Add(this.lblTitulo);
            this.Font = new System.Drawing.Font("Segoe UI", 9F);
            this.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245);
            this.Name = "FrmConfiguracion";
            this.Text = "Configuración";
            this.Load += new System.EventHandler(this.FrmConfiguracion_Load);
            this.panelConfiguracion.ResumeLayout(false);
            ((System.ComponentModel.ISupportInitialize)(this.nudSeguroSocial)).EndInit();
            ((System.ComponentModel.ISupportInitialize)(this.nudISR)).EndInit();
            ((System.ComponentModel.ISupportInitialize)(this.nudSeguroEducativo)).EndInit();
            ((System.ComponentModel.ISupportInitialize)(this.nudRecargoHoraExtra)).EndInit();
            ((System.ComponentModel.ISupportInitialize)(this.nudHorasMensualesBase)).EndInit();
            this.ResumeLayout(false);
        }
    }
}
```

UI/FrmDashboard.cs

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Drawing;
using System.Windows.Forms;
using SistemaGestionNomina.Helpers;
using SistemaGestionNomina.Models;
using SistemaGestionNomina.Services;

namespace SistemaGestionNomina.UI
{
    public partial class FrmDashboard : Form
    {
        private readonly Action openEmployees;
        private readonly Action openPayroll;
        private readonly EmpleadoService empleadoService = new EmpleadoService();
        private readonly NominaService nominaService = new NominaService();
        private readonly ComprobanteService comprobanteService = new ComprobanteService();

        public FrmDashboard()
            : this(null, null)
        {
        }

        public FrmDashboard(Action openEmployeesAction, Action openPayrollAction)
        {
            openEmployees = openEmployeesAction;
            openPayroll = openPayrollAction;
            InitializeComponent();
            ControlStyleHelper.ApplyModernForm(this);
        }

        private void FrmDashboard_Load(object sender, EventArgs e)
        {
            if (LicenseManager.UsageMode != LicenseUsageMode.Designtime)
            {
                RefreshDashboard();
            }
        }

        private void btnRegistrarEmpleado_Click(object sender, EventArgs e)
        {
            if (openEmployees != null)
            {
                openEmployees();
            }
        }

        private void btnProcesarNomina_Click(object sender, EventArgs e)
        {
            if (openPayroll != null)
            {
                openPayroll();
            }
        }

        private void RefreshDashboard()
        {
            List<Empleado> activos = empleadoService.GetActive(null);
            List<Nomina> nominas = nominaService.GetNominas();
```

UI/FrmDashboard.cs (continuacion)

```
            List<Comprobante> comprobantes = comprobanteService.GetAll(string.Empty);
            decimal totalNomina = nominas.Count > 0 ? nominas[0].TotalNeto : 0;
            decimal totalDeducciones = nominas.Count > 0 ? nominas[0].TotalDeducciones : 0;

            lblEmpleadosActivosValor.Text = activos.Count.ToString();
            SetMoneyLabel(lblNominaActualValor, totalNomina);
            lblComprobantesValor.Text = comprobantes.Count.ToString();
            SetMoneyLabel(lblAlertasValor, totalDeducciones);

            chartPagos.Series["Neto"].Points.Clear();
            string[] months = { "Ene", "Feb", "Mar", "Abr", "May", "Jun" };
            decimal baseValue = totalNomina > 0 ? totalNomina : 8500;
            for (int i = 0; i < months.Length; i++)
            {
                chartPagos.Series["Neto"].Points.AddXY(months[i], Convert.ToDouble(baseValue + (i * 420)));
            }
        }

        private static void SetMoneyLabel(Label label, decimal amount)
        {
            string value = "B/. " + amount.ToString("#,##0.00");
            float size = value.Length > 13 ? 15F : value.Length > 11 ? 17F : 20F;

            label.Font = new Font("Segoe UI", size, FontStyle.Bold);
            label.Text = value;
        }
    }
}
```

UI/FrmDashboard.Designer.cs

```
namespace SistemaGestionNomina.UI
{
    partial class FrmDashboard
    {
        private System.ComponentModel.IContainer components = null;
        private System.Windows.Forms.Label lblTitulo;
        private System.Windows.Forms.Label lblSubtitulo;
        private System.Windows.Forms.Button btnRegistrarEmpleado;
        private System.Windows.Forms.Button btnProcesarNomina;
        private System.Windows.Forms.Panel cardEmpleados;
        private System.Windows.Forms.Panel cardNomina;
        private System.Windows.Forms.Panel cardComprobantes;
        private System.Windows.Forms.Panel cardAlertas;
        private System.Windows.Forms.Label lblEmpleadosActivosValor;
        private System.Windows.Forms.Label lblNominaActualValor;
        private System.Windows.Forms.Label lblComprobantesValor;
        private System.Windows.Forms.Label lblAlertasValor;
        private System.Windows.Forms.Label lblCardEmpleados;
        private System.Windows.Forms.Label lblCardNomina;
        private System.Windows.Forms.Label lblCardComprobantes;
        private System.Windows.Forms.Label lblCardAlertas;
        private System.Windows.Forms.Label lblActividadTitulo;
        private System.Windows.Forms.Label lblActividadUno;
        private System.Windows.Forms.Label lblActividadDos;
        private System.Windows.Forms.Label lblActividadTres;
        private System.Windows.Forms.Panel panelGrafico;
        private System.Windows.Forms.Panel panelActividad;
        private System.Windows.Forms.DataVisualization.Charting.Chart chartPagos;

        protected override void Dispose(bool disposing)
        {
            if (disposing && (components != null))
            {
                components.Dispose();
            }
            base.Dispose(disposing);
        }

        private void InitializeComponent()
        {
            System.Windows.Forms.DataVisualization.Charting.ChartArea chartArea1 = new System.Windows.Forms.DataVisualization.Charting.ChartArea();
            System.Windows.Forms.DataVisualization.Charting.Series series1 = new System.Windows.Forms.DataVisualization.Charting.Series();
            this.lblTitulo = new System.Windows.Forms.Label();
            this.lblSubtitulo = new System.Windows.Forms.Label();
            this.btnRegistrarEmpleado = new System.Windows.Forms.Button();
            this.btnProcesarNomina = new System.Windows.Forms.Button();
            this.cardEmpleados = new System.Windows.Forms.Panel();
            this.lblCardEmpleados = new System.Windows.Forms.Label();
            this.lblEmpleadosActivosValor = new System.Windows.Forms.Label();
            this.cardNomina = new System.Windows.Forms.Panel();
            this.lblCardNomina = new System.Windows.Forms.Label();
            this.lblNominaActualValor = new System.Windows.Forms.Label();
            this.cardComprobantes = new System.Windows.Forms.Panel();
            this.lblCardComprobantes = new System.Windows.Forms.Label();
            this.lblComprobantesValor = new System.Windows.Forms.Label();
            this.cardAlertas = new System.Windows.Forms.Panel();
            this.lblCardAlertas = new System.Windows.Forms.Label();
            this.lblAlertasValor = new System.Windows.Forms.Label();
            this.panelGrafico = new System.Windows.Forms.Panel();
            this.chartPagos = new System.Windows.Forms.DataVisualization.Charting.Chart();
```

UI/FrmDashboard.Designer.cs (continuacion)

```
            this.panelActividad = new System.Windows.Forms.Panel();
            this.lblActividadTitulo = new System.Windows.Forms.Label();
            this.lblActividadUno = new System.Windows.Forms.Label();
            this.lblActividadDos = new System.Windows.Forms.Label();
            this.lblActividadTres = new System.Windows.Forms.Label();
            this.cardEmpleados.SuspendLayout();
            this.cardNomina.SuspendLayout();
            this.cardComprobantes.SuspendLayout();
            this.cardAlertas.SuspendLayout();
            this.panelGrafico.SuspendLayout();
            ((System.ComponentModel.ISupportInitialize)(this.chartPagos)).BeginInit();
            this.SuspendLayout();
            //
            // lblTitulo
            //
            this.lblTitulo.Font = new System.Drawing.Font("Segoe UI", 22F, System.Drawing.FontStyle.Bold);
            this.lblTitulo.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(229)))), ((int)(((byte)(228)))), ((int)(((byte)(245)))));
            this.lblTitulo.Location = new System.Drawing.Point(34, 28);
            this.lblTitulo.Name = "lblTitulo";
            this.lblTitulo.Size = new System.Drawing.Size(420, 42);
            this.lblTitulo.TabIndex = 0;
            this.lblTitulo.Text = "Panel Ejecutivo";
            //
            // lblSubtitulo
            //
            this.lblSubtitulo.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(170)))), ((int)(((byte)(170)))), ((int)(((byte)(185)))));
            this.lblSubtitulo.Location = new System.Drawing.Point(38, 70);
            this.lblSubtitulo.Name = "lblSubtitulo";
            this.lblSubtitulo.Size = new System.Drawing.Size(520, 25);
            this.lblSubtitulo.TabIndex = 1;
            this.lblSubtitulo.Text = "Resumen ejecutivo del periodo actual de nómina.";
            //
            // btnRegistrarEmpleado
            //
            this.btnRegistrarEmpleado.Anchor = ((System.Windows.Forms.AnchorStyles)((System.Windows.Forms.AnchorStyles.Top |  \
    System.Windows.Forms.AnchorStyles.Right)));
            this.btnRegistrarEmpleado.BackColor = System.Drawing.Color.Black;
            this.btnRegistrarEmpleado.FlatAppearance.BorderColor = System.Drawing.Color.FromArgb(((int)(((byte)(42)))), ((int)(((byte)(43)))),  \
    ((int)(((byte)(56)))));
            this.btnRegistrarEmpleado.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
            this.btnRegistrarEmpleado.Font = new System.Drawing.Font("Segoe UI", 9F, System.Drawing.FontStyle.Bold);
            this.btnRegistrarEmpleado.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(229)))), ((int)(((byte)(228)))), ((int)(((byte)(245)))));
            this.btnRegistrarEmpleado.Location = new System.Drawing.Point(650, 42);
            this.btnRegistrarEmpleado.Name = "btnRegistrarEmpleado";
            this.btnRegistrarEmpleado.Size = new System.Drawing.Size(170, 38);
            this.btnRegistrarEmpleado.TabIndex = 2;
            this.btnRegistrarEmpleado.Text = "+ Nuevo empleado";
            this.btnRegistrarEmpleado.UseVisualStyleBackColor = false;
            this.btnRegistrarEmpleado.Click += new System.EventHandler(this.btnRegistrarEmpleado_Click);
            //
            // btnProcesarNomina
            //
            this.btnProcesarNomina.Anchor = ((System.Windows.Forms.AnchorStyles)((System.Windows.Forms.AnchorStyles.Top |  \
    System.Windows.Forms.AnchorStyles.Right)));
            this.btnProcesarNomina.BackColor = System.Drawing.Color.FromArgb(((int)(((byte)(167)))), ((int)(((byte)(139)))), ((int)(((byte)(250)))));
            this.btnProcesarNomina.FlatAppearance.BorderSize = 0;
            this.btnProcesarNomina.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
            this.btnProcesarNomina.Font = new System.Drawing.Font("Segoe UI", 9F, System.Drawing.FontStyle.Bold);
            this.btnProcesarNomina.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(48)))), ((int)(((byte)(25)))), ((int)(((byte)(92)))));
            this.btnProcesarNomina.Location = new System.Drawing.Point(836, 42);
```

UI/FrmDashboard.Designer.cs (continuacion)

```
            this.btnProcesarNomina.Name = "btnProcesarNomina";
            this.btnProcesarNomina.Size = new System.Drawing.Size(170, 38);
            this.btnProcesarNomina.TabIndex = 3;
            this.btnProcesarNomina.Text = "Procesar nómina";
            this.btnProcesarNomina.UseVisualStyleBackColor = false;
            this.btnProcesarNomina.Click += new System.EventHandler(this.btnProcesarNomina_Click);
            //
            // cardEmpleados
            //
            this.cardEmpleados.BackColor = System.Drawing.Color.Black;
            this.cardEmpleados.BorderStyle = System.Windows.Forms.BorderStyle.FixedSingle;
            this.cardEmpleados.Controls.Add(this.lblCardEmpleados);
            this.cardEmpleados.Controls.Add(this.lblEmpleadosActivosValor);
            this.cardEmpleados.Location = new System.Drawing.Point(36, 120);
            this.cardEmpleados.Name = "cardEmpleados";
            this.cardEmpleados.Size = new System.Drawing.Size(230, 130);
            this.cardEmpleados.TabIndex = 4;
            //
            // lblCardEmpleados
            //
            this.lblCardEmpleados.Font = new System.Drawing.Font("Segoe UI", 10F, System.Drawing.FontStyle.Bold);
            this.lblCardEmpleados.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(170)))), ((int)(((byte)(170)))), ((int)(((byte)(185)))));
            this.lblCardEmpleados.Location = new System.Drawing.Point(18, 18);
            this.lblCardEmpleados.Name = "lblCardEmpleados";
            this.lblCardEmpleados.Size = new System.Drawing.Size(180, 24);
            this.lblCardEmpleados.TabIndex = 0;
            this.lblCardEmpleados.Text = "Empleados activos";
            //
            // lblEmpleadosActivosValor
            //
            this.lblEmpleadosActivosValor.Font = new System.Drawing.Font("Segoe UI", 22F, System.Drawing.FontStyle.Bold);
            this.lblEmpleadosActivosValor.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(59)))), ((int)(((byte)(130)))), ((int)(((byte)(246)))));
            this.lblEmpleadosActivosValor.Location = new System.Drawing.Point(18, 56);
            this.lblEmpleadosActivosValor.Name = "lblEmpleadosActivosValor";
            this.lblEmpleadosActivosValor.Size = new System.Drawing.Size(170, 44);
            this.lblEmpleadosActivosValor.TabIndex = 1;
            this.lblEmpleadosActivosValor.Text = "0";
            //
            // cardNomina
            //
            this.cardNomina.BackColor = System.Drawing.Color.Black;
            this.cardNomina.BorderStyle = System.Windows.Forms.BorderStyle.FixedSingle;
            this.cardNomina.Controls.Add(this.lblCardNomina);
            this.cardNomina.Controls.Add(this.lblNominaActualValor);
            this.cardNomina.Location = new System.Drawing.Point(282, 120);
            this.cardNomina.Name = "cardNomina";
            this.cardNomina.Size = new System.Drawing.Size(230, 130);
            this.cardNomina.TabIndex = 5;
            //
            // lblCardNomina
            //
            this.lblCardNomina.Font = new System.Drawing.Font("Segoe UI", 10F, System.Drawing.FontStyle.Bold);
            this.lblCardNomina.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(170)))), ((int)(((byte)(170)))), ((int)(((byte)(185)))));
            this.lblCardNomina.Location = new System.Drawing.Point(18, 18);
            this.lblCardNomina.Name = "lblCardNomina";
            this.lblCardNomina.Size = new System.Drawing.Size(180, 24);
            this.lblCardNomina.TabIndex = 0;
            this.lblCardNomina.Text = "Nómina actual";
            //
            // lblNominaActualValor
```

UI/FrmDashboard.Designer.cs (continuacion)

```
            //
            this.lblNominaActualValor.Font = new System.Drawing.Font("Segoe UI", 17F, System.Drawing.FontStyle.Bold);
            this.lblNominaActualValor.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(34)))), ((int)(((byte)(197)))), ((int)(((byte)(94)))));
            this.lblNominaActualValor.Location = new System.Drawing.Point(18, 56);
            this.lblNominaActualValor.Name = "lblNominaActualValor";
            this.lblNominaActualValor.Size = new System.Drawing.Size(206, 44);
            this.lblNominaActualValor.TabIndex = 1;
            this.lblNominaActualValor.Text = "B/. 0.00";
            //
            // cardComprobantes
            //
            this.cardComprobantes.BackColor = System.Drawing.Color.Black;
            this.cardComprobantes.BorderStyle = System.Windows.Forms.BorderStyle.FixedSingle;
            this.cardComprobantes.Controls.Add(this.lblCardComprobantes);
            this.cardComprobantes.Controls.Add(this.lblComprobantesValor);
            this.cardComprobantes.Location = new System.Drawing.Point(528, 120);
            this.cardComprobantes.Name = "cardComprobantes";
            this.cardComprobantes.Size = new System.Drawing.Size(230, 130);
            this.cardComprobantes.TabIndex = 6;
            //
            // lblCardComprobantes
            //
            this.lblCardComprobantes.Font = new System.Drawing.Font("Segoe UI", 10F, System.Drawing.FontStyle.Bold);
            this.lblCardComprobantes.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(170)))), ((int)(((byte)(170)))), ((int)(((byte)(185)))));
            this.lblCardComprobantes.Location = new System.Drawing.Point(18, 18);
            this.lblCardComprobantes.Name = "lblCardComprobantes";
            this.lblCardComprobantes.Size = new System.Drawing.Size(190, 24);
            this.lblCardComprobantes.TabIndex = 0;
            this.lblCardComprobantes.Text = "Comprobantes generados";
            //
            // lblComprobantesValor
            //
            this.lblComprobantesValor.Font = new System.Drawing.Font("Segoe UI", 22F, System.Drawing.FontStyle.Bold);
            this.lblComprobantesValor.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(139)))), ((int)(((byte)(92)))), ((int)(((byte)(246)))));
            this.lblComprobantesValor.Location = new System.Drawing.Point(18, 56);
            this.lblComprobantesValor.Name = "lblComprobantesValor";
            this.lblComprobantesValor.Size = new System.Drawing.Size(170, 44);
            this.lblComprobantesValor.TabIndex = 1;
            this.lblComprobantesValor.Text = "0";
            //
            // cardAlertas
            //
            this.cardAlertas.BackColor = System.Drawing.Color.Black;
            this.cardAlertas.BorderStyle = System.Windows.Forms.BorderStyle.FixedSingle;
            this.cardAlertas.Controls.Add(this.lblCardAlertas);
            this.cardAlertas.Controls.Add(this.lblAlertasValor);
            this.cardAlertas.Location = new System.Drawing.Point(774, 120);
            this.cardAlertas.Name = "cardAlertas";
            this.cardAlertas.Size = new System.Drawing.Size(230, 130);
            this.cardAlertas.TabIndex = 7;
            //
            // lblCardAlertas
            //
            this.lblCardAlertas.Font = new System.Drawing.Font("Segoe UI", 10F, System.Drawing.FontStyle.Bold);
            this.lblCardAlertas.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(170)))), ((int)(((byte)(170)))), ((int)(((byte)(185)))));
            this.lblCardAlertas.Location = new System.Drawing.Point(18, 18);
            this.lblCardAlertas.Name = "lblCardAlertas";
            this.lblCardAlertas.Size = new System.Drawing.Size(180, 24);
            this.lblCardAlertas.TabIndex = 0;
            this.lblCardAlertas.Text = "Deducciones del mes";
```

UI/FrmDashboard.Designer.cs (continuacion)

```
            //
            // lblAlertasValor
            //
            this.lblAlertasValor.Font = new System.Drawing.Font("Segoe UI", 17F, System.Drawing.FontStyle.Bold);
            this.lblAlertasValor.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(249)))), ((int)(((byte)(115)))), ((int)(((byte)(134)))));
            this.lblAlertasValor.Location = new System.Drawing.Point(18, 56);
            this.lblAlertasValor.Name = "lblAlertasValor";
            this.lblAlertasValor.Size = new System.Drawing.Size(206, 44);
            this.lblAlertasValor.TabIndex = 1;
            this.lblAlertasValor.Text = "B/. 0.00";
            //
            // panelGrafico
            //
            this.panelGrafico.BackColor = System.Drawing.Color.FromArgb(((int)(((byte)(24)))), ((int)(((byte)(25)))), ((int)(((byte)(34)))));
            this.panelGrafico.BorderStyle = System.Windows.Forms.BorderStyle.FixedSingle;
            this.panelGrafico.Controls.Add(this.chartPagos);
            this.panelGrafico.Location = new System.Drawing.Point(36, 290);
            this.panelGrafico.Name = "panelGrafico";
            this.panelGrafico.Size = new System.Drawing.Size(650, 330);
            this.panelGrafico.TabIndex = 8;
            //
            // chartPagos
            //
            this.chartPagos.BackColor = System.Drawing.Color.FromArgb(((int)(((byte)(24)))), ((int)(((byte)(25)))), ((int)(((byte)(34)))));
            chartArea1.AxisX.LabelStyle.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(170)))), ((int)(((byte)(170)))), ((int)(((byte)(185)))));
            chartArea1.AxisX.MajorGrid.LineColor = System.Drawing.Color.FromArgb(((int)(((byte)(42)))), ((int)(((byte)(43)))), ((int)(((byte)(56)))));
            chartArea1.AxisY.LabelStyle.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(170)))), ((int)(((byte)(170)))), ((int)(((byte)(185)))));
            chartArea1.AxisY.MajorGrid.LineColor = System.Drawing.Color.FromArgb(((int)(((byte)(42)))), ((int)(((byte)(43)))), ((int)(((byte)(56)))));
            chartArea1.BackColor = System.Drawing.Color.FromArgb(((int)(((byte)(24)))), ((int)(((byte)(25)))), ((int)(((byte)(34)))));
            chartArea1.Name = "Pagos";
            this.chartPagos.ChartAreas.Add(chartArea1);
            this.chartPagos.Location = new System.Drawing.Point(20, 42);
            this.chartPagos.Name = "chartPagos";
            series1.BorderColor = System.Drawing.Color.FromArgb(((int)(((byte)(167)))), ((int)(((byte)(139)))), ((int)(((byte)(250)))));
            series1.BorderWidth = 3;
            series1.ChartArea = "Pagos";
            series1.ChartType = System.Windows.Forms.DataVisualization.Charting.SeriesChartType.SplineArea;
            series1.Color = System.Drawing.Color.FromArgb(((int)(((byte)(120)))), ((int)(((byte)(167)))), ((int)(((byte)(139)))), ((int)(((byte)(250)))));
            series1.Name = "Neto";
            this.chartPagos.Series.Add(series1);
            this.chartPagos.Size = new System.Drawing.Size(600, 260);
            this.chartPagos.TabIndex = 0;
            this.chartPagos.Text = "Tendencia de pagos";
            //
            // panelActividad
            //
            this.panelActividad.BackColor = System.Drawing.Color.FromArgb(((int)(((byte)(24)))), ((int)(((byte)(25)))), ((int)(((byte)(34)))));
            this.panelActividad.BorderStyle = System.Windows.Forms.BorderStyle.FixedSingle;
            this.panelActividad.Controls.Add(this.lblActividadTitulo);
            this.panelActividad.Controls.Add(this.lblActividadUno);
            this.panelActividad.Controls.Add(this.lblActividadDos);
            this.panelActividad.Controls.Add(this.lblActividadTres);
            this.panelActividad.Location = new System.Drawing.Point(710, 290);
            this.panelActividad.Name = "panelActividad";
            this.panelActividad.Size = new System.Drawing.Size(294, 330);
            this.panelActividad.TabIndex = 9;
            //
            // lblActividadTitulo
            //
            this.lblActividadTitulo.Font = new System.Drawing.Font("Segoe UI", 15F, System.Drawing.FontStyle.Bold);
```

UI/FrmDashboard.Designer.cs (continuacion)

```
            this.lblActividadTitulo.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(229)))), ((int)(((byte)(228)))), ((int)(((byte)(245)))));
            this.lblActividadTitulo.Location = new System.Drawing.Point(20, 22);
            this.lblActividadTitulo.Name = "lblActividadTitulo";
            this.lblActividadTitulo.Size = new System.Drawing.Size(240, 30);
            this.lblActividadTitulo.TabIndex = 0;
            this.lblActividadTitulo.Text = "Actividad reciente";
            //
            // lblActividadUno
            //
            this.lblActividadUno.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(229)))), ((int)(((byte)(228)))), ((int)(((byte)(245)))));
            this.lblActividadUno.Location = new System.Drawing.Point(24, 82);
            this.lblActividadUno.Name = "lblActividadUno";
            this.lblActividadUno.Size = new System.Drawing.Size(230, 42);
            this.lblActividadUno.TabIndex = 1;
            this.lblActividadUno.Text = "Nómina lista para revisión\r\nHace 2 horas";
            //
            // lblActividadDos
            //
            this.lblActividadDos.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(229)))), ((int)(((byte)(228)))), ((int)(((byte)(245)))));
            this.lblActividadDos.Location = new System.Drawing.Point(24, 146);
            this.lblActividadDos.Name = "lblActividadDos";
            this.lblActividadDos.Size = new System.Drawing.Size(230, 42);
            this.lblActividadDos.TabIndex = 2;
            this.lblActividadDos.Text = "Nuevo empleado registrado\r\nDatos iniciales cargados";
            //
            // lblActividadTres
            //
            this.lblActividadTres.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(249)))), ((int)(((byte)(115)))), ((int)(((byte)(134)))));
            this.lblActividadTres.Location = new System.Drawing.Point(24, 210);
            this.lblActividadTres.Name = "lblActividadTres";
            this.lblActividadTres.Size = new System.Drawing.Size(230, 42);
            this.lblActividadTres.TabIndex = 3;
            this.lblActividadTres.Text = "Validar asistencia\r\nRevisar tardanzas";
            //
            // FrmDashboard
            //
            this.AutoScaleDimensions = new System.Drawing.SizeF(7F, 15F);
            this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font;
            this.AutoScroll = true;
            this.BackColor = System.Drawing.Color.FromArgb(((int)(((byte)(13)))), ((int)(((byte)(14)))), ((int)(((byte)(20)))));
            this.ClientSize = new System.Drawing.Size(1020, 742);
            this.Controls.Add(this.panelActividad);
            this.Controls.Add(this.panelGrafico);
            this.Controls.Add(this.cardAlertas);
            this.Controls.Add(this.cardComprobantes);
            this.Controls.Add(this.cardNomina);
            this.Controls.Add(this.cardEmpleados);
            this.Controls.Add(this.btnProcesarNomina);
            this.Controls.Add(this.btnRegistrarEmpleado);
            this.Controls.Add(this.lblSubtitulo);
            this.Controls.Add(this.lblTitulo);
            this.Font = new System.Drawing.Font("Segoe UI", 9F);
            this.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(229)))), ((int)(((byte)(228)))), ((int)(((byte)(245)))));
            this.Name = "FrmDashboard";
            this.Text = "Dashboard";
            this.Load += new System.EventHandler(this.FrmDashboard_Load);
            this.cardEmpleados.ResumeLayout(false);
            this.cardNomina.ResumeLayout(false);
            this.cardComprobantes.ResumeLayout(false);
            this.cardAlertas.ResumeLayout(false);
```

UI/FrmDashboard.Designer.cs (continuacion)

```
            this.panelGrafico.ResumeLayout(false);
            ((System.ComponentModel.ISupportInitialize)(this.chartPagos)).EndInit();
            this.ResumeLayout(false);
        }
    }
}
```

UI/FrmEmpleados.cs

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Windows.Forms;
using SistemaGestionNomina.Helpers;
using SistemaGestionNomina.Models;
using SistemaGestionNomina.Services;
using SistemaGestionNomina.Security;

namespace SistemaGestionNomina.UI
{
    public partial class FrmEmpleados : Form
    {
        private readonly EmpleadoService empleadoService = new EmpleadoService();
        private readonly ExcelExportService excelExportService = new ExcelExportService();
        private readonly PdfExportService pdfExportService = new PdfExportService();
        private readonly AuthorizationService authorizationService = new AuthorizationService();
        private int selectedId;
        private List<Empleado> currentEmployees = new List<Empleado>();

        public FrmEmpleados()
        {
            InitializeComponent();
            ControlStyleHelper.ApplyModernForm(this);
        }

        private void FrmEmpleados_Load(object sender, EventArgs e)
        {
            if (LicenseManager.UsageMode == LicenseUsageMode.Designtime)
            {
                return;
            }

            btnNuevo.Visible = authorizationService.HasPermission(Permissions.EmployeesCreate);
            btnEditar.Visible = authorizationService.HasPermission(Permissions.EmployeesEdit);
            btnDesactivar.Visible = authorizationService.HasPermission(Permissions.EmployeesDeactivate);
            btnExportarExcel.Visible = authorizationService.HasPermission(Permissions.EmployeesExport);
            btnExportarPdf.Visible = authorizationService.HasPermission(Permissions.EmployeesExport);
            btnHistorialLaboral.Visible = authorizationService.HasPermission(Permissions.EmployeeHistoryView);
            LoadDepartments();
            LoadEmployees();
        }

        private void LoadDepartments()
        {
            List<Departamento> departments = empleadoService.GetDepartments();
            cmbDepartamento.DataSource = new List<Departamento>(departments);
            cmbDepartamento.DisplayMember = "Nombre";
            cmbDepartamento.ValueMember = "IdDepartamento";

            List<Departamento> filters = new List<Departamento>();
            filters.Add(new Departamento { IdDepartamento = 0, Nombre = "Todos los departamentos" });
            filters.AddRange(departments);
            cmbFiltroDepartamento.DataSource = filters;
            cmbFiltroDepartamento.DisplayMember = "Nombre";
            cmbFiltroDepartamento.ValueMember = "IdDepartamento";
        }

        private void LoadEmployees()
        {
```

UI/FrmEmpleados.cs (continuacion)

```
            int? department = null;
            if (cmbFiltroDepartamento.SelectedValue is int && (int)cmbFiltroDepartamento.SelectedValue > 0)
            {
                department = (int)cmbFiltroDepartamento.SelectedValue;
            }

            string status = cmbFiltroEstado.SelectedItem == null || cmbFiltroEstado.SelectedItem.ToString() == "Todos"
                ? string.Empty
                : cmbFiltroEstado.SelectedItem.ToString();

            currentEmployees = empleadoService.GetAll(txtBuscar.Text, department, status);
            dgvEmpleados.Rows.Clear();
            foreach (Empleado e in currentEmployees)
            {
                dgvEmpleados.Rows.Add(e.IdEmpleado, e.Codigo, e.NombreCompleto, e.Cedula, e.Cargo,
                    e.DepartamentoNombre, "B/. " + e.SalarioBase.ToString("0.00"), e.Estado, "Editar");
            }
        }

        private void dgvEmpleados_SelectionChanged(object sender, EventArgs e)
        {
            if (dgvEmpleados.CurrentRow == null || dgvEmpleados.CurrentRow.Cells["IdEmpleado"].Value == null)
            {
                return;
            }

            selectedId = Convert.ToInt32(dgvEmpleados.CurrentRow.Cells["IdEmpleado"].Value);
            Empleado empleado = empleadoService.GetById(selectedId);
            if (empleado == null)
            {
                return;
            }

            txtCodigo.Text = empleado.Codigo;
            txtNombre.Text = empleado.Nombre;
            txtApellido.Text = empleado.Apellido;
            txtCedula.Text = empleado.Cedula;
            txtCargo.Text = empleado.Cargo;
            cmbDepartamento.SelectedValue = empleado.IdDepartamento;
            txtSalario.Text = empleado.SalarioBase.ToString("0.00");
            cmbEstado.SelectedItem = empleado.Estado;
            dtpIngreso.Value = empleado.FechaIngreso;
        }

        private void btnNuevo_Click(object sender, EventArgs e)
        {
            ClearForm();
        }

        private void btnEditar_Click(object sender, EventArgs e)
        {
            decimal salary;
            if (!ValidationHelper.RequireText(txtCodigo, "Código") ||
                !ValidationHelper.RequireText(txtNombre, "Nombre") ||
                !ValidationHelper.RequireText(txtApellido, "Apellido") ||
                !ValidationHelper.RequireText(txtCedula, "Cédula") ||
                !ValidationHelper.RequireText(txtCargo, "Cargo") ||
                !ValidationHelper.TryPositiveDecimal(txtSalario, "Salario base", out salary))
            {
                return;
```

UI/FrmEmpleados.cs (continuacion)

```
            }

            if (cmbDepartamento.SelectedValue == null)
            {
                MessageBox.Show("Seleccione un departamento.", "Empleados", MessageBoxButtons.OK, MessageBoxIcon.Warning);
                return;
            }

            if (cmbEstado.SelectedItem == null)
            {
                MessageBox.Show("Seleccione el estado del empleado.", "Empleados", MessageBoxButtons.OK, MessageBoxIcon.Warning);
                return;
            }

            try
            {
                Empleado empleado = new Empleado();
                empleado.IdEmpleado = selectedId;
                empleado.Codigo = txtCodigo.Text.Trim();
                empleado.Nombre = txtNombre.Text.Trim();
                empleado.Apellido = txtApellido.Text.Trim();
                empleado.Cedula = txtCedula.Text.Trim();
                empleado.Cargo = txtCargo.Text.Trim();
                empleado.IdDepartamento = Convert.ToInt32(cmbDepartamento.SelectedValue);
                empleado.SalarioBase = salary;
                empleado.Estado = cmbEstado.SelectedItem.ToString();
                empleado.FechaIngreso = dtpIngreso.Value.Date;
                EmployeeChangeContext changeContext = null;
                if (selectedId > 0)
                {
                    Empleado original = empleadoService.GetById(selectedId);
                    if (HasTrackedChanges(original, empleado))
                    {
                        using (FrmCambioLaboral dialog = new FrmCambioLaboral(original))
                        {
                            if (dialog.ShowDialog() != DialogResult.OK) return;
                            changeContext = dialog.ChangeContext;
                        }
                    }
                }
                EmployeeSaveResult result = empleadoService.Save(empleado, changeContext);
                string message = result.HasScheduledChanges
                    ? "El cambio laboral quedó programado para la fecha indicada."
                    : "Empleado guardado correctamente.";
                MessageBox.Show(message, "Empleados", MessageBoxButtons.OK, MessageBoxIcon.Information);
                ClearForm();
                LoadEmployees();
            }
            catch (Exception ex)
            {
                MessageBox.Show(ex.Message, "Empleados", MessageBoxButtons.OK, MessageBoxIcon.Warning);
            }
        }

        private void btnDesactivar_Click(object sender, EventArgs e)
        {
            if (selectedId == 0)
            {
                MessageBox.Show("Seleccione un empleado.", "Empleados", MessageBoxButtons.OK, MessageBoxIcon.Information);
                return;
```

UI/FrmEmpleados.cs (continuacion)

```
            }

            if (MessageBox.Show("¿Desea desactivar el empleado seleccionado?", "Confirmar",
                MessageBoxButtons.YesNo, MessageBoxIcon.Question) == DialogResult.Yes)
            {
                try
                {
                    Empleado employee = empleadoService.GetById(selectedId);
                    using (FrmCambioLaboral dialog = new FrmCambioLaboral(employee))
                    {
                        if (dialog.ShowDialog() != DialogResult.OK) return;
                        EmployeeSaveResult result = empleadoService.Deactivate(selectedId, dialog.ChangeContext);
                        MessageBox.Show(result.HasScheduledChanges
                            ? "La desactivación quedó programada." : "Empleado desactivado correctamente.",
                            "Empleados", MessageBoxButtons.OK, MessageBoxIcon.Information);
                    }
                    ClearForm();
                    LoadEmployees();
                }
                catch (Exception ex)
                {
                    MessageBox.Show(ex.Message, "Empleados", MessageBoxButtons.OK, MessageBoxIcon.Warning);
                }
            }
        }

        private void btnExportarExcel_Click(object sender, EventArgs e)
        {
            if (currentEmployees.Count == 0)
            {
                MessageBox.Show("No hay empleados para exportar.", "Empleados", MessageBoxButtons.OK, MessageBoxIcon.Information);
                return;
            }

            UiFactory.ShowExported(excelExportService.ExportarEmpleados(currentEmployees));
        }

        private void btnExportarPdf_Click(object sender, EventArgs e)
        {
            if (currentEmployees.Count == 0)
            {
                MessageBox.Show("No hay empleados para exportar.", "Empleados", MessageBoxButtons.OK, MessageBoxIcon.Information);
                return;
            }

            UiFactory.ShowExported(pdfExportService.ExportarEmpleados(currentEmployees));
        }

        private void FilterChanged(object sender, EventArgs e)
        {
            if (LicenseManager.UsageMode != LicenseUsageMode.Designtime && cmbFiltroDepartamento.DataSource != null)
            {
                LoadEmployees();
            }
        }

        private void btnHistorialLaboral_Click(object sender, EventArgs e)
        {
            using (FrmHistorialEmpleado form = new FrmHistorialEmpleado(selectedId > 0 ? (int?)selectedId : null))
                form.ShowDialog();
```

UI/FrmEmpleados.cs (continuacion)

```
        }

        private static bool HasTrackedChanges(Empleado original, Empleado updated)
        {
            return original != null && (original.SalarioBase != updated.SalarioBase ||
                original.IdDepartamento != updated.IdDepartamento ||
                !string.Equals(original.Cargo, updated.Cargo, StringComparison.Ordinal) ||
                !string.Equals(original.Estado, updated.Estado, StringComparison.OrdinalIgnoreCase));
        }

        private void ClearForm()
        {
            selectedId = 0;
            txtCodigo.Clear();
            txtNombre.Clear();
            txtApellido.Clear();
            txtCedula.Clear();
            txtCargo.Clear();
            txtSalario.Clear();
            if (cmbDepartamento.Items.Count > 0)
            {
                cmbDepartamento.SelectedIndex = 0;
            }
            cmbEstado.SelectedIndex = 0;
            dtpIngreso.Value = DateTime.Today;
        }
    }
}
```

UI/FrmEmpleados.Designer.cs

```
namespace SistemaGestionNomina.UI
{
    partial class FrmEmpleados
    {
        private System.ComponentModel.IContainer components = null;
        private System.Windows.Forms.Label lblTitulo;
        private System.Windows.Forms.Panel panelFormulario;
        private System.Windows.Forms.Panel panelFiltros;
        private System.Windows.Forms.TextBox txtBuscar;
        private System.Windows.Forms.ComboBox cmbFiltroDepartamento;
        private System.Windows.Forms.ComboBox cmbFiltroEstado;
        private System.Windows.Forms.Button btnNuevo;
        private System.Windows.Forms.Button btnEditar;
        private System.Windows.Forms.Button btnDesactivar;
        private System.Windows.Forms.Button btnExportarExcel;
        private System.Windows.Forms.Button btnExportarPdf;
        private System.Windows.Forms.Button btnHistorialLaboral;
        private System.Windows.Forms.TextBox txtCodigo;
        private System.Windows.Forms.TextBox txtNombre;
        private System.Windows.Forms.TextBox txtApellido;
        private System.Windows.Forms.TextBox txtCedula;
        private System.Windows.Forms.TextBox txtCargo;
        private System.Windows.Forms.TextBox txtSalario;
        private System.Windows.Forms.Label lblCodigo;
        private System.Windows.Forms.Label lblNombre;
        private System.Windows.Forms.Label lblApellido;
        private System.Windows.Forms.Label lblCedula;
        private System.Windows.Forms.Label lblCargo;
        private System.Windows.Forms.Label lblDepartamento;
        private System.Windows.Forms.Label lblSalario;
        private System.Windows.Forms.ComboBox cmbDepartamento;
        private System.Windows.Forms.ComboBox cmbEstado;
        private System.Windows.Forms.DateTimePicker dtpIngreso;
        private System.Windows.Forms.DataGridView dgvEmpleados;
        private System.Windows.Forms.DataGridViewTextBoxColumn colIdEmpleado;
        private System.Windows.Forms.DataGridViewTextBoxColumn colCodigo;
        private System.Windows.Forms.DataGridViewTextBoxColumn colNombre;
        private System.Windows.Forms.DataGridViewTextBoxColumn colCedula;
        private System.Windows.Forms.DataGridViewTextBoxColumn colCargo;
        private System.Windows.Forms.DataGridViewTextBoxColumn colDepartamento;
        private System.Windows.Forms.DataGridViewTextBoxColumn colSalarioBase;
        private System.Windows.Forms.DataGridViewTextBoxColumn colEstado;
        private System.Windows.Forms.DataGridViewTextBoxColumn colAcciones;

        protected override void Dispose(bool disposing)
        {
            if (disposing && (components != null)) components.Dispose();
            base.Dispose(disposing);
        }

        private void InitializeComponent()
        {
            this.lblTitulo = new System.Windows.Forms.Label();
            this.panelFormulario = new System.Windows.Forms.Panel();
            this.txtCodigo = new System.Windows.Forms.TextBox();
            this.txtNombre = new System.Windows.Forms.TextBox();
            this.txtApellido = new System.Windows.Forms.TextBox();
            this.txtCedula = new System.Windows.Forms.TextBox();
            this.txtCargo = new System.Windows.Forms.TextBox();
            this.lblCodigo = new System.Windows.Forms.Label();
```

UI/FrmEmpleados.Designer.cs (continuacion)

```
            this.lblNombre = new System.Windows.Forms.Label();
            this.lblApellido = new System.Windows.Forms.Label();
            this.lblCedula = new System.Windows.Forms.Label();
            this.lblCargo = new System.Windows.Forms.Label();
            this.lblDepartamento = new System.Windows.Forms.Label();
            this.lblSalario = new System.Windows.Forms.Label();
            this.cmbDepartamento = new System.Windows.Forms.ComboBox();
            this.txtSalario = new System.Windows.Forms.TextBox();
            this.cmbEstado = new System.Windows.Forms.ComboBox();
            this.dtpIngreso = new System.Windows.Forms.DateTimePicker();
            this.panelFiltros = new System.Windows.Forms.Panel();
            this.txtBuscar = new System.Windows.Forms.TextBox();
            this.cmbFiltroDepartamento = new System.Windows.Forms.ComboBox();
            this.cmbFiltroEstado = new System.Windows.Forms.ComboBox();
            this.btnNuevo = new System.Windows.Forms.Button();
            this.btnEditar = new System.Windows.Forms.Button();
            this.btnDesactivar = new System.Windows.Forms.Button();
            this.btnExportarExcel = new System.Windows.Forms.Button();
            this.btnExportarPdf = new System.Windows.Forms.Button();
            this.btnHistorialLaboral = new System.Windows.Forms.Button();
            this.dgvEmpleados = new System.Windows.Forms.DataGridView();
            this.colIdEmpleado = new System.Windows.Forms.DataGridViewTextBoxColumn();
            this.colCodigo = new System.Windows.Forms.DataGridViewTextBoxColumn();
            this.colNombre = new System.Windows.Forms.DataGridViewTextBoxColumn();
            this.colCedula = new System.Windows.Forms.DataGridViewTextBoxColumn();
            this.colCargo = new System.Windows.Forms.DataGridViewTextBoxColumn();
            this.colDepartamento = new System.Windows.Forms.DataGridViewTextBoxColumn();
            this.colSalarioBase = new System.Windows.Forms.DataGridViewTextBoxColumn();
            this.colEstado = new System.Windows.Forms.DataGridViewTextBoxColumn();
            this.colAcciones = new System.Windows.Forms.DataGridViewTextBoxColumn();
            this.panelFormulario.SuspendLayout();
            this.panelFiltros.SuspendLayout();
            ((System.ComponentModel.ISupportInitialize)(this.dgvEmpleados)).BeginInit();
            this.SuspendLayout();
            //
            // lblTitulo
            //
            this.lblTitulo.Font = new System.Drawing.Font("Segoe UI", 22F, System.Drawing.FontStyle.Bold);
            this.lblTitulo.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245);
            this.lblTitulo.Location = new System.Drawing.Point(34, 24);
            this.lblTitulo.Name = "lblTitulo";
            this.lblTitulo.Size = new System.Drawing.Size(420, 42);
            this.lblTitulo.TabIndex = 0;
            this.lblTitulo.Text = "Empleados";
            //
            // btnHistorialLaboral
            //
            this.btnHistorialLaboral.BackColor = System.Drawing.Color.FromArgb(17, 18, 26);
            this.btnHistorialLaboral.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
            this.btnHistorialLaboral.Location = new System.Drawing.Point(824, 28);
            this.btnHistorialLaboral.Name = "btnHistorialLaboral";
            this.btnHistorialLaboral.Size = new System.Drawing.Size(160, 34);
            this.btnHistorialLaboral.TabIndex = 20;
            this.btnHistorialLaboral.Text = "Historial laboral";
            this.btnHistorialLaboral.UseVisualStyleBackColor = false;
            this.btnHistorialLaboral.Click += new System.EventHandler(this.btnHistorialLaboral_Click);
            //
            // panelFormulario
            //
            this.panelFormulario.BackColor = System.Drawing.Color.FromArgb(17, 18, 26);
```

UI/FrmEmpleados.Designer.cs (continuacion)

```
            this.panelFormulario.BorderStyle = System.Windows.Forms.BorderStyle.FixedSingle;
            this.panelFormulario.Controls.Add(this.lblCodigo);
            this.panelFormulario.Controls.Add(this.lblNombre);
            this.panelFormulario.Controls.Add(this.lblApellido);
            this.panelFormulario.Controls.Add(this.lblCedula);
            this.panelFormulario.Controls.Add(this.lblCargo);
            this.panelFormulario.Controls.Add(this.lblDepartamento);
            this.panelFormulario.Controls.Add(this.lblSalario);
            this.panelFormulario.Controls.Add(this.txtCodigo);
            this.panelFormulario.Controls.Add(this.txtNombre);
            this.panelFormulario.Controls.Add(this.txtApellido);
            this.panelFormulario.Controls.Add(this.txtCedula);
            this.panelFormulario.Controls.Add(this.txtCargo);
            this.panelFormulario.Controls.Add(this.cmbDepartamento);
            this.panelFormulario.Controls.Add(this.txtSalario);
            this.panelFormulario.Controls.Add(this.cmbEstado);
            this.panelFormulario.Controls.Add(this.dtpIngreso);
            this.panelFormulario.Location = new System.Drawing.Point(36, 86);
            this.panelFormulario.Name = "panelFormulario";
            this.panelFormulario.Size = new System.Drawing.Size(948, 118);
            this.panelFormulario.TabIndex = 1;
            //
            // txtCodigo
            //
            this.txtCodigo.BackColor = System.Drawing.Color.FromArgb(24, 25, 34);
            this.txtCodigo.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245);
            this.txtCodigo.Location = new System.Drawing.Point(18, 36);
            this.txtCodigo.Name = "txtCodigo";
            this.txtCodigo.Size = new System.Drawing.Size(100, 23);
            this.txtCodigo.TabIndex = 0;
            this.txtCodigo.Text = "EMP-";
            //
            // txtNombre
            //
            this.txtNombre.BackColor = System.Drawing.Color.FromArgb(24, 25, 34);
            this.txtNombre.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245);
            this.txtNombre.Location = new System.Drawing.Point(128, 36);
            this.txtNombre.Name = "txtNombre";
            this.txtNombre.Size = new System.Drawing.Size(120, 23);
            this.txtNombre.TabIndex = 1;
            //
            // txtApellido
            //
            this.txtApellido.BackColor = System.Drawing.Color.FromArgb(24, 25, 34);
            this.txtApellido.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245);
            this.txtApellido.Location = new System.Drawing.Point(258, 36);
            this.txtApellido.Name = "txtApellido";
            this.txtApellido.Size = new System.Drawing.Size(120, 23);
            this.txtApellido.TabIndex = 2;
            //
            // txtCedula
            //
            this.txtCedula.BackColor = System.Drawing.Color.FromArgb(24, 25, 34);
            this.txtCedula.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245);
            this.txtCedula.Location = new System.Drawing.Point(388, 36);
            this.txtCedula.Name = "txtCedula";
            this.txtCedula.Size = new System.Drawing.Size(120, 23);
            this.txtCedula.TabIndex = 3;
            //
            // txtCargo
```

UI/FrmEmpleados.Designer.cs (continuacion)

```
            //
            this.txtCargo.BackColor = System.Drawing.Color.FromArgb(24, 25, 34);
            this.txtCargo.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245);
            this.txtCargo.Location = new System.Drawing.Point(518, 36);
            this.txtCargo.Name = "txtCargo";
            this.txtCargo.Size = new System.Drawing.Size(150, 23);
            this.txtCargo.TabIndex = 4;
            //
            // cmbDepartamento
            //
            this.cmbDepartamento.BackColor = System.Drawing.Color.FromArgb(24, 25, 34);
            this.cmbDepartamento.DropDownStyle = System.Windows.Forms.ComboBoxStyle.DropDownList;
            this.cmbDepartamento.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245);
            this.cmbDepartamento.Location = new System.Drawing.Point(678, 36);
            this.cmbDepartamento.Name = "cmbDepartamento";
            this.cmbDepartamento.Size = new System.Drawing.Size(150, 23);
            this.cmbDepartamento.TabIndex = 5;
            //
            // txtSalario
            //
            this.txtSalario.BackColor = System.Drawing.Color.FromArgb(24, 25, 34);
            this.txtSalario.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245);
            this.txtSalario.Location = new System.Drawing.Point(18, 84);
            this.txtSalario.Name = "txtSalario";
            this.txtSalario.Size = new System.Drawing.Size(120, 23);
            this.txtSalario.TabIndex = 6;
            //
            // input labels
            //
            this.lblCodigo.ForeColor = System.Drawing.Color.FromArgb(170, 170, 185);
            this.lblCodigo.Location = new System.Drawing.Point(18, 16);
            this.lblCodigo.Name = "lblCodigo";
            this.lblCodigo.Size = new System.Drawing.Size(100, 18);
            this.lblCodigo.TabIndex = 20;
            this.lblCodigo.Text = "Código";
            this.lblNombre.ForeColor = System.Drawing.Color.FromArgb(170, 170, 185);
            this.lblNombre.Location = new System.Drawing.Point(128, 16);
            this.lblNombre.Name = "lblNombre";
            this.lblNombre.Size = new System.Drawing.Size(100, 18);
            this.lblNombre.TabIndex = 21;
            this.lblNombre.Text = "Nombre";
            this.lblApellido.ForeColor = System.Drawing.Color.FromArgb(170, 170, 185);
            this.lblApellido.Location = new System.Drawing.Point(258, 16);
            this.lblApellido.Name = "lblApellido";
            this.lblApellido.Size = new System.Drawing.Size(100, 18);
            this.lblApellido.TabIndex = 22;
            this.lblApellido.Text = "Apellido";
            this.lblCedula.ForeColor = System.Drawing.Color.FromArgb(170, 170, 185);
            this.lblCedula.Location = new System.Drawing.Point(388, 16);
            this.lblCedula.Name = "lblCedula";
            this.lblCedula.Size = new System.Drawing.Size(100, 18);
            this.lblCedula.TabIndex = 23;
            this.lblCedula.Text = "Cédula";
            this.lblCargo.ForeColor = System.Drawing.Color.FromArgb(170, 170, 185);
            this.lblCargo.Location = new System.Drawing.Point(518, 16);
            this.lblCargo.Name = "lblCargo";
            this.lblCargo.Size = new System.Drawing.Size(100, 18);
            this.lblCargo.TabIndex = 24;
            this.lblCargo.Text = "Cargo";
            this.lblDepartamento.ForeColor = System.Drawing.Color.FromArgb(170, 170, 185);
```

UI/FrmEmpleados.Designer.cs (continuacion)

```
            this.lblDepartamento.Location = new System.Drawing.Point(678, 16);
            this.lblDepartamento.Name = "lblDepartamento";
            this.lblDepartamento.Size = new System.Drawing.Size(120, 18);
            this.lblDepartamento.TabIndex = 25;
            this.lblDepartamento.Text = "Departamento";
            this.lblSalario.ForeColor = System.Drawing.Color.FromArgb(170, 170, 185);
            this.lblSalario.Location = new System.Drawing.Point(18, 64);
            this.lblSalario.Name = "lblSalario";
            this.lblSalario.Size = new System.Drawing.Size(100, 18);
            this.lblSalario.TabIndex = 26;
            this.lblSalario.Text = "Salario base";
            //
            // cmbEstado
            //
            this.cmbEstado.BackColor = System.Drawing.Color.FromArgb(24, 25, 34);
            this.cmbEstado.DropDownStyle = System.Windows.Forms.ComboBoxStyle.DropDownList;
            this.cmbEstado.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245);
            this.cmbEstado.Items.AddRange(new object[] { "Activo", "Inactivo" });
            this.cmbEstado.Location = new System.Drawing.Point(150, 70);
            this.cmbEstado.Name = "cmbEstado";
            this.cmbEstado.SelectedIndex = 0;
            this.cmbEstado.Size = new System.Drawing.Size(120, 23);
            this.cmbEstado.TabIndex = 7;
            //
            // dtpIngreso
            //
            this.dtpIngreso.Format = System.Windows.Forms.DateTimePickerFormat.Short;
            this.dtpIngreso.Location = new System.Drawing.Point(282, 70);
            this.dtpIngreso.Name = "dtpIngreso";
            this.dtpIngreso.Size = new System.Drawing.Size(120, 23);
            this.dtpIngreso.TabIndex = 8;
            //
            // panelFiltros
            //
            this.panelFiltros.BackColor = System.Drawing.Color.FromArgb(17, 18, 26);
            this.panelFiltros.BorderStyle = System.Windows.Forms.BorderStyle.FixedSingle;
            this.panelFiltros.Controls.Add(this.txtBuscar);
            this.panelFiltros.Controls.Add(this.cmbFiltroDepartamento);
            this.panelFiltros.Controls.Add(this.cmbFiltroEstado);
            this.panelFiltros.Controls.Add(this.btnNuevo);
            this.panelFiltros.Controls.Add(this.btnEditar);
            this.panelFiltros.Controls.Add(this.btnDesactivar);
            this.panelFiltros.Controls.Add(this.btnExportarExcel);
            this.panelFiltros.Controls.Add(this.btnExportarPdf);
            this.panelFiltros.Location = new System.Drawing.Point(36, 224);
            this.panelFiltros.Name = "panelFiltros";
            this.panelFiltros.Size = new System.Drawing.Size(948, 70);
            this.panelFiltros.TabIndex = 2;
            //
            // txtBuscar
            //
            this.txtBuscar.BackColor = System.Drawing.Color.FromArgb(24, 25, 34);
            this.txtBuscar.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245);
            this.txtBuscar.Location = new System.Drawing.Point(18, 24);
            this.txtBuscar.Name = "txtBuscar";
            this.txtBuscar.Size = new System.Drawing.Size(190, 23);
            this.txtBuscar.TabIndex = 0;
            this.txtBuscar.TextChanged += new System.EventHandler(this.FilterChanged);
            //
            // cmbFiltroDepartamento
```

UI/FrmEmpleados.Designer.cs (continuacion)

```
            //
            this.cmbFiltroDepartamento.DropDownStyle = System.Windows.Forms.ComboBoxStyle.DropDownList;
            this.cmbFiltroDepartamento.Location = new System.Drawing.Point(222, 24);
            this.cmbFiltroDepartamento.Name = "cmbFiltroDepartamento";
            this.cmbFiltroDepartamento.Size = new System.Drawing.Size(180, 23);
            this.cmbFiltroDepartamento.TabIndex = 1;
            this.cmbFiltroDepartamento.SelectedIndexChanged += new System.EventHandler(this.FilterChanged);
            //
            // cmbFiltroEstado
            //
            this.cmbFiltroEstado.DropDownStyle = System.Windows.Forms.ComboBoxStyle.DropDownList;
            this.cmbFiltroEstado.Items.AddRange(new object[] { "Todos", "Activo", "Inactivo" });
            this.cmbFiltroEstado.Location = new System.Drawing.Point(414, 24);
            this.cmbFiltroEstado.Name = "cmbFiltroEstado";
            this.cmbFiltroEstado.SelectedIndex = 0;
            this.cmbFiltroEstado.Size = new System.Drawing.Size(110, 23);
            this.cmbFiltroEstado.TabIndex = 2;
            this.cmbFiltroEstado.SelectedIndexChanged += new System.EventHandler(this.FilterChanged);
            //
            // btnNuevo
            //
            this.btnNuevo.BackColor = System.Drawing.Color.FromArgb(167, 139, 250);
            this.btnNuevo.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
            this.btnNuevo.Location = new System.Drawing.Point(540, 20);
            this.btnNuevo.Name = "btnNuevo";
            this.btnNuevo.Size = new System.Drawing.Size(80, 30);
            this.btnNuevo.TabIndex = 3;
            this.btnNuevo.Text = "Nuevo";
            this.btnNuevo.UseVisualStyleBackColor = false;
            this.btnNuevo.Click += new System.EventHandler(this.btnNuevo_Click);
            //
            // btnEditar
            //
            this.btnEditar.BackColor = System.Drawing.Color.FromArgb(167, 139, 250);
            this.btnEditar.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
            this.btnEditar.Location = new System.Drawing.Point(626, 20);
            this.btnEditar.Name = "btnEditar";
            this.btnEditar.Size = new System.Drawing.Size(80, 30);
            this.btnEditar.TabIndex = 4;
            this.btnEditar.Text = "Guardar";
            this.btnEditar.UseVisualStyleBackColor = false;
            this.btnEditar.Click += new System.EventHandler(this.btnEditar_Click);
            //
            // btnDesactivar
            //
            this.btnDesactivar.BackColor = System.Drawing.Color.Black;
            this.btnDesactivar.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
            this.btnDesactivar.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245);
            this.btnDesactivar.Location = new System.Drawing.Point(712, 20);
            this.btnDesactivar.Name = "btnDesactivar";
            this.btnDesactivar.Size = new System.Drawing.Size(90, 30);
            this.btnDesactivar.TabIndex = 5;
            this.btnDesactivar.Text = "Desactivar";
            this.btnDesactivar.UseVisualStyleBackColor = false;
            this.btnDesactivar.Click += new System.EventHandler(this.btnDesactivar_Click);
            //
            // btnExportarExcel
            //
            this.btnExportarExcel.BackColor = System.Drawing.Color.Black;
            this.btnExportarExcel.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
```

UI/FrmEmpleados.Designer.cs (continuacion)

```
            this.btnExportarExcel.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245);
            this.btnExportarExcel.Location = new System.Drawing.Point(808, 20);
            this.btnExportarExcel.Name = "btnExportarExcel";
            this.btnExportarExcel.Size = new System.Drawing.Size(60, 30);
            this.btnExportarExcel.TabIndex = 6;
            this.btnExportarExcel.Text = "Excel";
            this.btnExportarExcel.UseVisualStyleBackColor = false;
            this.btnExportarExcel.Click += new System.EventHandler(this.btnExportarExcel_Click);
            //
            // btnExportarPdf
            //
            this.btnExportarPdf.BackColor = System.Drawing.Color.Black;
            this.btnExportarPdf.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
            this.btnExportarPdf.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245);
            this.btnExportarPdf.Location = new System.Drawing.Point(874, 20);
            this.btnExportarPdf.Name = "btnExportarPdf";
            this.btnExportarPdf.Size = new System.Drawing.Size(55, 30);
            this.btnExportarPdf.TabIndex = 7;
            this.btnExportarPdf.Text = "PDF";
            this.btnExportarPdf.UseVisualStyleBackColor = false;
            this.btnExportarPdf.Click += new System.EventHandler(this.btnExportarPdf_Click);
            //
            // dgvEmpleados
            //
            this.dgvEmpleados.AllowUserToAddRows = false;
            this.dgvEmpleados.AllowUserToDeleteRows = false;
            this.dgvEmpleados.BackgroundColor = System.Drawing.Color.Black;
            this.dgvEmpleados.ColumnHeadersHeightSizeMode = System.Windows.Forms.DataGridViewColumnHeadersHeightSizeMode.AutoSize;
            this.dgvEmpleados.Columns.AddRange(new System.Windows.Forms.DataGridViewColumn[] {
            this.colIdEmpleado,
            this.colCodigo,
            this.colNombre,
            this.colCedula,
            this.colCargo,
            this.colDepartamento,
            this.colSalarioBase,
            this.colEstado,
            this.colAcciones});
            this.dgvEmpleados.Location = new System.Drawing.Point(36, 318);
            this.dgvEmpleados.MultiSelect = false;
            this.dgvEmpleados.Name = "dgvEmpleados";
            this.dgvEmpleados.ReadOnly = true;
            this.dgvEmpleados.RowHeadersVisible = false;
            this.dgvEmpleados.SelectionMode = System.Windows.Forms.DataGridViewSelectionMode.FullRowSelect;
            this.dgvEmpleados.Size = new System.Drawing.Size(948, 360);
            this.dgvEmpleados.TabIndex = 3;
            this.dgvEmpleados.SelectionChanged += new System.EventHandler(this.dgvEmpleados_SelectionChanged);
            //
            // columns
            //
            this.colIdEmpleado.HeaderText = "Id";
            this.colIdEmpleado.Name = "IdEmpleado";
            this.colIdEmpleado.ReadOnly = true;
            this.colIdEmpleado.Visible = false;
            this.colCodigo.HeaderText = "Código";
            this.colCodigo.Name = "Codigo";
            this.colCodigo.ReadOnly = true;
            this.colNombre.HeaderText = "Nombre";
            this.colNombre.Name = "Nombre";
            this.colNombre.ReadOnly = true;
```

UI/FrmEmpleados.Designer.cs (continuacion)

```
            this.colCedula.HeaderText = "Cédula";
            this.colCedula.Name = "Cedula";
            this.colCedula.ReadOnly = true;
            this.colCargo.HeaderText = "Cargo";
            this.colCargo.Name = "Cargo";
            this.colCargo.ReadOnly = true;
            this.colDepartamento.HeaderText = "Departamento";
            this.colDepartamento.Name = "Departamento";
            this.colDepartamento.ReadOnly = true;
            this.colSalarioBase.HeaderText = "Salario Base";
            this.colSalarioBase.Name = "SalarioBase";
            this.colSalarioBase.ReadOnly = true;
            this.colEstado.HeaderText = "Estado";
            this.colEstado.Name = "Estado";
            this.colEstado.ReadOnly = true;
            this.colAcciones.HeaderText = "Acciones";
            this.colAcciones.Name = "Acciones";
            this.colAcciones.ReadOnly = true;
            //
            // FrmEmpleados
            //
            this.AutoScaleDimensions = new System.Drawing.SizeF(7F, 15F);
            this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font;
            this.BackColor = System.Drawing.Color.FromArgb(13, 14, 20);
            this.ClientSize = new System.Drawing.Size(1020, 742);
            this.Controls.Add(this.dgvEmpleados);
            this.Controls.Add(this.panelFiltros);
            this.Controls.Add(this.panelFormulario);
            this.Controls.Add(this.btnHistorialLaboral);
            this.Controls.Add(this.lblTitulo);
            this.Font = new System.Drawing.Font("Segoe UI", 9F);
            this.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245);
            this.Name = "FrmEmpleados";
            this.Text = "Empleados";
            this.Load += new System.EventHandler(this.FrmEmpleados_Load);
            this.panelFormulario.ResumeLayout(false);
            this.panelFormulario.PerformLayout();
            this.panelFiltros.ResumeLayout(false);
            this.panelFiltros.PerformLayout();
            ((System.ComponentModel.ISupportInitialize)(this.dgvEmpleados)).EndInit();
            this.ResumeLayout(false);
        }
    }
}
```

UI/FrmGestionAusencias.cs

```
using System;
using System.ComponentModel;
using System.Windows.Forms;
using SistemaGestionNomina.Helpers;
using SistemaGestionNomina.Models;
using SistemaGestionNomina.Security;
using SistemaGestionNomina.Services;

namespace SistemaGestionNomina.UI
{
    public partial class FrmGestionAusencias : Form
    {
        private readonly int requestId;
        private readonly bool cancelOnly;
        private readonly AbsenceRequestService service = new AbsenceRequestService();
        private readonly AuthorizationService authorizationService = new AuthorizationService();
        public bool RequestChanged { get; private set; }

        public FrmGestionAusencias() : this(0, false) { }
        public FrmGestionAusencias(int id, bool showCancelOnly)
        {
            requestId = id; cancelOnly = showCancelOnly; InitializeComponent(); ControlStyleHelper.ApplyModernForm(this);
        }

        private void FrmGestionAusencias_Load(object sender, EventArgs e)
        {
            if (LicenseManager.UsageMode == LicenseUsageMode.Designtime || requestId <= 0) return;
            try
            {
                SolicitudAusencia item = service.GetById(requestId);
                lblSolicitud.Text = item.CodigoEmpleado + " - " + item.EmpleadoNombre;
                lblDetalle.Text = item.Tipo + " | " + item.FechaInicio.ToString("dd/MM/yyyy") + " al " + item.FechaFin.ToString("dd/MM/yyyy") + " | " +  \
    item.Estado;
                txtMotivo.Text = item.Motivo;
                btnAprobar.Visible = !cancelOnly && authorizationService.HasPermission(Permissions.AbsencesApprove) && item.Estado == AbsenceStates.Pending;
                btnRechazar.Visible = !cancelOnly && authorizationService.HasPermission(Permissions.AbsencesReject) && item.Estado == AbsenceStates.Pending;
                btnCancelarSolicitud.Visible = (authorizationService.HasPermission(Permissions.AbsencesCancel) ||  \
    authorizationService.HasPermission(Permissions.OwnAbsencesCancel)) &&
                    (item.Estado == AbsenceStates.Pending || item.Estado == AbsenceStates.Approved);
            }
            catch (Exception ex) { MessageBox.Show(ex.Message, "Ausencias", MessageBoxButtons.OK, MessageBoxIcon.Warning); }
        }

        private void btnAprobar_Click(object sender, EventArgs e) { Execute("aprobar", delegate { service.Approve(requestId, txtObservacion.Text); }); }
        private void btnRechazar_Click(object sender, EventArgs e) { Execute("rechazar", delegate { service.Reject(requestId, txtObservacion.Text); }); }
        private void btnCancelarSolicitud_Click(object sender, EventArgs e) { Execute("cancelar", delegate { service.Cancel(requestId,  \
    txtObservacion.Text); }); }
        private void btnCerrar_Click(object sender, EventArgs e) { Close(); }

        private void Execute(string action, Action operation)
        {
            if (MessageBox.Show("¿Confirma que desea " + action + " esta solicitud?", "Ausencias", MessageBoxButtons.YesNo, MessageBoxIcon.Question) !=  \
    DialogResult.Yes) return;
            try { operation(); RequestChanged = true; DialogResult = DialogResult.OK; Close(); }
            catch (Exception ex) { MessageBox.Show(ex.Message, "Ausencias", MessageBoxButtons.OK, MessageBoxIcon.Warning); }
        }
    }
}
```

UI/FrmGestionAusencias.Designer.cs

```
namespace SistemaGestionNomina.UI
{
    partial class FrmGestionAusencias
    {
        private System.ComponentModel.IContainer components = null;
        private System.Windows.Forms.Label lblTitulo, lblSolicitud, lblDetalle, lblMotivo, lblObservacion;
        private System.Windows.Forms.TextBox txtMotivo, txtObservacion;
        private System.Windows.Forms.Button btnAprobar, btnRechazar, btnCancelarSolicitud, btnCerrar;
        protected override void Dispose(bool disposing) { if (disposing && components != null) components.Dispose(); base.Dispose(disposing); }
        private void InitializeComponent()
        {
            this.lblTitulo = new System.Windows.Forms.Label(); this.lblSolicitud = new System.Windows.Forms.Label(); this.lblDetalle = new  \
    System.Windows.Forms.Label(); this.lblMotivo = new System.Windows.Forms.Label(); this.lblObservacion = new System.Windows.Forms.Label();  \
    this.txtMotivo = new System.Windows.Forms.TextBox(); this.txtObservacion = new System.Windows.Forms.TextBox(); this.btnAprobar = new  \
    System.Windows.Forms.Button(); this.btnRechazar = new System.Windows.Forms.Button(); this.btnCancelarSolicitud = new System.Windows.Forms.Button();  \
    this.btnCerrar = new System.Windows.Forms.Button(); this.SuspendLayout();
            this.BackColor = System.Drawing.Color.FromArgb(18, 18, 27); this.ClientSize = new System.Drawing.Size(660, 460); this.Font = new  \
    System.Drawing.Font("Segoe UI", 9F); this.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245); this.FormBorderStyle =  \
    System.Windows.Forms.FormBorderStyle.FixedDialog; this.MaximizeBox = false; this.MinimizeBox = false; this.Name = "FrmGestionAusencias";  \
    this.StartPosition = System.Windows.Forms.FormStartPosition.CenterParent; this.Text = "Gestión de ausencia"; this.Load += new  \
    System.EventHandler(this.FrmGestionAusencias_Load);
            this.lblTitulo.AutoSize = true; this.lblTitulo.Font = new System.Drawing.Font("Segoe UI", 20F, System.Drawing.FontStyle.Bold);  \
    this.lblTitulo.Location = new System.Drawing.Point(28, 20); this.lblTitulo.Text = "Resolver solicitud";
            this.lblSolicitud.AutoSize = true; this.lblSolicitud.ForeColor = System.Drawing.Color.FromArgb(167, 139, 250); this.lblSolicitud.Location =  \
    new System.Drawing.Point(32, 72); this.lblSolicitud.Text = "Empleado";
            this.lblDetalle.AutoSize = true; this.lblDetalle.Location = new System.Drawing.Point(32, 99); this.lblDetalle.Text = "Detalle de la solicitud";
            this.lblMotivo.AutoSize = true; this.lblMotivo.Location = new System.Drawing.Point(32, 136); this.lblMotivo.Text = "Motivo del trabajador";  \
    this.txtMotivo.Location = new System.Drawing.Point(35, 158); this.txtMotivo.Multiline = true; this.txtMotivo.ReadOnly = true; this.txtMotivo.Size =  \
    new System.Drawing.Size(590, 72);
            this.lblObservacion.AutoSize = true; this.lblObservacion.Location = new System.Drawing.Point(32, 250); this.lblObservacion.Text =  \
    "Observación o motivo de resolución"; this.txtObservacion.Location = new System.Drawing.Point(35, 272); this.txtObservacion.MaxLength = 500;  \
    this.txtObservacion.Multiline = true; this.txtObservacion.Size = new System.Drawing.Size(590, 72);
            this.btnAprobar.BackColor = System.Drawing.Color.FromArgb(34, 197, 94); this.btnAprobar.FlatStyle = System.Windows.Forms.FlatStyle.Flat;  \
    this.btnAprobar.Location = new System.Drawing.Point(194, 380); this.btnAprobar.Size = new System.Drawing.Size(94, 34); this.btnAprobar.Text =  \
    "Aprobar"; this.btnAprobar.UseVisualStyleBackColor = false; this.btnAprobar.Click += new System.EventHandler(this.btnAprobar_Click);
            this.btnRechazar.BackColor = System.Drawing.Color.FromArgb(239, 68, 68); this.btnRechazar.FlatStyle = System.Windows.Forms.FlatStyle.Flat;  \
    this.btnRechazar.Location = new System.Drawing.Point(296, 380); this.btnRechazar.Size = new System.Drawing.Size(94, 34); this.btnRechazar.Text =  \
    "Rechazar"; this.btnRechazar.UseVisualStyleBackColor = false; this.btnRechazar.Click += new System.EventHandler(this.btnRechazar_Click);
            this.btnCancelarSolicitud.FlatStyle = System.Windows.Forms.FlatStyle.Flat; this.btnCancelarSolicitud.Location = new System.Drawing.Point(398,  \
    380); this.btnCancelarSolicitud.Size = new System.Drawing.Size(108, 34); this.btnCancelarSolicitud.Text = "Cancelar solicitud";  \
    this.btnCancelarSolicitud.Click += new System.EventHandler(this.btnCancelarSolicitud_Click);
            this.btnCerrar.FlatStyle = System.Windows.Forms.FlatStyle.Flat; this.btnCerrar.Location = new System.Drawing.Point(514, 380);  \
    this.btnCerrar.Size = new System.Drawing.Size(110, 34); this.btnCerrar.Text = "Cerrar"; this.btnCerrar.Click += new  \
    System.EventHandler(this.btnCerrar_Click);
            this.Controls.Add(this.lblTitulo); this.Controls.Add(this.lblSolicitud); this.Controls.Add(this.lblDetalle);  \
    this.Controls.Add(this.lblMotivo); this.Controls.Add(this.txtMotivo); this.Controls.Add(this.lblObservacion); this.Controls.Add(this.txtObservacion);  \
    this.Controls.Add(this.btnAprobar); this.Controls.Add(this.btnRechazar); this.Controls.Add(this.btnCancelarSolicitud);  \
    this.Controls.Add(this.btnCerrar); this.ResumeLayout(false); this.PerformLayout();
        }
    }
}
```

UI/FrmHistorialEmpleado.cs

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Windows.Forms;
using SistemaGestionNomina.Helpers;
using SistemaGestionNomina.Models;
using SistemaGestionNomina.Security;
using SistemaGestionNomina.Services;

namespace SistemaGestionNomina.UI
{
    public partial class FrmHistorialEmpleado : Form
    {
        private readonly int? initialEmployeeId;
        private readonly EmployeeHistoryService historyService = new EmployeeHistoryService();
        private readonly EmpleadoService employeeService = new EmpleadoService();
        private readonly ExcelExportService excelExportService = new ExcelExportService();
        private readonly PdfExportService pdfExportService = new PdfExportService();
        private readonly AuthorizationService authorizationService = new AuthorizationService();
        private List<HistorialEmpleado> currentHistory = new List<HistorialEmpleado>();

        public FrmHistorialEmpleado() : this(null) { }

        public FrmHistorialEmpleado(int? employeeId)
        {
            initialEmployeeId = employeeId;
            InitializeComponent();
            ControlStyleHelper.ApplyModernForm(this);
        }

        private void FrmHistorialEmpleado_Load(object sender, EventArgs e)
        {
            if (LicenseManager.UsageMode == LicenseUsageMode.Designtime) return;
            List<Empleado> employees = new List<Empleado>();
            employees.Add(new Empleado
            {
                IdEmpleado = 0, Codigo = "TODOS", Nombre = "Todos los empleados", Apellido = string.Empty,
                Cedula = "N/A", Cargo = "N/A", IdDepartamento = 1, SalarioBase = 1,
                Estado = "Activo", FechaIngreso = DateTime.Today
            });
            employees.AddRange(employeeService.GetAll(string.Empty, null, string.Empty));
            cmbEmpleado.DataSource = employees;
            cmbEmpleado.DisplayMember = "NombreCompleto";
            cmbEmpleado.ValueMember = "IdEmpleado";
            if (initialEmployeeId.HasValue) cmbEmpleado.SelectedValue = initialEmployeeId.Value;
            cmbCampo.Items.Add("Todos");
            cmbCampo.Items.Add(EmployeeHistoryFields.BaseSalary);
            cmbCampo.Items.Add(EmployeeHistoryFields.Position);
            cmbCampo.Items.Add(EmployeeHistoryFields.Department);
            cmbCampo.Items.Add(EmployeeHistoryFields.Status);
            cmbCampo.SelectedIndex = 0;
            btnExcel.Visible = authorizationService.HasPermission(Permissions.EmployeeHistoryExport);
            btnPdf.Visible = btnExcel.Visible;
            LoadHistory();
        }

        private void btnBuscar_Click(object sender, EventArgs e) { LoadHistory(); }

        private void LoadHistory()
        {
```

UI/FrmHistorialEmpleado.cs (continuacion)

```
            try
            {
                int employeeId = cmbEmpleado.SelectedValue is int ? (int)cmbEmpleado.SelectedValue : 0;
                currentHistory = historyService.Search(new EmployeeHistoryQuery
                {
                    EmployeeId = employeeId > 0 ? (int?)employeeId : null,
                    Field = cmbCampo.SelectedIndex > 0 ? Convert.ToString(cmbCampo.SelectedItem) : string.Empty,
                    User = txtUsuario.Text.Trim(),
                    From = dtpDesde.Checked ? (DateTime?)dtpDesde.Value.Date : null,
                    To = dtpHasta.Checked ? (DateTime?)dtpHasta.Value.Date : null
                });
                dgvHistorial.Rows.Clear();
                for (int i = 0; i < currentHistory.Count; i++)
                {
                    HistorialEmpleado item = currentHistory[i];
                    dgvHistorial.Rows.Add(item.CodigoEmpleado, item.EmpleadoNombre, item.CampoModificado,
                        item.ValorAnterior, item.ValorNuevo, item.FechaEfectiva.ToString("dd/MM/yyyy"),
                        item.FechaCambio.ToString("dd/MM/yyyy HH:mm"), item.UsuarioResponsable,
                        item.Motivo, item.Aplicado ? "Aplicado" : "Programado");
                }
                lblResultado.Text = currentHistory.Count + " cambios encontrados";
            }
            catch (Exception ex)
            {
                MessageBox.Show(ex.Message, "Historial laboral", MessageBoxButtons.OK, MessageBoxIcon.Warning);
            }
        }

        private void btnExcel_Click(object sender, EventArgs e)
        {
            if (currentHistory.Count == 0) return;
            UiFactory.ShowExported(excelExportService.ExportarHistorialEmpleados(currentHistory));
        }

        private void btnPdf_Click(object sender, EventArgs e)
        {
            if (currentHistory.Count == 0) return;
            UiFactory.ShowExported(pdfExportService.ExportarHistorialEmpleados(currentHistory));
        }

        private void btnCerrar_Click(object sender, EventArgs e) { Close(); }
    }
}
```

UI/FrmHistorialEmpleado.Designer.cs

```
namespace SistemaGestionNomina.UI
{
    partial class FrmHistorialEmpleado
    {
        private System.ComponentModel.IContainer components = null;
        private System.Windows.Forms.Label lblTitulo;
        private System.Windows.Forms.ComboBox cmbEmpleado;
        private System.Windows.Forms.ComboBox cmbCampo;
        private System.Windows.Forms.TextBox txtUsuario;
        private System.Windows.Forms.DateTimePicker dtpDesde;
        private System.Windows.Forms.DateTimePicker dtpHasta;
        private System.Windows.Forms.Button btnBuscar;
        private System.Windows.Forms.Button btnExcel;
        private System.Windows.Forms.Button btnPdf;
        private System.Windows.Forms.Button btnCerrar;
        private System.Windows.Forms.DataGridView dgvHistorial;
        private System.Windows.Forms.Label lblResultado;

        protected override void Dispose(bool disposing)
        {
            if (disposing && (components != null)) components.Dispose();
            base.Dispose(disposing);
        }

        private void InitializeComponent()
        {
            this.lblTitulo = new System.Windows.Forms.Label(); this.cmbEmpleado = new System.Windows.Forms.ComboBox();
            this.cmbCampo = new System.Windows.Forms.ComboBox(); this.txtUsuario = new System.Windows.Forms.TextBox();
            this.dtpDesde = new System.Windows.Forms.DateTimePicker(); this.dtpHasta = new System.Windows.Forms.DateTimePicker();
            this.btnBuscar = new System.Windows.Forms.Button(); this.btnExcel = new System.Windows.Forms.Button();
            this.btnPdf = new System.Windows.Forms.Button(); this.btnCerrar = new System.Windows.Forms.Button();
            this.dgvHistorial = new System.Windows.Forms.DataGridView(); this.lblResultado = new System.Windows.Forms.Label();
            ((System.ComponentModel.ISupportInitialize)(this.dgvHistorial)).BeginInit(); this.SuspendLayout();
            this.BackColor = System.Drawing.Color.FromArgb(12, 12, 18); this.ClientSize = new System.Drawing.Size(1120, 700);
            this.Font = new System.Drawing.Font("Segoe UI", 9F); this.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245);
            this.Name = "FrmHistorialEmpleado"; this.StartPosition = System.Windows.Forms.FormStartPosition.CenterParent;
            this.Text = "Historial laboral"; this.Load += new System.EventHandler(this.FrmHistorialEmpleado_Load);
            this.lblTitulo.AutoSize = true; this.lblTitulo.Font = new System.Drawing.Font("Segoe UI", 22F, System.Drawing.FontStyle.Bold);
            this.lblTitulo.Location = new System.Drawing.Point(28, 20); this.lblTitulo.Text = "Historial laboral";
            this.cmbEmpleado.DropDownStyle = System.Windows.Forms.ComboBoxStyle.DropDownList; this.cmbEmpleado.Location = new System.Drawing.Point(34,  \
    82); this.cmbEmpleado.Size = new System.Drawing.Size(220, 23);
            this.cmbCampo.DropDownStyle = System.Windows.Forms.ComboBoxStyle.DropDownList; this.cmbCampo.Location = new System.Drawing.Point(266, 82);  \
    this.cmbCampo.Size = new System.Drawing.Size(150, 23);
            this.txtUsuario.Location = new System.Drawing.Point(428, 82); this.txtUsuario.Size = new System.Drawing.Size(130, 23);
            this.dtpDesde.Format = System.Windows.Forms.DateTimePickerFormat.Short; this.dtpDesde.Location = new System.Drawing.Point(570, 82);  \
    this.dtpDesde.ShowCheckBox = true; this.dtpDesde.Size = new System.Drawing.Size(130, 23); this.dtpDesde.Checked = false;
            this.dtpHasta.Format = System.Windows.Forms.DateTimePickerFormat.Short; this.dtpHasta.Location = new System.Drawing.Point(712, 82);  \
    this.dtpHasta.ShowCheckBox = true; this.dtpHasta.Size = new System.Drawing.Size(130, 23); this.dtpHasta.Checked = false;
            this.btnBuscar.BackColor = System.Drawing.Color.FromArgb(124, 58, 237); this.btnBuscar.FlatStyle = System.Windows.Forms.FlatStyle.Flat;  \
    this.btnBuscar.Location = new System.Drawing.Point(854, 78); this.btnBuscar.Size = new System.Drawing.Size(78, 31); this.btnBuscar.Text = "Buscar";  \
    this.btnBuscar.UseVisualStyleBackColor = false; this.btnBuscar.Click += new System.EventHandler(this.btnBuscar_Click);
            this.btnExcel.FlatStyle = System.Windows.Forms.FlatStyle.Flat; this.btnExcel.Location = new System.Drawing.Point(938, 78); this.btnExcel.Size  \
    = new System.Drawing.Size(64, 31); this.btnExcel.Text = "Excel"; this.btnExcel.Click += new System.EventHandler(this.btnExcel_Click);
            this.btnPdf.FlatStyle = System.Windows.Forms.FlatStyle.Flat; this.btnPdf.Location = new System.Drawing.Point(1008, 78); this.btnPdf.Size =  \
    new System.Drawing.Size(64, 31); this.btnPdf.Text = "PDF"; this.btnPdf.Click += new System.EventHandler(this.btnPdf_Click);
            this.dgvHistorial.AllowUserToAddRows = false; this.dgvHistorial.AllowUserToDeleteRows = false; this.dgvHistorial.AutoSizeColumnsMode =  \
    System.Windows.Forms.DataGridViewAutoSizeColumnsMode.Fill; this.dgvHistorial.BackgroundColor = System.Drawing.Color.FromArgb(18, 18, 27);  \
    this.dgvHistorial.Location = new System.Drawing.Point(34, 130); this.dgvHistorial.ReadOnly = true; this.dgvHistorial.RowHeadersVisible = false;  \
    this.dgvHistorial.SelectionMode = System.Windows.Forms.DataGridViewSelectionMode.FullRowSelect; this.dgvHistorial.Size = new  \
    System.Drawing.Size(1038, 500);
```

UI/FrmHistorialEmpleado.Designer.cs (continuacion)

```
            this.dgvHistorial.Columns.Add("Codigo", "Código"); this.dgvHistorial.Columns.Add("Empleado", "Empleado");  \
    this.dgvHistorial.Columns.Add("Campo", "Campo"); this.dgvHistorial.Columns.Add("Anterior", "Anterior"); this.dgvHistorial.Columns.Add("Nuevo",  \
    "Nuevo"); this.dgvHistorial.Columns.Add("Efectiva", "Fecha efectiva"); this.dgvHistorial.Columns.Add("Registro", "Registrado");  \
    this.dgvHistorial.Columns.Add("Usuario", "Usuario"); this.dgvHistorial.Columns.Add("Motivo", "Motivo"); this.dgvHistorial.Columns.Add("Estado",  \
    "Aplicación");
            this.lblResultado.AutoSize = true; this.lblResultado.Location = new System.Drawing.Point(34, 650); this.lblResultado.Text = "0 cambios  \
    encontrados";
            this.btnCerrar.FlatStyle = System.Windows.Forms.FlatStyle.Flat; this.btnCerrar.Location = new System.Drawing.Point(976, 644);  \
    this.btnCerrar.Size = new System.Drawing.Size(96, 32); this.btnCerrar.Text = "Cerrar"; this.btnCerrar.Click += new  \
    System.EventHandler(this.btnCerrar_Click);
            this.Controls.Add(this.lblTitulo); this.Controls.Add(this.cmbEmpleado); this.Controls.Add(this.cmbCampo); this.Controls.Add(this.txtUsuario);  \
    this.Controls.Add(this.dtpDesde); this.Controls.Add(this.dtpHasta); this.Controls.Add(this.btnBuscar); this.Controls.Add(this.btnExcel);  \
    this.Controls.Add(this.btnPdf); this.Controls.Add(this.dgvHistorial); this.Controls.Add(this.lblResultado); this.Controls.Add(this.btnCerrar);
            ((System.ComponentModel.ISupportInitialize)(this.dgvHistorial)).EndInit(); this.ResumeLayout(false); this.PerformLayout();
        }
    }
}
```

UI/FrmHistorialNomina.cs

```
using System;
using System.Collections.Generic;
using System.Drawing;
using System.Windows.Forms;
using SistemaGestionNomina.Models;

namespace SistemaGestionNomina.UI
{
    public partial class FrmHistorialNomina : Form
    {
        private DataGridView dgvHistorial;
        private Button btnCerrar;

        public FrmHistorialNomina(List<NominaVersion> versiones)
        {
            InitializeComponent();
            CargarHistorial(versiones);
        }

        private void InitializeComponent()
        {
            this.dgvHistorial = new DataGridView();
            this.btnCerrar = new Button();
            ((System.ComponentModel.ISupportInitialize)(this.dgvHistorial)).BeginInit();
            this.SuspendLayout();

            this.dgvHistorial.AllowUserToAddRows = false;
            this.dgvHistorial.AllowUserToDeleteRows = false;
            this.dgvHistorial.BackgroundColor = Color.FromArgb(24, 25, 34);
            this.dgvHistorial.BorderStyle = BorderStyle.None;
            this.dgvHistorial.CellBorderStyle = DataGridViewCellBorderStyle.SingleHorizontal;
            this.dgvHistorial.ColumnHeadersHeightSizeMode = DataGridViewColumnHeadersHeightSizeMode.AutoSize;
            this.dgvHistorial.Location = new Point(16, 16);
            this.dgvHistorial.Name = "dgvHistorial";
            this.dgvHistorial.ReadOnly = true;
            this.dgvHistorial.RowHeadersVisible = false;
            this.dgvHistorial.Size = new Size(600, 300);
            this.dgvHistorial.TabIndex = 0;
            this.dgvHistorial.AutoSizeColumnsMode = DataGridViewAutoSizeColumnsMode.Fill;
            this.dgvHistorial.EnableHeadersVisualStyles = false;

            this.dgvHistorial.DefaultCellStyle.BackColor = Color.FromArgb(13, 14, 20);
            this.dgvHistorial.DefaultCellStyle.ForeColor = Color.FromArgb(229, 228, 245);
            this.dgvHistorial.DefaultCellStyle.SelectionBackColor = Color.FromArgb(167, 139, 250);
            this.dgvHistorial.DefaultCellStyle.SelectionForeColor = Color.Black;
            this.dgvHistorial.DefaultCellStyle.Font = new Font("Segoe UI", 9F);

            this.dgvHistorial.ColumnHeadersDefaultCellStyle.BackColor = Color.FromArgb(17, 18, 26);
            this.dgvHistorial.ColumnHeadersDefaultCellStyle.ForeColor = Color.FromArgb(167, 139, 250);
            this.dgvHistorial.ColumnHeadersDefaultCellStyle.Font = new Font("Segoe UI", 9F, FontStyle.Bold);
            this.dgvHistorial.ColumnHeadersDefaultCellStyle.Alignment = DataGridViewContentAlignment.MiddleCenter;

            this.dgvHistorial.AlternatingRowsDefaultCellStyle.BackColor = Color.FromArgb(20, 21, 30);

            this.dgvHistorial.GridColor = Color.FromArgb(30, 31, 40);

            this.btnCerrar.BackColor = Color.Black;
            this.btnCerrar.FlatStyle = FlatStyle.Flat;
            this.btnCerrar.ForeColor = Color.FromArgb(229, 228, 245);
            this.btnCerrar.Location = new Point(520, 332);
```

UI/FrmHistorialNomina.cs (continuacion)

```
            this.btnCerrar.Size = new Size(96, 32);
            this.btnCerrar.Text = "Cerrar";
            this.btnCerrar.UseVisualStyleBackColor = false;
            this.btnCerrar.Click += new EventHandler(this.btnCerrar_Click);

            this.AutoScaleDimensions = new SizeF(7F, 15F);
            this.AutoScaleMode = AutoScaleMode.Font;
            this.BackColor = Color.FromArgb(13, 14, 20);
            this.ClientSize = new Size(634, 381);
            this.Controls.Add(this.dgvHistorial);
            this.Controls.Add(this.btnCerrar);
            this.Font = new Font("Segoe UI", 9F);
            this.ForeColor = Color.FromArgb(229, 228, 245);
            this.FormBorderStyle = FormBorderStyle.FixedDialog;
            this.MaximizeBox = false;
            this.MinimizeBox = false;
            this.StartPosition = FormStartPosition.CenterParent;
            this.Text = "Historial de versiones";
            ((System.ComponentModel.ISupportInitialize)(this.dgvHistorial)).EndInit();
            this.ResumeLayout(false);
        }

        private void CargarHistorial(List<NominaVersion> versiones)
        {
            dgvHistorial.Columns.Clear();
            dgvHistorial.Columns.Add("Version", "Versión");
            dgvHistorial.Columns.Add("Direccion", "Original ? Nueva");
            dgvHistorial.Columns.Add("Motivo", "Motivo");
            dgvHistorial.Columns.Add("Usuario", "Usuario");
            dgvHistorial.Columns.Add("Fecha", "Fecha");

            for (int i = 0; i < versiones.Count; i++)
            {
                NominaVersion v = versiones[i];
                string direccion = v.IdNominaOriginal.ToString() + " ? " +
                    (v.IdNominaNueva.HasValue ? v.IdNominaNueva.Value.ToString() : "Pendiente");
                dgvHistorial.Rows.Add(
                    v.IdVersion.ToString(),
                    direccion,
                    v.MotivoCambio,
                    v.UsuarioResponsable,
                    v.FechaCambio.ToString("yyyy-MM-dd HH:mm")
                );
            }
        }

        private void btnCerrar_Click(object sender, EventArgs e)
        {
            Close();
        }
    }
}
```

UI/FrmLogin.cs

```
using System;
using System.ComponentModel;
using System.Drawing;
using System.Drawing.Drawing2D;
using System.Windows.Forms;
using SistemaGestionNomina.Helpers;
using SistemaGestionNomina.Models;
using SistemaGestionNomina.Services;

namespace SistemaGestionNomina.UI
{
    public partial class FrmLogin : Form
    {
        private readonly AuthService authService = new AuthService();

        public Usuario AuthenticatedUser { get; private set; }

        public FrmLogin()
        {
            InitializeComponent();
            ControlStyleHelper.ApplyModernForm(this);
            string rememberedUsername = RememberedUsernameHelper.Load();
            if (!string.IsNullOrWhiteSpace(rememberedUsername))
            {
                txtUsuario.Text = rememberedUsername;
                chkRecordar.Checked = true;
                txtPassword.Focus();
            }
        }

        private void FrmLogin_Paint(object sender, PaintEventArgs e)
        {
            if (LicenseManager.UsageMode == LicenseUsageMode.Designtime)
            {
                return;
            }

            using (LinearGradientBrush brush = new LinearGradientBrush(
                ClientRectangle,
                Color.FromArgb(13, 14, 20),
                Color.FromArgb(24, 18, 45),
                LinearGradientMode.ForwardDiagonal))
            {
                e.Graphics.FillRectangle(brush, ClientRectangle);
            }

            using (Pen pen = new Pen(Color.FromArgb(24, 29, 42)))
            {
                for (int x = 0; x < Width; x += 34)
                {
                    e.Graphics.DrawLine(pen, x, 0, x, Height);
                }

                for (int y = 0; y < Height; y += 34)
                {
                    e.Graphics.DrawLine(pen, 0, y, Width, y);
                }
            }
        }

```

UI/FrmLogin.cs (continuacion)

```
        private void btnIngresar_Click(object sender, EventArgs e)
        {
            if (string.IsNullOrWhiteSpace(txtUsuario.Text) || string.IsNullOrWhiteSpace(txtPassword.Text))
            {
                lblMensaje.Text = "Ingrese usuario y contraseña.";
                lblMensaje.ForeColor = ThemeHelper.Error;
                return;
            }

            try
            {
                AuthenticationResult result = authService.Authenticate(txtUsuario.Text.Trim(), txtPassword.Text);
                if (!result.IsSuccess)
                {
                    lblMensaje.Text = result.Message;
                    lblMensaje.ForeColor = ThemeHelper.Error;
                    txtPassword.Clear();
                    txtPassword.Focus();
                    return;
                }

                if (chkRecordar.Checked)
                {
                    RememberedUsernameHelper.Save(result.User.NombreUsuario);
                }
                else
                {
                    RememberedUsernameHelper.Clear();
                }

                AuthenticatedUser = result.User;
                DialogResult = DialogResult.OK;
                Close();
            }
            catch (Exception)
            {
                MessageBox.Show("No se pudo iniciar sesión. Revise la conexión con la base de datos e intente nuevamente.",
                    "Login", MessageBoxButtons.OK, MessageBoxIcon.Error);
            }
        }

        private void foxBigLabel1_Click(object sender, EventArgs e)
        {

        }
    }
}
```

UI/FrmLogin.Designer.cs

```
namespace SistemaGestionNomina.UI
{
    partial class FrmLogin
    {
        private System.ComponentModel.IContainer components = null;
        private System.Windows.Forms.Panel panelLogin;
        private System.Windows.Forms.Label lblSubtitulo;
        private System.Windows.Forms.Label lblUsuario;
        private System.Windows.Forms.Label lblPassword;
        private System.Windows.Forms.TextBox txtUsuario;
        private System.Windows.Forms.TextBox txtPassword;
        private System.Windows.Forms.CheckBox chkRecordar;
        private System.Windows.Forms.Button btnIngresar;
        private System.Windows.Forms.Label lblMensaje;

        protected override void Dispose(bool disposing)
        {
            if (disposing && (components != null))
            {
                components.Dispose();
            }
            base.Dispose(disposing);
        }

        private void InitializeComponent()
        {
            this.panelLogin = new System.Windows.Forms.Panel();
            this.foxBigLabel1 = new ReaLTaiizor.Controls.FoxBigLabel();
            this.lblSubtitulo = new System.Windows.Forms.Label();
            this.lblUsuario = new System.Windows.Forms.Label();
            this.lblPassword = new System.Windows.Forms.Label();
            this.txtUsuario = new System.Windows.Forms.TextBox();
            this.txtPassword = new System.Windows.Forms.TextBox();
            this.chkRecordar = new System.Windows.Forms.CheckBox();
            this.btnIngresar = new System.Windows.Forms.Button();
            this.lblMensaje = new System.Windows.Forms.Label();
            this.panelLogin.SuspendLayout();
            this.SuspendLayout();
            //
            // panelLogin
            //
            this.panelLogin.Anchor = System.Windows.Forms.AnchorStyles.None;
            this.panelLogin.BackColor = System.Drawing.Color.Black;
            this.panelLogin.BorderStyle = System.Windows.Forms.BorderStyle.FixedSingle;
            this.panelLogin.Controls.Add(this.foxBigLabel1);
            this.panelLogin.Controls.Add(this.lblSubtitulo);
            this.panelLogin.Controls.Add(this.lblUsuario);
            this.panelLogin.Controls.Add(this.lblPassword);
            this.panelLogin.Controls.Add(this.txtUsuario);
            this.panelLogin.Controls.Add(this.txtPassword);
            this.panelLogin.Controls.Add(this.chkRecordar);
            this.panelLogin.Controls.Add(this.btnIngresar);
            this.panelLogin.Controls.Add(this.lblMensaje);
            this.panelLogin.Location = new System.Drawing.Point(275, 78);
            this.panelLogin.Name = "panelLogin";
            this.panelLogin.Size = new System.Drawing.Size(430, 520);
            this.panelLogin.TabIndex = 0;
            //
            // foxBigLabel1
            //
```

UI/FrmLogin.Designer.cs (continuacion)

```
            this.foxBigLabel1.BackColor = System.Drawing.Color.Transparent;
            this.foxBigLabel1.Font = new System.Drawing.Font("Segoe UI Semibold", 20F);
            this.foxBigLabel1.ForeColor = System.Drawing.Color.Indigo;
            this.foxBigLabel1.Line = ReaLTaiizor.Controls.FoxBigLabel.Direction.Top;
            this.foxBigLabel1.LineColor = System.Drawing.Color.Transparent;
            this.foxBigLabel1.Location = new System.Drawing.Point(135, 81);
            this.foxBigLabel1.Name = "foxBigLabel1";
            this.foxBigLabel1.Size = new System.Drawing.Size(165, 41);
            this.foxBigLabel1.TabIndex = 10;
            this.foxBigLabel1.Text = "NomiCore";
            this.foxBigLabel1.Click += new System.EventHandler(this.foxBigLabel1_Click);
            //
            // lblSubtitulo
            //
            this.lblSubtitulo.Font = new System.Drawing.Font("Segoe UI", 10F);
            this.lblSubtitulo.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(170)))), ((int)(((byte)(170)))), ((int)(((byte)(185)))));
            this.lblSubtitulo.Location = new System.Drawing.Point(40, 125);
            this.lblSubtitulo.Name = "lblSubtitulo";
            this.lblSubtitulo.Size = new System.Drawing.Size(350, 24);
            this.lblSubtitulo.TabIndex = 2;
            this.lblSubtitulo.Text = "Acceso corporativo seguro";
            this.lblSubtitulo.TextAlign = System.Drawing.ContentAlignment.MiddleCenter;
            //
            // lblUsuario
            //
            this.lblUsuario.AutoSize = true;
            this.lblUsuario.Font = new System.Drawing.Font("Segoe UI", 9F, System.Drawing.FontStyle.Bold);
            this.lblUsuario.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(229)))), ((int)(((byte)(228)))), ((int)(((byte)(245)))));
            this.lblUsuario.Location = new System.Drawing.Point(40, 224);
            this.lblUsuario.Name = "lblUsuario";
            this.lblUsuario.Size = new System.Drawing.Size(63, 20);
            this.lblUsuario.TabIndex = 3;
            this.lblUsuario.Text = "Usuario";
            //
            // lblPassword
            //
            this.lblPassword.AutoSize = true;
            this.lblPassword.Font = new System.Drawing.Font("Segoe UI", 9F, System.Drawing.FontStyle.Bold);
            this.lblPassword.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(229)))), ((int)(((byte)(228)))), ((int)(((byte)(245)))));
            this.lblPassword.Location = new System.Drawing.Point(40, 302);
            this.lblPassword.Name = "lblPassword";
            this.lblPassword.Size = new System.Drawing.Size(88, 20);
            this.lblPassword.TabIndex = 5;
            this.lblPassword.Text = "Contraseña";
            //
            // txtUsuario
            //
            this.txtUsuario.BackColor = System.Drawing.Color.FromArgb(((int)(((byte)(24)))), ((int)(((byte)(25)))), ((int)(((byte)(34)))));
            this.txtUsuario.BorderStyle = System.Windows.Forms.BorderStyle.FixedSingle;
            this.txtUsuario.Font = new System.Drawing.Font("Segoe UI", 10F);
            this.txtUsuario.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(229)))), ((int)(((byte)(228)))), ((int)(((byte)(245)))));
            this.txtUsuario.Location = new System.Drawing.Point(40, 252);
            this.txtUsuario.Name = "txtUsuario";
            this.txtUsuario.Size = new System.Drawing.Size(350, 30);
            this.txtUsuario.TabIndex = 4;
            //
            // txtPassword
            //
            this.txtPassword.BackColor = System.Drawing.Color.FromArgb(((int)(((byte)(24)))), ((int)(((byte)(25)))), ((int)(((byte)(34)))));
            this.txtPassword.BorderStyle = System.Windows.Forms.BorderStyle.FixedSingle;
```

UI/FrmLogin.Designer.cs (continuacion)

```
            this.txtPassword.Font = new System.Drawing.Font("Segoe UI", 10F);
            this.txtPassword.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(229)))), ((int)(((byte)(228)))), ((int)(((byte)(245)))));
            this.txtPassword.Location = new System.Drawing.Point(40, 330);
            this.txtPassword.Name = "txtPassword";
            this.txtPassword.Size = new System.Drawing.Size(350, 30);
            this.txtPassword.TabIndex = 6;
            this.txtPassword.UseSystemPasswordChar = true;
            //
            // chkRecordar
            //
            this.chkRecordar.AutoSize = true;
            this.chkRecordar.BackColor = System.Drawing.Color.Transparent;
            this.chkRecordar.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(170)))), ((int)(((byte)(170)))), ((int)(((byte)(185)))));
            this.chkRecordar.Location = new System.Drawing.Point(40, 378);
            this.chkRecordar.Name = "chkRecordar";
            this.chkRecordar.Size = new System.Drawing.Size(145, 24);
            this.chkRecordar.TabIndex = 7;
            this.chkRecordar.Text = "Recordar usuario";
            this.chkRecordar.UseVisualStyleBackColor = false;
            //
            // btnIngresar
            //
            this.btnIngresar.BackColor = System.Drawing.Color.FromArgb(((int)(((byte)(167)))), ((int)(((byte)(139)))), ((int)(((byte)(250)))));
            this.btnIngresar.FlatAppearance.BorderSize = 0;
            this.btnIngresar.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
            this.btnIngresar.Font = new System.Drawing.Font("Segoe UI", 10F, System.Drawing.FontStyle.Bold);
            this.btnIngresar.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(48)))), ((int)(((byte)(25)))), ((int)(((byte)(92)))));
            this.btnIngresar.Location = new System.Drawing.Point(40, 426);
            this.btnIngresar.Name = "btnIngresar";
            this.btnIngresar.Size = new System.Drawing.Size(350, 42);
            this.btnIngresar.TabIndex = 8;
            this.btnIngresar.Text = "Iniciar sesión";
            this.btnIngresar.UseVisualStyleBackColor = false;
            this.btnIngresar.Click += new System.EventHandler(this.btnIngresar_Click);
            //
            // lblMensaje
            //
            this.lblMensaje.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(170)))), ((int)(((byte)(170)))), ((int)(((byte)(185)))));
            this.lblMensaje.Location = new System.Drawing.Point(40, 476);
            this.lblMensaje.Name = "lblMensaje";
            this.lblMensaje.Size = new System.Drawing.Size(350, 24);
            this.lblMensaje.TabIndex = 9;
            this.lblMensaje.Text = "Ingrese sus credenciales para continuar.";
            this.lblMensaje.TextAlign = System.Drawing.ContentAlignment.MiddleCenter;
            //
            // FrmLogin
            //
            this.AcceptButton = this.btnIngresar;
            this.AutoScaleDimensions = new System.Drawing.SizeF(8F, 20F);
            this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font;
            this.BackColor = System.Drawing.Color.FromArgb(((int)(((byte)(13)))), ((int)(((byte)(14)))), ((int)(((byte)(20)))));
            this.ClientSize = new System.Drawing.Size(980, 700);
            this.Controls.Add(this.panelLogin);
            this.Font = new System.Drawing.Font("Segoe UI", 9F);
            this.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(229)))), ((int)(((byte)(228)))), ((int)(((byte)(245)))));
            this.FormBorderStyle = System.Windows.Forms.FormBorderStyle.FixedSingle;
            this.MaximizeBox = false;
            this.MinimumSize = new System.Drawing.Size(900, 640);
            this.Name = "FrmLogin";
            this.StartPosition = System.Windows.Forms.FormStartPosition.CenterScreen;
```

UI/FrmLogin.Designer.cs (continuacion)

```
            this.Text = "ProyFinal_LPI_Eq01_NomiCore - Login";
            this.Paint += new System.Windows.Forms.PaintEventHandler(this.FrmLogin_Paint);
            this.panelLogin.ResumeLayout(false);
            this.panelLogin.PerformLayout();
            this.ResumeLayout(false);

        }

        private ReaLTaiizor.Controls.FoxBigLabel foxBigLabel1;
    }
}
```

UI/FrmMain.cs

```
using System;
using System.ComponentModel;
using System.Windows.Forms;
using FontAwesome.Sharp;
using SistemaGestionNomina.Helpers;
using SistemaGestionNomina.Models;
using SistemaGestionNomina.Security;
using SistemaGestionNomina.Services;

namespace SistemaGestionNomina.UI
{
    public partial class FrmMain : Form
    {
        private Usuario currentUser;
        private Form activeChild;
        private readonly AuthorizationService authorizationService = new AuthorizationService();
        private readonly AuditTrailService auditTrailService = new AuditTrailService();

        public bool LogoutRequested { get; private set; }

        public FrmMain()
        {
            InitializeComponent();
            ControlStyleHelper.ApplyModernForm(this);
        }

        public FrmMain(Usuario usuario)
            : this()
        {
            if (usuario == null) throw new ArgumentNullException("usuario");
            currentUser = usuario;
            if (!SessionContext.IsAuthenticated) SessionContext.Begin(usuario);
            ConfigureRuntime();
        }

        public void OpenChildForm(Form childForm)
        {
            if (activeChild != null)
            {
                activeChild.Close();
                activeChild.Dispose();
            }

            activeChild = childForm;
            childForm.TopLevel = false;
            childForm.FormBorderStyle = FormBorderStyle.None;
            childForm.Dock = DockStyle.Fill;
            panelContent.Controls.Clear();
            panelContent.Controls.Add(childForm);
            childForm.Show();
        }

        private void ConfigureRuntime()
        {
            lblUsuarioActual.Text = currentUser.NombreUsuario + " | " + currentUser.Rol;
            ApplyPermissions();
            if (!IsDesignTime())
            {
                OpenInitialModule();
            }
```

UI/FrmMain.cs (continuacion)

```
        }

        private void ApplyPermissions()
        {
            btnPortal.Visible = string.Equals(SessionContext.Role, Roles.Trabajador, StringComparison.OrdinalIgnoreCase) &&
                authorizationService.HasPermission(Permissions.PortalView);
            btnDashboard.Visible = authorizationService.HasPermission(Permissions.DashboardView);
            btnEmpleados.Visible = authorizationService.HasPermission(Permissions.EmployeesView);
            btnAsistencia.Visible = authorizationService.HasPermission(Permissions.AttendanceView);
            btnNomina.Visible = authorizationService.HasPermission(Permissions.PayrollView);
            btnComprobantes.Visible = authorizationService.HasPermission(Permissions.PayslipsView);
            btnReportes.Visible = authorizationService.HasPermission(Permissions.ReportsView);
            btnConfiguracion.Visible = authorizationService.HasPermission(Permissions.ConfigurationView);
            btnAuditoria.Visible = authorizationService.HasPermission(Permissions.AuditView);
            btnAusencias.Visible = authorizationService.HasPermission(Permissions.AbsencesView);
        }

        private void OpenInitialModule()
        {
            if (btnPortal.Visible) OpenPortal();
            else if (btnDashboard.Visible) OpenDashboard();
            else if (btnEmpleados.Visible) OpenAuthorized(Permissions.EmployeesView, btnEmpleados, new FrmEmpleados());
            else if (btnNomina.Visible) OpenAuthorized(Permissions.PayrollView, btnNomina, new FrmNomina());
            else if (btnAsistencia.Visible) OpenAuthorized(Permissions.AttendanceView, btnAsistencia, new FrmAsistencia());
            else
            {
                SetActiveButton(btnAcercaDe);
                OpenChildForm(new FrmAcercaDe());
            }
        }

        private static bool IsDesignTime()
        {
            return LicenseManager.UsageMode == LicenseUsageMode.Designtime;
        }

        private void btnDashboard_Click(object sender, EventArgs e)
        {
            SetActiveButton(btnDashboard);
            OpenDashboard();
        }

        private void btnPortal_Click(object sender, EventArgs e)
        {
            OpenPortal();
        }

        private void btnEmpleados_Click(object sender, EventArgs e)
        {
            OpenAuthorized(Permissions.EmployeesView, btnEmpleados, new FrmEmpleados());
        }

        private void btnAsistencia_Click(object sender, EventArgs e)
        {
            OpenAuthorized(Permissions.AttendanceView, btnAsistencia, new FrmAsistencia());
        }

        private void btnNomina_Click(object sender, EventArgs e)
        {
            OpenAuthorized(Permissions.PayrollView, btnNomina, new FrmNomina());
```

UI/FrmMain.cs (continuacion)

```
        }

        private void btnComprobantes_Click(object sender, EventArgs e)
        {
            OpenAuthorized(Permissions.PayslipsView, btnComprobantes, new FrmComprobantes());
        }

        private void btnReportes_Click(object sender, EventArgs e)
        {
            OpenAuthorized(Permissions.ReportsView, btnReportes, new FrmReportes());
        }

        private void btnConfiguracion_Click(object sender, EventArgs e)
        {
            OpenAuthorized(Permissions.ConfigurationView, btnConfiguracion, new FrmConfiguracion());
        }

        private void btnAcercaDe_Click(object sender, EventArgs e)
        {
            SetActiveButton(btnAcercaDe);
            OpenChildForm(new FrmAcercaDe());
        }

        private void btnCerrarSesion_Click(object sender, EventArgs e)
        {
            auditTrailService.RegistrarAccion("Seguridad", "Cerrar sesión", string.Empty);
            SessionContext.Clear();
            LogoutRequested = true;
            Close();
        }

        private void btnSalir_Click(object sender, EventArgs e)
        {
            DialogResult result = MessageBox.Show("¿Desea salir de la aplicación?",
                "Salir", MessageBoxButtons.YesNo, MessageBoxIcon.Question);
            if (result == DialogResult.Yes)
            {
                auditTrailService.RegistrarAccion("Seguridad", "Salir de la aplicación", string.Empty);
                SessionContext.Clear();
                Application.Exit();
            }
        }

        private void btnAuditoria_Click(object sender, EventArgs e)
        {
            OpenAuthorized(Permissions.AuditView, btnAuditoria, new FrmAuditoria());
        }

        private void btnAusencias_Click(object sender, EventArgs e)
        {
            OpenAuthorized(Permissions.AbsencesView, btnAusencias, new FrmSolicitudesAusencia());
        }

        private void OpenDashboard()
        {
            authorizationService.DemandPermission(Permissions.DashboardView);
            SetActiveButton(btnDashboard);
            OpenChildForm(new FrmDashboard(
                delegate { OpenChildForm(new FrmEmpleados()); },
                delegate { OpenChildForm(new FrmNomina()); }));
```

UI/FrmMain.cs (continuacion)

```
        }

        private void OpenPortal()
        {
            authorizationService.DemandPermission(Permissions.PortalView);
            SetActiveButton(btnPortal);
            OpenChildForm(new FrmPortalTrabajador(
                delegate { OpenPortalChild(Permissions.OwnProfileView, new FrmMiPerfil()); },
                delegate { OpenPortalChild(Permissions.OwnAttendanceView, new FrmMisAsistencias()); },
                delegate { OpenPortalChild(Permissions.OwnPayslipsView, new FrmMisComprobantes()); },
                delegate { OpenPortalChild(Permissions.OwnPasswordChange, new FrmCambiarPassword()); },
                delegate { OpenPortalChild(Permissions.OwnAbsencesView, new FrmSolicitudesAusencia()); }));
        }

        private void OpenPortalChild(string permission, Form form)
        {
            try
            {
                authorizationService.DemandPermission(permission);
                SetActiveButton(btnPortal);
                OpenChildForm(form);
            }
            catch (UnauthorizedAccessException ex)
            {
                form.Dispose();
                MessageBox.Show(ex.Message, "Acceso restringido", MessageBoxButtons.OK, MessageBoxIcon.Warning);
            }
        }

        private void OpenAuthorized(string permission, IconButton button, Form form)
        {
            try
            {
                authorizationService.DemandPermission(permission);
                SetActiveButton(button);
                OpenChildForm(form);
            }
            catch (UnauthorizedAccessException ex)
            {
                form.Dispose();
                MessageBox.Show(ex.Message, "Acceso restringido", MessageBoxButtons.OK, MessageBoxIcon.Warning);
            }
        }

        private void SetActiveButton(IconButton button)
        {
            ControlStyleHelper.SetActiveSidebarButton(panelSidebar, button);
        }
    }
}
```

UI/FrmMain.Designer.cs

```
namespace SistemaGestionNomina.UI
{
    partial class FrmMain
    {
        private System.ComponentModel.IContainer components = null;
        public System.Windows.Forms.Panel panelSidebar;
        private System.Windows.Forms.Panel panelTopbar;
        public System.Windows.Forms.Panel panelContent;
        private System.Windows.Forms.Panel panelBrand;
        private System.Windows.Forms.FlowLayoutPanel flowMenu;
        private System.Windows.Forms.FlowLayoutPanel flowAccount;
        private System.Windows.Forms.Panel panelAccountSeparator;
        private System.Windows.Forms.Label lblMarca;
        private System.Windows.Forms.Label lblSubmarca;
        private System.Windows.Forms.Label lblUsuarioActual;
        public FontAwesome.Sharp.IconButton btnPortal;
        public FontAwesome.Sharp.IconButton btnDashboard;
        public FontAwesome.Sharp.IconButton btnEmpleados;
        public FontAwesome.Sharp.IconButton btnAsistencia;
        public FontAwesome.Sharp.IconButton btnNomina;
        public FontAwesome.Sharp.IconButton btnComprobantes;
        public FontAwesome.Sharp.IconButton btnReportes;
        public FontAwesome.Sharp.IconButton btnConfiguracion;
        public FontAwesome.Sharp.IconButton btnAcercaDe;
        public FontAwesome.Sharp.IconButton btnAuditoria;
        public FontAwesome.Sharp.IconButton btnAusencias;
        public FontAwesome.Sharp.IconButton btnSalir;
        public FontAwesome.Sharp.IconButton btnCerrarSesion;

        protected override void Dispose(bool disposing)
        {
            if (disposing && (components != null))
            {
                components.Dispose();
            }
            base.Dispose(disposing);
        }

        private void InitializeComponent()
        {
            this.panelSidebar = new System.Windows.Forms.Panel();
            this.panelBrand = new System.Windows.Forms.Panel();
            this.lblMarca = new System.Windows.Forms.Label();
            this.lblSubmarca = new System.Windows.Forms.Label();
            this.flowMenu = new System.Windows.Forms.FlowLayoutPanel();
            this.btnPortal = new FontAwesome.Sharp.IconButton();
            this.btnDashboard = new FontAwesome.Sharp.IconButton();
            this.btnEmpleados = new FontAwesome.Sharp.IconButton();
            this.btnAsistencia = new FontAwesome.Sharp.IconButton();
            this.btnNomina = new FontAwesome.Sharp.IconButton();
            this.btnComprobantes = new FontAwesome.Sharp.IconButton();
            this.btnReportes = new FontAwesome.Sharp.IconButton();
            this.btnConfiguracion = new FontAwesome.Sharp.IconButton();
            this.btnAcercaDe = new FontAwesome.Sharp.IconButton();
            this.btnAuditoria = new FontAwesome.Sharp.IconButton();
            this.btnAusencias = new FontAwesome.Sharp.IconButton();
            this.btnSalir = new FontAwesome.Sharp.IconButton();
            this.btnCerrarSesion = new FontAwesome.Sharp.IconButton();
            this.flowAccount = new System.Windows.Forms.FlowLayoutPanel();
            this.panelAccountSeparator = new System.Windows.Forms.Panel();
```

UI/FrmMain.Designer.cs (continuacion)

```
            this.panelTopbar = new System.Windows.Forms.Panel();
            this.lblUsuarioActual = new System.Windows.Forms.Label();
            this.panelContent = new System.Windows.Forms.Panel();
            this.panelSidebar.SuspendLayout();
            this.panelBrand.SuspendLayout();
            this.flowMenu.SuspendLayout();
            this.flowAccount.SuspendLayout();
            this.panelTopbar.SuspendLayout();
            this.SuspendLayout();
            //
            // panelSidebar
            //
            this.panelSidebar.BackColor = System.Drawing.Color.FromArgb(((int)(((byte)(11)))), ((int)(((byte)(15)))), ((int)(((byte)(25)))));
            this.panelSidebar.Controls.Add(this.flowMenu);
            this.panelSidebar.Controls.Add(this.flowAccount);
            this.panelSidebar.Controls.Add(this.panelBrand);
            this.panelSidebar.Dock = System.Windows.Forms.DockStyle.Left;
            this.panelSidebar.Location = new System.Drawing.Point(0, 0);
            this.panelSidebar.Name = "panelSidebar";
            this.panelSidebar.Size = new System.Drawing.Size(260, 820);
            this.panelSidebar.TabIndex = 0;
            //
            // panelBrand
            //
            this.panelBrand.BackColor = System.Drawing.Color.FromArgb(((int)(((byte)(11)))), ((int)(((byte)(15)))), ((int)(((byte)(25)))));
            this.panelBrand.Controls.Add(this.lblMarca);
            this.panelBrand.Controls.Add(this.lblSubmarca);
            this.panelBrand.Dock = System.Windows.Forms.DockStyle.Top;
            this.panelBrand.Location = new System.Drawing.Point(0, 0);
            this.panelBrand.Name = "panelBrand";
            this.panelBrand.Size = new System.Drawing.Size(260, 112);
            this.panelBrand.TabIndex = 0;
            //
            // lblMarca
            //
            this.lblMarca.Font = new System.Drawing.Font("Segoe UI", 16F, System.Drawing.FontStyle.Bold);
            this.lblMarca.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(243)))), ((int)(((byte)(244)))), ((int)(((byte)(246)))));
            this.lblMarca.Location = new System.Drawing.Point(24, 28);
            this.lblMarca.Name = "lblMarca";
            this.lblMarca.Size = new System.Drawing.Size(210, 32);
            this.lblMarca.TabIndex = 0;
            this.lblMarca.Text = "NomiCore";
            //
            // lblSubmarca
            //
            this.lblSubmarca.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(170)))), ((int)(((byte)(170)))), ((int)(((byte)(185)))));
            this.lblSubmarca.Location = new System.Drawing.Point(26, 63);
            this.lblSubmarca.Name = "lblSubmarca";
            this.lblSubmarca.Size = new System.Drawing.Size(210, 22);
            this.lblSubmarca.TabIndex = 1;
            this.lblSubmarca.Text = "Sistema de nómina";
            //
            // flowMenu
            //
            this.flowMenu.AutoScroll = true;
            this.flowMenu.BackColor = System.Drawing.Color.FromArgb(((int)(((byte)(11)))), ((int)(((byte)(15)))), ((int)(((byte)(25)))));
            this.flowMenu.Controls.Add(this.btnPortal);
            this.flowMenu.Controls.Add(this.btnDashboard);
            this.flowMenu.Controls.Add(this.btnEmpleados);
            this.flowMenu.Controls.Add(this.btnAsistencia);
```

UI/FrmMain.Designer.cs (continuacion)

```
            this.flowMenu.Controls.Add(this.btnNomina);
            this.flowMenu.Controls.Add(this.btnComprobantes);
            this.flowMenu.Controls.Add(this.btnReportes);
            this.flowMenu.Controls.Add(this.btnAusencias);
            this.flowMenu.Controls.Add(this.btnConfiguracion);
            this.flowMenu.Controls.Add(this.btnAuditoria);
            this.flowMenu.Controls.Add(this.btnAcercaDe);
            this.flowMenu.Dock = System.Windows.Forms.DockStyle.Fill;
            this.flowMenu.FlowDirection = System.Windows.Forms.FlowDirection.TopDown;
            this.flowMenu.Location = new System.Drawing.Point(0, 112);
            this.flowMenu.Name = "flowMenu";
            this.flowMenu.Padding = new System.Windows.Forms.Padding(12, 20, 12, 12);
            this.flowMenu.Size = new System.Drawing.Size(260, 592);
            this.flowMenu.TabIndex = 1;
            this.flowMenu.WrapContents = false;
            //
            // btnPortal
            //
            this.btnPortal.FlatAppearance.BorderSize = 0;
            this.btnPortal.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
            this.btnPortal.Font = new System.Drawing.Font("Segoe UI", 10F, System.Drawing.FontStyle.Bold);
            this.btnPortal.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(229)))), ((int)(((byte)(228)))), ((int)(((byte)(245)))));
            this.btnPortal.IconChar = FontAwesome.Sharp.IconChar.User;
            this.btnPortal.IconColor = System.Drawing.Color.FromArgb(((int)(((byte)(167)))), ((int)(((byte)(139)))), ((int)(((byte)(250)))));
            this.btnPortal.IconFont = FontAwesome.Sharp.IconFont.Auto;
            this.btnPortal.IconSize = 20;
            this.btnPortal.ImageAlign = System.Drawing.ContentAlignment.MiddleLeft;
            this.btnPortal.Margin = new System.Windows.Forms.Padding(0, 0, 0, 8);
            this.btnPortal.Name = "btnPortal";
            this.btnPortal.Padding = new System.Windows.Forms.Padding(14, 0, 0, 0);
            this.btnPortal.Size = new System.Drawing.Size(228, 44);
            this.btnPortal.TabIndex = 2;
            this.btnPortal.Text = "  Mi portal";
            this.btnPortal.TextAlign = System.Drawing.ContentAlignment.MiddleLeft;
            this.btnPortal.TextImageRelation = System.Windows.Forms.TextImageRelation.ImageBeforeText;
            this.btnPortal.UseVisualStyleBackColor = true;
            this.btnPortal.Visible = false;
            this.btnPortal.Click += new System.EventHandler(this.btnPortal_Click);
            //
            // btnDashboard
            //
            this.btnDashboard.FlatAppearance.BorderSize = 0;
            this.btnDashboard.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
            this.btnDashboard.Font = new System.Drawing.Font("Segoe UI", 10F, System.Drawing.FontStyle.Bold);
            this.btnDashboard.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(229)))), ((int)(((byte)(228)))), ((int)(((byte)(245)))));
            this.btnDashboard.IconChar = FontAwesome.Sharp.IconChar.BorderAll;
            this.btnDashboard.IconColor = System.Drawing.Color.FromArgb(((int)(((byte)(167)))), ((int)(((byte)(139)))), ((int)(((byte)(250)))));
            this.btnDashboard.IconFont = FontAwesome.Sharp.IconFont.Auto;
            this.btnDashboard.IconSize = 20;
            this.btnDashboard.ImageAlign = System.Drawing.ContentAlignment.MiddleLeft;
            this.btnDashboard.Margin = new System.Windows.Forms.Padding(0, 0, 0, 8);
            this.btnDashboard.Name = "btnDashboard";
            this.btnDashboard.Padding = new System.Windows.Forms.Padding(14, 0, 0, 0);
            this.btnDashboard.Size = new System.Drawing.Size(228, 44);
            this.btnDashboard.TabIndex = 2;
            this.btnDashboard.Text = "  Dashboard";
            this.btnDashboard.TextAlign = System.Drawing.ContentAlignment.MiddleLeft;
            this.btnDashboard.TextImageRelation = System.Windows.Forms.TextImageRelation.ImageBeforeText;
            this.btnDashboard.UseVisualStyleBackColor = true;
            this.btnDashboard.Click += new System.EventHandler(this.btnDashboard_Click);
```

UI/FrmMain.Designer.cs (continuacion)

```
            //
            // btnEmpleados
            //
            this.btnEmpleados.FlatAppearance.BorderSize = 0;
            this.btnEmpleados.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
            this.btnEmpleados.Font = new System.Drawing.Font("Segoe UI", 10F, System.Drawing.FontStyle.Bold);
            this.btnEmpleados.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(170)))), ((int)(((byte)(170)))), ((int)(((byte)(185)))));
            this.btnEmpleados.IconChar = FontAwesome.Sharp.IconChar.Users;
            this.btnEmpleados.IconColor = System.Drawing.Color.FromArgb(((int)(((byte)(170)))), ((int)(((byte)(170)))), ((int)(((byte)(185)))));
            this.btnEmpleados.IconFont = FontAwesome.Sharp.IconFont.Auto;
            this.btnEmpleados.IconSize = 20;
            this.btnEmpleados.ImageAlign = System.Drawing.ContentAlignment.MiddleLeft;
            this.btnEmpleados.Margin = new System.Windows.Forms.Padding(0, 0, 0, 8);
            this.btnEmpleados.Name = "btnEmpleados";
            this.btnEmpleados.Padding = new System.Windows.Forms.Padding(14, 0, 0, 0);
            this.btnEmpleados.Size = new System.Drawing.Size(228, 44);
            this.btnEmpleados.TabIndex = 3;
            this.btnEmpleados.Text = "  Empleados";
            this.btnEmpleados.TextAlign = System.Drawing.ContentAlignment.MiddleLeft;
            this.btnEmpleados.TextImageRelation = System.Windows.Forms.TextImageRelation.ImageBeforeText;
            this.btnEmpleados.UseVisualStyleBackColor = true;
            this.btnEmpleados.Click += new System.EventHandler(this.btnEmpleados_Click);
            //
            // btnAsistencia
            //
            this.btnAsistencia.FlatAppearance.BorderSize = 0;
            this.btnAsistencia.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
            this.btnAsistencia.Font = new System.Drawing.Font("Segoe UI", 10F, System.Drawing.FontStyle.Bold);
            this.btnAsistencia.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(170)))), ((int)(((byte)(170)))), ((int)(((byte)(185)))));
            this.btnAsistencia.IconChar = FontAwesome.Sharp.IconChar.CalendarCheck;
            this.btnAsistencia.IconColor = System.Drawing.Color.FromArgb(((int)(((byte)(170)))), ((int)(((byte)(170)))), ((int)(((byte)(185)))));
            this.btnAsistencia.IconFont = FontAwesome.Sharp.IconFont.Auto;
            this.btnAsistencia.IconSize = 20;
            this.btnAsistencia.ImageAlign = System.Drawing.ContentAlignment.MiddleLeft;
            this.btnAsistencia.Margin = new System.Windows.Forms.Padding(0, 0, 0, 8);
            this.btnAsistencia.Name = "btnAsistencia";
            this.btnAsistencia.Padding = new System.Windows.Forms.Padding(14, 0, 0, 0);
            this.btnAsistencia.Size = new System.Drawing.Size(228, 44);
            this.btnAsistencia.TabIndex = 4;
            this.btnAsistencia.Text = "  Asistencia";
            this.btnAsistencia.TextAlign = System.Drawing.ContentAlignment.MiddleLeft;
            this.btnAsistencia.TextImageRelation = System.Windows.Forms.TextImageRelation.ImageBeforeText;
            this.btnAsistencia.UseVisualStyleBackColor = true;
            this.btnAsistencia.Click += new System.EventHandler(this.btnAsistencia_Click);
            //
            // btnNomina
            //
            this.btnNomina.FlatAppearance.BorderSize = 0;
            this.btnNomina.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
            this.btnNomina.Font = new System.Drawing.Font("Segoe UI", 10F, System.Drawing.FontStyle.Bold);
            this.btnNomina.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(170)))), ((int)(((byte)(170)))), ((int)(((byte)(185)))));
            this.btnNomina.IconChar = FontAwesome.Sharp.IconChar.MoneyCheckDollar;
            this.btnNomina.IconColor = System.Drawing.Color.FromArgb(((int)(((byte)(170)))), ((int)(((byte)(170)))), ((int)(((byte)(185)))));
            this.btnNomina.IconFont = FontAwesome.Sharp.IconFont.Auto;
            this.btnNomina.IconSize = 20;
            this.btnNomina.ImageAlign = System.Drawing.ContentAlignment.MiddleLeft;
            this.btnNomina.Margin = new System.Windows.Forms.Padding(0, 0, 0, 8);
            this.btnNomina.Name = "btnNomina";
            this.btnNomina.Padding = new System.Windows.Forms.Padding(14, 0, 0, 0);
            this.btnNomina.Size = new System.Drawing.Size(228, 44);
```

UI/FrmMain.Designer.cs (continuacion)

```
            this.btnNomina.TabIndex = 5;
            this.btnNomina.Text = "  Nómina";
            this.btnNomina.TextAlign = System.Drawing.ContentAlignment.MiddleLeft;
            this.btnNomina.TextImageRelation = System.Windows.Forms.TextImageRelation.ImageBeforeText;
            this.btnNomina.UseVisualStyleBackColor = true;
            this.btnNomina.Click += new System.EventHandler(this.btnNomina_Click);
            //
            // btnComprobantes
            //
            this.btnComprobantes.FlatAppearance.BorderSize = 0;
            this.btnComprobantes.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
            this.btnComprobantes.Font = new System.Drawing.Font("Segoe UI", 10F, System.Drawing.FontStyle.Bold);
            this.btnComprobantes.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(170)))), ((int)(((byte)(170)))), ((int)(((byte)(185)))));
            this.btnComprobantes.IconChar = FontAwesome.Sharp.IconChar.Receipt;
            this.btnComprobantes.IconColor = System.Drawing.Color.FromArgb(((int)(((byte)(170)))), ((int)(((byte)(170)))), ((int)(((byte)(185)))));
            this.btnComprobantes.IconFont = FontAwesome.Sharp.IconFont.Auto;
            this.btnComprobantes.IconSize = 20;
            this.btnComprobantes.ImageAlign = System.Drawing.ContentAlignment.MiddleLeft;
            this.btnComprobantes.Margin = new System.Windows.Forms.Padding(0, 0, 0, 8);
            this.btnComprobantes.Name = "btnComprobantes";
            this.btnComprobantes.Padding = new System.Windows.Forms.Padding(14, 0, 0, 0);
            this.btnComprobantes.Size = new System.Drawing.Size(228, 44);
            this.btnComprobantes.TabIndex = 6;
            this.btnComprobantes.Text = "  Comprobantes";
            this.btnComprobantes.TextAlign = System.Drawing.ContentAlignment.MiddleLeft;
            this.btnComprobantes.TextImageRelation = System.Windows.Forms.TextImageRelation.ImageBeforeText;
            this.btnComprobantes.UseVisualStyleBackColor = true;
            this.btnComprobantes.Click += new System.EventHandler(this.btnComprobantes_Click);
            //
            // btnReportes
            //
            this.btnReportes.FlatAppearance.BorderSize = 0;
            this.btnReportes.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
            this.btnReportes.Font = new System.Drawing.Font("Segoe UI", 10F, System.Drawing.FontStyle.Bold);
            this.btnReportes.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(170)))), ((int)(((byte)(170)))), ((int)(((byte)(185)))));
            this.btnReportes.IconChar = FontAwesome.Sharp.IconChar.ChartSimple;
            this.btnReportes.IconColor = System.Drawing.Color.FromArgb(((int)(((byte)(170)))), ((int)(((byte)(170)))), ((int)(((byte)(185)))));
            this.btnReportes.IconFont = FontAwesome.Sharp.IconFont.Auto;
            this.btnReportes.IconSize = 20;
            this.btnReportes.ImageAlign = System.Drawing.ContentAlignment.MiddleLeft;
            this.btnReportes.Margin = new System.Windows.Forms.Padding(0, 0, 0, 8);
            this.btnReportes.Name = "btnReportes";
            this.btnReportes.Padding = new System.Windows.Forms.Padding(14, 0, 0, 0);
            this.btnReportes.Size = new System.Drawing.Size(228, 44);
            this.btnReportes.TabIndex = 7;
            this.btnReportes.Text = "  Reportes";
            this.btnReportes.TextAlign = System.Drawing.ContentAlignment.MiddleLeft;
            this.btnReportes.TextImageRelation = System.Windows.Forms.TextImageRelation.ImageBeforeText;
            this.btnReportes.UseVisualStyleBackColor = true;
            this.btnReportes.Click += new System.EventHandler(this.btnReportes_Click);
            //
            // btnConfiguracion
            //
            this.btnConfiguracion.FlatAppearance.BorderSize = 0;
            this.btnConfiguracion.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
            this.btnConfiguracion.Font = new System.Drawing.Font("Segoe UI", 10F, System.Drawing.FontStyle.Bold);
            this.btnConfiguracion.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(170)))), ((int)(((byte)(170)))), ((int)(((byte)(185)))));
            this.btnConfiguracion.IconChar = FontAwesome.Sharp.IconChar.Cog;
            this.btnConfiguracion.IconColor = System.Drawing.Color.FromArgb(((int)(((byte)(170)))), ((int)(((byte)(170)))), ((int)(((byte)(185)))));
            this.btnConfiguracion.IconFont = FontAwesome.Sharp.IconFont.Auto;
```

UI/FrmMain.Designer.cs (continuacion)

```
            this.btnConfiguracion.IconSize = 20;
            this.btnConfiguracion.ImageAlign = System.Drawing.ContentAlignment.MiddleLeft;
            this.btnConfiguracion.Margin = new System.Windows.Forms.Padding(0, 0, 0, 8);
            this.btnConfiguracion.Name = "btnConfiguracion";
            this.btnConfiguracion.Padding = new System.Windows.Forms.Padding(14, 0, 0, 0);
            this.btnConfiguracion.Size = new System.Drawing.Size(228, 44);
            this.btnConfiguracion.TabIndex = 8;
            this.btnConfiguracion.Text = "  Configuración";
            this.btnConfiguracion.TextAlign = System.Drawing.ContentAlignment.MiddleLeft;
            this.btnConfiguracion.TextImageRelation = System.Windows.Forms.TextImageRelation.ImageBeforeText;
            this.btnConfiguracion.UseVisualStyleBackColor = true;
            this.btnConfiguracion.Click += new System.EventHandler(this.btnConfiguracion_Click);
            //
            // btnAcercaDe
            //
            this.btnAcercaDe.FlatAppearance.BorderSize = 0;
            this.btnAcercaDe.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
            this.btnAcercaDe.Font = new System.Drawing.Font("Segoe UI", 10F, System.Drawing.FontStyle.Bold);
            this.btnAcercaDe.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(170)))), ((int)(((byte)(170)))), ((int)(((byte)(185)))));
            this.btnAcercaDe.IconChar = FontAwesome.Sharp.IconChar.CircleInfo;
            this.btnAcercaDe.IconColor = System.Drawing.Color.FromArgb(((int)(((byte)(170)))), ((int)(((byte)(170)))), ((int)(((byte)(185)))));
            this.btnAcercaDe.IconFont = FontAwesome.Sharp.IconFont.Auto;
            this.btnAcercaDe.IconSize = 20;
            this.btnAcercaDe.ImageAlign = System.Drawing.ContentAlignment.MiddleLeft;
            this.btnAcercaDe.Margin = new System.Windows.Forms.Padding(0, 0, 0, 8);
            this.btnAcercaDe.Name = "btnAcercaDe";
            this.btnAcercaDe.Padding = new System.Windows.Forms.Padding(14, 0, 0, 0);
            this.btnAcercaDe.Size = new System.Drawing.Size(228, 44);
            this.btnAcercaDe.TabIndex = 9;
            this.btnAcercaDe.Text = "  Acerca de";
            this.btnAcercaDe.TextAlign = System.Drawing.ContentAlignment.MiddleLeft;
            this.btnAcercaDe.TextImageRelation = System.Windows.Forms.TextImageRelation.ImageBeforeText;
            this.btnAcercaDe.UseVisualStyleBackColor = true;
            this.btnAcercaDe.Click += new System.EventHandler(this.btnAcercaDe_Click);
            //
            // btnAuditoria
            //
            this.btnAuditoria.FlatAppearance.BorderSize = 0;
            this.btnAuditoria.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
            this.btnAuditoria.Font = new System.Drawing.Font("Segoe UI", 10F, System.Drawing.FontStyle.Bold);
            this.btnAuditoria.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(170)))), ((int)(((byte)(170)))), ((int)(((byte)(185)))));
            this.btnAuditoria.IconChar = FontAwesome.Sharp.IconChar.Clipboard;
            this.btnAuditoria.IconColor = System.Drawing.Color.FromArgb(((int)(((byte)(170)))), ((int)(((byte)(170)))), ((int)(((byte)(185)))));
            this.btnAuditoria.IconFont = FontAwesome.Sharp.IconFont.Auto;
            this.btnAuditoria.IconSize = 20;
            this.btnAuditoria.ImageAlign = System.Drawing.ContentAlignment.MiddleLeft;
            this.btnAuditoria.Margin = new System.Windows.Forms.Padding(0, 0, 0, 8);
            this.btnAuditoria.Name = "btnAuditoria";
            this.btnAuditoria.Padding = new System.Windows.Forms.Padding(14, 0, 0, 0);
            this.btnAuditoria.Size = new System.Drawing.Size(228, 44);
            this.btnAuditoria.TabIndex = 10;
            this.btnAuditoria.Text = "  Auditoría";
            this.btnAuditoria.TextAlign = System.Drawing.ContentAlignment.MiddleLeft;
            this.btnAuditoria.TextImageRelation = System.Windows.Forms.TextImageRelation.ImageBeforeText;
            this.btnAuditoria.UseVisualStyleBackColor = true;
            this.btnAuditoria.Click += new System.EventHandler(this.btnAuditoria_Click);
            //
            // btnAusencias
            //
            this.btnAusencias.FlatAppearance.BorderSize = 0;
```

```
            this.btnAusencias.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
            this.btnAusencias.Font = new System.Drawing.Font("Segoe UI", 10F, System.Drawing.FontStyle.Bold);
            this.btnAusencias.ForeColor = System.Drawing.Color.FromArgb(170, 170, 185);
            this.btnAusencias.IconChar = FontAwesome.Sharp.IconChar.CalendarMinus;
            this.btnAusencias.IconColor = System.Drawing.Color.FromArgb(170, 170, 185);
            this.btnAusencias.IconFont = FontAwesome.Sharp.IconFont.Auto;
            this.btnAusencias.IconSize = 20;
            this.btnAusencias.ImageAlign = System.Drawing.ContentAlignment.MiddleLeft;
            this.btnAusencias.Margin = new System.Windows.Forms.Padding(0, 0, 0, 8);
            this.btnAusencias.Name = "btnAusencias";
            this.btnAusencias.Padding = new System.Windows.Forms.Padding(14, 0, 0, 0);
            this.btnAusencias.Size = new System.Drawing.Size(228, 44);
            this.btnAusencias.TabIndex = 11;
            this.btnAusencias.Text = "  Ausencias";
            this.btnAusencias.TextAlign = System.Drawing.ContentAlignment.MiddleLeft;
            this.btnAusencias.TextImageRelation = System.Windows.Forms.TextImageRelation.ImageBeforeText;
            this.btnAusencias.UseVisualStyleBackColor = true;
            this.btnAusencias.Click += new System.EventHandler(this.btnAusencias_Click);
            //
            // btnSalir
            //
            this.btnSalir.FlatAppearance.BorderSize = 0;
            this.btnSalir.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
            this.btnSalir.Font = new System.Drawing.Font("Segoe UI", 10F, System.Drawing.FontStyle.Bold);
            this.btnSalir.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(170)))), ((int)(((byte)(170)))), ((int)(((byte)(185)))));
            this.btnSalir.IconChar = FontAwesome.Sharp.IconChar.PowerOff;
            this.btnSalir.IconColor = System.Drawing.Color.FromArgb(((int)(((byte)(170)))), ((int)(((byte)(170)))), ((int)(((byte)(185)))));
            this.btnSalir.IconFont = FontAwesome.Sharp.IconFont.Auto;
            this.btnSalir.IconSize = 20;
            this.btnSalir.ImageAlign = System.Drawing.ContentAlignment.MiddleLeft;
            this.btnSalir.Margin = new System.Windows.Forms.Padding(0, 0, 0, 4);
            this.btnSalir.Name = "btnSalir";
            this.btnSalir.Padding = new System.Windows.Forms.Padding(14, 0, 0, 0);
            this.btnSalir.Size = new System.Drawing.Size(228, 44);
            this.btnSalir.TabIndex = 10;
            this.btnSalir.Text = "  Salir";
            this.btnSalir.TextAlign = System.Drawing.ContentAlignment.MiddleLeft;
            this.btnSalir.TextImageRelation = System.Windows.Forms.TextImageRelation.ImageBeforeText;
            this.btnSalir.UseVisualStyleBackColor = true;
            this.btnSalir.Click += new System.EventHandler(this.btnSalir_Click);
            //
            // btnCerrarSesion
            //
            this.btnCerrarSesion.FlatAppearance.BorderSize = 0;
            this.btnCerrarSesion.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
            this.btnCerrarSesion.Font = new System.Drawing.Font("Segoe UI", 10F, System.Drawing.FontStyle.Bold);
            this.btnCerrarSesion.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(170)))), ((int)(((byte)(170)))), ((int)(((byte)(185)))));
            this.btnCerrarSesion.IconChar = FontAwesome.Sharp.IconChar.RightFromBracket;
            this.btnCerrarSesion.IconColor = System.Drawing.Color.FromArgb(((int)(((byte)(170)))), ((int)(((byte)(170)))), ((int)(((byte)(185)))));
            this.btnCerrarSesion.IconFont = FontAwesome.Sharp.IconFont.Auto;
            this.btnCerrarSesion.IconSize = 20;
            this.btnCerrarSesion.ImageAlign = System.Drawing.ContentAlignment.MiddleLeft;
            this.btnCerrarSesion.Margin = new System.Windows.Forms.Padding(0, 0, 0, 4);
            this.btnCerrarSesion.Name = "btnCerrarSesion";
            this.btnCerrarSesion.Padding = new System.Windows.Forms.Padding(14, 0, 0, 0);
            this.btnCerrarSesion.Size = new System.Drawing.Size(228, 44);
            this.btnCerrarSesion.TabIndex = 11;
            this.btnCerrarSesion.Text = "  Cerrar sesión";
            this.btnCerrarSesion.TextAlign = System.Drawing.ContentAlignment.MiddleLeft;
            this.btnCerrarSesion.TextImageRelation = System.Windows.Forms.TextImageRelation.ImageBeforeText;
```

UI/FrmMain.Designer.cs (continuacion)

```
            this.btnCerrarSesion.UseVisualStyleBackColor = true;
            this.btnCerrarSesion.Click += new System.EventHandler(this.btnCerrarSesion_Click);
            //
            // flowAccount
            //
            this.flowAccount.BackColor = System.Drawing.Color.FromArgb(((int)(((byte)(11)))), ((int)(((byte)(15)))), ((int)(((byte)(25)))));
            this.flowAccount.Controls.Add(this.panelAccountSeparator);
            this.flowAccount.Controls.Add(this.btnCerrarSesion);
            this.flowAccount.Controls.Add(this.btnSalir);
            this.flowAccount.Dock = System.Windows.Forms.DockStyle.Bottom;
            this.flowAccount.FlowDirection = System.Windows.Forms.FlowDirection.TopDown;
            this.flowAccount.Location = new System.Drawing.Point(0, 704);
            this.flowAccount.Name = "flowAccount";
            this.flowAccount.Padding = new System.Windows.Forms.Padding(12, 8, 12, 8);
            this.flowAccount.Size = new System.Drawing.Size(260, 116);
            this.flowAccount.TabIndex = 2;
            this.flowAccount.WrapContents = false;
            //
            // panelAccountSeparator
            //
            this.panelAccountSeparator.BackColor = System.Drawing.Color.FromArgb(((int)(((byte)(50)))), ((int)(((byte)(54)))), ((int)(((byte)(68)))));
            this.panelAccountSeparator.Margin = new System.Windows.Forms.Padding(0, 0, 0, 7);
            this.panelAccountSeparator.Name = "panelAccountSeparator";
            this.panelAccountSeparator.Size = new System.Drawing.Size(228, 1);
            this.panelAccountSeparator.TabIndex = 0;
            //
            // panelTopbar
            //
            this.panelTopbar.BackColor = System.Drawing.Color.FromArgb(((int)(((byte)(17)))), ((int)(((byte)(18)))), ((int)(((byte)(26)))));
            this.panelTopbar.Controls.Add(this.lblUsuarioActual);
            this.panelTopbar.Dock = System.Windows.Forms.DockStyle.Top;
            this.panelTopbar.Location = new System.Drawing.Point(260, 0);
            this.panelTopbar.Name = "panelTopbar";
            this.panelTopbar.Size = new System.Drawing.Size(1020, 78);
            this.panelTopbar.TabIndex = 1;
            //
            // lblUsuarioActual
            //
            this.lblUsuarioActual.Anchor = ((System.Windows.Forms.AnchorStyles)((System.Windows.Forms.AnchorStyles.Top |  \
        System.Windows.Forms.AnchorStyles.Right)));
            this.lblUsuarioActual.Font = new System.Drawing.Font("Segoe UI", 9F, System.Drawing.FontStyle.Bold);
            this.lblUsuarioActual.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(161)))), ((int)(((byte)(161)))), ((int)(((byte)(170)))));
            this.lblUsuarioActual.Location = new System.Drawing.Point(708, 25);
            this.lblUsuarioActual.Name = "lblUsuarioActual";
            this.lblUsuarioActual.Size = new System.Drawing.Size(282, 24);
            this.lblUsuarioActual.TabIndex = 1;
            this.lblUsuarioActual.Text = "Sin sesión";
            this.lblUsuarioActual.TextAlign = System.Drawing.ContentAlignment.MiddleRight;
            //
            // panelContent
            //
            this.panelContent.BackColor = System.Drawing.Color.FromArgb(((int)(((byte)(13)))), ((int)(((byte)(14)))), ((int)(((byte)(20)))));
            this.panelContent.Dock = System.Windows.Forms.DockStyle.Fill;
            this.panelContent.Location = new System.Drawing.Point(260, 78);
            this.panelContent.Name = "panelContent";
            this.panelContent.Size = new System.Drawing.Size(1020, 742);
            this.panelContent.TabIndex = 2;
            //
            // FrmMain
            //
```

UI/FrmMain.Designer.cs (continuacion)

```
            this.AutoScaleDimensions = new System.Drawing.SizeF(8F, 20F);
            this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font;
            this.BackColor = System.Drawing.Color.FromArgb(((int)(((byte)(13)))), ((int)(((byte)(14)))), ((int)(((byte)(20)))));
            this.ClientSize = new System.Drawing.Size(1280, 820);
            this.Controls.Add(this.panelContent);
            this.Controls.Add(this.panelTopbar);
            this.Controls.Add(this.panelSidebar);
            this.Font = new System.Drawing.Font("Segoe UI", 9F);
            this.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(229)))), ((int)(((byte)(228)))), ((int)(((byte)(245)))));
            this.MinimumSize = new System.Drawing.Size(1120, 720);
            this.Name = "FrmMain";
            this.StartPosition = System.Windows.Forms.FormStartPosition.CenterScreen;
            this.Text = "ProyFinal_LPI_Eq01_NomiCore";
            this.panelSidebar.ResumeLayout(false);
            this.panelBrand.ResumeLayout(false);
            this.flowMenu.ResumeLayout(false);
            this.flowAccount.ResumeLayout(false);
            this.panelTopbar.ResumeLayout(false);
            this.ResumeLayout(false);

        }
    }
}
```

UI/FrmMiPerfil.cs

```
using System;
using System.ComponentModel;
using System.Windows.Forms;
using SistemaGestionNomina.Helpers;
using SistemaGestionNomina.Models;
using SistemaGestionNomina.Services;

namespace SistemaGestionNomina.UI
{
    public partial class FrmMiPerfil : Form
    {
        private readonly EmployeePortalService portalService = new EmployeePortalService();

        public FrmMiPerfil()
        {
            InitializeComponent();
            ControlStyleHelper.ApplyModernForm(this);
        }

        private void FrmMiPerfil_Load(object sender, EventArgs e)
        {
            if (LicenseManager.UsageMode == LicenseUsageMode.Designtime) return;
            try
            {
                EmployeeProfileViewModel profile = portalService.GetMyProfile();
                lblNombreValor.Text = profile.NombreCompleto;
                lblCodigoValor.Text = profile.Codigo;
                lblCedulaValor.Text = profile.Cedula;
                lblCargoValor.Text = profile.Cargo;
                lblDepartamentoValor.Text = profile.Departamento;
                lblSalarioValor.Text = "B/. " + profile.SalarioBase.ToString("#,##0.00");
                lblEstadoValor.Text = profile.Estado;
                lblIngresoValor.Text = profile.FechaIngreso.ToString("dd/MM/yyyy");
            }
            catch (Exception ex)
            {
                MessageBox.Show(ex.Message, "Mi perfil", MessageBoxButtons.OK, MessageBoxIcon.Warning);
            }
        }
    }
}
```

UI/FrmMiPerfil.Designer.cs

```
namespace SistemaGestionNomina.UI
{
    partial class FrmMiPerfil
    {
        private System.ComponentModel.IContainer components = null;
        private System.Windows.Forms.Label lblTitulo;
        private System.Windows.Forms.Label lblSubtitulo;
        private System.Windows.Forms.TableLayoutPanel tablePerfil;
        private System.Windows.Forms.Label lblNombre;
        private System.Windows.Forms.Label lblNombreValor;
        private System.Windows.Forms.Label lblCodigo;
        private System.Windows.Forms.Label lblCodigoValor;
        private System.Windows.Forms.Label lblCedula;
        private System.Windows.Forms.Label lblCedulaValor;
        private System.Windows.Forms.Label lblCargo;
        private System.Windows.Forms.Label lblCargoValor;
        private System.Windows.Forms.Label lblDepartamento;
        private System.Windows.Forms.Label lblDepartamentoValor;
        private System.Windows.Forms.Label lblSalario;
        private System.Windows.Forms.Label lblSalarioValor;
        private System.Windows.Forms.Label lblEstado;
        private System.Windows.Forms.Label lblEstadoValor;
        private System.Windows.Forms.Label lblIngreso;
        private System.Windows.Forms.Label lblIngresoValor;

        protected override void Dispose(bool disposing) { if (disposing && components != null) components.Dispose(); base.Dispose(disposing); }

        private void InitializeComponent()
        {
            this.lblTitulo = new System.Windows.Forms.Label(); this.lblSubtitulo = new System.Windows.Forms.Label(); this.tablePerfil = new  \
    System.Windows.Forms.TableLayoutPanel();
            this.lblNombre = new System.Windows.Forms.Label(); this.lblNombreValor = new System.Windows.Forms.Label(); this.lblCodigo = new  \
    System.Windows.Forms.Label(); this.lblCodigoValor = new System.Windows.Forms.Label(); this.lblCedula = new System.Windows.Forms.Label();  \
    this.lblCedulaValor = new System.Windows.Forms.Label(); this.lblCargo = new System.Windows.Forms.Label(); this.lblCargoValor = new  \
    System.Windows.Forms.Label(); this.lblDepartamento = new System.Windows.Forms.Label(); this.lblDepartamentoValor = new System.Windows.Forms.Label();  \
    this.lblSalario = new System.Windows.Forms.Label(); this.lblSalarioValor = new System.Windows.Forms.Label(); this.lblEstado = new  \
    System.Windows.Forms.Label(); this.lblEstadoValor = new System.Windows.Forms.Label(); this.lblIngreso = new System.Windows.Forms.Label();  \
    this.lblIngresoValor = new System.Windows.Forms.Label();
            this.tablePerfil.SuspendLayout(); this.SuspendLayout();
            this.lblTitulo.AutoSize = true; this.lblTitulo.Font = new System.Drawing.Font("Segoe UI", 23F, System.Drawing.FontStyle.Bold);  \
    this.lblTitulo.Location = new System.Drawing.Point(32, 24); this.lblTitulo.Text = "Mi perfil laboral";
            this.lblSubtitulo.AutoSize = true; this.lblSubtitulo.ForeColor = System.Drawing.Color.FromArgb(161, 161, 175); this.lblSubtitulo.Location =  \
    new System.Drawing.Point(36, 78); this.lblSubtitulo.Text = "Informacion registrada por Recursos Humanos.";
            this.tablePerfil.Anchor = ((System.Windows.Forms.AnchorStyles)(((System.Windows.Forms.AnchorStyles.Top |  \
    System.Windows.Forms.AnchorStyles.Left) | System.Windows.Forms.AnchorStyles.Right))); this.tablePerfil.BackColor = System.Drawing.Color.FromArgb(24,  \
    25, 34); this.tablePerfil.ColumnCount = 2; this.tablePerfil.ColumnStyles.Add(new  \
    System.Windows.Forms.ColumnStyle(System.Windows.Forms.SizeType.Percent, 32F)); this.tablePerfil.ColumnStyles.Add(new  \
    System.Windows.Forms.ColumnStyle(System.Windows.Forms.SizeType.Percent, 68F)); this.tablePerfil.Controls.Add(this.lblNombre, 0, 0);  \
    this.tablePerfil.Controls.Add(this.lblNombreValor, 1, 0); this.tablePerfil.Controls.Add(this.lblCodigo, 0, 1);  \
    this.tablePerfil.Controls.Add(this.lblCodigoValor, 1, 1); this.tablePerfil.Controls.Add(this.lblCedula, 0, 2);  \
    this.tablePerfil.Controls.Add(this.lblCedulaValor, 1, 2); this.tablePerfil.Controls.Add(this.lblCargo, 0, 3);  \
    this.tablePerfil.Controls.Add(this.lblCargoValor, 1, 3); this.tablePerfil.Controls.Add(this.lblDepartamento, 0, 4);  \
    this.tablePerfil.Controls.Add(this.lblDepartamentoValor, 1, 4); this.tablePerfil.Controls.Add(this.lblSalario, 0, 5);  \
    this.tablePerfil.Controls.Add(this.lblSalarioValor, 1, 5); this.tablePerfil.Controls.Add(this.lblEstado, 0, 6);  \
    this.tablePerfil.Controls.Add(this.lblEstadoValor, 1, 6); this.tablePerfil.Controls.Add(this.lblIngreso, 0, 7);  \
    this.tablePerfil.Controls.Add(this.lblIngresoValor, 1, 7); this.tablePerfil.Location = new System.Drawing.Point(38, 126); this.tablePerfil.Name =  \
    "tablePerfil"; this.tablePerfil.Padding = new System.Windows.Forms.Padding(24); this.tablePerfil.RowCount = 8; for (int i = 0; i < 8; i++)  \
    this.tablePerfil.RowStyles.Add(new System.Windows.Forms.RowStyle(System.Windows.Forms.SizeType.Absolute, 58F)); this.tablePerfil.Size = new  \
    System.Drawing.Size(944, 514);
            ConfigureCaption(this.lblNombre, "Nombre completo"); ConfigureValue(this.lblNombreValor); ConfigureCaption(this.lblCodigo, "Codigo de  \
```

UI/FrmMiPerfil.Designer.cs (continuacion)

```
    empleado"); ConfigureValue(this.lblCodigoValor); ConfigureCaption(this.lblCedula, "Cedula"); ConfigureValue(this.lblCedulaValor);  \
    ConfigureCaption(this.lblCargo, "Cargo"); ConfigureValue(this.lblCargoValor); ConfigureCaption(this.lblDepartamento, "Departamento");  \
    ConfigureValue(this.lblDepartamentoValor); ConfigureCaption(this.lblSalario, "Salario base"); ConfigureValue(this.lblSalarioValor);  \
    ConfigureCaption(this.lblEstado, "Estado laboral"); ConfigureValue(this.lblEstadoValor); ConfigureCaption(this.lblIngreso, "Fecha de ingreso");  \
    ConfigureValue(this.lblIngresoValor);
            this.AutoScaleDimensions = new System.Drawing.SizeF(8F, 20F); this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font; this.BackColor =  \
    System.Drawing.Color.FromArgb(13, 14, 20); this.ClientSize = new System.Drawing.Size(1020, 742); this.Controls.Add(this.tablePerfil);  \
    this.Controls.Add(this.lblSubtitulo); this.Controls.Add(this.lblTitulo); this.Font = new System.Drawing.Font("Segoe UI", 9F); this.ForeColor =  \
    System.Drawing.Color.FromArgb(229, 228, 245); this.Name = "FrmMiPerfil"; this.Text = "Mi perfil"; this.Load += new  \
    System.EventHandler(this.FrmMiPerfil_Load); this.tablePerfil.ResumeLayout(false); this.ResumeLayout(false); this.PerformLayout();
        }

        private static void ConfigureCaption(System.Windows.Forms.Label label, string text) { label.Dock = System.Windows.Forms.DockStyle.Fill;  \
    label.ForeColor = System.Drawing.Color.FromArgb(161, 161, 175); label.Text = text; label.TextAlign = System.Drawing.ContentAlignment.MiddleLeft; }
        private static void ConfigureValue(System.Windows.Forms.Label label) { label.AutoEllipsis = true; label.Dock =  \
    System.Windows.Forms.DockStyle.Fill; label.Font = new System.Drawing.Font("Segoe UI", 10F, System.Drawing.FontStyle.Bold); label.Text = "-";  \
    label.TextAlign = System.Drawing.ContentAlignment.MiddleLeft; }
    }
}
```

UI/FrmMisAsistencias.cs

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Windows.Forms;
using SistemaGestionNomina.Helpers;
using SistemaGestionNomina.Models;
using SistemaGestionNomina.Services;

namespace SistemaGestionNomina.UI
{
    public partial class FrmMisAsistencias : Form
    {
        private readonly EmployeePortalService portalService = new EmployeePortalService();

        public FrmMisAsistencias()
        {
            InitializeComponent();
            ControlStyleHelper.ApplyModernForm(this);
        }

        private void FrmMisAsistencias_Load(object sender, EventArgs e)
        {
            if (LicenseManager.UsageMode == LicenseUsageMode.Designtime) return;
            dtpDesde.Value = new DateTime(DateTime.Today.Year, DateTime.Today.Month, 1);
            dtpHasta.Value = DateTime.Today;
            cmbEstado.SelectedIndex = 0;
            LoadAttendance();
        }

        private void btnFiltrar_Click(object sender, EventArgs e) { LoadAttendance(); }

        private void btnLimpiar_Click(object sender, EventArgs e)
        {
            dtpDesde.Checked = false;
            dtpHasta.Checked = false;
            cmbEstado.SelectedIndex = 0;
            LoadAttendance();
        }

        private void LoadAttendance()
        {
            try
            {
                DateTime? start = dtpDesde.Checked ? dtpDesde.Value.Date : (DateTime?)null;
                DateTime? end = dtpHasta.Checked ? dtpHasta.Value.Date : (DateTime?)null;
                List<Asistencia> items = portalService.GetMyAttendance(start, end,
                    Convert.ToString(cmbEstado.SelectedItem));

                dgvAsistencias.Rows.Clear();
                foreach (Asistencia item in items)
                {
                    dgvAsistencias.Rows.Add(item.Fecha.ToString("dd/MM/yyyy"),
                        item.HoraEntrada.HasValue ? item.HoraEntrada.Value.ToString(@"hh\:mm") : "-",
                        item.HoraSalida.HasValue ? item.HoraSalida.Value.ToString(@"hh\:mm") : "-",
                        item.HorasTrabajadas.ToString("0.00"), item.Estado);
                }

                EmployeeAttendanceSummaryViewModel summary = EmployeePortalService.SummarizeAttendance(items);
                lblPuntualesValor.Text = summary.Puntuales + "\nPuntuales";
                lblTardanzasValor.Text = summary.Tardanzas + "\nTardanzas";
```

UI/FrmMisAsistencias.cs (continuacion)

```
                lblFaltasValor.Text = summary.Faltas + "\nFaltas";
                lblPermisosValor.Text = summary.Permisos + "\nPermisos";
                lblHorasExtraValor.Text = summary.HorasExtra.ToString("0.00") + " h\nHoras extra";
            }
            catch (Exception ex)
            {
                MessageBox.Show(ex.Message, "Mis asistencias", MessageBoxButtons.OK, MessageBoxIcon.Warning);
            }
        }
    }
}
```

UI/FrmMisAsistencias.Designer.cs

```
namespace SistemaGestionNomina.UI
{
    partial class FrmMisAsistencias
    {
        private System.ComponentModel.IContainer components = null;
        private System.Windows.Forms.Label lblTitulo;
        private System.Windows.Forms.Panel panelFiltros;
        private System.Windows.Forms.DateTimePicker dtpDesde;
        private System.Windows.Forms.DateTimePicker dtpHasta;
        private System.Windows.Forms.ComboBox cmbEstado;
        private System.Windows.Forms.Button btnFiltrar;
        private System.Windows.Forms.Button btnLimpiar;
        private System.Windows.Forms.FlowLayoutPanel panelResumen;
        private System.Windows.Forms.Label lblPuntualesValor;
        private System.Windows.Forms.Label lblTardanzasValor;
        private System.Windows.Forms.Label lblFaltasValor;
        private System.Windows.Forms.Label lblPermisosValor;
        private System.Windows.Forms.Label lblHorasExtraValor;
        private System.Windows.Forms.DataGridView dgvAsistencias;
        private System.Windows.Forms.DataGridViewTextBoxColumn colFecha;
        private System.Windows.Forms.DataGridViewTextBoxColumn colEntrada;
        private System.Windows.Forms.DataGridViewTextBoxColumn colSalida;
        private System.Windows.Forms.DataGridViewTextBoxColumn colHoras;
        private System.Windows.Forms.DataGridViewTextBoxColumn colEstado;

        protected override void Dispose(bool disposing) { if (disposing && components != null) components.Dispose(); base.Dispose(disposing); }

        private void InitializeComponent()
        {
            this.lblTitulo = new System.Windows.Forms.Label(); this.panelFiltros = new System.Windows.Forms.Panel(); this.dtpDesde = new  \
    System.Windows.Forms.DateTimePicker(); this.dtpHasta = new System.Windows.Forms.DateTimePicker(); this.cmbEstado = new  \
    System.Windows.Forms.ComboBox(); this.btnFiltrar = new System.Windows.Forms.Button(); this.btnLimpiar = new System.Windows.Forms.Button();  \
    this.panelResumen = new System.Windows.Forms.FlowLayoutPanel(); this.lblPuntualesValor = new System.Windows.Forms.Label(); this.lblTardanzasValor =  \
    new System.Windows.Forms.Label(); this.lblFaltasValor = new System.Windows.Forms.Label(); this.lblPermisosValor = new System.Windows.Forms.Label();  \
    this.lblHorasExtraValor = new System.Windows.Forms.Label(); this.dgvAsistencias = new System.Windows.Forms.DataGridView(); this.colFecha = new  \
    System.Windows.Forms.DataGridViewTextBoxColumn(); this.colEntrada = new System.Windows.Forms.DataGridViewTextBoxColumn(); this.colSalida = new  \
    System.Windows.Forms.DataGridViewTextBoxColumn(); this.colHoras = new System.Windows.Forms.DataGridViewTextBoxColumn(); this.colEstado = new  \
    System.Windows.Forms.DataGridViewTextBoxColumn();
            this.panelFiltros.SuspendLayout(); this.panelResumen.SuspendLayout();  \
    ((System.ComponentModel.ISupportInitialize)(this.dgvAsistencias)).BeginInit(); this.SuspendLayout();
            this.lblTitulo.AutoSize = true; this.lblTitulo.Font = new System.Drawing.Font("Segoe UI", 23F, System.Drawing.FontStyle.Bold);  \
    this.lblTitulo.Location = new System.Drawing.Point(28, 20); this.lblTitulo.Text = "Mis asistencias";
            this.panelFiltros.Anchor = ((System.Windows.Forms.AnchorStyles)(((System.Windows.Forms.AnchorStyles.Top |  \
    System.Windows.Forms.AnchorStyles.Left) | System.Windows.Forms.AnchorStyles.Right))); this.panelFiltros.BackColor = System.Drawing.Color.FromArgb(24,  \
    25, 34); this.panelFiltros.Controls.Add(this.dtpDesde); this.panelFiltros.Controls.Add(this.dtpHasta);  \
    this.panelFiltros.Controls.Add(this.cmbEstado); this.panelFiltros.Controls.Add(this.btnFiltrar); this.panelFiltros.Controls.Add(this.btnLimpiar);  \
    this.panelFiltros.Location = new System.Drawing.Point(34, 86); this.panelFiltros.Size = new System.Drawing.Size(952, 74);
            this.dtpDesde.Format = System.Windows.Forms.DateTimePickerFormat.Short; this.dtpDesde.Location = new System.Drawing.Point(18, 22);  \
    this.dtpDesde.Name = "dtpDesde"; this.dtpDesde.ShowCheckBox = true; this.dtpDesde.Size = new System.Drawing.Size(190, 27);
            this.dtpHasta.Format = System.Windows.Forms.DateTimePickerFormat.Short; this.dtpHasta.Location = new System.Drawing.Point(224, 22);  \
    this.dtpHasta.Name = "dtpHasta"; this.dtpHasta.ShowCheckBox = true; this.dtpHasta.Size = new System.Drawing.Size(190, 27);
            this.cmbEstado.DropDownStyle = System.Windows.Forms.ComboBoxStyle.DropDownList; this.cmbEstado.Items.AddRange(new object[] { "Todos",  \
    "Puntual", "Tardanza", "Falta", "Permiso" }); this.cmbEstado.Location = new System.Drawing.Point(430, 21); this.cmbEstado.Name = "cmbEstado";  \
    this.cmbEstado.Size = new System.Drawing.Size(176, 28);
            this.btnFiltrar.Location = new System.Drawing.Point(630, 17); this.btnFiltrar.Name = "btnFiltrar"; this.btnFiltrar.Size = new  \
    System.Drawing.Size(142, 38); this.btnFiltrar.Text = "Aplicar filtros"; this.btnFiltrar.Click += new System.EventHandler(this.btnFiltrar_Click);
            this.btnLimpiar.Location = new System.Drawing.Point(790, 17); this.btnLimpiar.Name = "btnLimpiar"; this.btnLimpiar.Size = new  \
    System.Drawing.Size(142, 38); this.btnLimpiar.Text = "Limpiar"; this.btnLimpiar.Click += new System.EventHandler(this.btnLimpiar_Click);
            this.panelResumen.Anchor = ((System.Windows.Forms.AnchorStyles)(((System.Windows.Forms.AnchorStyles.Top |  \
    System.Windows.Forms.AnchorStyles.Left) | System.Windows.Forms.AnchorStyles.Right))); this.panelResumen.Controls.Add(this.lblPuntualesValor);  \
```

UI/FrmMisAsistencias.Designer.cs (continuacion)

```
    this.panelResumen.Controls.Add(this.lblTardanzasValor); this.panelResumen.Controls.Add(this.lblFaltasValor);  \
    this.panelResumen.Controls.Add(this.lblPermisosValor); this.panelResumen.Controls.Add(this.lblHorasExtraValor); this.panelResumen.FlowDirection =  \
    System.Windows.Forms.FlowDirection.LeftToRight; this.panelResumen.Location = new System.Drawing.Point(34, 178); this.panelResumen.Name =  \
    "panelResumen"; this.panelResumen.Size = new System.Drawing.Size(952, 80); this.panelResumen.WrapContents = false;
            ConfigureSummary(this.lblPuntualesValor, "Puntuales", System.Drawing.Color.FromArgb(52, 211, 153)); ConfigureSummary(this.lblTardanzasValor,  \
    "Tardanzas", System.Drawing.Color.FromArgb(251, 191, 36)); ConfigureSummary(this.lblFaltasValor, "Faltas", System.Drawing.Color.FromArgb(248, 113,  \
    113)); ConfigureSummary(this.lblPermisosValor, "Permisos", System.Drawing.Color.FromArgb(96, 165, 250)); ConfigureSummary(this.lblHorasExtraValor,  \
    "Horas extra", System.Drawing.Color.FromArgb(167, 139, 250));
            this.dgvAsistencias.AllowUserToAddRows = false; this.dgvAsistencias.AllowUserToDeleteRows = false; this.dgvAsistencias.Anchor =  \
    ((System.Windows.Forms.AnchorStyles)((((System.Windows.Forms.AnchorStyles.Top | System.Windows.Forms.AnchorStyles.Bottom) |  \
    System.Windows.Forms.AnchorStyles.Left) | System.Windows.Forms.AnchorStyles.Right))); this.dgvAsistencias.AutoSizeColumnsMode =  \
    System.Windows.Forms.DataGridViewAutoSizeColumnsMode.Fill; this.dgvAsistencias.BackgroundColor = System.Drawing.Color.FromArgb(18, 19, 27);  \
    this.dgvAsistencias.BorderStyle = System.Windows.Forms.BorderStyle.None; this.dgvAsistencias.ColumnHeadersHeight = 40;  \
    this.dgvAsistencias.Columns.AddRange(new System.Windows.Forms.DataGridViewColumn[] { this.colFecha, this.colEntrada, this.colSalida, this.colHoras,  \
    this.colEstado }); this.dgvAsistencias.Location = new System.Drawing.Point(34, 276); this.dgvAsistencias.Name = "dgvAsistencias";  \
    this.dgvAsistencias.ReadOnly = true; this.dgvAsistencias.RowHeadersVisible = false; this.dgvAsistencias.RowTemplate.Height = 36;  \
    this.dgvAsistencias.SelectionMode = System.Windows.Forms.DataGridViewSelectionMode.FullRowSelect; this.dgvAsistencias.Size = new  \
    System.Drawing.Size(952, 430);
            this.colFecha.HeaderText = "Fecha"; this.colEntrada.HeaderText = "Entrada"; this.colSalida.HeaderText = "Salida"; this.colHoras.HeaderText =  \
    "Horas trabajadas"; this.colEstado.HeaderText = "Estado";
            this.AutoScaleDimensions = new System.Drawing.SizeF(8F, 20F); this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font; this.BackColor =  \
    System.Drawing.Color.FromArgb(13, 14, 20); this.ClientSize = new System.Drawing.Size(1020, 742); this.Controls.Add(this.dgvAsistencias);  \
    this.Controls.Add(this.panelResumen); this.Controls.Add(this.panelFiltros); this.Controls.Add(this.lblTitulo); this.Font = new  \
    System.Drawing.Font("Segoe UI", 9F); this.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245); this.Name = "FrmMisAsistencias"; this.Text = "Mis  \
    asistencias"; this.Load += new System.EventHandler(this.FrmMisAsistencias_Load); this.panelFiltros.ResumeLayout(false);  \
    this.panelResumen.ResumeLayout(false); ((System.ComponentModel.ISupportInitialize)(this.dgvAsistencias)).EndInit(); this.ResumeLayout(false);  \
    this.PerformLayout();
        }

        private static void ConfigureSummary(System.Windows.Forms.Label label, string title, System.Drawing.Color color) { label.BackColor =  \
    System.Drawing.Color.FromArgb(24, 25, 34); label.Font = new System.Drawing.Font("Segoe UI", 13F, System.Drawing.FontStyle.Bold); label.ForeColor =  \
    color; label.Margin = new System.Windows.Forms.Padding(0, 0, 12, 0); label.Size = new System.Drawing.Size(178, 76); label.Text = "0\n" + title;  \
    label.TextAlign = System.Drawing.ContentAlignment.MiddleCenter; }
    }
}
```

UI/FrmMisComprobantes.cs

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Diagnostics;
using System.Drawing.Printing;
using System.Windows.Forms;
using SistemaGestionNomina.Helpers;
using SistemaGestionNomina.Models;
using SistemaGestionNomina.Services;

namespace SistemaGestionNomina.UI
{
    public partial class FrmMisComprobantes : Form
    {
        private readonly EmployeePortalService portalService = new EmployeePortalService();
        private readonly PrintDocument printDocument = new PrintDocument();
        private List<Comprobante> currentItems = new List<Comprobante>();
        private Comprobante selectedPayslip;

        public FrmMisComprobantes()
        {
            InitializeComponent();
            ControlStyleHelper.ApplyModernForm(this);
            printDocument.PrintPage += PrintDocument_PrintPage;
        }

        private void FrmMisComprobantes_Load(object sender, EventArgs e)
        {
            if (LicenseManager.UsageMode != LicenseUsageMode.Designtime) LoadPayslips();
        }

        private void txtBuscar_TextChanged(object sender, EventArgs e)
        {
            if (LicenseManager.UsageMode != LicenseUsageMode.Designtime) LoadPayslips();
        }

        private void dgvComprobantes_SelectionChanged(object sender, EventArgs e)
        {
            if (dgvComprobantes.CurrentRow == null || dgvComprobantes.CurrentRow.Cells["colId"].Value == null) return;
            try
            {
                selectedPayslip = portalService.GetMyPayslipById(
                    Convert.ToInt32(dgvComprobantes.CurrentRow.Cells["colId"].Value));
                RenderPreview();
            }
            catch (Exception ex)
            {
                selectedPayslip = null;
                lblVistaPrevia.Text = ex.Message;
            }
        }

        private void LoadPayslips()
        {
            try
            {
                currentItems = portalService.GetMyPayslips(txtBuscar.Text);
                dgvComprobantes.Rows.Clear();
                foreach (Comprobante item in currentItems)
                {
```

UI/FrmMisComprobantes.cs (continuacion)

```
                    dgvComprobantes.Rows.Add(item.IdComprobante, item.NumeroComprobante,
                        item.PeriodoNombre, item.FechaGeneracion.ToString("dd/MM/yyyy"),
                        item.TotalIngresos.ToString("0.00"), item.TotalDeducciones.ToString("0.00"),
                        item.NetoPagar.ToString("0.00"));
                }

                if (currentItems.Count == 0)
                {
                    selectedPayslip = null;
                    lblVistaPrevia.Text = "No hay comprobantes disponibles.";
                }
            }
            catch (Exception ex)
            {
                MessageBox.Show(ex.Message, "Mis comprobantes", MessageBoxButtons.OK, MessageBoxIcon.Warning);
            }
        }

        private void RenderPreview()
        {
            if (selectedPayslip == null) return;
            lblVistaPrevia.Text = "COMPROBANTE DE PAGO\n\n" +
                "Numero: " + selectedPayslip.NumeroComprobante + "\n" +
                "Periodo: " + selectedPayslip.PeriodoNombre + "\n" +
                "Fecha: " + selectedPayslip.FechaGeneracion.ToString("dd/MM/yyyy") + "\n\n" +
                "Ingresos: B/. " + selectedPayslip.TotalIngresos.ToString("#,##0.00") + "\n" +
                "Deducciones: B/. " + selectedPayslip.TotalDeducciones.ToString("#,##0.00") + "\n\n" +
                "NETO A PAGAR: B/. " + selectedPayslip.NetoPagar.ToString("#,##0.00");
        }

        private bool EnsureSelected()
        {
            if (selectedPayslip != null) return true;
            MessageBox.Show("Seleccione un comprobante.", "Mis comprobantes",
                MessageBoxButtons.OK, MessageBoxIcon.Information);
            return false;
        }

        private void btnDescargar_Click(object sender, EventArgs e)
        {
            if (!EnsureSelected()) return;
            try
            {
                string path = portalService.DownloadMyPayslipPdf(selectedPayslip.IdComprobante);
                if (string.IsNullOrWhiteSpace(path)) return;
                selectedPayslip.RutaPdf = path;
                UiFactory.ShowExported(path);
            }
            catch (Exception ex)
            {
                MessageBox.Show(ex.Message, "Descargar PDF", MessageBoxButtons.OK, MessageBoxIcon.Warning);
            }
        }

        private void btnAbrirPdf_Click(object sender, EventArgs e)
        {
            if (!EnsureSelected()) return;
            try
            {
                string path = portalService.GetExistingMyPayslipPdf(selectedPayslip.IdComprobante);
```

UI/FrmMisComprobantes.cs (continuacion)

```
                Process.Start(new ProcessStartInfo(path) { UseShellExecute = true });
            }
            catch (Exception ex)
            {
                MessageBox.Show(ex.Message, "Abrir PDF", MessageBoxButtons.OK, MessageBoxIcon.Warning);
            }
        }

        private void btnImprimir_Click(object sender, EventArgs e)
        {
            if (!EnsureSelected()) return;
            try
            {
                selectedPayslip = portalService.GetMyPayslipForPrint(selectedPayslip.IdComprobante);
                using (PrintPreviewDialog preview = new PrintPreviewDialog())
                {
                    preview.Document = printDocument;
                    preview.Width = 900;
                    preview.Height = 700;
                    preview.StartPosition = FormStartPosition.CenterParent;
                    preview.ShowDialog(this);
                }
            }
            catch (Exception ex)
            {
                MessageBox.Show(ex.Message, "Imprimir comprobante", MessageBoxButtons.OK, MessageBoxIcon.Warning);
            }
        }

        private void PrintDocument_PrintPage(object sender, PrintPageEventArgs e)
        {
            if (selectedPayslip == null)
            {
                e.HasMorePages = false;
                return;
            }

            ComprobantePrintRenderer.Draw(e.Graphics, e.MarginBounds, selectedPayslip);
            e.HasMorePages = false;
        }
    }
}
```

UI/FrmMisComprobantes.Designer.cs

```
namespace SistemaGestionNomina.UI
{
    partial class FrmMisComprobantes
    {
        private System.ComponentModel.IContainer components = null;
        private System.Windows.Forms.Label lblTitulo;
        private System.Windows.Forms.TextBox txtBuscar;
        private System.Windows.Forms.DataGridView dgvComprobantes;
        private System.Windows.Forms.DataGridViewTextBoxColumn colId;
        private System.Windows.Forms.DataGridViewTextBoxColumn colNumero;
        private System.Windows.Forms.DataGridViewTextBoxColumn colPeriodo;
        private System.Windows.Forms.DataGridViewTextBoxColumn colFecha;
        private System.Windows.Forms.DataGridViewTextBoxColumn colIngresos;
        private System.Windows.Forms.DataGridViewTextBoxColumn colDeducciones;
        private System.Windows.Forms.DataGridViewTextBoxColumn colNeto;
        private System.Windows.Forms.Panel panelVistaPrevia;
        private System.Windows.Forms.Label lblVistaPrevia;
        private System.Windows.Forms.Button btnDescargar;
        private System.Windows.Forms.Button btnAbrirPdf;
        private System.Windows.Forms.Button btnImprimir;

        protected override void Dispose(bool disposing) { if (disposing && components != null) components.Dispose(); base.Dispose(disposing); }

        private void InitializeComponent()
        {
            this.lblTitulo = new System.Windows.Forms.Label(); this.txtBuscar = new System.Windows.Forms.TextBox(); this.dgvComprobantes = new  \
    System.Windows.Forms.DataGridView(); this.colId = new System.Windows.Forms.DataGridViewTextBoxColumn(); this.colNumero = new  \
    System.Windows.Forms.DataGridViewTextBoxColumn(); this.colPeriodo = new System.Windows.Forms.DataGridViewTextBoxColumn(); this.colFecha = new  \
    System.Windows.Forms.DataGridViewTextBoxColumn(); this.colIngresos = new System.Windows.Forms.DataGridViewTextBoxColumn(); this.colDeducciones = new  \
    System.Windows.Forms.DataGridViewTextBoxColumn(); this.colNeto = new System.Windows.Forms.DataGridViewTextBoxColumn(); this.panelVistaPrevia = new  \
    System.Windows.Forms.Panel(); this.lblVistaPrevia = new System.Windows.Forms.Label(); this.btnDescargar = new System.Windows.Forms.Button();  \
    this.btnAbrirPdf = new System.Windows.Forms.Button(); this.btnImprimir = new System.Windows.Forms.Button();
            ((System.ComponentModel.ISupportInitialize)(this.dgvComprobantes)).BeginInit(); this.panelVistaPrevia.SuspendLayout(); this.SuspendLayout();
            this.lblTitulo.AutoSize = true; this.lblTitulo.Font = new System.Drawing.Font("Segoe UI", 23F, System.Drawing.FontStyle.Bold);  \
    this.lblTitulo.Location = new System.Drawing.Point(28, 20); this.lblTitulo.Text = "Mis comprobantes";
            this.txtBuscar.Anchor = ((System.Windows.Forms.AnchorStyles)((System.Windows.Forms.AnchorStyles.Top |  \
    System.Windows.Forms.AnchorStyles.Right))); this.txtBuscar.Location = new System.Drawing.Point(666, 36); this.txtBuscar.Name = "txtBuscar";  \
    this.txtBuscar.Size = new System.Drawing.Size(320, 27); this.txtBuscar.TextChanged += new System.EventHandler(this.txtBuscar_TextChanged);
            this.dgvComprobantes.AllowUserToAddRows = false; this.dgvComprobantes.AllowUserToDeleteRows = false; this.dgvComprobantes.Anchor =  \
    ((System.Windows.Forms.AnchorStyles)(((System.Windows.Forms.AnchorStyles.Top | System.Windows.Forms.AnchorStyles.Left) |  \
    System.Windows.Forms.AnchorStyles.Right))); this.dgvComprobantes.AutoSizeColumnsMode = System.Windows.Forms.DataGridViewAutoSizeColumnsMode.Fill;  \
    this.dgvComprobantes.BackgroundColor = System.Drawing.Color.FromArgb(18, 19, 27); this.dgvComprobantes.BorderStyle =  \
    System.Windows.Forms.BorderStyle.None; this.dgvComprobantes.ColumnHeadersHeight = 40; this.dgvComprobantes.Columns.AddRange(new  \
    System.Windows.Forms.DataGridViewColumn[] { this.colId, this.colNumero, this.colPeriodo, this.colFecha, this.colIngresos, this.colDeducciones,  \
    this.colNeto }); this.dgvComprobantes.Location = new System.Drawing.Point(34, 92); this.dgvComprobantes.MultiSelect = false;  \
    this.dgvComprobantes.Name = "dgvComprobantes"; this.dgvComprobantes.ReadOnly = true; this.dgvComprobantes.RowHeadersVisible = false;  \
    this.dgvComprobantes.RowTemplate.Height = 34; this.dgvComprobantes.SelectionMode = System.Windows.Forms.DataGridViewSelectionMode.FullRowSelect;  \
    this.dgvComprobantes.Size = new System.Drawing.Size(952, 262); this.dgvComprobantes.SelectionChanged += new  \
    System.EventHandler(this.dgvComprobantes_SelectionChanged);
            this.colId.HeaderText = "Id"; this.colId.Name = "colId"; this.colId.ReadOnly = true; this.colId.Visible = false; this.colNumero.HeaderText =  \
    "Numero"; this.colPeriodo.HeaderText = "Periodo"; this.colFecha.HeaderText = "Fecha"; this.colIngresos.HeaderText = "Ingresos";  \
    this.colDeducciones.HeaderText = "Deducciones"; this.colNeto.HeaderText = "Neto";
            this.panelVistaPrevia.Anchor = ((System.Windows.Forms.AnchorStyles)((((System.Windows.Forms.AnchorStyles.Top |  \
    System.Windows.Forms.AnchorStyles.Bottom) | System.Windows.Forms.AnchorStyles.Left) | System.Windows.Forms.AnchorStyles.Right)));  \
    this.panelVistaPrevia.BackColor = System.Drawing.Color.FromArgb(24, 25, 34); this.panelVistaPrevia.Controls.Add(this.lblVistaPrevia);  \
    this.panelVistaPrevia.Location = new System.Drawing.Point(34, 374); this.panelVistaPrevia.Name = "panelVistaPrevia"; this.panelVistaPrevia.Size = new  \
    System.Drawing.Size(694, 332);
            this.lblVistaPrevia.Dock = System.Windows.Forms.DockStyle.Fill; this.lblVistaPrevia.Font = new System.Drawing.Font("Consolas", 11F);  \
    this.lblVistaPrevia.Location = new System.Drawing.Point(0, 0); this.lblVistaPrevia.Name = "lblVistaPrevia"; this.lblVistaPrevia.Padding = new  \
    System.Windows.Forms.Padding(28); this.lblVistaPrevia.Size = new System.Drawing.Size(694, 332); this.lblVistaPrevia.Text = "Seleccione un comprobante  \
```

```
    para ver el detalle.";
            this.btnDescargar.Anchor = ((System.Windows.Forms.AnchorStyles)((System.Windows.Forms.AnchorStyles.Top |  \
    System.Windows.Forms.AnchorStyles.Right))); this.btnDescargar.Location = new System.Drawing.Point(754, 390); this.btnDescargar.Name = "btnDescargar";  \
    this.btnDescargar.Size = new System.Drawing.Size(232, 44); this.btnDescargar.Text = "Descargar PDF"; this.btnDescargar.Click += new  \
    System.EventHandler(this.btnDescargar_Click);
            this.btnAbrirPdf.Anchor = ((System.Windows.Forms.AnchorStyles)((System.Windows.Forms.AnchorStyles.Top |  \
    System.Windows.Forms.AnchorStyles.Right))); this.btnAbrirPdf.Location = new System.Drawing.Point(754, 452); this.btnAbrirPdf.Name = "btnAbrirPdf";  \
    this.btnAbrirPdf.Size = new System.Drawing.Size(232, 44); this.btnAbrirPdf.Text = "Abrir PDF existente"; this.btnAbrirPdf.Click += new  \
    System.EventHandler(this.btnAbrirPdf_Click);
            this.btnImprimir.Anchor = ((System.Windows.Forms.AnchorStyles)((System.Windows.Forms.AnchorStyles.Top |  \
    System.Windows.Forms.AnchorStyles.Right))); this.btnImprimir.Location = new System.Drawing.Point(754, 514); this.btnImprimir.Name = "btnImprimir";  \
    this.btnImprimir.Size = new System.Drawing.Size(232, 44); this.btnImprimir.Text = "Vista previa e imprimir"; this.btnImprimir.Click += new  \
    System.EventHandler(this.btnImprimir_Click);
            this.AutoScaleDimensions = new System.Drawing.SizeF(8F, 20F); this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font; this.BackColor =  \
    System.Drawing.Color.FromArgb(13, 14, 20); this.ClientSize = new System.Drawing.Size(1020, 742); this.Controls.Add(this.btnImprimir);  \
    this.Controls.Add(this.btnAbrirPdf); this.Controls.Add(this.btnDescargar); this.Controls.Add(this.panelVistaPrevia);  \
    this.Controls.Add(this.dgvComprobantes); this.Controls.Add(this.txtBuscar); this.Controls.Add(this.lblTitulo); this.Font = new  \
    System.Drawing.Font("Segoe UI", 9F); this.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245); this.Name = "FrmMisComprobantes"; this.Text =  \
    "Mis comprobantes"; this.Load += new System.EventHandler(this.FrmMisComprobantes_Load); this.panelVistaPrevia.ResumeLayout(false);  \
    ((System.ComponentModel.ISupportInitialize)(this.dgvComprobantes)).EndInit(); this.ResumeLayout(false); this.PerformLayout();
        }
    }
}
```

UI/FrmNomina.cs

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Windows.Forms;
using SistemaGestionNomina.Helpers;
using SistemaGestionNomina.Models;
using SistemaGestionNomina.Services;
using SistemaGestionNomina.Security;

namespace SistemaGestionNomina.UI
{
    public partial class FrmNomina : Form
    {
        private readonly NominaService nominaService = new NominaService();
        private readonly EmpleadoService empleadoService = new EmpleadoService();
        private readonly ExcelExportService excelExportService = new ExcelExportService();
        private readonly PdfExportService pdfExportService = new PdfExportService();
        private readonly AuthorizationService authorizationService = new AuthorizationService();
        private readonly PayrollLifecycleService payrollLifecycleService = new PayrollLifecycleService();
        private Nomina currentNomina;
        private int? currentNominaId;

        public FrmNomina()
        {
            InitializeComponent();
            ControlStyleHelper.ApplyModernForm(this);
        }

        private void FrmNomina_Load(object sender, EventArgs e)
        {
            if (LicenseManager.UsageMode == LicenseUsageMode.Designtime) return;
            btnCalcular.Visible = authorizationService.HasPermission(Permissions.PayrollCalculate);
            btnConfirmarPago.Visible = authorizationService.HasPermission(Permissions.PayrollConfirm);
            btnExportarExcel.Visible = authorizationService.HasPermission(Permissions.PayrollExport);
            btnExportarPdf.Visible = authorizationService.HasPermission(Permissions.PayrollExport);
            btnPagar.Visible = authorizationService.HasPermission(Permissions.PayrollPay);
            btnAnular.Visible = authorizationService.HasPermission(Permissions.PayrollAnnul);
            btnRecalcular.Visible = authorizationService.HasPermission(Permissions.PayrollRecalculate);
            btnVerHistorial.Visible = authorizationService.HasPermission(Permissions.PayrollHistoryView);
            LoadDepartments();
            ActualizarEstadoBotones();
        }

        private void LoadDepartments()
        {
            List<Departamento> departments = new List<Departamento>();
            departments.Add(new Departamento { IdDepartamento = 0, Nombre = "Todos los departamentos" });
            departments.AddRange(empleadoService.GetDepartments());
            cmbDepartamento.DataSource = departments;
            cmbDepartamento.DisplayMember = "Nombre";
            cmbDepartamento.ValueMember = "IdDepartamento";
            cmbPeriodo.Text = "Nómina " + DateTime.Today.ToString("MMMM yyyy");
        }

        private void btnCalcular_Click(object sender, EventArgs e)
        {
            if (!ValidationHelper.ValidateDateRange(dtpFechaInicio.Value.Date, dtpFechaFin.Value.Date))
            {
                return;
            }
```

UI/FrmNomina.cs (continuacion)

```

            try
            {
                int? dep = null;
                if (cmbDepartamento.SelectedValue is int && (int)cmbDepartamento.SelectedValue > 0)
                {
                    dep = (int)cmbDepartamento.SelectedValue;
                }
                currentNomina = nominaService.CalcularNomina(dtpFechaInicio.Value.Date, dtpFechaFin.Value.Date, dep);
                if (!string.IsNullOrWhiteSpace(cmbPeriodo.Text))
                {
                    currentNomina.PeriodoNombre = cmbPeriodo.Text.Trim();
                }
                LoadNominaGrid();
            }
            catch (Exception ex)
            {
                MessageBox.Show(ex.Message, "Nómina", MessageBoxButtons.OK, MessageBoxIcon.Warning);
            }
        }

        private void LoadNominaGrid()
        {
            dgvNominaDetalle.Rows.Clear();
            foreach (NominaDetalle d in currentNomina.Detalles)
            {
                dgvNominaDetalle.Rows.Add(d.CodigoEmpleado, d.EmpleadoNombre, "B/. " + d.SueldoBase.ToString("0.00"),
                    "B/. " + d.Bonos.ToString("0.00"), d.HorasExtra.ToString("0.00"),
                    "B/. " + d.TotalDeducciones.ToString("0.00"), "B/. " + d.NetoPagar.ToString("0.00"));
            }

            lblTotalIngresos.Text = "Ingresos: B/. " + currentNomina.TotalIngresos.ToString("0.00");
            lblTotalDeducciones.Text = "Deducciones: B/. " + currentNomina.TotalDeducciones.ToString("0.00");
            lblTotalNeto.Text = "B/. " + currentNomina.TotalNeto.ToString("0.00");
            lblProcesados.Text = currentNomina.Detalles.Count + " empleados procesados";
            lblEstadoNomina.Text = "Estado: " + (currentNomina.Estado ?? "Calculada");
            ActualizarEstadoBotones();
        }

        private void btnConfirmarPago_Click(object sender, EventArgs e)
        {
            if (!EnsureCalculated()) return;
            if (MessageBox.Show("¿Desea confirmar el pago y generar comprobantes?", "Confirmar pago",
                MessageBoxButtons.YesNo, MessageBoxIcon.Question) != DialogResult.Yes)
            {
                return;
            }

            try
            {
                int id = nominaService.ConfirmarPago(currentNomina, dtpFechaInicio.Value.Date, dtpFechaFin.Value.Date);
                currentNominaId = id;
                currentNomina.Estado = PayrollStates.Confirmed;
                MessageBox.Show("Nómina confirmada correctamente. Id: " + id + "\nSe generaron los comprobantes.\nEl período ha sido cerrado.",
                    "Nómina", MessageBoxButtons.OK, MessageBoxIcon.Information);
                ActualizarEstadoBotones();
            }
            catch (Exception ex)
            {
                MessageBox.Show(ex.Message, "Nómina", MessageBoxButtons.OK, MessageBoxIcon.Warning);
```

UI/FrmNomina.cs (continuacion)

```
            }
        }

        private void btnPagar_Click(object sender, EventArgs e)
        {
            if (!currentNominaId.HasValue)
            {
                MessageBox.Show("Primero debe confirmar la nómina.", "Pago", MessageBoxButtons.OK, MessageBoxIcon.Information);
                return;
            }

            if (MessageBox.Show("¿Registrar el pago de esta nómina? Esta acción no se puede deshacer.",
                "Registrar pago", MessageBoxButtons.YesNo, MessageBoxIcon.Question) != DialogResult.Yes)
            {
                return;
            }

            try
            {
                nominaService.PagarNomina(currentNominaId.Value);
                currentNomina.Estado = PayrollStates.Paid;
                MessageBox.Show("Pago registrado correctamente.", "Nómina", MessageBoxButtons.OK, MessageBoxIcon.Information);
                ActualizarEstadoBotones();
            }
            catch (Exception ex)
            {
                MessageBox.Show(ex.Message, "Nómina", MessageBoxButtons.OK, MessageBoxIcon.Warning);
            }
        }

        private void btnAnular_Click(object sender, EventArgs e)
        {
            if (!currentNominaId.HasValue)
            {
                MessageBox.Show("Primero debe confirmar la nómina.", "Anular", MessageBoxButtons.OK, MessageBoxIcon.Information);
                return;
            }

            using (FrmAnularNomina dialogo = new FrmAnularNomina(currentNomina))
            {
                if (dialogo.ShowDialog() != DialogResult.OK) return;

                try
                {
                    nominaService.AnularNomina(currentNominaId.Value, dialogo.Motivo);
                    currentNomina.Estado = PayrollStates.Annulled;
                    MessageBox.Show("Nómina anulada correctamente.\nEl período ha sido reabierto para corrección.",
                        "Nómina", MessageBoxButtons.OK, MessageBoxIcon.Information);
                    ActualizarEstadoBotones();
                }
                catch (Exception ex)
                {
                    MessageBox.Show(ex.Message, "Nómina", MessageBoxButtons.OK, MessageBoxIcon.Warning);
                }
            }
        }

        private void btnRecalcular_Click(object sender, EventArgs e)
        {
            if (!currentNominaId.HasValue)
```

UI/FrmNomina.cs (continuacion)

```
            {
                MessageBox.Show("Primero debe anular la nómina.", "Recalcular", MessageBoxButtons.OK, MessageBoxIcon.Information);
                return;
            }

            try
            {
                int? dep = null;
                if (cmbDepartamento.SelectedValue is int && (int)cmbDepartamento.SelectedValue > 0)
                    dep = (int)cmbDepartamento.SelectedValue;

                Nomina nuevaNomina = nominaService.CalcularNomina(dtpFechaInicio.Value.Date, dtpFechaFin.Value.Date, dep);
                if (!string.IsNullOrWhiteSpace(cmbPeriodo.Text))
                    nuevaNomina.PeriodoNombre = cmbPeriodo.Text.Trim();

                int idNueva = nominaService.RecalcularNomina(currentNominaId.Value,
                    dtpFechaInicio.Value.Date, dtpFechaFin.Value.Date, nuevaNomina);
                currentNomina = nuevaNomina;
                currentNomina.Estado = PayrollStates.Confirmed;
                currentNominaId = idNueva;
                LoadNominaGrid();
                MessageBox.Show("Nómina recalculada y confirmada correctamente. Id: " + idNueva,
                    "Nómina", MessageBoxButtons.OK, MessageBoxIcon.Information);
                ActualizarEstadoBotones();
            }
            catch (Exception ex)
            {
                MessageBox.Show(ex.Message, "Nómina", MessageBoxButtons.OK, MessageBoxIcon.Warning);
            }
        }

        private void btnVerHistorial_Click(object sender, EventArgs e)
        {
            if (!currentNominaId.HasValue)
            {
                MessageBox.Show("Primero debe confirmar la nómina.", "Historial", MessageBoxButtons.OK, MessageBoxIcon.Information);
                return;
            }

            List<NominaVersion> versiones = nominaService.ObtenerHistorialNomina(currentNominaId.Value);
            if (versiones.Count == 0)
            {
                MessageBox.Show("No hay versiones registradas para esta nómina.", "Historial",
                    MessageBoxButtons.OK, MessageBoxIcon.Information);
                return;
            }

            using (FrmHistorialNomina dialogo = new FrmHistorialNomina(versiones))
            {
                dialogo.ShowDialog();
            }
        }

        private void ActualizarEstadoBotones()
        {
            string estado = currentNomina?.Estado ?? string.Empty;
            bool annulled = string.Equals(estado, PayrollStates.Annulled, StringComparison.OrdinalIgnoreCase);
            btnCalcular.Enabled = authorizationService.HasPermission(Permissions.PayrollCalculate) &&
                (!currentNominaId.HasValue || annulled);
            btnConfirmarPago.Enabled = authorizationService.HasPermission(Permissions.PayrollConfirm) &&
```

UI/FrmNomina.cs (continuacion)

```
                PayrollStateMachine.CanTransition(estado, PayrollStates.Confirmed);
            btnPagar.Enabled = authorizationService.HasPermission(Permissions.PayrollPay) &&
                PayrollStateMachine.CanTransition(estado, PayrollStates.Paid);
            btnAnular.Enabled = authorizationService.HasPermission(Permissions.PayrollAnnul) &&
                PayrollStateMachine.CanTransition(estado, PayrollStates.Annulled);
            btnRecalcular.Enabled = authorizationService.HasPermission(Permissions.PayrollRecalculate) && annulled;
        }

        private void btnExportarExcel_Click(object sender, EventArgs e)
        {
            if (EnsureCalculated()) UiFactory.ShowExported(excelExportService.ExportarNomina(currentNomina));
        }

        private void btnExportarPdf_Click(object sender, EventArgs e)
        {
            if (EnsureCalculated()) UiFactory.ShowExported(pdfExportService.ExportarNomina(currentNomina));
        }

        private bool EnsureCalculated()
        {
            if (currentNomina != null && currentNomina.Detalles.Count > 0) return true;
            MessageBox.Show("Primero calcule la nómina.", "Nómina", MessageBoxButtons.OK, MessageBoxIcon.Information);
            return false;
        }
    }
}
```

UI/FrmNomina.Designer.cs

```
namespace SistemaGestionNomina.UI
{
    partial class FrmNomina
    {
        private System.ComponentModel.IContainer components = null;
        private System.Windows.Forms.Label lblTitulo;
        private System.Windows.Forms.Panel panelParametros;
        private System.Windows.Forms.ComboBox cmbPeriodo;
        private System.Windows.Forms.Label lblPeriodo;
        private System.Windows.Forms.Label lblFechaInicio;
        private System.Windows.Forms.Label lblFechaFin;
        private System.Windows.Forms.Label lblDepartamento;
        private System.Windows.Forms.Label lblResumen;
        private System.Windows.Forms.DateTimePicker dtpFechaInicio;
        private System.Windows.Forms.DateTimePicker dtpFechaFin;
        private System.Windows.Forms.ComboBox cmbDepartamento;
        private System.Windows.Forms.Button btnCalcular;
        private System.Windows.Forms.Panel cardTotales;
        private System.Windows.Forms.Label lblTotalNeto;
        private System.Windows.Forms.Label lblProcesados;
        private System.Windows.Forms.Label lblTotalIngresos;
        private System.Windows.Forms.Label lblTotalDeducciones;
        private System.Windows.Forms.Button btnConfirmarPago;
        private System.Windows.Forms.Button btnExportarExcel;
        private System.Windows.Forms.Button btnExportarPdf;
        private System.Windows.Forms.Button btnPagar;
        private System.Windows.Forms.Button btnAnular;
        private System.Windows.Forms.Button btnRecalcular;
        private System.Windows.Forms.Button btnVerHistorial;
        private System.Windows.Forms.Label lblEstadoNomina;
        private System.Windows.Forms.DataGridView dgvNominaDetalle;

        protected override void Dispose(bool disposing)
        {
            if (disposing)
            {
                if (components != null) components.Dispose();
                if (btnPagar != null) btnPagar.Dispose();
                if (btnAnular != null) btnAnular.Dispose();
                if (btnRecalcular != null) btnRecalcular.Dispose();
                if (btnVerHistorial != null) btnVerHistorial.Dispose();
                if (lblEstadoNomina != null) lblEstadoNomina.Dispose();
            }
            base.Dispose(disposing);
        }

        private void InitializeComponent()
        {
            this.lblTitulo = new System.Windows.Forms.Label();
            this.panelParametros = new System.Windows.Forms.Panel();
            this.cmbPeriodo = new System.Windows.Forms.ComboBox();
            this.lblPeriodo = new System.Windows.Forms.Label();
            this.lblFechaInicio = new System.Windows.Forms.Label();
            this.lblFechaFin = new System.Windows.Forms.Label();
            this.lblDepartamento = new System.Windows.Forms.Label();
            this.lblResumen = new System.Windows.Forms.Label();
            this.dtpFechaInicio = new System.Windows.Forms.DateTimePicker();
            this.dtpFechaFin = new System.Windows.Forms.DateTimePicker();
            this.cmbDepartamento = new System.Windows.Forms.ComboBox();
            this.btnCalcular = new System.Windows.Forms.Button();
```

UI/FrmNomina.Designer.cs (continuacion)

```
            this.cardTotales = new System.Windows.Forms.Panel();
            this.lblTotalNeto = new System.Windows.Forms.Label();
            this.lblProcesados = new System.Windows.Forms.Label();
            this.lblTotalIngresos = new System.Windows.Forms.Label();
            this.lblTotalDeducciones = new System.Windows.Forms.Label();
            this.btnConfirmarPago = new System.Windows.Forms.Button();
            this.btnExportarExcel = new System.Windows.Forms.Button();
            this.btnExportarPdf = new System.Windows.Forms.Button();
            this.btnPagar = new System.Windows.Forms.Button();
            this.btnAnular = new System.Windows.Forms.Button();
            this.btnRecalcular = new System.Windows.Forms.Button();
            this.btnVerHistorial = new System.Windows.Forms.Button();
            this.lblEstadoNomina = new System.Windows.Forms.Label();
            this.dgvNominaDetalle = new System.Windows.Forms.DataGridView();
            this.panelParametros.SuspendLayout();
            this.cardTotales.SuspendLayout();
            ((System.ComponentModel.ISupportInitialize)(this.dgvNominaDetalle)).BeginInit();
            this.SuspendLayout();
            //
            // lblTitulo
            //
            this.lblTitulo.Font = new System.Drawing.Font("Segoe UI", 22F, System.Drawing.FontStyle.Bold);
            this.lblTitulo.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245);
            this.lblTitulo.Location = new System.Drawing.Point(34, 24);
            this.lblTitulo.Name = "lblTitulo";
            this.lblTitulo.Size = new System.Drawing.Size(500, 42);
            this.lblTitulo.TabIndex = 0;
            this.lblTitulo.Text = "Cálculo de nómina";
            //
            // panelParametros
            //
            this.panelParametros.BackColor = System.Drawing.Color.Black;
            this.panelParametros.BorderStyle = System.Windows.Forms.BorderStyle.FixedSingle;
            this.panelParametros.Controls.Add(this.lblPeriodo);
            this.panelParametros.Controls.Add(this.lblFechaInicio);
            this.panelParametros.Controls.Add(this.lblFechaFin);
            this.panelParametros.Controls.Add(this.lblDepartamento);
            this.panelParametros.Controls.Add(this.cmbPeriodo);
            this.panelParametros.Controls.Add(this.dtpFechaInicio);
            this.panelParametros.Controls.Add(this.dtpFechaFin);
            this.panelParametros.Controls.Add(this.cmbDepartamento);
            this.panelParametros.Controls.Add(this.btnCalcular);
            this.panelParametros.Location = new System.Drawing.Point(36, 96);
            this.panelParametros.Name = "panelParametros";
            this.panelParametros.Size = new System.Drawing.Size(330, 250);
            this.panelParametros.TabIndex = 1;
            //
            // cmbPeriodo
            //
            this.cmbPeriodo.BackColor = System.Drawing.Color.FromArgb(24, 25, 34);
            this.cmbPeriodo.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245);
            this.cmbPeriodo.Items.AddRange(new object[] { "Nómina mensual", "Primera quincena", "Segunda quincena" });
            this.cmbPeriodo.Location = new System.Drawing.Point(22, 50);
            this.cmbPeriodo.Name = "cmbPeriodo";
            this.cmbPeriodo.Size = new System.Drawing.Size(280, 23);
            this.cmbPeriodo.TabIndex = 0;
            //
            // dtpFechaInicio
            //
            this.dtpFechaInicio.Format = System.Windows.Forms.DateTimePickerFormat.Short;
```

UI/FrmNomina.Designer.cs (continuacion)

```
            this.dtpFechaInicio.Location = new System.Drawing.Point(22, 104);
            this.dtpFechaInicio.Name = "dtpFechaInicio";
            this.dtpFechaInicio.Size = new System.Drawing.Size(130, 23);
            this.dtpFechaInicio.TabIndex = 1;
            //
            // dtpFechaFin
            //
            this.dtpFechaFin.Format = System.Windows.Forms.DateTimePickerFormat.Short;
            this.dtpFechaFin.Location = new System.Drawing.Point(172, 104);
            this.dtpFechaFin.Name = "dtpFechaFin";
            this.dtpFechaFin.Size = new System.Drawing.Size(130, 23);
            this.dtpFechaFin.TabIndex = 2;
            //
            // cmbDepartamento
            //
            this.cmbDepartamento.DropDownStyle = System.Windows.Forms.ComboBoxStyle.DropDownList;
            this.cmbDepartamento.Location = new System.Drawing.Point(22, 160);
            this.cmbDepartamento.Name = "cmbDepartamento";
            this.cmbDepartamento.Size = new System.Drawing.Size(280, 23);
            this.cmbDepartamento.TabIndex = 3;
            //
            // btnCalcular
            //
            this.btnCalcular.BackColor = System.Drawing.Color.FromArgb(167, 139, 250);
            this.btnCalcular.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
            this.btnCalcular.Font = new System.Drawing.Font("Segoe UI", 10F, System.Drawing.FontStyle.Bold);
            this.btnCalcular.Location = new System.Drawing.Point(22, 198);
            this.btnCalcular.Name = "btnCalcular";
            this.btnCalcular.Size = new System.Drawing.Size(280, 38);
            this.btnCalcular.TabIndex = 4;
            this.btnCalcular.Text = "Calcular nómina";
            this.btnCalcular.UseVisualStyleBackColor = false;
            this.btnCalcular.Click += new System.EventHandler(this.btnCalcular_Click);
            this.lblPeriodo.ForeColor = System.Drawing.Color.FromArgb(170, 170, 185);
            this.lblPeriodo.Location = new System.Drawing.Point(22, 28);
            this.lblPeriodo.Name = "lblPeriodo";
            this.lblPeriodo.Size = new System.Drawing.Size(130, 18);
            this.lblPeriodo.TabIndex = 10;
            this.lblPeriodo.Text = "Periodo";
            this.lblFechaInicio.ForeColor = System.Drawing.Color.FromArgb(170, 170, 185);
            this.lblFechaInicio.Location = new System.Drawing.Point(22, 82);
            this.lblFechaInicio.Name = "lblFechaInicio";
            this.lblFechaInicio.Size = new System.Drawing.Size(130, 18);
            this.lblFechaInicio.TabIndex = 11;
            this.lblFechaInicio.Text = "Fecha inicio";
            this.lblFechaFin.ForeColor = System.Drawing.Color.FromArgb(170, 170, 185);
            this.lblFechaFin.Location = new System.Drawing.Point(172, 82);
            this.lblFechaFin.Name = "lblFechaFin";
            this.lblFechaFin.Size = new System.Drawing.Size(130, 18);
            this.lblFechaFin.TabIndex = 12;
            this.lblFechaFin.Text = "Fecha fin";
            this.lblDepartamento.ForeColor = System.Drawing.Color.FromArgb(170, 170, 185);
            this.lblDepartamento.Location = new System.Drawing.Point(22, 138);
            this.lblDepartamento.Name = "lblDepartamento";
            this.lblDepartamento.Size = new System.Drawing.Size(130, 18);
            this.lblDepartamento.TabIndex = 13;
            this.lblDepartamento.Text = "Departamento";
            //
            // cardTotales
            //
```

UI/FrmNomina.Designer.cs (continuacion)

```
            this.cardTotales.BackColor = System.Drawing.Color.FromArgb(17, 18, 26);
            this.cardTotales.BorderStyle = System.Windows.Forms.BorderStyle.FixedSingle;
            this.cardTotales.Controls.Add(this.lblResumen);
            this.cardTotales.Controls.Add(this.lblTotalNeto);
            this.cardTotales.Controls.Add(this.lblProcesados);
            this.cardTotales.Controls.Add(this.lblTotalIngresos);
            this.cardTotales.Controls.Add(this.lblTotalDeducciones);
            this.cardTotales.Controls.Add(this.btnConfirmarPago);
            this.cardTotales.Controls.Add(this.btnPagar);
            this.cardTotales.Controls.Add(this.btnAnular);
            this.cardTotales.Controls.Add(this.btnRecalcular);
            this.cardTotales.Controls.Add(this.btnVerHistorial);
            this.cardTotales.Controls.Add(this.btnExportarExcel);
            this.cardTotales.Controls.Add(this.btnExportarPdf);
            this.cardTotales.Controls.Add(this.lblEstadoNomina);
            this.cardTotales.Location = new System.Drawing.Point(392, 96);
            this.cardTotales.Name = "cardTotales";
            this.cardTotales.Size = new System.Drawing.Size(592, 250);
            this.cardTotales.TabIndex = 2;
            //
            // lblTotalNeto
            //
            this.lblTotalNeto.Font = new System.Drawing.Font("Segoe UI", 24F, System.Drawing.FontStyle.Bold);
            this.lblTotalNeto.ForeColor = System.Drawing.Color.FromArgb(139, 92, 246);
            this.lblTotalNeto.Location = new System.Drawing.Point(24, 54);
            this.lblTotalNeto.Name = "lblTotalNeto";
            this.lblTotalNeto.Size = new System.Drawing.Size(280, 48);
            this.lblTotalNeto.TabIndex = 0;
            this.lblTotalNeto.Text = "B/. 0.00";
            //
            // lblProcesados
            //
            this.lblProcesados.ForeColor = System.Drawing.Color.FromArgb(170, 170, 185);
            this.lblProcesados.Location = new System.Drawing.Point(28, 104);
            this.lblProcesados.Name = "lblProcesados";
            this.lblProcesados.Size = new System.Drawing.Size(250, 24);
            this.lblProcesados.TabIndex = 1;
            this.lblProcesados.Text = "0 empleados procesados";
            //
            // lblTotalIngresos
            //
            this.lblTotalIngresos.ForeColor = System.Drawing.Color.FromArgb(34, 197, 94);
            this.lblTotalIngresos.Location = new System.Drawing.Point(28, 146);
            this.lblTotalIngresos.Name = "lblTotalIngresos";
            this.lblTotalIngresos.Size = new System.Drawing.Size(230, 26);
            this.lblTotalIngresos.TabIndex = 2;
            this.lblTotalIngresos.Text = "Ingresos: B/. 0.00";
            //
            // lblTotalDeducciones
            //
            this.lblTotalDeducciones.ForeColor = System.Drawing.Color.FromArgb(239, 68, 68);
            this.lblTotalDeducciones.Location = new System.Drawing.Point(28, 180);
            this.lblTotalDeducciones.Name = "lblTotalDeducciones";
            this.lblTotalDeducciones.Size = new System.Drawing.Size(250, 26);
            this.lblTotalDeducciones.TabIndex = 3;
            this.lblTotalDeducciones.Text = "Deducciones: B/. 0.00";
            this.lblResumen.Font = new System.Drawing.Font("Segoe UI", 12F, System.Drawing.FontStyle.Bold);
            this.lblResumen.ForeColor = System.Drawing.Color.FromArgb(170, 170, 185);
            this.lblResumen.Location = new System.Drawing.Point(24, 20);
            this.lblResumen.Name = "lblResumen";
```

UI/FrmNomina.Designer.cs (continuacion)

```
            this.lblResumen.Size = new System.Drawing.Size(260, 26);
            this.lblResumen.TabIndex = 7;
            this.lblResumen.Text = "Neto a pagar";
            //
            // btnConfirmarPago
            //
            this.btnConfirmarPago.BackColor = System.Drawing.Color.FromArgb(167, 139, 250);
            this.btnConfirmarPago.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
            this.btnConfirmarPago.Location = new System.Drawing.Point(390, 34);
            this.btnConfirmarPago.Name = "btnConfirmarPago";
            this.btnConfirmarPago.Size = new System.Drawing.Size(150, 36);
            this.btnConfirmarPago.TabIndex = 4;
            this.btnConfirmarPago.Text = "Confirmar pago";
            this.btnConfirmarPago.UseVisualStyleBackColor = false;
            this.btnConfirmarPago.Click += new System.EventHandler(this.btnConfirmarPago_Click);
            //
            // btnExportarExcel
            //
            this.btnExportarExcel.BackColor = System.Drawing.Color.Black;
            this.btnExportarExcel.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
            this.btnExportarExcel.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245);
            this.btnExportarExcel.Location = new System.Drawing.Point(390, 96);
            this.btnExportarExcel.Name = "btnExportarExcel";
            this.btnExportarExcel.Size = new System.Drawing.Size(80, 32);
            this.btnExportarExcel.TabIndex = 5;
            this.btnExportarExcel.Text = "Excel";
            this.btnExportarExcel.UseVisualStyleBackColor = false;
            this.btnExportarExcel.Click += new System.EventHandler(this.btnExportarExcel_Click);
            //
            // btnExportarPdf
            //
            this.btnExportarPdf.BackColor = System.Drawing.Color.Black;
            this.btnExportarPdf.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
            this.btnExportarPdf.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245);
            this.btnExportarPdf.Location = new System.Drawing.Point(480, 96);
            this.btnExportarPdf.Name = "btnExportarPdf";
            this.btnExportarPdf.Size = new System.Drawing.Size(60, 32);
            this.btnExportarPdf.TabIndex = 6;
            this.btnExportarPdf.Text = "PDF";
            this.btnExportarPdf.UseVisualStyleBackColor = false;
            this.btnExportarPdf.Click += new System.EventHandler(this.btnExportarPdf_Click);
            //
            // btnPagar
            //
            this.btnPagar.BackColor = System.Drawing.Color.FromArgb(34, 197, 94);
            this.btnPagar.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
            this.btnPagar.Font = new System.Drawing.Font("Segoe UI", 9F, System.Drawing.FontStyle.Bold);
            this.btnPagar.ForeColor = System.Drawing.Color.White;
            this.btnPagar.Location = new System.Drawing.Point(390, 140);
            this.btnPagar.Name = "btnPagar";
            this.btnPagar.Size = new System.Drawing.Size(80, 32);
            this.btnPagar.TabIndex = 7;
            this.btnPagar.Text = "Pagar";
            this.btnPagar.UseVisualStyleBackColor = false;
            this.btnPagar.Click += new System.EventHandler(this.btnPagar_Click);
            //
            // btnAnular
            //
            this.btnAnular.BackColor = System.Drawing.Color.FromArgb(239, 68, 68);
            this.btnAnular.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
```

UI/FrmNomina.Designer.cs (continuacion)

```
            this.btnAnular.Font = new System.Drawing.Font("Segoe UI", 9F, System.Drawing.FontStyle.Bold);
            this.btnAnular.ForeColor = System.Drawing.Color.White;
            this.btnAnular.Location = new System.Drawing.Point(480, 140);
            this.btnAnular.Name = "btnAnular";
            this.btnAnular.Size = new System.Drawing.Size(80, 32);
            this.btnAnular.TabIndex = 8;
            this.btnAnular.Text = "Anular";
            this.btnAnular.UseVisualStyleBackColor = false;
            this.btnAnular.Click += new System.EventHandler(this.btnAnular_Click);
            //
            // btnRecalcular
            //
            this.btnRecalcular.BackColor = System.Drawing.Color.FromArgb(250, 204, 21);
            this.btnRecalcular.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
            this.btnRecalcular.Font = new System.Drawing.Font("Segoe UI", 9F, System.Drawing.FontStyle.Bold);
            this.btnRecalcular.ForeColor = System.Drawing.Color.Black;
            this.btnRecalcular.Location = new System.Drawing.Point(390, 180);
            this.btnRecalcular.Name = "btnRecalcular";
            this.btnRecalcular.Size = new System.Drawing.Size(100, 32);
            this.btnRecalcular.TabIndex = 9;
            this.btnRecalcular.Text = "Recalcular";
            this.btnRecalcular.UseVisualStyleBackColor = false;
            this.btnRecalcular.Click += new System.EventHandler(this.btnRecalcular_Click);
            //
            // btnVerHistorial
            //
            this.btnVerHistorial.BackColor = System.Drawing.Color.Black;
            this.btnVerHistorial.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
            this.btnVerHistorial.ForeColor = System.Drawing.Color.FromArgb(167, 139, 250);
            this.btnVerHistorial.Location = new System.Drawing.Point(500, 180);
            this.btnVerHistorial.Name = "btnVerHistorial";
            this.btnVerHistorial.Size = new System.Drawing.Size(80, 32);
            this.btnVerHistorial.TabIndex = 10;
            this.btnVerHistorial.Text = "Historial";
            this.btnVerHistorial.UseVisualStyleBackColor = false;
            this.btnVerHistorial.Click += new System.EventHandler(this.btnVerHistorial_Click);
            //
            // lblEstadoNomina
            //
            this.lblEstadoNomina.Font = new System.Drawing.Font("Segoe UI", 9F, System.Drawing.FontStyle.Bold);
            this.lblEstadoNomina.ForeColor = System.Drawing.Color.FromArgb(167, 139, 250);
            this.lblEstadoNomina.Location = new System.Drawing.Point(24, 214);
            this.lblEstadoNomina.Name = "lblEstadoNomina";
            this.lblEstadoNomina.Size = new System.Drawing.Size(260, 20);
            this.lblEstadoNomina.TabIndex = 11;
            this.lblEstadoNomina.Text = "Estado: —";
            //
            // dgvNominaDetalle
            //
            this.dgvNominaDetalle.AllowUserToAddRows = false;
            this.dgvNominaDetalle.AllowUserToDeleteRows = false;
            this.dgvNominaDetalle.BackgroundColor = System.Drawing.Color.Black;
            this.dgvNominaDetalle.ColumnHeadersHeightSizeMode = System.Windows.Forms.DataGridViewColumnHeadersHeightSizeMode.AutoSize;
            this.dgvNominaDetalle.Location = new System.Drawing.Point(36, 382);
            this.dgvNominaDetalle.Name = "dgvNominaDetalle";
            this.dgvNominaDetalle.ReadOnly = true;
            this.dgvNominaDetalle.RowHeadersVisible = false;
            this.dgvNominaDetalle.Size = new System.Drawing.Size(948, 318);
            this.dgvNominaDetalle.TabIndex = 3;
            this.dgvNominaDetalle.Columns.Add("Codigo", "Código");
```

UI/FrmNomina.Designer.cs (continuacion)

```
            this.dgvNominaDetalle.Columns.Add("Empleado", "Empleado");
            this.dgvNominaDetalle.Columns.Add("SueldoBase", "Sueldo base");
            this.dgvNominaDetalle.Columns.Add("Bonos", "Bonos");
            this.dgvNominaDetalle.Columns.Add("HorasExtra", "Horas extra");
            this.dgvNominaDetalle.Columns.Add("Deducciones", "Deducciones");
            this.dgvNominaDetalle.Columns.Add("Neto", "Neto a pagar");
            //
            // FrmNomina
            //
            this.AutoScaleDimensions = new System.Drawing.SizeF(7F, 15F);
            this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font;
            this.BackColor = System.Drawing.Color.FromArgb(13, 14, 20);
            this.ClientSize = new System.Drawing.Size(1020, 742);
            this.Controls.Add(this.dgvNominaDetalle);
            this.Controls.Add(this.cardTotales);
            this.Controls.Add(this.panelParametros);
            this.Controls.Add(this.lblTitulo);
            this.Font = new System.Drawing.Font("Segoe UI", 9F);
            this.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245);
            this.Name = "FrmNomina";
            this.Text = "Nómina";
            this.Load += new System.EventHandler(this.FrmNomina_Load);
            this.panelParametros.ResumeLayout(false);
            this.cardTotales.ResumeLayout(false);
            ((System.ComponentModel.ISupportInitialize)(this.dgvNominaDetalle)).EndInit();
            this.ResumeLayout(false);
        }
    }
}
```

UI/FrmPortalTrabajador.cs

```
using System;
using System.ComponentModel;
using System.Windows.Forms;
using SistemaGestionNomina.Helpers;
using SistemaGestionNomina.Models;
using SistemaGestionNomina.Services;

namespace SistemaGestionNomina.UI
{
    public partial class FrmPortalTrabajador : Form
    {
        private readonly EmployeePortalService portalService = new EmployeePortalService();
        private readonly Action openProfile;
        private readonly Action openAttendance;
        private readonly Action openPayslips;
        private readonly Action openPassword;
        private readonly Action openAbsences;

        public FrmPortalTrabajador() : this(null, null, null, null, null) { }

        public FrmPortalTrabajador(Action profile, Action attendance, Action payslips, Action password,
            Action absences)
        {
            openProfile = profile;
            openAttendance = attendance;
            openPayslips = payslips;
            openPassword = password;
            openAbsences = absences;
            InitializeComponent();
            ControlStyleHelper.ApplyModernForm(this);
        }

        private void FrmPortalTrabajador_Load(object sender, EventArgs e)
        {
            if (LicenseManager.UsageMode == LicenseUsageMode.Designtime) return;
            try
            {
                EmployeePortalDashboardViewModel model = portalService.GetDashboard();
                lblBienvenida.Text = "Hola, " + model.Profile.NombreCompleto;
                lblCargoValor.Text = model.Profile.Cargo;
                lblDepartamentoValor.Text = model.Profile.Departamento;
                lblCodigoValor.Text = model.Profile.Codigo;
                lblNetoValor.Text = model.LatestPayslip == null
                    ? "Sin comprobantes"
                    : "B/. " + model.LatestPayslip.NetoPagar.ToString("#,##0.00");
                lblPeriodoValor.Text = model.LatestPayslip == null ? "No disponible" : model.LatestPayslip.PeriodoNombre;
                lblAsistenciaValor.Text = model.AttendanceSummary.Puntuales + " puntuales / " +
                    model.AttendanceSummary.Tardanzas + " tardanzas";

                dgvRecientes.Rows.Clear();
                foreach (Asistencia item in model.RecentAttendance)
                {
                    dgvRecientes.Rows.Add(item.Fecha.ToString("dd/MM/yyyy"), item.Estado,
                        item.HoraEntrada.HasValue ? item.HoraEntrada.Value.ToString(@"hh\:mm") : "-",
                        item.HorasTrabajadas.ToString("0.00"));
                }
            }
            catch (Exception ex)
            {
                MessageBox.Show(ex.Message, "Portal del trabajador", MessageBoxButtons.OK, MessageBoxIcon.Warning);
```

UI/FrmPortalTrabajador.cs (continuacion)

```
            }
        }

        private void btnMiPerfil_Click(object sender, EventArgs e) { if (openProfile != null) openProfile(); }
        private void btnMisAsistencias_Click(object sender, EventArgs e) { if (openAttendance != null) openAttendance(); }
        private void btnMisComprobantes_Click(object sender, EventArgs e) { if (openPayslips != null) openPayslips(); }
        private void btnCambiarPassword_Click(object sender, EventArgs e) { if (openPassword != null) openPassword(); }
        private void btnMisSolicitudes_Click(object sender, EventArgs e) { if (openAbsences != null) openAbsences(); }
    }
}
```

UI/FrmPortalTrabajador.Designer.cs

```
namespace SistemaGestionNomina.UI
{
    partial class FrmPortalTrabajador
    {
        private System.ComponentModel.IContainer components = null;
        private System.Windows.Forms.Label lblBienvenida;
        private System.Windows.Forms.Label lblSubtitulo;
        private System.Windows.Forms.Panel panelPerfil;
        private System.Windows.Forms.Label lblPerfilTitulo;
        private System.Windows.Forms.Label lblCargoValor;
        private System.Windows.Forms.Label lblDepartamentoValor;
        private System.Windows.Forms.Label lblCodigoValor;
        private System.Windows.Forms.Panel panelPago;
        private System.Windows.Forms.Label lblPagoTitulo;
        private System.Windows.Forms.Label lblNetoValor;
        private System.Windows.Forms.Label lblPeriodoValor;
        private System.Windows.Forms.Panel panelAsistencia;
        private System.Windows.Forms.Label lblAsistenciaTitulo;
        private System.Windows.Forms.Label lblAsistenciaValor;
        private System.Windows.Forms.DataGridView dgvRecientes;
        private System.Windows.Forms.DataGridViewTextBoxColumn colFecha;
        private System.Windows.Forms.DataGridViewTextBoxColumn colEstado;
        private System.Windows.Forms.DataGridViewTextBoxColumn colEntrada;
        private System.Windows.Forms.DataGridViewTextBoxColumn colHoras;
        private System.Windows.Forms.Button btnMiPerfil;
        private System.Windows.Forms.Button btnMisAsistencias;
        private System.Windows.Forms.Button btnMisComprobantes;
        private System.Windows.Forms.Button btnCambiarPassword;
        private System.Windows.Forms.Button btnMisSolicitudes;

        protected override void Dispose(bool disposing)
        {
            if (disposing && components != null) components.Dispose();
            base.Dispose(disposing);
        }

        private void InitializeComponent()
        {
            this.lblBienvenida = new System.Windows.Forms.Label();
            this.lblSubtitulo = new System.Windows.Forms.Label();
            this.panelPerfil = new System.Windows.Forms.Panel();
            this.lblPerfilTitulo = new System.Windows.Forms.Label();
            this.lblCargoValor = new System.Windows.Forms.Label();
            this.lblDepartamentoValor = new System.Windows.Forms.Label();
            this.lblCodigoValor = new System.Windows.Forms.Label();
            this.panelPago = new System.Windows.Forms.Panel();
            this.lblPagoTitulo = new System.Windows.Forms.Label();
            this.lblNetoValor = new System.Windows.Forms.Label();
            this.lblPeriodoValor = new System.Windows.Forms.Label();
            this.panelAsistencia = new System.Windows.Forms.Panel();
            this.lblAsistenciaTitulo = new System.Windows.Forms.Label();
            this.lblAsistenciaValor = new System.Windows.Forms.Label();
            this.dgvRecientes = new System.Windows.Forms.DataGridView();
            this.colFecha = new System.Windows.Forms.DataGridViewTextBoxColumn();
            this.colEstado = new System.Windows.Forms.DataGridViewTextBoxColumn();
            this.colEntrada = new System.Windows.Forms.DataGridViewTextBoxColumn();
            this.colHoras = new System.Windows.Forms.DataGridViewTextBoxColumn();
            this.btnMiPerfil = new System.Windows.Forms.Button();
            this.btnMisAsistencias = new System.Windows.Forms.Button();
            this.btnMisComprobantes = new System.Windows.Forms.Button();
```

```
            this.btnCambiarPassword = new System.Windows.Forms.Button();
            this.btnMisSolicitudes = new System.Windows.Forms.Button();
            this.panelPerfil.SuspendLayout();
            this.panelPago.SuspendLayout();
            this.panelAsistencia.SuspendLayout();
            ((System.ComponentModel.ISupportInitialize)(this.dgvRecientes)).BeginInit();
            this.SuspendLayout();
            this.lblBienvenida.AutoSize = true; this.lblBienvenida.Font = new System.Drawing.Font("Segoe UI", 23F, System.Drawing.FontStyle.Bold);  \
    this.lblBienvenida.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245); this.lblBienvenida.Location = new System.Drawing.Point(30, 22);  \
    this.lblBienvenida.Name = "lblBienvenida"; this.lblBienvenida.Size = new System.Drawing.Size(355, 52); this.lblBienvenida.Text = "Portal del  \
    trabajador";
            this.lblSubtitulo.AutoSize = true; this.lblSubtitulo.ForeColor = System.Drawing.Color.FromArgb(161, 161, 175); this.lblSubtitulo.Location =  \
    new System.Drawing.Point(34, 76); this.lblSubtitulo.Name = "lblSubtitulo"; this.lblSubtitulo.Size = new System.Drawing.Size(352, 20);  \
    this.lblSubtitulo.Text = "Consulta tu informacion laboral y comprobantes.";
            this.panelPerfil.BackColor = System.Drawing.Color.FromArgb(24, 25, 34); this.panelPerfil.Controls.Add(this.lblPerfilTitulo);  \
    this.panelPerfil.Controls.Add(this.lblCargoValor); this.panelPerfil.Controls.Add(this.lblDepartamentoValor);  \
    this.panelPerfil.Controls.Add(this.lblCodigoValor); this.panelPerfil.Location = new System.Drawing.Point(34, 116); this.panelPerfil.Name =  \
    "panelPerfil"; this.panelPerfil.Size = new System.Drawing.Size(292, 154);
            this.lblPerfilTitulo.AutoSize = true; this.lblPerfilTitulo.Font = new System.Drawing.Font("Segoe UI", 11F, System.Drawing.FontStyle.Bold);  \
    this.lblPerfilTitulo.Location = new System.Drawing.Point(18, 17); this.lblPerfilTitulo.Text = "Resumen laboral";
            this.lblCargoValor.AutoEllipsis = true; this.lblCargoValor.Font = new System.Drawing.Font("Segoe UI", 10F, System.Drawing.FontStyle.Bold);  \
    this.lblCargoValor.Location = new System.Drawing.Point(18, 56); this.lblCargoValor.Size = new System.Drawing.Size(252, 25); this.lblCargoValor.Text =  \
    "Cargo";
            this.lblDepartamentoValor.AutoEllipsis = true; this.lblDepartamentoValor.ForeColor = System.Drawing.Color.FromArgb(161, 161, 175);  \
    this.lblDepartamentoValor.Location = new System.Drawing.Point(18, 86); this.lblDepartamentoValor.Size = new System.Drawing.Size(252, 24);  \
    this.lblDepartamentoValor.Text = "Departamento";
            this.lblCodigoValor.ForeColor = System.Drawing.Color.FromArgb(161, 161, 175); this.lblCodigoValor.Location = new System.Drawing.Point(18,  \
    116); this.lblCodigoValor.Size = new System.Drawing.Size(252, 24); this.lblCodigoValor.Text = "Codigo";
            this.panelPago.BackColor = System.Drawing.Color.FromArgb(24, 25, 34); this.panelPago.Controls.Add(this.lblPagoTitulo);  \
    this.panelPago.Controls.Add(this.lblNetoValor); this.panelPago.Controls.Add(this.lblPeriodoValor); this.panelPago.Location = new  \
    System.Drawing.Point(346, 116); this.panelPago.Name = "panelPago"; this.panelPago.Size = new System.Drawing.Size(292, 154);
            this.lblPagoTitulo.AutoSize = true; this.lblPagoTitulo.Font = new System.Drawing.Font("Segoe UI", 11F, System.Drawing.FontStyle.Bold);  \
    this.lblPagoTitulo.Location = new System.Drawing.Point(18, 17); this.lblPagoTitulo.Text = "Ultimo comprobante";
            this.lblNetoValor.AutoEllipsis = true; this.lblNetoValor.Font = new System.Drawing.Font("Segoe UI", 19F, System.Drawing.FontStyle.Bold);  \
    this.lblNetoValor.ForeColor = System.Drawing.Color.FromArgb(52, 211, 153); this.lblNetoValor.Location = new System.Drawing.Point(18, 52);  \
    this.lblNetoValor.Size = new System.Drawing.Size(252, 44); this.lblNetoValor.Text = "Sin comprobantes";
            this.lblPeriodoValor.ForeColor = System.Drawing.Color.FromArgb(161, 161, 175); this.lblPeriodoValor.Location = new System.Drawing.Point(18,  \
    111); this.lblPeriodoValor.Size = new System.Drawing.Size(252, 24); this.lblPeriodoValor.Text = "Periodo";
            this.panelAsistencia.Anchor = ((System.Windows.Forms.AnchorStyles)((System.Windows.Forms.AnchorStyles.Top |  \
    System.Windows.Forms.AnchorStyles.Right))); this.panelAsistencia.BackColor = System.Drawing.Color.FromArgb(24, 25, 34);  \
    this.panelAsistencia.Controls.Add(this.lblAsistenciaTitulo); this.panelAsistencia.Controls.Add(this.lblAsistenciaValor);  \
    this.panelAsistencia.Location = new System.Drawing.Point(658, 116); this.panelAsistencia.Name = "panelAsistencia"; this.panelAsistencia.Size = new  \
    System.Drawing.Size(328, 154);
            this.lblAsistenciaTitulo.AutoSize = true; this.lblAsistenciaTitulo.Font = new System.Drawing.Font("Segoe UI", 11F,  \
    System.Drawing.FontStyle.Bold); this.lblAsistenciaTitulo.Location = new System.Drawing.Point(18, 17); this.lblAsistenciaTitulo.Text = "Asistencia del  \
    mes";
            this.lblAsistenciaValor.Font = new System.Drawing.Font("Segoe UI", 13F, System.Drawing.FontStyle.Bold); this.lblAsistenciaValor.ForeColor =  \
    System.Drawing.Color.FromArgb(167, 139, 250); this.lblAsistenciaValor.Location = new System.Drawing.Point(18, 59); this.lblAsistenciaValor.Size = new  \
    System.Drawing.Size(286, 62); this.lblAsistenciaValor.Text = "0 puntuales / 0 tardanzas";
            this.btnMiPerfil.Location = new System.Drawing.Point(34, 292); this.btnMiPerfil.Name = "btnMiPerfil"; this.btnMiPerfil.Size = new  \
    System.Drawing.Size(176, 42); this.btnMiPerfil.Text = "Mi perfil"; this.btnMiPerfil.Click += new System.EventHandler(this.btnMiPerfil_Click);
            this.btnMisAsistencias.Location = new System.Drawing.Point(228, 292); this.btnMisAsistencias.Name = "btnMisAsistencias";  \
    this.btnMisAsistencias.Size = new System.Drawing.Size(176, 42); this.btnMisAsistencias.Text = "Mis asistencias"; this.btnMisAsistencias.Click += new  \
    System.EventHandler(this.btnMisAsistencias_Click);
            this.btnMisComprobantes.Location = new System.Drawing.Point(422, 292); this.btnMisComprobantes.Name = "btnMisComprobantes";  \
    this.btnMisComprobantes.Size = new System.Drawing.Size(176, 42); this.btnMisComprobantes.Text = "Mis comprobantes"; this.btnMisComprobantes.Click +=  \
    new System.EventHandler(this.btnMisComprobantes_Click);
            this.btnMisSolicitudes.Location = new System.Drawing.Point(616, 292); this.btnMisSolicitudes.Name = "btnMisSolicitudes";  \
    this.btnMisSolicitudes.Size = new System.Drawing.Size(176, 42); this.btnMisSolicitudes.Text = "Mis solicitudes"; this.btnMisSolicitudes.Click += new  \
    System.EventHandler(this.btnMisSolicitudes_Click);
```

UI/FrmPortalTrabajador.Designer.cs (continuacion)

```
            this.btnCambiarPassword.Location = new System.Drawing.Point(810, 292); this.btnCambiarPassword.Name = "btnCambiarPassword";  \
    this.btnCambiarPassword.Size = new System.Drawing.Size(176, 42); this.btnCambiarPassword.Text = "Cambiar contraseña"; this.btnCambiarPassword.Click  \
    += new System.EventHandler(this.btnCambiarPassword_Click);
            this.dgvRecientes.AllowUserToAddRows = false; this.dgvRecientes.AllowUserToDeleteRows = false; this.dgvRecientes.Anchor =  \
    ((System.Windows.Forms.AnchorStyles)((((System.Windows.Forms.AnchorStyles.Top | System.Windows.Forms.AnchorStyles.Bottom) |  \
    System.Windows.Forms.AnchorStyles.Left) | System.Windows.Forms.AnchorStyles.Right))); this.dgvRecientes.AutoSizeColumnsMode =  \
    System.Windows.Forms.DataGridViewAutoSizeColumnsMode.Fill; this.dgvRecientes.BackgroundColor = System.Drawing.Color.FromArgb(18, 19, 27);  \
    this.dgvRecientes.BorderStyle = System.Windows.Forms.BorderStyle.None; this.dgvRecientes.ColumnHeadersHeight = 40;  \
    this.dgvRecientes.Columns.AddRange(new System.Windows.Forms.DataGridViewColumn[] { this.colFecha, this.colEstado, this.colEntrada, this.colHoras });  \
    this.dgvRecientes.Location = new System.Drawing.Point(34, 362); this.dgvRecientes.Name = "dgvRecientes"; this.dgvRecientes.ReadOnly = true;  \
    this.dgvRecientes.RowHeadersVisible = false; this.dgvRecientes.RowTemplate.Height = 36; this.dgvRecientes.SelectionMode =  \
    System.Windows.Forms.DataGridViewSelectionMode.FullRowSelect; this.dgvRecientes.Size = new System.Drawing.Size(952, 340);
            this.colFecha.HeaderText = "Fecha"; this.colFecha.Name = "colFecha"; this.colFecha.ReadOnly = true;
            this.colEstado.HeaderText = "Estado"; this.colEstado.Name = "colEstado"; this.colEstado.ReadOnly = true;
            this.colEntrada.HeaderText = "Entrada"; this.colEntrada.Name = "colEntrada"; this.colEntrada.ReadOnly = true;
            this.colHoras.HeaderText = "Horas"; this.colHoras.Name = "colHoras"; this.colHoras.ReadOnly = true;
            this.AutoScaleDimensions = new System.Drawing.SizeF(8F, 20F); this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font; this.BackColor =  \
    System.Drawing.Color.FromArgb(13, 14, 20); this.ClientSize = new System.Drawing.Size(1020, 742); this.Controls.Add(this.dgvRecientes);  \
    this.Controls.Add(this.btnCambiarPassword); this.Controls.Add(this.btnMisSolicitudes); this.Controls.Add(this.btnMisComprobantes);  \
    this.Controls.Add(this.btnMisAsistencias); this.Controls.Add(this.btnMiPerfil); this.Controls.Add(this.panelAsistencia);  \
    this.Controls.Add(this.panelPago); this.Controls.Add(this.panelPerfil); this.Controls.Add(this.lblSubtitulo); this.Controls.Add(this.lblBienvenida);  \
    this.Font = new System.Drawing.Font("Segoe UI", 9F); this.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245); this.Name =  \
    "FrmPortalTrabajador"; this.Text = "Portal del trabajador"; this.Load += new System.EventHandler(this.FrmPortalTrabajador_Load);
            this.panelPerfil.ResumeLayout(false); this.panelPerfil.PerformLayout(); this.panelPago.ResumeLayout(false); this.panelPago.PerformLayout();  \
    this.panelAsistencia.ResumeLayout(false); this.panelAsistencia.PerformLayout();  \
    ((System.ComponentModel.ISupportInitialize)(this.dgvRecientes)).EndInit(); this.ResumeLayout(false); this.PerformLayout();
        }
    }
}
```

UI/FrmReportes.cs

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Windows.Forms;
using SistemaGestionNomina.Helpers;
using SistemaGestionNomina.Models;
using SistemaGestionNomina.Services;
using SistemaGestionNomina.Security;

namespace SistemaGestionNomina.UI
{
    public partial class FrmReportes : Form
    {
        private readonly ReporteService reporteService = new ReporteService();
        private readonly EmpleadoService empleadoService = new EmpleadoService();
        private readonly ExcelExportService excelExportService = new ExcelExportService();
        private readonly PdfExportService pdfExportService = new PdfExportService();
        private readonly AuthorizationService authorizationService = new AuthorizationService();

        public FrmReportes()
        {
            InitializeComponent();
            ControlStyleHelper.ApplyModernForm(this);
        }

        private void FrmReportes_Load(object sender, EventArgs e)
        {
            if (LicenseManager.UsageMode != LicenseUsageMode.Designtime)
            {
                bool personal = authorizationService.HasPermission(Permissions.ReportsPersonal);
                bool financial = authorizationService.HasPermission(Permissions.ReportsFinancial);
                cardEmpleados.Visible = personal;
                cardNomina.Visible = financial;
                cardPagos.Visible = financial;
                cardDeducciones.Visible = financial;
                btnExportarExcel.Visible = personal && financial;
                btnExportarPdf.Visible = personal && financial;
                LoadReports();
            }
        }

        private void btnReporteNomina_Click(object sender, EventArgs e)
        {
            authorizationService.DemandPermission(Permissions.ReportsFinancial);
            GenerateReport("Reporte general de nómina", "General", "PDF");
        }

        private void btnReporteEmpleados_Click(object sender, EventArgs e)
        {
            authorizationService.DemandPermission(Permissions.ReportsPersonal);
            GenerateReport("Empleados activos", "Personal", "PDF");
        }

        private void btnReportePagos_Click(object sender, EventArgs e)
        {
            authorizationService.DemandPermission(Permissions.ReportsFinancial);
            GenerateReport("Historial de pagos", "Histórico", "PDF");
        }

        private void btnReporteDeducciones_Click(object sender, EventArgs e)
```

UI/FrmReportes.cs (continuacion)

```
        {
            authorizationService.DemandPermission(Permissions.ReportsFinancial);
            GenerateReport("Resumen de deducciones", "Deducciones", "PDF");
        }

        private void btnExportarExcel_Click(object sender, EventArgs e)
        {
            GenerateReport("Reportes administrativos", "General", "Excel");
        }

        private void btnExportarPdf_Click(object sender, EventArgs e)
        {
            GenerateReport("Reportes administrativos", "General", "PDF");
        }

        private void GenerateReport(string name, string type, string format)
        {
            try
            {
                string path;
                if (name == "Empleados activos" && format == "Excel")
                {
                    List<Empleado> empleados = empleadoService.GetActive(null);
                    if (empleados.Count == 0)
                    {
                        MessageBox.Show("No hay empleados activos para generar el reporte.", "Reportes", MessageBoxButtons.OK, MessageBoxIcon.Information);
                        return;
                    }

                    path = excelExportService.ExportarEmpleados(empleados);
                }
                else if (name == "Empleados activos")
                {
                    List<Empleado> empleados = empleadoService.GetActive(null);
                    if (empleados.Count == 0)
                    {
                        MessageBox.Show("No hay empleados activos para generar el reporte.", "Reportes", MessageBoxButtons.OK, MessageBoxIcon.Information);
                        return;
                    }

                    path = pdfExportService.ExportarEmpleados(empleados);
                }
                else if (format == "Excel")
                {
                    List<ReporteGenerado> reportes = reporteService.GetAll();
                    if (name == "Reportes administrativos" && reportes.Count == 0)
                    {
                        MessageBox.Show("No hay reportes registrados para exportar.", "Reportes", MessageBoxButtons.OK, MessageBoxIcon.Information);
                        return;
                    }

                    path = excelExportService.ExportarReportes(reportes);
                }
                else
                {
                    List<ReporteGenerado> reportes = reporteService.GetAll();
                    if (name == "Reportes administrativos" && reportes.Count == 0)
                    {
                        MessageBox.Show("No hay reportes registrados para exportar.", "Reportes", MessageBoxButtons.OK, MessageBoxIcon.Information);
                        return;
```

UI/FrmReportes.cs (continuacion)

```
                    }

                    path = pdfExportService.ExportarReportes(reportes);
                }

                if (string.IsNullOrWhiteSpace(path)) return;

                reporteService.Register(name, type + " / " + format, path);
                UiFactory.ShowExported(path);
                LoadReports();
            }
            catch (Exception ex)
            {
                MessageBox.Show(ex.Message, "Reportes", MessageBoxButtons.OK, MessageBoxIcon.Warning);
            }
        }

        private void LoadReports()
        {
            List<ReporteGenerado> reports = reporteService.GetAll();
            dgvReportesRecientes.Rows.Clear();
            foreach (ReporteGenerado r in reports)
            {
                dgvReportesRecientes.Rows.Add(r.NombreReporte, r.Tipo, r.GeneradoPor,
                    r.FechaGeneracion.ToString("dd/MM/yyyy HH:mm"), r.RutaArchivo);
            }
        }
    }
}
```

UI/FrmReportes.Designer.cs

```
namespace SistemaGestionNomina.UI
{
    partial class FrmReportes
    {
        private System.ComponentModel.IContainer components = null;
        private System.Windows.Forms.Label lblTitulo;
        private System.Windows.Forms.Panel cardNomina;
        private System.Windows.Forms.Panel cardEmpleados;
        private System.Windows.Forms.Panel cardPagos;
        private System.Windows.Forms.Panel cardDeducciones;
        private System.Windows.Forms.Label lblCardNomina;
        private System.Windows.Forms.Label lblCardEmpleados;
        private System.Windows.Forms.Label lblCardPagos;
        private System.Windows.Forms.Label lblCardDeducciones;
        private System.Windows.Forms.Button btnReporteNomina;
        private System.Windows.Forms.Button btnReporteEmpleados;
        private System.Windows.Forms.Button btnReportePagos;
        private System.Windows.Forms.Button btnReporteDeducciones;
        private System.Windows.Forms.Button btnExportarExcel;
        private System.Windows.Forms.Button btnExportarPdf;
        private System.Windows.Forms.DataGridView dgvReportesRecientes;

        protected override void Dispose(bool disposing)
        {
            if (disposing && (components != null)) components.Dispose();
            base.Dispose(disposing);
        }

        private void InitializeComponent()
        {
            this.lblTitulo = new System.Windows.Forms.Label();
            this.cardNomina = new System.Windows.Forms.Panel();
            this.lblCardNomina = new System.Windows.Forms.Label();
            this.btnReporteNomina = new System.Windows.Forms.Button();
            this.cardEmpleados = new System.Windows.Forms.Panel();
            this.lblCardEmpleados = new System.Windows.Forms.Label();
            this.btnReporteEmpleados = new System.Windows.Forms.Button();
            this.cardPagos = new System.Windows.Forms.Panel();
            this.lblCardPagos = new System.Windows.Forms.Label();
            this.btnReportePagos = new System.Windows.Forms.Button();
            this.cardDeducciones = new System.Windows.Forms.Panel();
            this.lblCardDeducciones = new System.Windows.Forms.Label();
            this.btnReporteDeducciones = new System.Windows.Forms.Button();
            this.btnExportarExcel = new System.Windows.Forms.Button();
            this.btnExportarPdf = new System.Windows.Forms.Button();
            this.dgvReportesRecientes = new System.Windows.Forms.DataGridView();
            this.dataGridViewTextBoxColumn1 = new System.Windows.Forms.DataGridViewTextBoxColumn();
            this.dataGridViewTextBoxColumn2 = new System.Windows.Forms.DataGridViewTextBoxColumn();
            this.dataGridViewTextBoxColumn3 = new System.Windows.Forms.DataGridViewTextBoxColumn();
            this.dataGridViewTextBoxColumn4 = new System.Windows.Forms.DataGridViewTextBoxColumn();
            this.dataGridViewTextBoxColumn5 = new System.Windows.Forms.DataGridViewTextBoxColumn();
            this.cardNomina.SuspendLayout();
            this.cardEmpleados.SuspendLayout();
            this.cardPagos.SuspendLayout();
            this.cardDeducciones.SuspendLayout();
            ((System.ComponentModel.ISupportInitialize)(this.dgvReportesRecientes)).BeginInit();
            this.SuspendLayout();
            //
            // lblTitulo
            //
```

UI/FrmReportes.Designer.cs (continuacion)

```
            this.lblTitulo.Font = new System.Drawing.Font("Segoe UI", 22F, System.Drawing.FontStyle.Bold);
            this.lblTitulo.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(229)))), ((int)(((byte)(228)))), ((int)(((byte)(245)))));
            this.lblTitulo.Location = new System.Drawing.Point(34, 24);
            this.lblTitulo.Name = "lblTitulo";
            this.lblTitulo.Size = new System.Drawing.Size(520, 42);
            this.lblTitulo.TabIndex = 0;
            this.lblTitulo.Text = "Centro de reportes";
            //
            // cardNomina
            //
            this.cardNomina.BackColor = System.Drawing.Color.Black;
            this.cardNomina.BorderStyle = System.Windows.Forms.BorderStyle.FixedSingle;
            this.cardNomina.Controls.Add(this.lblCardNomina);
            this.cardNomina.Controls.Add(this.btnReporteNomina);
            this.cardNomina.Location = new System.Drawing.Point(36, 100);
            this.cardNomina.Name = "cardNomina";
            this.cardNomina.Size = new System.Drawing.Size(220, 150);
            this.cardNomina.TabIndex = 1;
            //
            // lblCardNomina
            //
            this.lblCardNomina.Font = new System.Drawing.Font("Segoe UI", 12F, System.Drawing.FontStyle.Bold);
            this.lblCardNomina.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(229)))), ((int)(((byte)(228)))), ((int)(((byte)(245)))));
            this.lblCardNomina.Location = new System.Drawing.Point(24, 22);
            this.lblCardNomina.Name = "lblCardNomina";
            this.lblCardNomina.Size = new System.Drawing.Size(170, 58);
            this.lblCardNomina.TabIndex = 1;
            this.lblCardNomina.Text = "Reporte general\r\nde nómina";
            //
            // btnReporteNomina
            //
            this.btnReporteNomina.BackColor = System.Drawing.Color.FromArgb(((int)(((byte)(167)))), ((int)(((byte)(139)))), ((int)(((byte)(250)))));
            this.btnReporteNomina.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
            this.btnReporteNomina.Location = new System.Drawing.Point(28, 92);
            this.btnReporteNomina.Name = "btnReporteNomina";
            this.btnReporteNomina.Size = new System.Drawing.Size(160, 34);
            this.btnReporteNomina.TabIndex = 0;
            this.btnReporteNomina.Text = "Generar PDF";
            this.btnReporteNomina.UseVisualStyleBackColor = false;
            this.btnReporteNomina.Click += new System.EventHandler(this.btnReporteNomina_Click);
            //
            // cardEmpleados
            //
            this.cardEmpleados.BackColor = System.Drawing.Color.Black;
            this.cardEmpleados.BorderStyle = System.Windows.Forms.BorderStyle.FixedSingle;
            this.cardEmpleados.Controls.Add(this.lblCardEmpleados);
            this.cardEmpleados.Controls.Add(this.btnReporteEmpleados);
            this.cardEmpleados.Location = new System.Drawing.Point(278, 100);
            this.cardEmpleados.Name = "cardEmpleados";
            this.cardEmpleados.Size = new System.Drawing.Size(220, 150);
            this.cardEmpleados.TabIndex = 2;
            //
            // lblCardEmpleados
            //
            this.lblCardEmpleados.Font = new System.Drawing.Font("Segoe UI", 12F, System.Drawing.FontStyle.Bold);
            this.lblCardEmpleados.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(229)))), ((int)(((byte)(228)))), ((int)(((byte)(245)))));
            this.lblCardEmpleados.Location = new System.Drawing.Point(24, 22);
            this.lblCardEmpleados.Name = "lblCardEmpleados";
            this.lblCardEmpleados.Size = new System.Drawing.Size(170, 58);
            this.lblCardEmpleados.TabIndex = 1;
```

UI/FrmReportes.Designer.cs (continuacion)

```
            this.lblCardEmpleados.Text = "Empleados\r\nactivos";
            //
            // btnReporteEmpleados
            //
            this.btnReporteEmpleados.BackColor = System.Drawing.Color.FromArgb(((int)(((byte)(167)))), ((int)(((byte)(139)))), ((int)(((byte)(250)))));
            this.btnReporteEmpleados.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
            this.btnReporteEmpleados.Location = new System.Drawing.Point(28, 92);
            this.btnReporteEmpleados.Name = "btnReporteEmpleados";
            this.btnReporteEmpleados.Size = new System.Drawing.Size(160, 34);
            this.btnReporteEmpleados.TabIndex = 0;
            this.btnReporteEmpleados.Text = "Generar PDF";
            this.btnReporteEmpleados.UseVisualStyleBackColor = false;
            this.btnReporteEmpleados.Click += new System.EventHandler(this.btnReporteEmpleados_Click);
            //
            // cardPagos
            //
            this.cardPagos.BackColor = System.Drawing.Color.Black;
            this.cardPagos.BorderStyle = System.Windows.Forms.BorderStyle.FixedSingle;
            this.cardPagos.Controls.Add(this.lblCardPagos);
            this.cardPagos.Controls.Add(this.btnReportePagos);
            this.cardPagos.Location = new System.Drawing.Point(520, 100);
            this.cardPagos.Name = "cardPagos";
            this.cardPagos.Size = new System.Drawing.Size(220, 150);
            this.cardPagos.TabIndex = 3;
            //
            // lblCardPagos
            //
            this.lblCardPagos.Font = new System.Drawing.Font("Segoe UI", 12F, System.Drawing.FontStyle.Bold);
            this.lblCardPagos.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(229)))), ((int)(((byte)(228)))), ((int)(((byte)(245)))));
            this.lblCardPagos.Location = new System.Drawing.Point(24, 22);
            this.lblCardPagos.Name = "lblCardPagos";
            this.lblCardPagos.Size = new System.Drawing.Size(170, 67);
            this.lblCardPagos.TabIndex = 1;
            this.lblCardPagos.Text = "Historial\r\nde pagos";
            //
            // btnReportePagos
            //
            this.btnReportePagos.BackColor = System.Drawing.Color.FromArgb(((int)(((byte)(167)))), ((int)(((byte)(139)))), ((int)(((byte)(250)))));
            this.btnReportePagos.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
            this.btnReportePagos.Location = new System.Drawing.Point(28, 92);
            this.btnReportePagos.Name = "btnReportePagos";
            this.btnReportePagos.Size = new System.Drawing.Size(160, 34);
            this.btnReportePagos.TabIndex = 0;
            this.btnReportePagos.Text = "Generar PDF";
            this.btnReportePagos.UseVisualStyleBackColor = false;
            this.btnReportePagos.Click += new System.EventHandler(this.btnReportePagos_Click);
            //
            // cardDeducciones
            //
            this.cardDeducciones.BackColor = System.Drawing.Color.Black;
            this.cardDeducciones.BorderStyle = System.Windows.Forms.BorderStyle.FixedSingle;
            this.cardDeducciones.Controls.Add(this.lblCardDeducciones);
            this.cardDeducciones.Controls.Add(this.btnReporteDeducciones);
            this.cardDeducciones.Location = new System.Drawing.Point(762, 100);
            this.cardDeducciones.Name = "cardDeducciones";
            this.cardDeducciones.Size = new System.Drawing.Size(220, 150);
            this.cardDeducciones.TabIndex = 4;
            //
            // lblCardDeducciones
            //
```

UI/FrmReportes.Designer.cs (continuacion)

```
            this.lblCardDeducciones.Font = new System.Drawing.Font("Segoe UI", 12F, System.Drawing.FontStyle.Bold);
            this.lblCardDeducciones.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(229)))), ((int)(((byte)(228)))), ((int)(((byte)(245)))));
            this.lblCardDeducciones.Location = new System.Drawing.Point(24, 22);
            this.lblCardDeducciones.Name = "lblCardDeducciones";
            this.lblCardDeducciones.Size = new System.Drawing.Size(170, 58);
            this.lblCardDeducciones.TabIndex = 1;
            this.lblCardDeducciones.Text = "Resumen de\r\ndeducciones";
            //
            // btnReporteDeducciones
            //
            this.btnReporteDeducciones.BackColor = System.Drawing.Color.FromArgb(((int)(((byte)(167)))), ((int)(((byte)(139)))), ((int)(((byte)(250)))));
            this.btnReporteDeducciones.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
            this.btnReporteDeducciones.Location = new System.Drawing.Point(28, 92);
            this.btnReporteDeducciones.Name = "btnReporteDeducciones";
            this.btnReporteDeducciones.Size = new System.Drawing.Size(160, 34);
            this.btnReporteDeducciones.TabIndex = 0;
            this.btnReporteDeducciones.Text = "Generar PDF";
            this.btnReporteDeducciones.UseVisualStyleBackColor = false;
            this.btnReporteDeducciones.Click += new System.EventHandler(this.btnReporteDeducciones_Click);
            //
            // btnExportarExcel
            //
            this.btnExportarExcel.BackColor = System.Drawing.Color.Black;
            this.btnExportarExcel.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
            this.btnExportarExcel.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(229)))), ((int)(((byte)(228)))), ((int)(((byte)(245)))));
            this.btnExportarExcel.Location = new System.Drawing.Point(776, 284);
            this.btnExportarExcel.Name = "btnExportarExcel";
            this.btnExportarExcel.Size = new System.Drawing.Size(95, 34);
            this.btnExportarExcel.TabIndex = 5;
            this.btnExportarExcel.Text = "Excel";
            this.btnExportarExcel.UseVisualStyleBackColor = false;
            this.btnExportarExcel.Click += new System.EventHandler(this.btnExportarExcel_Click);
            //
            // btnExportarPdf
            //
            this.btnExportarPdf.BackColor = System.Drawing.Color.Black;
            this.btnExportarPdf.FlatStyle = System.Windows.Forms.FlatStyle.Flat;
            this.btnExportarPdf.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(229)))), ((int)(((byte)(228)))), ((int)(((byte)(245)))));
            this.btnExportarPdf.Location = new System.Drawing.Point(886, 284);
            this.btnExportarPdf.Name = "btnExportarPdf";
            this.btnExportarPdf.Size = new System.Drawing.Size(95, 34);
            this.btnExportarPdf.TabIndex = 6;
            this.btnExportarPdf.Text = "PDF";
            this.btnExportarPdf.UseVisualStyleBackColor = false;
            this.btnExportarPdf.Click += new System.EventHandler(this.btnExportarPdf_Click);
            //
            // dgvReportesRecientes
            //
            this.dgvReportesRecientes.AllowUserToAddRows = false;
            this.dgvReportesRecientes.AllowUserToDeleteRows = false;
            this.dgvReportesRecientes.BackgroundColor = System.Drawing.Color.Black;
            this.dgvReportesRecientes.ColumnHeadersHeightSizeMode = System.Windows.Forms.DataGridViewColumnHeadersHeightSizeMode.AutoSize;
            this.dgvReportesRecientes.Columns.AddRange(new System.Windows.Forms.DataGridViewColumn[] {
            this.dataGridViewTextBoxColumn1,
            this.dataGridViewTextBoxColumn2,
            this.dataGridViewTextBoxColumn3,
            this.dataGridViewTextBoxColumn4,
            this.dataGridViewTextBoxColumn5});
            this.dgvReportesRecientes.Location = new System.Drawing.Point(36, 334);
            this.dgvReportesRecientes.Name = "dgvReportesRecientes";
```

UI/FrmReportes.Designer.cs (continuacion)

```
            this.dgvReportesRecientes.ReadOnly = true;
            this.dgvReportesRecientes.RowHeadersVisible = false;
            this.dgvReportesRecientes.RowHeadersWidth = 51;
            this.dgvReportesRecientes.Size = new System.Drawing.Size(946, 330);
            this.dgvReportesRecientes.TabIndex = 7;
            //
            // dataGridViewTextBoxColumn1
            //
            this.dataGridViewTextBoxColumn1.HeaderText = "Nombre del reporte";
            this.dataGridViewTextBoxColumn1.MinimumWidth = 6;
            this.dataGridViewTextBoxColumn1.Name = "dataGridViewTextBoxColumn1";
            this.dataGridViewTextBoxColumn1.ReadOnly = true;
            this.dataGridViewTextBoxColumn1.Width = 125;
            //
            // dataGridViewTextBoxColumn2
            //
            this.dataGridViewTextBoxColumn2.HeaderText = "Tipo";
            this.dataGridViewTextBoxColumn2.MinimumWidth = 6;
            this.dataGridViewTextBoxColumn2.Name = "dataGridViewTextBoxColumn2";
            this.dataGridViewTextBoxColumn2.ReadOnly = true;
            this.dataGridViewTextBoxColumn2.Width = 125;
            //
            // dataGridViewTextBoxColumn3
            //
            this.dataGridViewTextBoxColumn3.HeaderText = "Generado por";
            this.dataGridViewTextBoxColumn3.MinimumWidth = 6;
            this.dataGridViewTextBoxColumn3.Name = "dataGridViewTextBoxColumn3";
            this.dataGridViewTextBoxColumn3.ReadOnly = true;
            this.dataGridViewTextBoxColumn3.Width = 125;
            //
            // dataGridViewTextBoxColumn4
            //
            this.dataGridViewTextBoxColumn4.HeaderText = "Fecha";
            this.dataGridViewTextBoxColumn4.MinimumWidth = 6;
            this.dataGridViewTextBoxColumn4.Name = "dataGridViewTextBoxColumn4";
            this.dataGridViewTextBoxColumn4.ReadOnly = true;
            this.dataGridViewTextBoxColumn4.Width = 125;
            //
            // dataGridViewTextBoxColumn5
            //
            this.dataGridViewTextBoxColumn5.HeaderText = "Ruta";
            this.dataGridViewTextBoxColumn5.MinimumWidth = 6;
            this.dataGridViewTextBoxColumn5.Name = "dataGridViewTextBoxColumn5";
            this.dataGridViewTextBoxColumn5.ReadOnly = true;
            this.dataGridViewTextBoxColumn5.Width = 125;
            //
            // FrmReportes
            //
            this.AutoScaleDimensions = new System.Drawing.SizeF(8F, 20F);
            this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font;
            this.BackColor = System.Drawing.Color.FromArgb(((int)(((byte)(13)))), ((int)(((byte)(14)))), ((int)(((byte)(20)))));
            this.ClientSize = new System.Drawing.Size(1020, 742);
            this.Controls.Add(this.dgvReportesRecientes);
            this.Controls.Add(this.btnExportarPdf);
            this.Controls.Add(this.btnExportarExcel);
            this.Controls.Add(this.cardDeducciones);
            this.Controls.Add(this.cardPagos);
            this.Controls.Add(this.cardEmpleados);
            this.Controls.Add(this.cardNomina);
            this.Controls.Add(this.lblTitulo);
```

UI/FrmReportes.Designer.cs (continuacion)

```
            this.Font = new System.Drawing.Font("Segoe UI", 9F);
            this.ForeColor = System.Drawing.Color.FromArgb(((int)(((byte)(229)))), ((int)(((byte)(228)))), ((int)(((byte)(245)))));
            this.Name = "FrmReportes";
            this.Text = "Reportes";
            this.Load += new System.EventHandler(this.FrmReportes_Load);
            this.cardNomina.ResumeLayout(false);
            this.cardEmpleados.ResumeLayout(false);
            this.cardPagos.ResumeLayout(false);
            this.cardDeducciones.ResumeLayout(false);
            ((System.ComponentModel.ISupportInitialize)(this.dgvReportesRecientes)).EndInit();
            this.ResumeLayout(false);

        }

        private System.Windows.Forms.DataGridViewTextBoxColumn dataGridViewTextBoxColumn1;
        private System.Windows.Forms.DataGridViewTextBoxColumn dataGridViewTextBoxColumn2;
        private System.Windows.Forms.DataGridViewTextBoxColumn dataGridViewTextBoxColumn3;
        private System.Windows.Forms.DataGridViewTextBoxColumn dataGridViewTextBoxColumn4;
        private System.Windows.Forms.DataGridViewTextBoxColumn dataGridViewTextBoxColumn5;
    }
}
```

UI/FrmSolicitudAusenciaDetalle.cs

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Windows.Forms;
using SistemaGestionNomina.Helpers;
using SistemaGestionNomina.Models;
using SistemaGestionNomina.Security;
using SistemaGestionNomina.Services;

namespace SistemaGestionNomina.UI
{
    public partial class FrmSolicitudAusenciaDetalle : Form
    {
        private readonly SolicitudAusencia existingRequest;
        private readonly AbsenceRequestService absenceService = new AbsenceRequestService();
        private readonly EmpleadoService employeeService = new EmpleadoService();
        public bool RequestCreated { get; private set; }

        public FrmSolicitudAusenciaDetalle() : this(null) { }

        public FrmSolicitudAusenciaDetalle(SolicitudAusencia item)
        {
            existingRequest = item;
            InitializeComponent();
            ControlStyleHelper.ApplyModernForm(this);
        }

        private void FrmSolicitudAusenciaDetalle_Load(object sender, EventArgs e)
        {
            if (LicenseManager.UsageMode == LicenseUsageMode.Designtime) return;
            cmbTipo.DataSource = new List<string>(AbsenceTypes.All);
            if (existingRequest != null)
            {
                Text = "Detalle de solicitud";
                lblTitulo.Text = "Detalle de la solicitud";
                cmbEmpleado.Items.Add(existingRequest.CodigoEmpleado + " - " + existingRequest.EmpleadoNombre);
                cmbEmpleado.SelectedIndex = 0;
                cmbTipo.SelectedItem = existingRequest.Tipo;
                dtpInicio.Value = existingRequest.FechaInicio;
                dtpFin.Value = existingRequest.FechaFin;
                txtMotivo.Text = existingRequest.Motivo;
                txtResolucion.Text = existingRequest.ObservacionResolucion;
                lblEstado.Text = "Estado: " + existingRequest.Estado;
                SetReadOnly();
                return;
            }

            lblEstado.Text = "Estado inicial: Pendiente";
            if (string.Equals(SessionContext.Role, Roles.Trabajador, StringComparison.OrdinalIgnoreCase))
            {
                cmbEmpleado.Items.Add("Empleado asociado a la sesión");
                cmbEmpleado.SelectedIndex = 0;
                cmbEmpleado.Enabled = false;
            }
            else
            {
                List<Empleado> employees = employeeService.GetActive(null);
                cmbEmpleado.DataSource = employees;
                cmbEmpleado.DisplayMember = "NombreCompleto";
                cmbEmpleado.ValueMember = "IdEmpleado";
```

UI/FrmSolicitudAusenciaDetalle.cs (continuacion)

```
            }
        }

        private void btnGuardar_Click(object sender, EventArgs e)
        {
            try
            {
                string type = Convert.ToString(cmbTipo.SelectedItem);
                if (string.Equals(SessionContext.Role, Roles.Trabajador, StringComparison.OrdinalIgnoreCase))
                {
                    absenceService.CreateOwn(new OwnAbsenceRequest
                    {
                        Type = type,
                        StartDate = dtpInicio.Value.Date,
                        EndDate = dtpFin.Value.Date,
                        Reason = txtMotivo.Text
                    });
                }
                else
                {
                    if (!(cmbEmpleado.SelectedValue is int))
                        throw new ArgumentException("Seleccione un empleado.");
                    absenceService.CreateForEmployee(new SolicitudAusencia
                    {
                        IdEmpleado = (int)cmbEmpleado.SelectedValue,
                        Tipo = type,
                        FechaInicio = dtpInicio.Value.Date,
                        FechaFin = dtpFin.Value.Date,
                        Motivo = txtMotivo.Text
                    });
                }
                RequestCreated = true;
                MessageBox.Show("Solicitud registrada en estado Pendiente.", "Ausencias",
                    MessageBoxButtons.OK, MessageBoxIcon.Information);
                DialogResult = DialogResult.OK;
                Close();
            }
            catch (Exception ex)
            {
                MessageBox.Show(ex.Message, "Ausencias", MessageBoxButtons.OK, MessageBoxIcon.Warning);
            }
        }

        private void btnCerrar_Click(object sender, EventArgs e) { Close(); }

        private void SetReadOnly()
        {
            cmbEmpleado.Enabled = false;
            cmbTipo.Enabled = false;
            dtpInicio.Enabled = false;
            dtpFin.Enabled = false;
            txtMotivo.ReadOnly = true;
            txtResolucion.ReadOnly = true;
            btnGuardar.Visible = false;
        }
    }
}
```

UI/FrmSolicitudAusenciaDetalle.Designer.cs

```
namespace SistemaGestionNomina.UI
{
    partial class FrmSolicitudAusenciaDetalle
    {
        private System.ComponentModel.IContainer components = null;
        private System.Windows.Forms.Label lblTitulo, lblEmpleado, lblTipo, lblInicio, lblFin, lblMotivo, lblResolucion, lblEstado;
        private System.Windows.Forms.ComboBox cmbEmpleado, cmbTipo;
        private System.Windows.Forms.DateTimePicker dtpInicio, dtpFin;
        private System.Windows.Forms.TextBox txtMotivo, txtResolucion;
        private System.Windows.Forms.Button btnGuardar, btnCerrar;
        protected override void Dispose(bool disposing) { if (disposing && components != null) components.Dispose(); base.Dispose(disposing); }
        private void InitializeComponent()
        {
            this.lblTitulo = new System.Windows.Forms.Label(); this.lblEmpleado = new System.Windows.Forms.Label(); this.lblTipo = new  \
    System.Windows.Forms.Label(); this.lblInicio = new System.Windows.Forms.Label(); this.lblFin = new System.Windows.Forms.Label(); this.lblMotivo = new  \
    System.Windows.Forms.Label(); this.lblResolucion = new System.Windows.Forms.Label(); this.lblEstado = new System.Windows.Forms.Label();
            this.cmbEmpleado = new System.Windows.Forms.ComboBox(); this.cmbTipo = new System.Windows.Forms.ComboBox(); this.dtpInicio = new  \
    System.Windows.Forms.DateTimePicker(); this.dtpFin = new System.Windows.Forms.DateTimePicker(); this.txtMotivo = new System.Windows.Forms.TextBox();  \
    this.txtResolucion = new System.Windows.Forms.TextBox(); this.btnGuardar = new System.Windows.Forms.Button(); this.btnCerrar = new  \
    System.Windows.Forms.Button(); this.SuspendLayout();
            this.BackColor = System.Drawing.Color.FromArgb(18, 18, 27); this.ClientSize = new System.Drawing.Size(650, 510); this.Font = new  \
    System.Drawing.Font("Segoe UI", 9F); this.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245); this.FormBorderStyle =  \
    System.Windows.Forms.FormBorderStyle.FixedDialog; this.MaximizeBox = false; this.MinimizeBox = false; this.Name = "FrmSolicitudAusenciaDetalle";  \
    this.StartPosition = System.Windows.Forms.FormStartPosition.CenterParent; this.Text = "Nueva solicitud de ausencia"; this.Load += new  \
    System.EventHandler(this.FrmSolicitudAusenciaDetalle_Load);
            this.lblTitulo.AutoSize = true; this.lblTitulo.Font = new System.Drawing.Font("Segoe UI", 20F, System.Drawing.FontStyle.Bold);  \
    this.lblTitulo.Location = new System.Drawing.Point(28, 20); this.lblTitulo.Text = "Nueva solicitud de ausencia";
            this.lblEstado.AutoSize = true; this.lblEstado.ForeColor = System.Drawing.Color.FromArgb(167, 139, 250); this.lblEstado.Location = new  \
    System.Drawing.Point(32, 66); this.lblEstado.Text = "Estado inicial: Pendiente";
            this.lblEmpleado.AutoSize = true; this.lblEmpleado.Location = new System.Drawing.Point(32, 104); this.lblEmpleado.Text = "Empleado";  \
    this.cmbEmpleado.DropDownStyle = System.Windows.Forms.ComboBoxStyle.DropDownList; this.cmbEmpleado.Location = new System.Drawing.Point(35, 126);  \
    this.cmbEmpleado.Size = new System.Drawing.Size(278, 23);
            this.lblTipo.AutoSize = true; this.lblTipo.Location = new System.Drawing.Point(329, 104); this.lblTipo.Text = "Tipo";  \
    this.cmbTipo.DropDownStyle = System.Windows.Forms.ComboBoxStyle.DropDownList; this.cmbTipo.Location = new System.Drawing.Point(332, 126);  \
    this.cmbTipo.Size = new System.Drawing.Size(278, 23);
            this.lblInicio.AutoSize = true; this.lblInicio.Location = new System.Drawing.Point(32, 170); this.lblInicio.Text = "Fecha inicial";  \
    this.dtpInicio.Format = System.Windows.Forms.DateTimePickerFormat.Short; this.dtpInicio.Location = new System.Drawing.Point(35, 192);  \
    this.dtpInicio.Size = new System.Drawing.Size(180, 23);
            this.lblFin.AutoSize = true; this.lblFin.Location = new System.Drawing.Point(230, 170); this.lblFin.Text = "Fecha final"; this.dtpFin.Format  \
    = System.Windows.Forms.DateTimePickerFormat.Short; this.dtpFin.Location = new System.Drawing.Point(233, 192); this.dtpFin.Size = new  \
    System.Drawing.Size(180, 23);
            this.lblMotivo.AutoSize = true; this.lblMotivo.Location = new System.Drawing.Point(32, 236); this.lblMotivo.Text = "Motivo";  \
    this.txtMotivo.Location = new System.Drawing.Point(35, 258); this.txtMotivo.MaxLength = 500; this.txtMotivo.Multiline = true; this.txtMotivo.Size =  \
    new System.Drawing.Size(575, 76);
            this.lblResolucion.AutoSize = true; this.lblResolucion.Location = new System.Drawing.Point(32, 350); this.lblResolucion.Text = "Observación  \
    de resolución"; this.txtResolucion.Location = new System.Drawing.Point(35, 372); this.txtResolucion.Multiline = true; this.txtResolucion.ReadOnly =  \
    true; this.txtResolucion.Size = new System.Drawing.Size(575, 58);
            this.btnGuardar.BackColor = System.Drawing.Color.FromArgb(124, 58, 237); this.btnGuardar.FlatStyle = System.Windows.Forms.FlatStyle.Flat;  \
    this.btnGuardar.Location = new System.Drawing.Point(414, 452); this.btnGuardar.Size = new System.Drawing.Size(94, 34); this.btnGuardar.Text =  \
    "Registrar"; this.btnGuardar.UseVisualStyleBackColor = false; this.btnGuardar.Click += new System.EventHandler(this.btnGuardar_Click);
            this.btnCerrar.FlatStyle = System.Windows.Forms.FlatStyle.Flat; this.btnCerrar.Location = new System.Drawing.Point(516, 452);  \
    this.btnCerrar.Size = new System.Drawing.Size(94, 34); this.btnCerrar.Text = "Cerrar"; this.btnCerrar.Click += new  \
    System.EventHandler(this.btnCerrar_Click);
            this.Controls.Add(this.lblTitulo); this.Controls.Add(this.lblEstado); this.Controls.Add(this.lblEmpleado);  \
    this.Controls.Add(this.cmbEmpleado); this.Controls.Add(this.lblTipo); this.Controls.Add(this.cmbTipo); this.Controls.Add(this.lblInicio);  \
    this.Controls.Add(this.dtpInicio); this.Controls.Add(this.lblFin); this.Controls.Add(this.dtpFin); this.Controls.Add(this.lblMotivo);  \
    this.Controls.Add(this.txtMotivo); this.Controls.Add(this.lblResolucion); this.Controls.Add(this.txtResolucion); this.Controls.Add(this.btnGuardar);  \
    this.Controls.Add(this.btnCerrar); this.ResumeLayout(false); this.PerformLayout();
        }
    }
```

UI/FrmSolicitudAusenciaDetalle.Designer.cs (continuacion)

```
}
```

UI/FrmSolicitudesAusencia.cs

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Windows.Forms;
using SistemaGestionNomina.Helpers;
using SistemaGestionNomina.Models;
using SistemaGestionNomina.Security;
using SistemaGestionNomina.Services;

namespace SistemaGestionNomina.UI
{
    public partial class FrmSolicitudesAusencia : Form
    {
        private readonly AbsenceRequestService service = new AbsenceRequestService();
        private readonly EmpleadoService employeeService = new EmpleadoService();
        private readonly ExcelExportService excelExportService = new ExcelExportService();
        private readonly PdfExportService pdfExportService = new PdfExportService();
        private readonly AuthorizationService authorizationService = new AuthorizationService();
        private List<SolicitudAusencia> currentRequests = new List<SolicitudAusencia>();

        public FrmSolicitudesAusencia()
        {
            InitializeComponent();
            ControlStyleHelper.ApplyModernForm(this);
        }

        private void FrmSolicitudesAusencia_Load(object sender, EventArgs e)
        {
            if (LicenseManager.UsageMode == LicenseUsageMode.Designtime) return;
            cmbTipo.Items.Add("Todos");
            cmbTipo.Items.AddRange(AbsenceTypes.All);
            cmbTipo.SelectedIndex = 0;
            cmbEstado.Items.Add("Todos");
            cmbEstado.Items.AddRange(AbsenceStates.All);
            cmbEstado.SelectedIndex = 0;

            bool worker = string.Equals(SessionContext.Role, Roles.Trabajador, StringComparison.OrdinalIgnoreCase);
            if (worker)
            {
                cmbDepartamento.Items.Add("Mi información");
                cmbDepartamento.SelectedIndex = 0;
                cmbDepartamento.Enabled = false;
                btnNuevo.Visible = authorizationService.HasPermission(Permissions.OwnAbsencesCreate);
                btnGestionar.Visible = false;
                btnCancelar.Visible = authorizationService.HasPermission(Permissions.OwnAbsencesCancel);
                btnExcel.Visible = false;
                btnPdf.Visible = false;
                lblTitulo.Text = "Mis solicitudes de ausencia";
            }
            else
            {
                List<Departamento> departments = new List<Departamento>();
                departments.Add(new Departamento { IdDepartamento = 0, Nombre = "Todos los departamentos" });
                departments.AddRange(employeeService.GetDepartments());
                cmbDepartamento.DataSource = departments;
                cmbDepartamento.DisplayMember = "Nombre";
                cmbDepartamento.ValueMember = "IdDepartamento";
                btnNuevo.Visible = authorizationService.HasPermission(Permissions.AbsencesCreate);
                btnGestionar.Visible = authorizationService.HasPermission(Permissions.AbsencesApprove) ||
                    authorizationService.HasPermission(Permissions.AbsencesReject);
```

UI/FrmSolicitudesAusencia.cs (continuacion)

```
                btnCancelar.Visible = authorizationService.HasPermission(Permissions.AbsencesCancel);
                btnExcel.Visible = authorizationService.HasPermission(Permissions.AbsencesExport);
                btnPdf.Visible = btnExcel.Visible;
            }
            LoadRequests();
        }

        private void btnBuscar_Click(object sender, EventArgs e) { LoadRequests(); }

        private void LoadRequests()
        {
            try
            {
                int? departmentId = null;
                if (cmbDepartamento.SelectedValue is int && (int)cmbDepartamento.SelectedValue > 0)
                    departmentId = (int)cmbDepartamento.SelectedValue;
                currentRequests = service.Search(new AbsenceQuery
                {
                    DepartmentId = departmentId,
                    Type = cmbTipo.SelectedIndex > 0 ? Convert.ToString(cmbTipo.SelectedItem) : string.Empty,
                    Status = cmbEstado.SelectedIndex > 0 ? Convert.ToString(cmbEstado.SelectedItem) : string.Empty,
                    From = dtpDesde.Checked ? (DateTime?)dtpDesde.Value.Date : null,
                    To = dtpHasta.Checked ? (DateTime?)dtpHasta.Value.Date : null,
                    Search = txtBuscar.Text.Trim()
                });
                dgvSolicitudes.Rows.Clear();
                for (int i = 0; i < currentRequests.Count; i++)
                {
                    SolicitudAusencia item = currentRequests[i];
                    dgvSolicitudes.Rows.Add(item.IdSolicitud, item.CodigoEmpleado, item.EmpleadoNombre,
                        item.DepartamentoNombre, item.Tipo, item.FechaInicio.ToString("dd/MM/yyyy"),
                        item.FechaFin.ToString("dd/MM/yyyy"), item.Estado, item.UsuarioSolicitante,
                        item.FechaCreacion.ToString("dd/MM/yyyy HH:mm"));
                }
                lblResultado.Text = currentRequests.Count + " solicitudes";
            }
            catch (Exception ex)
            {
                MessageBox.Show(ex.Message, "Ausencias", MessageBoxButtons.OK, MessageBoxIcon.Warning);
            }
        }

        private void btnNuevo_Click(object sender, EventArgs e)
        {
            using (FrmSolicitudAusenciaDetalle form = new FrmSolicitudAusenciaDetalle())
            {
                if (form.ShowDialog() == DialogResult.OK) LoadRequests();
            }
        }

        private void btnVer_Click(object sender, EventArgs e)
        {
            int id = SelectedRequestId();
            if (id == 0) return;
            try
            {
                using (FrmSolicitudAusenciaDetalle form = new FrmSolicitudAusenciaDetalle(service.GetById(id)))
                    form.ShowDialog();
            }
            catch (Exception ex) { MessageBox.Show(ex.Message, "Ausencias", MessageBoxButtons.OK, MessageBoxIcon.Warning); }
```

UI/FrmSolicitudesAusencia.cs (continuacion)

```
        }

        private void btnGestionar_Click(object sender, EventArgs e) { OpenManagement(false); }
        private void btnCancelar_Click(object sender, EventArgs e) { OpenManagement(true); }

        private void OpenManagement(bool cancelOnly)
        {
            int id = SelectedRequestId();
            if (id == 0) return;
            using (FrmGestionAusencias form = new FrmGestionAusencias(id, cancelOnly))
            {
                if (form.ShowDialog() == DialogResult.OK) LoadRequests();
            }
        }

        private int SelectedRequestId()
        {
            if (dgvSolicitudes.CurrentRow == null || dgvSolicitudes.CurrentRow.Cells["IdSolicitud"].Value == null)
            {
                MessageBox.Show("Seleccione una solicitud.", "Ausencias", MessageBoxButtons.OK, MessageBoxIcon.Information);
                return 0;
            }
            return Convert.ToInt32(dgvSolicitudes.CurrentRow.Cells["IdSolicitud"].Value);
        }

        private void btnExcel_Click(object sender, EventArgs e)
        {
            if (currentRequests.Count > 0) UiFactory.ShowExported(excelExportService.ExportarAusencias(currentRequests));
        }

        private void btnPdf_Click(object sender, EventArgs e)
        {
            if (currentRequests.Count > 0) UiFactory.ShowExported(pdfExportService.ExportarAusencias(currentRequests));
        }
    }
}
```

UI/FrmSolicitudesAusencia.Designer.cs

```
namespace SistemaGestionNomina.UI
{
    partial class FrmSolicitudesAusencia
    {
        private System.ComponentModel.IContainer components = null;
        private System.Windows.Forms.Label lblTitulo, lblResultado;
        private System.Windows.Forms.TextBox txtBuscar;
        private System.Windows.Forms.ComboBox cmbTipo, cmbEstado, cmbDepartamento;
        private System.Windows.Forms.DateTimePicker dtpDesde, dtpHasta;
        private System.Windows.Forms.Button btnBuscar, btnNuevo, btnVer, btnGestionar, btnCancelar, btnExcel, btnPdf;
        private System.Windows.Forms.DataGridView dgvSolicitudes;
        protected override void Dispose(bool disposing) { if (disposing && components != null) components.Dispose(); base.Dispose(disposing); }
        private void InitializeComponent()
        {
            this.lblTitulo = new System.Windows.Forms.Label(); this.lblResultado = new System.Windows.Forms.Label(); this.txtBuscar = new  \
    System.Windows.Forms.TextBox(); this.cmbTipo = new System.Windows.Forms.ComboBox(); this.cmbEstado = new System.Windows.Forms.ComboBox();  \
    this.cmbDepartamento = new System.Windows.Forms.ComboBox(); this.dtpDesde = new System.Windows.Forms.DateTimePicker(); this.dtpHasta = new  \
    System.Windows.Forms.DateTimePicker(); this.btnBuscar = new System.Windows.Forms.Button(); this.btnNuevo = new System.Windows.Forms.Button();  \
    this.btnVer = new System.Windows.Forms.Button(); this.btnGestionar = new System.Windows.Forms.Button(); this.btnCancelar = new  \
    System.Windows.Forms.Button(); this.btnExcel = new System.Windows.Forms.Button(); this.btnPdf = new System.Windows.Forms.Button();  \
    this.dgvSolicitudes = new System.Windows.Forms.DataGridView(); ((System.ComponentModel.ISupportInitialize)(this.dgvSolicitudes)).BeginInit();  \
    this.SuspendLayout();
            this.BackColor = System.Drawing.Color.FromArgb(12, 12, 18); this.ClientSize = new System.Drawing.Size(1120, 720); this.Font = new  \
    System.Drawing.Font("Segoe UI", 9F); this.ForeColor = System.Drawing.Color.FromArgb(229, 228, 245); this.Name = "FrmSolicitudesAusencia"; this.Text =  \
    "Solicitudes de ausencia"; this.Load += new System.EventHandler(this.FrmSolicitudesAusencia_Load);
            this.lblTitulo.AutoSize = true; this.lblTitulo.Font = new System.Drawing.Font("Segoe UI", 24F, System.Drawing.FontStyle.Bold);  \
    this.lblTitulo.Location = new System.Drawing.Point(28, 20); this.lblTitulo.Text = "Solicitudes de ausencia";
            this.txtBuscar.Location = new System.Drawing.Point(34, 82); this.txtBuscar.Size = new System.Drawing.Size(160, 23);
            this.cmbDepartamento.DropDownStyle = System.Windows.Forms.ComboBoxStyle.DropDownList; this.cmbDepartamento.Location = new  \
    System.Drawing.Point(204, 82); this.cmbDepartamento.Size = new System.Drawing.Size(170, 23);
            this.cmbTipo.DropDownStyle = System.Windows.Forms.ComboBoxStyle.DropDownList; this.cmbTipo.Location = new System.Drawing.Point(384, 82);  \
    this.cmbTipo.Size = new System.Drawing.Size(155, 23);
            this.cmbEstado.DropDownStyle = System.Windows.Forms.ComboBoxStyle.DropDownList; this.cmbEstado.Location = new System.Drawing.Point(549, 82);  \
    this.cmbEstado.Size = new System.Drawing.Size(112, 23);
            this.dtpDesde.Format = System.Windows.Forms.DateTimePickerFormat.Short; this.dtpDesde.Location = new System.Drawing.Point(671, 82);  \
    this.dtpDesde.ShowCheckBox = true; this.dtpDesde.Checked = false; this.dtpDesde.Size = new System.Drawing.Size(126, 23);
            this.dtpHasta.Format = System.Windows.Forms.DateTimePickerFormat.Short; this.dtpHasta.Location = new System.Drawing.Point(807, 82);  \
    this.dtpHasta.ShowCheckBox = true; this.dtpHasta.Checked = false; this.dtpHasta.Size = new System.Drawing.Size(126, 23);
            this.btnBuscar.BackColor = System.Drawing.Color.FromArgb(124, 58, 237); this.btnBuscar.FlatStyle = System.Windows.Forms.FlatStyle.Flat;  \
    this.btnBuscar.Location = new System.Drawing.Point(943, 78); this.btnBuscar.Size = new System.Drawing.Size(80, 31); this.btnBuscar.Text = "Buscar";  \
    this.btnBuscar.UseVisualStyleBackColor = false; this.btnBuscar.Click += new System.EventHandler(this.btnBuscar_Click);
            this.btnNuevo.BackColor = System.Drawing.Color.FromArgb(124, 58, 237); this.btnNuevo.FlatStyle = System.Windows.Forms.FlatStyle.Flat;  \
    this.btnNuevo.Location = new System.Drawing.Point(34, 126); this.btnNuevo.Size = new System.Drawing.Size(100, 32); this.btnNuevo.Text = "Nueva";  \
    this.btnNuevo.UseVisualStyleBackColor = false; this.btnNuevo.Click += new System.EventHandler(this.btnNuevo_Click);
            this.btnVer.FlatStyle = System.Windows.Forms.FlatStyle.Flat; this.btnVer.Location = new System.Drawing.Point(142, 126); this.btnVer.Size =  \
    new System.Drawing.Size(100, 32); this.btnVer.Text = "Ver detalle"; this.btnVer.Click += new System.EventHandler(this.btnVer_Click);
            this.btnGestionar.FlatStyle = System.Windows.Forms.FlatStyle.Flat; this.btnGestionar.Location = new System.Drawing.Point(250, 126);  \
    this.btnGestionar.Size = new System.Drawing.Size(100, 32); this.btnGestionar.Text = "Gestionar"; this.btnGestionar.Click += new  \
    System.EventHandler(this.btnGestionar_Click);
            this.btnCancelar.FlatStyle = System.Windows.Forms.FlatStyle.Flat; this.btnCancelar.Location = new System.Drawing.Point(358, 126);  \
    this.btnCancelar.Size = new System.Drawing.Size(100, 32); this.btnCancelar.Text = "Cancelar"; this.btnCancelar.Click += new  \
    System.EventHandler(this.btnCancelar_Click);
            this.btnExcel.FlatStyle = System.Windows.Forms.FlatStyle.Flat; this.btnExcel.Location = new System.Drawing.Point(884, 126);  \
    this.btnExcel.Size = new System.Drawing.Size(66, 32); this.btnExcel.Text = "Excel"; this.btnExcel.Click += new System.EventHandler(this.btnExcel_Click);
            this.btnPdf.FlatStyle = System.Windows.Forms.FlatStyle.Flat; this.btnPdf.Location = new System.Drawing.Point(958, 126); this.btnPdf.Size =  \
    new System.Drawing.Size(66, 32); this.btnPdf.Text = "PDF"; this.btnPdf.Click += new System.EventHandler(this.btnPdf_Click);
            this.dgvSolicitudes.AllowUserToAddRows = false; this.dgvSolicitudes.AllowUserToDeleteRows = false; this.dgvSolicitudes.AutoSizeColumnsMode =  \
    System.Windows.Forms.DataGridViewAutoSizeColumnsMode.Fill; this.dgvSolicitudes.BackgroundColor = System.Drawing.Color.FromArgb(18, 18, 27);  \
    this.dgvSolicitudes.Location = new System.Drawing.Point(34, 176); this.dgvSolicitudes.MultiSelect = false; this.dgvSolicitudes.ReadOnly = true;  \
    this.dgvSolicitudes.RowHeadersVisible = false; this.dgvSolicitudes.SelectionMode = System.Windows.Forms.DataGridViewSelectionMode.FullRowSelect;  \
```

UI/FrmSolicitudesAusencia.Designer.cs (continuacion)

```
    this.dgvSolicitudes.Size = new System.Drawing.Size(990, 478);
            this.dgvSolicitudes.Columns.Add("IdSolicitud", "Id"); this.dgvSolicitudes.Columns["IdSolicitud"].Visible = false;  \
    this.dgvSolicitudes.Columns.Add("Codigo", "Código"); this.dgvSolicitudes.Columns.Add("Empleado", "Empleado");  \
    this.dgvSolicitudes.Columns.Add("Departamento", "Departamento"); this.dgvSolicitudes.Columns.Add("Tipo", "Tipo");  \
    this.dgvSolicitudes.Columns.Add("Desde", "Desde"); this.dgvSolicitudes.Columns.Add("Hasta", "Hasta"); this.dgvSolicitudes.Columns.Add("Estado",  \
    "Estado"); this.dgvSolicitudes.Columns.Add("Solicitante", "Solicitante"); this.dgvSolicitudes.Columns.Add("Creacion", "Creada");
            this.lblResultado.AutoSize = true; this.lblResultado.Location = new System.Drawing.Point(34, 674); this.lblResultado.Text = "0 solicitudes";
            this.Controls.Add(this.lblTitulo); this.Controls.Add(this.txtBuscar); this.Controls.Add(this.cmbDepartamento);  \
    this.Controls.Add(this.cmbTipo); this.Controls.Add(this.cmbEstado); this.Controls.Add(this.dtpDesde); this.Controls.Add(this.dtpHasta);  \
    this.Controls.Add(this.btnBuscar); this.Controls.Add(this.btnNuevo); this.Controls.Add(this.btnVer); this.Controls.Add(this.btnGestionar);  \
    this.Controls.Add(this.btnCancelar); this.Controls.Add(this.btnExcel); this.Controls.Add(this.btnPdf); this.Controls.Add(this.dgvSolicitudes);  \
    this.Controls.Add(this.lblResultado); ((System.ComponentModel.ISupportInitialize)(this.dgvSolicitudes)).EndInit(); this.ResumeLayout(false);  \
    this.PerformLayout();
        }
    }
}
```

UI/UiFactory.cs

```
using System.Drawing;
using System.Windows.Forms;
using SistemaGestionNomina.Helpers;
using SistemaGestionNomina.UI.Controls;

namespace SistemaGestionNomina.UI
{
    public static class UiFactory
    {
        public static Label Title(string text, int x, int y, int width)
        {
            return ThemeHelper.Label(text, x, y, width, 40, 20F, FontStyle.Bold, ThemeHelper.Text);
        }

        public static Label Subtitle(string text, int x, int y, int width)
        {
            return ThemeHelper.Label(text, x, y, width, 28, 10.5F, FontStyle.Regular, ThemeHelper.TextMuted);
        }

        public static RoundedPanel Card(int x, int y, int width, int height)
        {
            RoundedPanel panel = new RoundedPanel();
            panel.Location = new Point(x, y);
            panel.Size = new Size(width, height);
            return panel;
        }

        public static RoundedButton PrimaryButton(string text, int x, int y, int width)
        {
            RoundedButton button = new RoundedButton();
            button.Text = text;
            button.Location = new Point(x, y);
            button.Width = width;
            button.BackColor = ThemeHelper.Accent;
            button.ForeColor = Color.FromArgb(48, 25, 92);
            button.BorderColor = ThemeHelper.Accent;
            return button;
        }

        public static RoundedButton SecondaryButton(string text, int x, int y, int width)
        {
            RoundedButton button = PrimaryButton(text, x, y, width);
            button.BackColor = Color.Black;
            button.ForeColor = ThemeHelper.Text;
            button.BorderColor = ThemeHelper.Border;
            return button;
        }

        public static DateTimePicker DatePicker(string name, int x, int y, int width)
        {
            DateTimePicker picker = new DateTimePicker();
            picker.Name = name;
            picker.Location = new Point(x, y);
            picker.Size = new Size(width, 30);
            picker.Format = DateTimePickerFormat.Short;
            picker.CalendarForeColor = ThemeHelper.Text;
            picker.CalendarMonthBackground = ThemeHelper.CardAlt;
            return picker;
        }

```

UI/UiFactory.cs (continuacion)

```
        public static DataGridView Grid(int x, int y, int width, int height)
        {
            DataGridView grid = new DataGridView();
            grid.Location = new Point(x, y);
            grid.Size = new Size(width, height);
            ThemeHelper.ApplyGrid(grid);
            return grid;
        }

        public static void ShowExported(string path)
        {
            if (string.IsNullOrWhiteSpace(path))
            {
                return;
            }

            MessageBox.Show("Archivo generado correctamente:\n" + path, "Exportación",
                MessageBoxButtons.OK, MessageBoxIcon.Information);
        }
    }
}
```